

Relatório Técnico do Pacote `gravmag_codes`

Uma plataforma computacional em Python para ensino, modelagem e processamento de dados geofísicos

Nelson Ribeiro Filho

31 de maio de 2026

Resumo

Este relatório apresenta a descrição técnica do pacote computacional `gravmag_codes`, desenvolvido em linguagem Python para aplicações em Geofísica Computacional, com ênfase em métodos potenciais, modelagem direta, filtragem, regressão, inversão, geração de modelos sintéticos, geomagnetismo, métodos elétricos, eletromagnéticos, sísmicos, radiométricos e perfilagem de poços. O documento descreve a estrutura geral do pacote, seus módulos, funções, formulações matemáticas, entradas, saídas e possibilidades de aplicação em ensino e pesquisa.

Sumário

1	Visão geral do pacote <code>gravmag_codes</code>	2
2	Descrição técnica do módulo <code>auxiliars.py</code>	7
2.1	Funções de validação e conversão para arrays	8
2.1.1	<code>my_asarray</code>	8
2.1.2	<code>my_check_same_shape</code>	9
2.1.3	<code>my_check_same_size</code>	9
2.2	Conversões angulares e funções trigonométricas	10
2.2.1	<code>my_deg2rad</code>	10
2.2.2	<code>my_rad2deg</code>	10
2.2.3	<code>my_trigonometrics_deg</code>	11

2.2.4	<code>my_trigonometrics_rad</code>	11
2.2.5	<code>my_asin</code> , <code>my_acos</code> e <code>my_atan</code>	12
2.2.6	<code>my_sqrt</code> , <code>my_log</code> e <code>my_safe_divide</code>	13
2.3	Funções de álgebra linear	14
2.3.1	<code>my_dot</code>	14
2.3.2	<code>my_hadamard</code>	15
2.3.3	<code>my_outer</code>	15
2.3.4	<code>my_inverse</code> e <code>my_pinv</code>	16
2.3.5	<code>my_LU</code>	17
2.4	Rotações tridimensionais e transformação de coordenadas	17
2.4.1	<code>my_xrotation</code>	17
2.4.2	<code>my_yrotation</code>	18
2.4.3	<code>my_zrotation</code>	18
2.4.4	<code>rotate3D_xyz</code>	19
2.4.5	<code>my_spherical2cartesian</code>	19
2.5	Ruído, resíduos e estatísticas básicas	20
2.5.1	<code>my_normalnoise</code>	20
2.5.2	<code>my_uniformnoise</code>	21
2.5.3	<code>my_residual</code>	21
2.5.4	<code>my_rms</code> , <code>my_rmse</code> e <code>my_mae</code>	22
2.6	Funções associadas à direção magnética	24
2.6.1	<code>my_dircos</code>	24
2.6.2	<code>my_regional</code>	24
2.6.3	<code>my_theta</code>	25
2.7	Funções associadas ao número de onda	26
2.7.1	<code>my_grid_spacing</code>	26
2.7.2	<code>my_wavenumber</code>	27
2.7.3	<code>my_wavenumber_modulus</code>	27
2.8	Funções de taper e padding para filtros FFT	28
2.8.1	<code>my_cosine_taper_1d</code>	28

2.8.2	<code>my_cosine_taper_2d</code>	29
2.8.3	<code>my_pad_grid</code>	29
2.8.4	<code>my_crop_grid</code>	30
2.9	Funções legadas de preenchimento	31
2.9.1	<code>my_padzeros</code>	31
2.9.2	<code>my_padones</code>	31
2.10	Funções de compatibilidade	31
2.10.1	<code>my_shape</code>	32
2.10.2	<code>my_size</code>	32
2.11	Síntese técnica do módulo	32
3	Descrição técnica do módulo <code>constants.py</code>	33
3.1	Constantes matemáticas fundamentais	34
3.2	Constantes gravimétricas	35
3.3	Constantes magnéticas	37
3.4	Constantes geodésicas e terrestres	38
3.5	Constantes de densidade	40
3.6	Conversões de comprimento, área e volume	42
3.7	Constantes físicas gerais	44
3.8	Pequenos valores numéricos	45
3.9	Função <code>my_show_constants</code>	46
3.10	Funções de conversão de densidade	46
3.10.1	<code>my_density_gcm3_to_kgm3</code>	46
3.10.2	<code>my_density_kgm3_to_gcm3</code>	47
3.11	Funções de conversão de distância	47
3.11.1	<code>my_m_to_km</code>	47
3.11.2	<code>my_km_to_m</code>	48
3.12	Funções de conversão gravimétrica	48
3.12.1	<code>my_si_to_mgal</code>	48
3.12.2	<code>my_mgal_to_si</code>	49

3.13	Funções de conversão magnética	49
3.13.1	my_tesla_to_nt	49
3.13.2	my_nt_to_tesla	50
3.14	Controle de exportação do módulo	50
3.15	Síntese técnica do módulo	51
4	Descrição técnica do módulo <code>cylinder.py</code>	52
4.1	Formulação geométrica do cilindro vertical	52
4.2	Funções internas de suporte	54
4.2.1	_check_observation_arrays	54
4.2.2	_prepare_z	54
4.2.3	_get_property	55
4.2.4	_dircos	55
4.3	Criação e propriedades geométricas do cilindro	56
4.3.1	my_cylinder_model	56
4.3.2	my_cylinder_volume	57
4.3.3	my_cylinder_cells	57
4.3.4	my_cylinder_to_prisms	59
4.4	Modelagem gravimétrica de um cilindro	60
4.4.1	my_cylinder_potential	60
4.4.2	my_cylinder_gx	61
4.4.3	my_cylinder_gy	62
4.4.4	my_cylinder_gz	62
4.4.5	my_cylinder_gxyz	63
4.5	Modelagem magnética de um cilindro	63
4.5.1	my_cylinder_bxyz	64
4.5.2	my_cylinder_bx, my_cylinder_by e my_cylinder_bz	65
4.5.3	my_cylinder_tfa	66
4.5.4	my_cylinder_tf	67
4.6	Modelagem de múltiplos cilindros	67

4.6.1	<code>my_cylinders_potential</code>	67
4.6.2	<code>my_cylinders_gx</code> , <code>my_cylinders_gy</code> e <code>my_cylinders_gz</code>	68
4.6.3	<code>my_cylinders_gxyz</code>	68
4.6.4	<code>my_cylinders_bxyz</code>	69
4.6.5	<code>my_cylinders_tfa</code>	69
4.6.6	<code>my_cylinders_tf</code>	70
4.7	Aliases de compatibilidade	70
4.8	Exemplo conceitual de uso	72
4.9	Síntese técnica do módulo	73
5	Descrição técnica do módulo <code>derivative.py</code>	73
5.1	Fundamentos espectrais das derivadas	74
5.2	Funções internas de validação, padding e FFT	75
5.2.1	<code>_check_grid_inputs</code>	75
5.2.2	<code>_as_output</code>	76
5.2.3	<code>_grid_spacing_from_coordinates</code>	76
5.2.4	<code>_get_wavenumbers</code>	77
5.2.5	<code>_cosine_taper_1d</code> e <code>_cosine_taper_2d</code>	77
5.2.6	<code>_pad_data</code> e <code>_crop_data</code>	78
5.2.7	<code>_fft_derivative_operator</code>	78
5.3	Derivadas no domínio da frequência	79
5.3.1	<code>my_xderiv</code>	79
5.3.2	<code>my_yderiv</code>	79
5.3.3	<code>my_zderiv</code>	80
5.3.4	<code>my_hgrad</code> e <code>my_totalgrad</code>	80
5.4	Funções com saída em grade	81
5.5	Derivadas mistas e de ordem superior	82
5.5.1	<code>my_mixed_deriv_fft</code>	82
5.5.2	<code>my_xzderiv</code> , <code>my_yzderiv</code> , <code>my_xxderiv</code> , <code>my_yyderiv</code> e <code>my_zzderiv</code> .	83
5.6	Derivadas no domínio espacial	83

5.6.1	<code>my_xderiv_space</code> e <code>my_yderiv_space</code>	83
5.6.2	<code>my_hgrad_space</code> , <code>my_laplacian_space</code> e <code>my_totalgrad_space</code> . .	84
5.7	Atributos derivados: <code>tilt</code> , <code>theta map</code> e <code>THDR</code>	85
5.7.1	<code>my_tilt_derivative</code>	85
5.7.2	<code>my_tilt_vertical_derivative</code>	85
5.7.3	<code>my_theta_map</code>	86
5.7.4	<code>my_thdr_tilt</code>	86
5.8	Aliases de compatibilidade	87
5.9	Síntese técnica do módulo	87
6	Descrição técnica do módulo <code>electrical_methods.py</code>	87
6.1	Funções auxiliares internas e salvamento de figuras	88
6.1.1	<code>_arr</code>	88
6.1.2	<code>my_save_figure</code>	89
6.2	Resistividade, condutividade e resistividade aparente	89
6.2.1	<code>my_resistivity_from_conductivity</code>	89
6.2.2	<code>my_conductivity_from_resistivity</code>	90
6.2.3	<code>my_apparent_resistivity_from_k</code>	90
6.3	Fatores geométricos Wenner e Schlumberger	91
6.3.1	<code>my_wenner_geometric_factor</code>	91
6.3.2	<code>my_schlumberger_geometric_factor</code>	91
6.4	Modelo de Terra em camadas e curva SEV	92
6.4.1	<code>my_layered_earth_model</code>	92
6.4.2	<code>my_sev_forward_smooth_transition</code>	92
6.4.3	<code>my_classify_sev_curve</code>	93
6.5	Potencial elétrico e campo elétrico	94
6.5.1	<code>my_point_current_potential</code>	94
6.5.2	<code>my_surface_point_current_potential</code>	95
6.5.3	<code>my_dipole_potential_surface</code>	95
6.5.4	<code>my_electric_field_from_potential</code>	96

6.6	Seção geoeétrica	96
6.6.1	my_geoelectric_section	96
6.7	Funções de visualização	97
6.8	Exemplo conceitual de uso	98
6.9	Síntese técnica do módulo	99
7	Descrição técnica do módulo <code>electromagnetic_methods.py</code>	99
7.1	Funções auxiliares internas	100
7.1.1	_arr	100
7.1.2	my_save_figure	101
7.2	Frequência angular, número de onda e impedância	101
7.2.1	my_angular_frequency	101
7.2.2	my_wave_number	102
7.2.3	my_intrinsic_impedance	102
7.3	Profundidade de penetração, atenuação e velocidade de fase	103
7.3.1	my_skin_depth	103
7.3.2	my_plane_wave_attenuation	104
7.3.3	my_phase_velocity	104
7.4	Campo de dipolo magnético e bobinas	105
7.4.1	my_magnetic_dipole_field	105
7.4.2	my_loop_axis_field	105
7.4.3	my_coil_moment	106
7.4.4	my_coil_dipole_field	106
7.5	Funções associadas ao GPR	107
7.5.1	my_gpr_velocity	107
7.5.2	my_gpr_two_way_time	108
7.5.3	my_gpr_depth_from_time	108
7.5.4	my_gpr_vertical_resolution	108
7.6	Resposta de indução eletromagnética e imagem sintética	109
7.6.1	my_em_induction_response	109

7.6.2	<code>my_em_image_from_anomalies</code>	109
7.7	Difusão eletromagnética unidimensional	110
7.7.1	<code>my_1d_em_diffusion_step</code>	110
7.7.2	<code>my_1d_em_diffusion</code>	111
7.8	Funções de visualização	111
7.9	Síntese técnica do módulo	112
8	Descrição técnica do módulo <code>equivalentlayer.py</code>	112
8.1	Constantes internas e funções auxiliares	113
8.1.1	<code>_gravity_constant</code> , <code>_si2mgal</code> , <code>_cm_constant</code> e <code>_t2nt</code>	114
8.1.2	<code>_as_array_1d</code>	114
8.1.3	<code>_check_observations</code>	114
8.1.4	<code>_check_dataset</code>	115
8.1.5	<code>_solve_tikhonov</code> e <code>my_tikhonov_solve</code>	115
8.2	Construção da camada equivalente	116
8.2.1	<code>my_layer</code>	116
8.2.2	<code>my_layer_array</code>	117
8.3	Matrizes de sensibilidade gravimétrica	117
8.3.1	<code>my_gz_layer</code>	117
8.3.2	<code>my_gxyz_layer</code>	118
8.4	Ajuste e predição gravimétrica	118
8.4.1	<code>my_predict_grav</code>	118
8.4.2	<code>my_fitdata_grav</code> e <code>my_fitdata_grav_full</code>	119
8.5	Camada equivalente magnética	119
8.5.1	<code>my_totalfield_layer</code>	120
8.5.2	<code>my_predict_mag</code>	120
8.5.3	<code>my_fitdata_mag</code> e <code>my_fitdata_mag_full</code>	121
8.6	Transformações magnéticas	121
8.6.1	<code>my_transform_layer</code>	121
8.6.2	<code>my_rtp_layer</code>	122

8.6.3	<code>my_rte_layer</code>	122
8.7	Resíduos e métricas de ajuste	123
8.7.1	<code>my_residual</code>	123
8.7.2	<code>my_misfit</code>	123
8.8	Exemplo conceitual de uso	124
8.9	Síntese técnica do módulo	125
9	Descrição técnica do módulo <code>filtering.py</code>	125
9.1	Base matemática da filtragem espectral	126
9.2	Funções internas de validação, número de onda, padding e FFT	127
9.2.1	<code>_check_grid</code>	127
9.2.2	<code>_as_grid</code>	127
9.2.3	<code>_grid_spacing</code> e <code>_wavenumbers</code>	128
9.2.4	<code>_cosine_taper_1d</code> , <code>_cosine_taper_2d</code> , <code>_pad_grid</code> e <code>_crop_grid</code> .	128
9.2.5	<code>_apply_fft_operator</code>	129
9.3	Continuação de campos potenciais	130
9.3.1	<code>my_continuation_grid</code> e <code>my_continuation</code>	130
9.3.2	<code>my_upward_continuation</code>	130
9.3.3	<code>my_downward_continuation_regularized</code>	131
9.4	Filtros Gaussianos	132
9.4.1	<code>my_lowpass_gaussian</code>	132
9.4.2	<code>my_highpass_gaussian</code>	132
9.4.3	<code>my_bandpass_gaussian</code>	133
9.5	Derivada direcional	133
9.6	Transformações magnéticas: redução ao polo e redução ao equador	134
9.6.1	<code>reduction_grid</code> e <code>reduction</code>	134
9.6.2	<code>my_rtp</code> e <code>my_rte</code>	135
9.7	Pseudogravidade	135
9.8	Atributos de realce: tilt, THDR, theta map e cosseno do tilt	136
9.8.1	<code>my_tilt_grid</code> e <code>my_tilt</code>	136

9.8.2	<code>my_tilt_vdr_grid</code> e <code>my_tilt_vdr</code>	137
9.8.3	<code>my_thdr_grid</code> e <code>my_thdr</code>	137
9.8.4	<code>my_hyperbolic_tilt_grid</code> e <code>my_hyperbolic_tilt</code>	137
9.8.5	<code>my_thetamap_grid</code> , <code>my_thetamap</code> , <code>my_cos_tilt_grid</code> e <code>my_cos_tilt</code>	137
9.9	Estimativas aproximadas de profundidade pelo tilt	138
9.10	Síntese técnica do módulo	138
10	Descrição técnica do módulo <code>geomagnetism.py</code>	139
10.1	Funções auxiliares de entrada, salvamento e conversão angular	140
10.2	Conversões de coordenadas	140
10.2.1	<code>my_geodetic_to_ecef</code>	140
10.2.2	<code>my_ecef_to_spherical</code> , <code>my_geodetic_to_spherical</code> e <code>my_spherical_to_geocentric_latlon</code>	141
10.3	Componentes, elementos e direções do campo geomagnético	142
10.3.1	<code>my_local_unit_vectors</code>	142
10.3.2	<code>my_direction_cosines</code> , <code>my_components_from_fid</code> e <code>my_elements_from_components</code>	142
10.3.3	<code>my_total_field_anomaly_from_vector</code>	143
10.4	Campo de dipolo magnético e dipolo axial centrado	143
10.4.1	<code>my_magnetic_dipole_field_cartesian</code>	143
10.4.2	<code>my_centered_axial_dipole_elements</code>	144
10.5	Harmônicos esféricos e coeficientes de Gauss	144
10.5.1	<code>my_schmidt_quasi_normalization</code> , <code>my_associated_legendre_schmidt</code> e <code>my_associated_legendre_unnormalized</code>	145
10.5.2	<code>my_empty_gauss_coefficients</code> , <code>my_set_gauss_coefficient</code> , <code>my_synthetic_gauss_coefficients</code> e <code>my_gauss_coefficients_from_arrays</code>	145
10.5.3	<code>my_internal_potential_spherical</code> , <code>my_internal_field_spherical</code> e <code>my_internal_field_elements</code>	146
10.6	Grades, campos sintéticos e variação secular	146
10.7	Filtros magnéticos no domínio da frequência	147

10.8	Perfis, mapas e visualização geomagnética	148
10.9	Anomalias crustais, campo observado e campo residual	149
10.10	Extração e salvamento de dados geomagnéticos	150
10.11	Exemplo conceitual de uso	151
10.12	Síntese técnica do módulo	152
11	Descrição técnica do módulo <code>gravity.py</code>	152
11.1	Funções auxiliares internas	153
11.1.1	<code>_asarray_float</code>	153
11.1.2	<code>_return_like_input</code>	153
11.1.3	<code>_check_same_shape</code>	154
11.2	Elipsoides de referência	154
11.2.1	<code>my_WGS84</code>	154
11.2.2	<code>my_GRS80</code>	155
11.2.3	<code>my_get_ellipsoid</code>	155
11.3	Gravidade normal	155
11.3.1	<code>my_somigliana</code>	156
11.3.2	<code>my_closedform</code>	156
11.3.3	<code>my_normal_gravity</code>	157
11.4	Correção ar-livre e anomalia ar-livre	157
11.4.1	<code>my_freeair_correction</code>	157
11.4.2	<code>my_freeair_anomaly</code>	158
11.5	Correção de Bouguer e anomalia de Bouguer	158
11.5.1	<code>my_bouguer_correction</code>	158
11.5.2	<code>my_bouguer_anomaly</code>	159
11.6	Modelo isostático de Airy e Moho	160
11.6.1	<code>my_Airy</code>	160
11.6.2	<code>my_moho_airy</code>	160
11.7	Correção e anomalia isostática	161
11.7.1	<code>my_isostatic_correction</code>	161

11.7.2	<code>my_isostatic_anomaly</code>	161
11.8	Distúrbio da gravidade	162
11.9	Correção de Eötvös	162
11.10	Aliases de compatibilidade	163
11.11	Síntese técnica do módulo	164
12	Descrição técnica do módulo <code>grids.py</code>	164
12.1	Funções internas de validação	165
12.1.1	<code>_check_area</code>	165
12.1.2	<code>_check_shape</code>	165
12.1.3	<code>_as_level_array</code>	166
12.2	Informações básicas de malhas	166
12.2.1	<code>my_shape_to_size</code>	166
12.2.2	<code>my_grid_spacing</code>	167
12.2.3	<code>my_grid_area</code>	167
12.2.4	<code>my_grid_shape_from_vectors</code>	167
12.3	Conversão entre grade e vetor	168
12.3.1	<code>my_as_grid</code>	168
12.3.2	<code>my_as_vector</code>	168
12.4	Malhas regulares e irregulares	168
12.4.1	<code>my_regular</code> e <code>my_regular_grid</code>	169
12.4.2	<code>my_irregular</code> e <code>my_random</code>	169
12.5	Perfis e interpolação	170
12.5.1	<code>my_profile</code>	170
12.5.2	<code>my_1Dinterpolation</code>	170
12.5.3	<code>my_griddata</code>	171
12.6	Grades especiais	171
12.7	Transformações geométricas	172
12.7.1	<code>my_rotate_grid</code>	172
12.7.2	<code>my_translate_grid</code>	173

12.8	Reamostragem, subgrades e ponto mais próximo	173
12.8.1	my_resample_grid	173
12.8.2	my_extract_subgrid	173
12.8.3	my_nearest_point	174
12.9	Conversão para tabela, salvamento e leitura	174
12.9.1	my_grid_to_table	174
12.9.2	my_save_grid e my_load_grid	175
12.10	Aliases de compatibilidade	175
12.11	Exemplo conceitual de uso	175
12.12	Síntese técnica do módulo	176
13	Descrição técnica do módulo hexagonalprism.py	177
13.1	Formulação geométrica do prisma hexagonal	177
13.2	Funções internas de suporte	178
13.2.1	_as_array_like_z	178
13.2.2	_check_observation_points	179
13.2.3	_get_physical_property	179
13.2.4	_direction_cosines	180
13.2.5	_safe_radius	180
13.3	Criação do modelo e propriedades geométricas	180
13.3.1	my_hexagonal_prism_model	181
13.3.2	my_hexagonal_vertices	181
13.3.3	my_hexagonal_area, my_hexagonal_volume e my_hexagonal_center	182
13.4	Discretização do prisma hexagonal	182
13.4.1	my_point_in_polygon	182
13.4.2	my_discretize_hexagonal_prism	183
13.5	Modelagem gravimétrica	183
13.5.1	my_hexagonal_prism_potential	184
13.5.2	my_hexagonal_prism_gx, my_hexagonal_prism_gy e my_hexagonal_prism_gz	184

13.5.3	<code>my_hexagonal_prism_gxyz</code>	185
13.6	Modelagem magnética	185
13.6.1	<code>my_hexagonal_prism_bxyz</code>	186
13.6.2	<code>my_hexagonal_prism_bx</code> , <code>my_hexagonal_prism_by</code> e <code>my_hexagonal_prism_bz</code>	186
13.6.3	<code>my_hexagonal_prism_tfa</code> e <code>my_hexagonal_prism_tf</code>	186
13.7	Modelagem de múltiplos prismas hexagonais	187
13.8	Conversão para prismas retangulares	188
13.8.1	<code>my_hexagonal_prism_to_prisms</code>	188
13.9	Síntese técnica do módulo	188
14	Descrição técnica do módulo <code>inversion.py</code>	189
14.1	Funções internas de preparação e métricas seguras	189
14.1.1	<code>_as_1d</code> e <code>_as_2d</code>	189
14.1.2	<code>_check_matrix_vector</code>	190
14.1.3	<code>_safe_norm</code> e <code>_safe_corr</code>	190
14.2	Resíduos e métricas de erro	190
14.2.1	<code>my_residual</code>	190
14.2.2	<code>my_rmse</code> , <code>my_mae</code> e <code>my_misfit_report</code>	191
14.3	Matrizes de regularização	191
14.3.1	<code>my_damping_matrix</code>	191
14.3.2	<code>my_first_difference_matrix</code> e <code>my_second_difference_matrix</code>	192
14.3.3	<code>my_2d_first_difference_matrix</code> e <code>my_2d_laplacian_matrix</code>	192
14.3.4	<code>my_regularization_matrix</code>	193
14.4	Inversão linear	193
14.4.1	<code>my_least_squares</code>	193
14.4.2	<code>my_weighted_least_squares</code>	193
14.4.3	<code>my_tikhonov</code>	194
14.4.4	<code>my_linear_inversion</code>	194
14.5	Ajuste polinomial e linearização	195

14.5.1	<code>my_polynomial_design_matrix</code>	195
14.5.2	<code>my_polynomial_fit</code>	195
14.5.3	<code>my_curve_fit_linearized</code>	196
14.6	Derivadas numéricas e função objetivo	196
14.6.1	<code>my_numerical_jacobian</code>	196
14.6.2	<code>my_numerical_gradient</code>	197
14.6.3	<code>my_objective_function</code>	197
14.7	Inversão não linear	197
14.7.1	<code>my_gauss_newton</code>	197
14.7.2	<code>my_levenberg_marquardt</code>	198
14.7.3	<code>my_nonlinear_inversion</code>	198
14.8	Curva L	199
14.8.1	<code>my_lcurve</code>	199
14.8.2	<code>my_best_lcurve_corner</code>	199
14.9	Matrizes diferenciais e elementos finitos	200
14.10	Salvamento de resultados	200
14.11	Exemplo conceitual de uso	200
14.12	Síntese técnica do módulo	201
15	Descrição técnica do módulo <code>kernel.py</code>	202
15.1	Funções internas de preparação	203
15.1.1	<code>_as_array</code>	203
15.1.2	<code>_check_observation_arrays</code>	203
15.1.3	<code>_parse_model</code>	204
15.2	Volume de esfera e coordenadas relativas	204
15.2.1	<code>my_sphere_volume</code>	204
15.2.2	<code>my_kernel_coordinates</code>	205
15.2.3	<code>my_kernel_r</code>	205
15.3	Núcleo fundamental e derivadas de primeira ordem	206
15.3.1	<code>my_kernel0</code> e <code>my_kernel_potential</code>	206

15.3.2	<code>my_kernelx</code> , <code>my_kernely</code> e <code>my_kernelz</code>	206
15.4	Derivadas de segunda ordem e tensor Hessiano	207
15.4.1	<code>my_kernelxx</code> , <code>my_kernelxy</code> , <code>my_kernelxz</code> , <code>my_kernelyy</code> , <code>my_kernelyz</code> e <code>my_kernelzz</code>	207
15.4.2	<code>my_kernel_gradient</code> , <code>my_kernel_hessian</code> , <code>my_kernel_tensor_matrix</code> e <code>my_kernel_laplacian</code>	208
15.5	Derivadas de terceira ordem	209
15.6	Operadores direcionais	209
15.6.1	<code>my_direction_vector</code>	209
15.6.2	<code>my_directional_first_kernel</code>	210
15.6.3	<code>my_directional_second_kernel</code>	210
15.7	Kernels gravimétricos e magnéticos	211
15.7.1	<code>my_gravity_gx_kernel</code> , <code>my_gravity_gy_kernel</code> e <code>my_gravity_gz_kernel</code>	211
15.7.2	<code>my_gravity_gradient_kernel</code>	211
15.7.3	<code>my_totalfield_kernel</code> e <code>my_magnetic_totalfield_kernel</code>	212
15.8	Matrizes de kernels	212
15.8.1	<code>my_kernel_matrix</code>	212
15.8.2	<code>my_gravity_kernel_matrix</code>	213
15.8.3	<code>my_totalfield_kernel_matrix</code>	213
15.9	Síntese técnica do módulo	214
16	Descrição técnica do módulo <code>plotting.py</code>	214
16.1	Funções internas de preparação	215
16.1.1	<code>_to_array</code>	215
16.1.2	<code>_finite_data</code>	215
16.1.3	<code>_check_same_shape</code>	216
16.1.4	<code>_get_fig_ax</code>	216
16.1.5	<code>_format_axis</code>	216
16.1.6	<code>_reshape_grid</code>	217
16.2	Colormaps, normalização e barra de cores	217

16.2.1	<code>my_get_colormap</code>	217
16.2.2	<code>my_get_norm</code>	217
16.2.3	<code>my_add_colorbar</code>	218
16.3	Desenho de formas geométricas	218
16.4	Gráficos básicos	219
16.4.1	<code>my_plot</code>	219
16.4.2	<code>my_scatter</code>	219
16.4.3	<code>my_histogram</code>	220
16.5	Mapas em grade regular	220
16.5.1	<code>my_contourf</code>	220
16.5.2	<code>my_contour</code>	221
16.5.3	<code>my_pcolormesh</code>	221
16.6	Mapas com triangulação irregular	221
16.6.1	<code>my_tricontourf</code> e <code>my_tricontour</code>	221
16.7	Funções cartográficas com Cartopy	222
16.7.1	<code>_import_cartopy</code>	222
16.7.2	<code>_infer_area</code>	223
16.7.3	<code>my_cartopy_basemap</code>	223
16.7.4	<code>my_cartopy_scatter</code> , <code>my_cartopy_tricontourf</code> e <code>my_cartopy_tricontour</code>	223
16.8	Salvamento de figuras	224
16.8.1	<code>my_save_figure</code>	224
16.9	Exemplo conceitual de uso	224
16.10	Síntese técnica do módulo	225
17	Descrição técnica do módulo <code>polygonalprism.py</code>	226
17.1	Formulação geométrica do prisma poligonal	226
17.2	Funções internas de suporte	227
17.2.1	<code>_as_array_like_z</code>	227
17.2.2	<code>_check_observation_points</code>	228

17.2.3	<code>_get_physical_property</code>	228
17.2.4	<code>_direction_cosines</code>	229
17.3	Criação do modelo e propriedades geométricas	229
17.3.1	<code>my_polygonal_prism_model</code>	229
17.3.2	<code>my_polygonal_area</code>	229
17.3.3	<code>my_polygonal_centroid</code>	230
17.3.4	<code>my_polygonal_bounds</code> , <code>my_polygonal_volume</code> e <code>my_polygonal_center</code>	230
17.4	Discretização do prisma poligonal	231
17.4.1	<code>my_point_in_polygon</code>	231
17.4.2	<code>my_discretize_polygonal_prism</code>	231
17.5	Modelagem gravimétrica	232
17.5.1	<code>my_polygonal_prism_potential</code>	232
17.5.2	<code>my_polygonal_prism_gx</code> , <code>my_polygonal_prism_gy</code> e <code>my_polygonal_prism_gz</code>	233
17.5.3	<code>my_polygonal_prism_gxyz</code>	233
17.6	Modelagem magnética	234
17.6.1	<code>my_polygonal_prism_bxyz</code>	234
17.6.2	<code>my_polygonal_prism_bx</code> , <code>my_polygonal_prism_by</code> e <code>my_polygonal_prism_bz</code>	235
17.6.3	<code>my_polygonal_prism_tfa</code> e <code>my_polygonal_prism_tf</code>	235
17.7	Modelagem de múltiplos prismas poligonais	235
17.8	Conversão para prismas retangulares	236
17.8.1	<code>my_polygonal_prism_to_prisms</code>	236
17.9	Síntese técnica do módulo	237
18	Descrição técnica do módulo <code>prism.py</code>	237
18.1	Formulação geométrica do prisma retangular	238
18.2	Funções internas de suporte	239
18.2.1	<code>_check_prism</code>	239
18.2.2	<code>_check_observation_arrays</code>	239

18.2.3	<code>_get_property</code>	239
18.2.4	<code>_log</code> e <code>_atan2</code>	240
18.3	Criação, volume e centro do prisma	240
18.3.1	<code>my_prism_model</code>	240
18.3.2	<code>my_prism_volume</code>	240
18.3.3	<code>my_prism_center</code>	241
18.4	Modelagem gravimétrica de um prisma	241
18.4.1	<code>my_prism_potential</code>	241
18.4.2	<code>my_prism_gx</code> , <code>my_prism_gy</code> e <code>my_prism_gz</code>	242
18.4.3	<code>my_prism_gxyz</code>	242
18.5	Tensor de gradiente gravimétrico	243
18.5.1	<code>my_prism_gxx</code> , <code>my_prism_gxy</code> , <code>my_prism_gxz</code> , <code>my_prism_gyy</code> , <code>my_prism_gyz</code> e <code>my_prism_gzz</code>	243
18.6	Modelagem magnética de um prisma	244
18.6.1	<code>my_prism_bxyz</code>	244
18.6.2	<code>my_prism_bx</code> , <code>my_prism_by</code> e <code>my_prism_bz</code>	244
18.6.3	<code>my_prism_tfa</code> e <code>my_prism_tf</code>	245
18.7	Modelagem de múltiplos prismas	245
18.8	Aliases de compatibilidade	246
18.9	Exemplo conceitual de uso	247
18.10	Síntese técnica do módulo	247
19	Descrição técnica do módulo <code>radiometry_welllog_methods.py</code>	248
19.1	Funções auxiliares internas e salvamento	249
19.1.1	<code>_arr</code>	249
19.1.2	<code>_safe_divide</code>	249
19.1.3	<code>_normalize_01</code>	250
19.1.4	<code>my_save_figure</code>	250
19.2	Processamento básico e estatísticas	250
19.3	Radiometria gama	251

19.3.1	<code>my_total_count</code>	251
19.3.2	<code>my_radiometric_ratios</code>	252
19.3.3	<code>my_f_parameter</code>	252
19.3.4	<code>my_ternary_rgb</code>	253
19.3.5	<code>my_classify_radiometric_domain</code>	253
19.4	Mapas radiométricos sintéticos	254
19.5	Perfilagem de poços e propriedades petrofísicas	254
19.5.1	<code>my_gamma_ray_index</code> e <code>my_vshale_linear</code>	254
19.5.2	<code>my_density_porosity</code>	255
19.5.3	<code>my_sonic_porosity</code>	255
19.5.4	<code>my_velocity_from_sonic</code>	256
19.5.5	<code>my_archie_water_saturation</code>	256
19.5.6	<code>my_acoustic_impedance</code> e <code>my_reflection_coefficient</code>	257
19.6	Indicadores de reservatório	257
19.7	Perfis sintéticos de poço	258
19.8	Funções de visualização	258
19.9	Exemplo conceitual de uso	258
19.10	Síntese técnica do módulo	259
20	Descrição técnica do módulo <code>regres.py</code>	260
20.1	Funções auxiliares internas	261
20.2	Métricas de ajuste e resíduos	261
20.2.1	<code>my_residual</code>	261
20.2.2	<code>my_sse</code> , <code>my_mse</code> , <code>my_rmse</code> e <code>my_mae</code>	261
20.2.3	<code>my_r2_score</code> e <code>my_correlation</code>	262
20.3	Mínimos quadrados e regularização	262
20.3.1	<code>my_least_squares</code>	262
20.3.2	<code>my_weighted_least_squares</code>	263
20.3.3	<code>my_tikhonov</code>	263
20.4	Regressão linear simples e múltipla	264

20.4.1	<code>my_linear_regression</code>	264
20.4.2	<code>my_multiple_linear_regression</code>	264
20.5	Matrizes de desenho polinomial	265
20.5.1	<code>my_polynomial_matrix_1d</code>	265
20.5.2	<code>my_polynomial_matrix_2d</code>	265
20.6	Ajuste polinomial	266
20.6.1	<code>my_polynomial_fit_1d</code>	266
20.6.2	<code>my_polynomial_fit_2d</code>	266
20.7	Separação regional-residual	266
20.7.1	<code>my_remove_polynomial_regional_2d</code>	267
20.8	Regressão robusta	267
20.9	Interpolação de Lagrange	268
20.9.1	<code>my_lagrange_basis</code> e <code>my_lagrange_interpolation</code>	268
20.10	Salvamento de resultados	268
20.11	Exemplo conceitual de uso	269
20.12	Síntese técnica do módulo	269
21	Descrição técnica do módulo <code>seismic_methods.py</code>	270
21.1	Funções auxiliares e salvamento de figuras	271
21.2	Wavelets e traços sintéticos	271
21.2.1	<code>my_ricker_wavelet</code>	271
21.2.2	<code>my_synthetic_trace</code>	272
21.3	Propriedades elásticas e velocidades sísmicas	272
21.3.1	<code>my_vp_from_elastic</code>	273
21.3.2	<code>my_vs_from_elastic</code>	273
21.3.3	<code>my_poisson_ratio</code>	273
21.3.4	<code>my_rayleigh_velocity_approx</code> e <code>my_love_wave_velocity_simple</code>	274
21.4	Tempos de viagem e reflexão sísmica	274
21.4.1	<code>my_two_way_time</code>	274
21.4.2	<code>my_depth_from_two_way_time</code>	275

21.4.3	<code>my_normal_moveout</code>	275
21.5	Impedância acústica e coeficientes de reflexão	275
21.5.1	<code>my_acoustic_impedance</code>	275
21.5.2	<code>my_reflection_coefficient</code>	276
21.5.3	<code>my_transmission_coefficient</code>	276
21.6	Atenuação, energia e momento sísmico	276
21.6.1	<code>my_attenuation_exponential</code>	277
21.6.2	<code>my_seismic_energy</code>	277
21.6.3	<code>my_moment_magnitude</code>	277
21.7	Osciladores e equação da onda	278
21.7.1	<code>my_harmonic_oscillator</code>	278
21.7.2	<code>my_wave_equation_1d_step</code>	278
21.7.3	<code>my_wave_equation_2d_step</code>	278
21.8	Tensor momento e padrões de radiação	279
21.9	Visualização sísmica	279
21.10	Exemplo conceitual de uso	280
21.11	Síntese técnica do módulo	280
22	Descrição técnica do módulo <code>sphere.py</code>	281
22.1	Formulação geométrica da esfera	281
22.2	Funções internas de suporte	282
22.3	Criação do modelo, volume e distância	282
22.3.1	<code>my_sphere_model</code>	282
22.3.2	<code>my_sphere_volume</code>	283
22.3.3	<code>my_sphere_distance</code>	283
22.4	Modelagem gravimétrica de uma esfera	284
22.4.1	<code>my_sphere_potential</code>	284
22.4.2	<code>my_sphere_gx</code> , <code>my_sphere_gy</code> e <code>my_sphere_gz</code>	284
22.4.3	<code>my_sphere_gxyz</code>	285
22.5	Tensor de gradiente gravimétrico	285

22.6	Modelagem magnética de uma esfera	286
22.6.1	my_sphere_bxyz	287
22.6.2	my_sphere_bx, my_sphere_by e my_sphere_bz	287
22.6.3	my_sphere_tfa e my_sphere_tf	288
22.7	Modelagem de múltiplas esferas	288
22.8	Aliases de compatibilidade	289
22.9	Exemplo conceitual de uso	289
22.10	Síntese técnica do módulo	290
23	Descrição técnica do módulo <code>statistical.py</code>	290
23.1	Estatísticas descritivas básicas	291
23.2	Normalização e padronização	292
23.3	Resíduos e métricas de erro	293
23.4	Correlação e covariância	294
23.5	Identificação de outliers	294
23.6	Relatórios estatísticos	295
23.7	Aplicações em testes sintéticos	296
23.8	Síntese técnica do módulo	296
24	Descrição técnica do módulo <code>synthetic.py</code>	297
24.1	Criação de grades de observação	297
24.2	Modelos sintéticos de fontes simples	299
24.3	Modelos compostos	299
24.4	Topografia sintética	300
24.5	Campos sintéticos de densidade e magnetização	301
24.6	Geração de dados gravimétricos sintéticos	301
24.7	Geração de dados magnéticos sintéticos	302
24.8	Adição de ruído sintético	302
24.9	Comparação entre dados verdadeiros e preditos	303
24.10	Exemplo conceitual de uso	304
24.11	Organização de experimentos sintéticos	305

24.12	Síntese técnica do módulo	305
25	Descrição técnica do módulo <code>transform.py</code>	306
25.1	Transformações por continuação vertical	307
25.2	Derivadas espectrais	307
25.3	Gradiente horizontal, gradiente total e sinal analítico	308
25.4	Redução ao polo e redução ao equador	309
25.5	Pseudogravidade	310
25.6	Separação regional-residual	310
25.7	Atributos tilt e theta map	311
25.8	Exemplo conceitual de uso	311
25.9	Síntese técnica do módulo	312
26	Descrição técnica do módulo <code>utils.py</code>	313
26.1	Validação de entradas	313
26.2	Conversão e preparação de arrays	314
26.3	Criação de diretórios e organização de saídas	314
26.4	Salvamento de tabelas e resultados	315
26.5	Mensagens, relatórios e impressão organizada	315
26.6	Controle de dependências	316
26.7	Medição de tempo de execução	316
26.8	Reprodutibilidade	316
26.9	Exemplo conceitual de uso	317
26.10	Síntese técnica do módulo	318
27	Síntese integrada do pacote <code>gravmag</code>	318
27.1	Estrutura conceitual do pacote	319
27.2	Fluxo computacional geral	319
27.3	Contribuição didática	320
27.4	Contribuição científica	321
28	Organização recomendada para disponibilização no GitHub e instalação	

via pip	321
28.1 Estrutura básica sugerida	322
28.2 Arquivo <code>__init__.py</code>	323
28.3 Arquivo <code>pyproject.toml</code>	324
28.4 Arquivo <code>README.md</code>	326
28.5 Instalação local para testes	326
28.6 Testes automatizados	327
28.7 Licença e citação	328
28.8 Síntese da organização para distribuição	328

1 Visão geral do pacote *gravmag_codes*

O pacote *gravmag_codes* foi desenvolvido como uma plataforma computacional em linguagem Python voltada ao ensino, à modelagem, ao processamento e à interpretação de dados geofísicos. Sua estrutura reúne um conjunto amplo de rotinas aplicadas principalmente aos métodos potenciais, com ênfase em gravimetria e magnetometria, mas também contempla módulos relacionados a geomagnetismo, sísmica, métodos elétricos, métodos eletromagnéticos, radiometria, perfilagem de poços, regressão, inversão geofísica, geração de modelos sintéticos e visualização científica.

Do ponto de vista educacional, o pacote busca aproximar conceitos físicos, matemáticos e computacionais em um único ambiente de programação. Essa integração permite que estudantes e pesquisadores compreendam não apenas o uso operacional das funções, mas também os fundamentos matemáticos envolvidos em cada etapa do processamento geofísico. Assim, o pacote pode ser utilizado em disciplinas de Geofísica Geral, Geofísica Aplicada, Métodos Potenciais, Modelagem Matemática, Métodos Numéricos, Programação Científica e Inversão Geofísica.

Finalidade geral do pacote

O *gravmag_codes* tem como finalidade fornecer um conjunto organizado de funções para construir modelos sintéticos, calcular respostas gravimétricas e magnéticas, aplicar filtros e derivadas, realizar ajustes matemáticos, resolver problemas inversos, visualizar campos geofísicos e apoiar atividades de ensino e pesquisa em Geofísica Computacional.

A organização modular do pacote permite que cada arquivo *.py* concentre um grupo específico de operações. Essa estrutura favorece a manutenção do código, a reutilização das funções e a expansão futura do projeto. Além disso, a separação dos módulos facilita o uso didático do pacote, pois cada conjunto de funções pode ser estudado de forma independente, de acordo com o conteúdo abordado em sala de aula ou em atividades de pesquisa.

Em linhas gerais, o pacote pode ser compreendido como uma biblioteca de apoio para três grandes finalidades. A primeira é a modelagem direta, na qual propriedades físicas e geometrias de fontes são fornecidas para calcular o campo geofísico produzido. A segunda é o processamento e a transformação de dados, incluindo filtros, derivadas, separação regional-residual, continuação de campos e operadores de realce. A terceira é a estimação de parâmetros, envolvendo regressão, ajuste polinomial, inversão linear, inversão não linear e regularização.

A modelagem direta gravimétrica e magnética é um dos eixos centrais do pacote. Em Geofísica, a modelagem direta consiste em calcular a resposta física esperada a partir de um modelo conhecido do subsolo. No caso gravimétrico, parte-se de uma distribuição de densidade e calcula-se o campo gravitacional produzido por essa distribuição. No caso magnético, parte-se de uma distribuição de magnetização e calcula-se a anomalia magnética associada. Em ambos os casos, o objetivo é compreender como corpos geológicos com diferentes geometrias, profundidades e propriedades físicas afetam o campo observado na superfície ou em outro nível de observação.

A formulação física geral da gravimetria baseia-se na lei da gravitação universal. Para uma distribuição volumétrica de densidade ρ , o potencial gravitacional em um ponto de observação $\mathbf{r}$ pode ser escrito como

$$V(\mathbf{r}) = G \int_{\Omega} \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} dV', \quad (1)$$

em que G é a constante gravitacional universal, Ω representa o domínio ocupado pela fonte, $\mathbf{r}'$ é a posição do elemento de volume da fonte e $|\mathbf{r} - \mathbf{r}'|$ é a distância entre o ponto de observação e o elemento fonte. A partir do potencial, as componentes do campo gravitacional são obtidas por derivação espacial:

$$g_x = \frac{\partial V}{\partial x}, \quad g_y = \frac{\partial V}{\partial y}, \quad g_z = \frac{\partial V}{\partial z}. \quad (2)$$

Em aplicações gravimétricas, a componente vertical g_z é frequentemente a mais utilizada, pois está diretamente relacionada às anomalias de gravidade observadas em levantamentos terrestres, aéreos, marinhos ou orbitais. No pacote, diferentes geometrias podem ser utilizadas para representar as fontes, incluindo prismas retangulares, esferas, cilindros, prismas hexagonais e prismas poligonais. Essas geometrias permitem construir modelos sintéticos simples ou representar aproximações discretizadas de corpos geológicos mais complexos.

No caso da magnetometria, a formulação geral considera o campo produzido por corpos magnetizados. A resposta magnética depende da intensidade de magnetização, da direção da magnetização e da direção do campo geomagnético regional. Uma aproximação frequentemente utilizada em métodos magnéticos consiste em calcular a anomalia de campo total como a projeção do campo magnético anômalo $\Delta\mathbf{B}$ na direção do campo principal da Terra:

$$\Delta T \approx \Delta\mathbf{B} \cdot \hat{\mathbf{F}}, \quad (3)$$

em que $\hat{\mathbf{F}}$ é o vetor unitário na direção do campo geomagnético regional. Essa

relação é importante para transformar componentes vetoriais do campo magnético em uma grandeza escalar comparável aos dados medidos por magnetômetros de campo total.

Convenção de unidades

Ao longo do pacote, as coordenadas espaciais são geralmente fornecidas em metros, as anomalias gravimétricas são expressas em miligal mGal, as grandezas magnéticas em nanoTesla nT e as densidades podem ser fornecidas em g/cm³, sendo convertidas internamente quando necessário. Essa padronização é fundamental para evitar inconsistências dimensionais nos cálculos.

Outro eixo importante do pacote é o processamento de campos potenciais. Dados gravimétricos e magnéticos são geralmente interpretados a partir de transformações que realçam determinadas características das fontes, como bordas, profundidades relativas, contatos laterais, tendências regionais e anomalias residuais. Para isso, o pacote inclui rotinas de derivadas espaciais, gradientes, filtros espectrais, continuação para cima, continuação para baixo, redução ao polo, redução ao equador, pseudogravidade, tilt derivative, theta map e separação regional-residual.

Grande parte dessas transformações pode ser formulada no domínio da frequência por meio da transformada de Fourier. Se $f(x, y)$ representa um campo potencial observado em uma malha regular, sua transformada bidimensional pode ser representada por

$$F(k_x, k_y) = \mathcal{F}\{f(x, y)\}, \quad (4)$$

em que k_x e k_y são os números de onda nas direções x e y . Derivadas horizontais podem ser calculadas pela multiplicação da transformada do campo por operadores espectrais:

$$\mathcal{F}\left\{\frac{\partial f}{\partial x}\right\} = ik_x F(k_x, k_y), \quad (5)$$

$$\mathcal{F}\left\{\frac{\partial f}{\partial y}\right\} = ik_y F(k_x, k_y). \quad (6)$$

A derivada vertical, em campos potenciais, pode ser associada ao módulo do vetor de onda,

$$k = \sqrt{k_x^2 + k_y^2}. \quad (7)$$

Desse modo, operadores espectrais podem ser empregados para realçar fontes rasas, suavizar componentes de alta frequência ou separar componentes regionais e residuais do campo observado.

Importância dos operadores espectrais

Os operadores espectrais são fundamentais em métodos potenciais porque permitem transformar um campo observado em produtos interpretativos. A partir deles, é possível calcular derivadas, gradientes, continuações, filtros de frequência e transformações magnéticas de forma eficiente, especialmente quando os dados estão organizados em malhas regulares.

Além da modelagem direta e do processamento de dados, o pacote também inclui funções voltadas à regressão e à inversão geofísica. A regressão aparece, por exemplo, no ajuste de superfícies regionais, na separação regional-residual e na estimativa de tendências matemáticas presentes nos dados. A inversão geofísica, por sua vez, busca estimar parâmetros do modelo a partir dos dados observados. Em sua forma linear, um problema inverso pode ser representado por

$$\mathbf{d} = \mathbf{G}\mathbf{m} + \mathbf{e}, \quad (8)$$

em que $\mathbf{d}$ é o vetor de dados observados, $\mathbf{G}$ é a matriz de sensibilidade, $\mathbf{m}$ é o vetor de parâmetros do modelo e $\mathbf{e}$ representa os erros ou resíduos. A solução clássica por mínimos quadrados procura minimizar a norma dos resíduos:

$$\Phi(\mathbf{m}) = \|\mathbf{G}\mathbf{m} - \mathbf{d}\|_2^2. \quad (9)$$

Quando o problema é instável ou mal condicionado, pode-se utilizar regularização de Tikhonov, cuja função objetivo é escrita como

$$\Phi(\mathbf{m}) = \|\mathbf{G}\mathbf{m} - \mathbf{d}\|_2^2 + \lambda \|\mathbf{L}\mathbf{m}\|_2^2, \quad (10)$$

em que λ é o parâmetro de regularização e $\mathbf{L}$ é uma matriz que impõe restrições ao modelo, como suavidade, amortecimento ou proximidade em relação a um modelo de referência.

A solução regularizada pode ser escrita como

$$\mathbf{m} = \left(\mathbf{G}^T \mathbf{G} + \lambda \mathbf{L}^T \mathbf{L} \right)^{-1} \mathbf{G}^T \mathbf{d}. \quad (11)$$

Esses conceitos são relevantes para o ensino de Geofísica porque permitem demonstrar a relação entre dados observados, modelos físicos, formulações matemáticas e soluções computacionais. Ao implementar essas etapas em Python, o estudante consegue visualizar a influência do ruído, da geometria das fontes, da discretização, da escolha dos parâmetros e das restrições impostas ao problema.

O pacote também apresenta potencial para construção de modelos sintéticos. Em Geofísica, modelos sintéticos são essenciais para validar algoritmos, testar hipóteses, avaliar sensibilidade e desenvolver atividades didáticas. Como os parâmetros verdadeiros são conhecidos, é possível comparar diretamente os resultados calculados ou invertidos com o modelo de referência. Essa abordagem é particularmente útil para ensinar conceitos como erro, resolução, ambiguidade, não unicidade, instabilidade e regularização.

Papel dos modelos sintéticos

Os modelos sintéticos permitem construir situações controladas nas quais a geometria da fonte, as propriedades físicas e os dados calculados são conhecidos. Isso possibilita avaliar o desempenho dos métodos computacionais antes da aplicação em dados reais, nos quais o modelo verdadeiro geralmente não é conhecido.

Do ponto de vista computacional, o pacote utiliza principalmente estruturas baseadas em arrays, especialmente por meio da biblioteca NumPy. Essa escolha é adequada para aplicações científicas porque permite operações vetorizadas, manipulação eficiente de matrizes e integração com bibliotecas de processamento, visualização e otimização. O uso de funções modulares também favorece a organização do código e facilita sua reutilização em notebooks, scripts, aulas práticas e projetos de pesquisa.

A visualização científica constitui outro aspecto relevante do pacote. Em métodos geofísicos, mapas, perfis, seções, histogramas e gráficos comparativos são fundamentais para interpretar os resultados. O pacote inclui rotinas que auxiliam na criação de mapas em coordenadas cartesianas e geográficas, gráficos de contorno, mapas de cores, perfis, histogramas, imagens radiométricas e representações globais com apoio de bibliotecas como Matplotlib e Cartopy. Isso permite que os resultados numéricos sejam convertidos em produtos visuais adequados para análise, ensino e publicação científica.

A estrutura do pacote também aponta para uma futura disponibilização em repositório público, como o GitHub, e posterior instalação via `pip`. Para isso, o pacote já contém elementos importantes, como arquivos de configuração, dependências, exemplos e testes básicos. A continuidade do projeto deverá envolver a revisão das docstrings, a padronização das entradas e saídas, a construção de tutoriais, a ampliação dos testes

automatizados, a definição de licença e a organização da documentação técnica.

Perspectiva de distribuição

A disponibilização do pacote no GitHub e sua futura instalação via `pip` exigirão uma documentação clara, exemplos reproduzíveis, testes básicos, descrição das dependências obrigatórias e opcionais, além de orientações para instalação em diferentes sistemas operacionais. Essa etapa será importante para transformar o pacote em uma ferramenta acessível a estudantes, professores e pesquisadores.

Em síntese, o `gravmag_codes` pode ser compreendido como uma biblioteca didático-científica para Geofísica Computacional. Sua principal contribuição está na integração entre teoria física, formulação matemática, implementação computacional e visualização dos resultados. Essa integração favorece uma abordagem ativa de aprendizagem, na qual o estudante pode modificar parâmetros, construir modelos, processar dados, interpretar resultados e compreender, de forma prática, os fundamentos dos métodos geofísicos.

2 Descrição técnica do módulo `auxiliars.py`

O arquivo `auxiliars.py` reúne funções auxiliares de caráter matemático, numérico, geométrico e geofísico utilizadas por diferentes partes do pacote `gravmag_codes`. Esse módulo funciona como uma camada de suporte para operações recorrentes, como conversão de unidades angulares, funções trigonométricas, validação de arrays, álgebra linear, rotação de coordenadas, geração de ruído, cálculo de resíduos, métricas de erro, cossenos diretores magnéticos, operadores espectrais e preparação de grades para processamento no domínio de Fourier.

Papel do módulo `auxiliars.py`

O módulo `auxiliars.py` fornece funções básicas e reutilizáveis para os demais módulos do pacote. Seu objetivo principal é evitar repetição de código, padronizar operações matemáticas e garantir maior estabilidade numérica em cálculos geofísicos, especialmente aqueles associados à modelagem gravimétrica, magnetométrica e ao processamento espectral de campos potenciais.

Do ponto de vista computacional, o módulo utiliza principalmente as bibliotecas

NumPy e SciPy. A biblioteca NumPy é empregada para operações vetorizadas, manipulação de arrays, cálculo de funções trigonométricas, transformadas discretas e estatísticas básicas. A biblioteca SciPy, por sua vez, é utilizada especificamente para decomposição LU, por meio da função `scipy.linalg.lu`.

Em termos de aplicação geofísica, esse módulo é especialmente importante porque muitas equações utilizadas em métodos potenciais envolvem funções logarítmicas, arctangentes, distâncias, rotações de coordenadas, direções magnéticas e operadores no domínio da frequência. Pequenas inconsistências nessas operações podem comprometer a estabilidade dos cálculos ou produzir resultados fisicamente incoerentes. Por isso, a existência de funções auxiliares padronizadas contribui para a robustez do pacote.

2.1 Funções de validação e conversão para arrays

As primeiras funções do módulo estão relacionadas à preparação e validação de dados. Em aplicações geofísicas, é comum trabalhar com escalares, listas, vetores ou matrizes. Portanto, é importante converter as entradas para arrays e verificar se diferentes variáveis possuem dimensões compatíveis.

2.1.1 `my_asarray`

A função `my_asarray` converte uma entrada escalar, lista ou array em um objeto do tipo `numpy.ndarray`. Essa função é utilizada para garantir que os dados possam ser manipulados de forma vetorizada pelas demais funções do pacote.

Sintaxe

```
1 my_asarray(x, dtype=float)
```

Entrada: `x` representa a entrada a ser convertida, podendo ser um escalar, lista ou array. O argumento `dtype` define o tipo numérico desejado, sendo `float` o valor padrão.

Saída: Retorna `arr`, um array NumPy com o tipo especificado.

Matematicamente, essa função não altera o conteúdo dos dados. Ela apenas transforma sua representação computacional para permitir operações vetorizadas. Se (x) representa um conjunto de valores,

$$x = x_1, x_2, \dots, x_n, \quad (12)$$

então a função retorna uma estrutura computacional equivalente, adequada para operações matriciais e vetoriais.

2.1.2 `my_check_same_shape`

A função `my_check_same_shape` verifica se todos os arrays fornecidos possuem exatamente a mesma forma, isto é, a mesma quantidade de linhas, colunas e dimensões.

Sintaxe

```
1 my_check_same_shape(*arrays)
```

Entrada: `arrays` representa uma sequência de objetos que serão convertidos internamente em arrays NumPy para verificação de suas dimensões.

Saída: Não retorna valor quando as formas são compatíveis. Caso contrário, gera um erro do tipo `ValueError`.

Essa verificação é importante em operações ponto a ponto. Por exemplo, ao calcular resíduos entre dados observados e preditos, ambos devem possuir a mesma forma:

$$\mathbf{r} = \mathbf{d}_{obs} - \mathbf{d}_{pred}. \quad (13)$$

Caso $\mathbf{d}_{obs}$ e $\mathbf{d}_{pred}$ tenham dimensões diferentes, a operação perde significado físico e computacional.

2.1.3 `my_check_same_size`

A função `my_check_same_size` verifica se todos os arrays fornecidos possuem o mesmo número total de elementos, ainda que suas formas possam ser diferentes.

Sintaxe

```
1 my_check_same_size(*arrays)
```

Entrada: `arrays` representa uma sequência de arrays, listas ou escalares.

Saída: Não retorna valor quando todos os objetos possuem o mesmo tamanho. Caso contrário, gera um erro do tipo `ValueError`.

Essa função é útil quando se deseja comparar vetores ou matrizes que podem ser reformatados, mas que precisam conter a mesma quantidade de pontos. Em uma malha geofísica, por exemplo, as coordenadas (x), (y) e os dados observados (d) devem possuir o mesmo número de amostras.

2.2 Conversões angulares e funções trigonométricas

Em Geofísica, ângulos aparecem em diferentes contextos, como inclinação magnética, declinação magnética, azimuth, coordenadas geográficas, rotações espaciais e direção de magnetização. Como muitas funções trigonométricas em Python utilizam radianos, torna-se necessário converter frequentemente entre graus e radianos.

2.2.1 `my_deg2rad`

A função `my_deg2rad` converte ângulos de graus para radianos.

Sintaxe

```
1 my_deg2rad(angle)
```

Entrada: `angle` é um escalar ou array contendo ângulos em graus.

Saída: Retorna o ângulo correspondente em radianos.

A conversão é dada por

$$\theta_{rad} = \theta_{graus} \frac{\pi}{180}. \quad (14)$$

Essa função também possui o alias `deg2rad`, preservando compatibilidade com códigos anteriores.

2.2.2 `my_rad2deg`

A função `my_rad2deg` converte ângulos de radianos para graus.

Sintaxe

```
1 my_rad2deg(argument)
```

Entrada: `argument` é um escalar ou array contendo ângulos em radianos.

Saída: Retorna o ângulo correspondente em graus.

A conversão é dada por

$$\theta_{\text{graus}} = \theta_{\text{rad}} \frac{180}{\pi}. \quad (15)$$

Essa função também possui o alias `rad2deg`.

2.2.3 my_trigonometrics_deg

A função `my_trigonometrics_deg` calcula seno, cosseno e tangente de um ângulo fornecido em graus.

Sintaxe

```
1 my_trigonometrics_deg(angle)
```

Entrada: `angle` é um escalar ou array contendo ângulos em graus.

Saída: Retorna três valores: `mysin`, `mycos` e `mytan`.

A função primeiro converte o ângulo para radianos e depois calcula

$$\sin(\theta), \quad \cos(\theta), \quad \tan(\theta). \quad (16)$$

Essa função é útil quando os parâmetros geofísicos são fornecidos em graus, como inclinação magnética, declinação magnética ou azimuth.

2.2.4 my_trigonometrics_rad

A função `my_trigonometrics_rad` calcula seno, cosseno e tangente de um ângulo fornecido diretamente em radianos.

Sintaxe

```
1 my_trigonometrics_rad(angle)
```

Entrada: `angle` é um escalar ou array contendo ângulos em radianos.

Saída: Retorna `mysin`, `mycos` e `mytan`.

A formulação matemática é dada por

$$\text{mysin} = \sin(\theta), \quad \text{mycos} = \cos(\theta), \quad \text{mytan} = \tan(\theta). \quad (17)$$

2.2.5 `my_asin`, `my_acos` e `my_atan`

As funções `my_asin`, `my_acos` e `my_atan` calculam funções trigonométricas inversas.

Sintaxe

```
1 my_asin(x)
2 my_acos(x)
3 my_atan(x, y)
```

A função `my_asin` retorna

$$\arcsin(x), \quad (18)$$

enquanto `my_acos` retorna

$$\arccos(x). \quad (19)$$

A função `my_atan` calcula uma arctangente estável usando internamente `numpy.arctan2`. Ela é particularmente importante em fórmulas fechadas de prismas, nas quais termos arctangentes aparecem com diferentes combinações de sinais.

Entrada: Em `my_atan`, `x` e `y` podem ser escalares ou arrays.

Saída: Retorna o valor da arctangente ajustado segundo a convenção adotada no código.

A forma geral é

$$\theta = \arctan 2(x, y). \quad (20)$$

Entretanto, a função aplica ajustes para preservar uma convenção antiga do pacote, na qual valores com ($x=0$) retornam zero. Também são realizados ajustes nos quadrantes quando ($y<0$), evitando inconsistências em expressões de modelagem por prismas.

Uso em fórmulas de prismas

A função `my_atan` é importante porque fórmulas analíticas de prismas retangulares envolvem termos com arctangentes avaliadas nos vértices do corpo. Nesses casos, o tratamento correto dos quadrantes é essencial para evitar descontinuidades ou sinais incorretos na resposta gravimétrica e magnética.

2.2.6 `my_sqrt`, `my_log` e `my_safe_divide`

A função `my_sqrt` calcula a raiz quadrada de uma entrada escalar ou vetorial:

$$y = \sqrt{x}. \quad (21)$$

Sintaxe

```
1 my_sqrt(x)
```

A função `my_log` calcula o logaritmo natural, com tratamento especial para valores iguais a zero.

Sintaxe

```
1 my_log(x)
```

A formulação usual é

$$y = \ln(x). \quad (22)$$

Entretanto, quando ($x=0$), a função substitui o valor indefinido por zero. Esse comportamento é útil em fórmulas analíticas de métodos potenciais, nas quais produtos envolvendo termos logarítmicos podem apresentar limites finitos, mesmo quando o argumento do logaritmo tende a zero.

A função `my_safe_divide` realiza divisão segura entre numerador e denominador:

Sintaxe

```
1 my_safe_divide(numerator, denominator, fill_value  
                =0.0)
```

Entrada: `numerator` é o numerador, `denominator` é o denominador e `fill_value` é o valor usado quando a divisão produz infinito ou valor não definido.

Saída: Retorna o resultado da divisão, substituindo valores não finitos por `fill_value`.

Matematicamente, a operação corresponde a

$$q_i = \begin{cases} \frac{a_i}{b_i}, & b_i \neq 0, \\ q_0, & b_i = 0, \end{cases} \quad (23)$$

em que q_0 é o valor definido por `fill_value`.

2.3 Funções de álgebra linear

O módulo também contém funções simples de álgebra linear utilizadas em diferentes partes do pacote. Essas funções encapsulam operações do NumPy e do SciPy, mantendo uma nomenclatura padronizada.

2.3.1 `my_dot`

A função `my_dot` calcula o produto escalar ou produto matricial entre duas entradas.

Sintaxe

```
1 my_dot(x, y)
```

Matematicamente, para vetores $\mathbf{x}$ e $\mathbf{y}$, o produto escalar é

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i. \quad (24)$$

Para matrizes, a função segue o comportamento da função `numpy.dot`.

2.3.2 my_hadamard

A função `my_hadamard` calcula o produto elemento a elemento entre dois arrays.

Sintaxe

```
1 my_hadamard(x, y)
```

O produto de Hadamard é dado por

$$\mathbf{C} = \mathbf{A} \circ \mathbf{B}, \quad (25)$$

com

$$c_{ij} = a_{ij} b_{ij}. \quad (26)$$

Essa operação é comum em filtros, janelas espectrais, ponderações e produtos ponto a ponto entre campos.

2.3.3 my_outer

A função `my_outer` calcula o produto externo entre dois vetores.

Sintaxe

```
1 my_outer(x, y)
```

Para dois vetores $\mathbf{x}$ e $\mathbf{y}$, o produto externo é

$$\mathbf{A} = \mathbf{xy}^T, \quad (27)$$

ou, em componentes,

$$a_{ij} = x_i y_j. \quad (28)$$

Essa operação é usada, por exemplo, na construção de janelas bidimensionais a partir de janelas unidimensionais.

2.3.4 `my_inverse` e `my_pinv`

A função `my_inverse` calcula a inversa de uma matriz quadrada:

Sintaxe

```
1 my_inverse(mat)
```

A matriz inversa $\mathbf{A}^{-1}$ satisfaz

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{AA}^{-1} = \mathbf{I}. \quad (29)$$

A função `my_pinv` calcula a pseudoinversa de Moore-Penrose:

Sintaxe

```
1 my_pinv(mat, rcond=1.0e-15)
```

A pseudoinversa é especialmente útil em problemas inversos e em sistemas mal condicionados, nos quais a matriz não é quadrada ou não possui inversa exata. Em problemas de mínimos quadrados, a solução pode ser escrita como

$$\mathbf{m} = \mathbf{G}^+ \mathbf{d}, \quad (30)$$

em que $\mathbf{G}^+$ representa a pseudoinversa da matriz de sensibilidade.

2.3.5 my_LU

A função `my_LU` calcula a decomposição LU de uma matriz.

Sintaxe

```
1 my_LU(mat)
```

Entrada: `mat` é uma matriz bidimensional.

Saída: Retorna as matrizes `p`, `l` e `u`, correspondentes à matriz de permutação, matriz triangular inferior e matriz triangular superior.

A decomposição pode ser representada por

$$\mathbf{A} = \mathbf{PLU}, \quad (31)$$

dependendo da convenção da biblioteca utilizada. Esse tipo de decomposição é útil na solução de sistemas lineares e em análises numéricas.

2.4 Rotações tridimensionais e transformação de coordenadas

Em modelagem geofísica, rotações podem ser necessárias para alterar sistemas de referência, orientar modelos, aplicar azimutes ou modificar a geometria de fontes. O módulo contém funções para gerar matrizes de rotação em torno dos eixos (x), (y) e (z).

2.4.1 my_xrotation

A função `my_xrotation` retorna a matriz de rotação em torno do eixo (x).

Sintaxe

```
1 my_xrotation(angle)
```

Entrada: `angle` é o ângulo de rotação em graus.

Saída: Retorna a matriz de rotação `rotx`.

A matriz utilizada no código é

$$\mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 & 0 & \cos \theta & \sin \theta & 0 & -\sin \theta & \cos \theta \end{bmatrix}. \quad (32)$$

2.4.2 `my_yrotation`

A função `my_yrotation` retorna a matriz de rotação em torno do eixo (y).

Sintaxe

```
1 my_yrotation(angle)
```

A matriz é dada por

$$\mathbf{R}_y = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 & 1 & 0 & -\sin \theta & 0 & \cos \theta \end{bmatrix}. \quad (33)$$

2.4.3 `my_zrotation`

A função `my_zrotation` retorna a matriz de rotação em torno do eixo (z).

Sintaxe

```
1 my_zrotation(angle)
```

A matriz é dada por

$$\mathbf{R}_z = \begin{bmatrix} \cos \theta & \sin \theta & 0 & -\sin \theta & \cos \theta & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (34)$$

Convenção de sinal

As matrizes de rotação seguem a convenção implementada no código. Em aplicações geofísicas, é importante manter a mesma convenção ao comparar resultados com outros programas ou sistemas de coordenadas, pois diferentes convenções de rotação podem alterar o sinal de componentes horizontais.

2.4.4 rotate3D_xyz

A função `rotate3D_xyz` aplica uma rotação tridimensional a coordenadas (x), (y) e (z).

Sintaxe

```
1 rotate3D_xyz(x, y, z, angle, direction='z')
```

Entrada: `x`, `y` e `z` são arrays de coordenadas. O parâmetro `angle` é o ângulo de rotação em graus. O parâmetro `direction` define o eixo de rotação, podendo assumir os valores `'x'`, `'y'` ou `'z'`.

Saída: Retorna `xr`, `yr` e `zr`, correspondentes às coordenadas rotacionadas.

A operação é realizada pela multiplicação matricial

$$\begin{bmatrix} x_r & y_r & z_r \end{bmatrix} = \mathbf{R} \begin{bmatrix} x & y & z \end{bmatrix}, \quad (35)$$

em que $\mathbf{R}$ é a matriz de rotação escolhida.

2.4.5 my_spherical2cartesian

A função `my_spherical2cartesian` converte coordenadas esféricas geográficas para coordenadas cartesianas geocêntricas.

Sintaxe

```
1 my_spherical2cartesian(longitude, latitude, level)
```

Entrada: `longitude` e `latitude` são fornecidas em graus. O parâmetro `level` representa a altura em metros acima do raio terrestre de referência.

Saída: Retorna as coordenadas cartesianas geocêntricas $\mathbf{x}$, $\mathbf{y}$ e $\mathbf{z}$.

O raio utilizado é

$$R = 6378137 + h, \quad (36)$$

em que (h) é o valor de `level`. As coordenadas cartesianas são calculadas por

$$x = R \cos \varphi \cos \lambda, \quad (37)$$

$$y = R \cos \varphi \sin \lambda, \quad (38)$$

$$z = R \sin \varphi, \quad (39)$$

em que λ é a longitude e φ é a latitude, ambas em radianos.

2.5 Ruído, resíduos e estatísticas básicas

Esta parte do módulo contém funções voltadas à contaminação de dados sintéticos com ruído e à avaliação quantitativa de erros. Essas funções são úteis para testes sintéticos, validação de algoritmos e comparação entre dados observados e calculados.

2.5.1 `my_normalnoise`

A função `my_normalnoise` adiciona ruído Gaussiano a um conjunto de dados.

Sintaxe

1

```
my_normalnoise(xi, vi=0.0, std=0.0, seed=None)
```

Entrada: `xi` representa os dados originais. O parâmetro `vi` define a média do ruído, `std` define o desvio padrão e `seed` controla a reprodutibilidade da geração aleatória.

Saída: Retorna os dados contaminados por ruído Gaussiano.

A operação pode ser representada por

$$x_i^{\text{ruidoso}} = x_i + \epsilon_i, \quad (40)$$

em que

$$\epsilon_i \sim \mathcal{N}(\mu, \sigma^2). \quad (41)$$

No código, μ é representado por `vi` e σ por `std`.

2.5.2 `my_uniformnoise`

A função `my_uniformnoise` adiciona ruído com distribuição uniforme a um conjunto de dados.

Sintaxe

1

```
my_uniformnoise(xi, vmin, vmax, seed=None)
```

Entrada: `xi` representa os dados originais. Os parâmetros `vmin` e `vmax` definem os limites inferior e superior da distribuição uniforme. O parâmetro `seed` controla a reprodutibilidade.

Saída: Retorna os dados contaminados por ruído uniforme.

A operação pode ser representada por

$$x_i^{\text{ruidoso}} = x_i + \epsilon_i, \quad (42)$$

com

$$\epsilon_i \sim U(a, b), \quad (43)$$

em que $a = \text{vmin}$ e $b = \text{vmax}$.

2.5.3 `my_residual`

A função `my_residual` calcula o resíduo entre dados observados e dados preditos.

Sintaxe

```
1 my_residual(do, dp, eps=1.0e-15)
```

Entrada: `do` representa os dados observados e `dp` representa os dados preditos. O parâmetro `eps` evita divisões ou normalizações instáveis quando o desvio padrão é muito pequeno.

Saída: Retorna quatro elementos: o vetor de resíduos `res`, a média dos resíduos `mean`, o resíduo normalizado `norm` e o desvio padrão dos resíduos `std`.

O resíduo é calculado por

$$r_i = d_i^{obs} - d_i^{pred}. \quad (44)$$

A média dos resíduos é

$$\bar{r} = \frac{1}{n} \sum_{i=1}^n r_i, \quad (45)$$

e o desvio padrão é

$$\sigma_r = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_i - \bar{r})^2}. \quad (46)$$

Quando σ_r é suficientemente diferente de zero, o resíduo normalizado é calculado por

$$r_i^{norm} = \frac{r_i - \bar{r}}{\sigma_r}. \quad (47)$$

2.5.4 my_rms, my_rmse e my_mae

A função `my_rms` calcula a raiz da média quadrática de um conjunto de dados.

Sintaxe

```
1 my_rms(data)
```

A formulação é

$$\text{RMS} = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}. \quad (48)$$

A função `my_rmse` calcula o erro quadrático médio entre dados observados e preditos.

Sintaxe

```
1 my_rmse(observed, predicted)
```

A formulação é

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (d_i^{\text{obs}} - d_i^{\text{pred}})^2}. \quad (49)$$

A função `my_mae` calcula o erro absoluto médio.

Sintaxe

```
1 my_mae(observed, predicted)
```

A formulação é

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |d_i^{\text{obs}} - d_i^{\text{pred}}|. \quad (50)$$

Uso das métricas de erro

As métricas RMS, RMSE e MAE são importantes para avaliar a qualidade de modelos sintéticos, ajustes, filtros e inversões. O RMSE penaliza mais fortemente erros grandes, enquanto o MAE fornece uma medida média mais direta da magnitude dos desvios.

2.6 Funções associadas à direção magnética

Em magnetometria, a direção do campo geomagnético e a direção da magnetização das fontes são descritas por inclinação, declinação e, em alguns casos, azimuth de correção. O módulo **auxiliars.py** contém funções para converter essas direções angulares em componentes cartesianas.

2.6.1 my_dircos

A função **my_dircos** calcula os cossenos diretores associados a uma direção definida por inclinação e declinação.

Sintaxe

```
1 my_dircos(inc, dec, azm=0.0)
```

Entrada: **inc** é a inclinação em graus, **dec** é a declinação em graus e **azm** é uma correção azimutal em graus.

Saída: Retorna **xdir**, **ydir** e **zdir**, que representam as componentes unitárias da direção.

A formulação implementada é

$$x_{dir} = \cos I \cos(D - A), \quad (51)$$

$$y_{dir} = \cos I \sin(D - A), \quad (52)$$

$$z_{dir} = \sin I, \quad (53)$$

em que (I) é a inclinação, (D) é a declinação e (A) é o azimuth de correção.

2.6.2 my_regional

A função **my_regional** calcula as componentes cartesianas do campo magnético regional.

Sintaxe

```
1 my_regional(field, inc, dec, azm=0.0)
```

Entrada: `field` representa a intensidade do campo magnético regional. Os parâmetros `inc`, `dec` e `azm` definem a orientação do campo.

Saída: Retorna as componentes `fx`, `fy` e `fz`.

A partir dos cossenos diretores, as componentes são calculadas por

$$F_x = F x_{dir}, \quad (54)$$

$$F_y = F y_{dir}, \quad (55)$$

$$F_z = F z_{dir}. \quad (56)$$

Essa função é utilizada em modelagem magnética para projetar o campo anômalo na direção do campo regional.

2.6.3 my_theta

A função `my_theta` calcula um operador direcional no domínio de Fourier associado à direção do campo ou da magnetização.

Sintaxe

```
1 my_theta(inc, dec, u, v, azim=0.0, eps=1.0e-15)
```

Entrada: `inc` e `dec` são inclinação e declinação em graus. Os arrays `u` e `v` representam os números de onda nas direções (x) e (y). O parâmetro `azim` representa uma correção azimutal.

Saída: Retorna `theta`, um array complexo que representa o operador direcional no domínio espectral.

O módulo do número de onda é calculado por

$$k = \sqrt{u^2 + v^2}. \quad (57)$$

Com os cossenos diretores x_{dir} , y_{dir} e z_{dir} , o operador é dado por

$$\Theta(u, v) = z_{dir} + i \left(\frac{x_{dir}u + y_{dir}v}{k} \right). \quad (58)$$

O termo de divisão é calculado de forma segura para evitar problemas no ponto ($k=0$).

Interpretação geofísica do operador Θ

O operador Θ aparece em transformações magnéticas no domínio da frequência, como redução ao polo, redução ao equador e pseudogravidade. Ele descreve a influência da direção do campo magnético ou da magnetização sobre o espectro do campo observado.

2.7 Funções associadas ao número de onda

O processamento de campos potenciais no domínio da frequência requer a construção de números de onda nas direções (x) e (y). Essas grandezas são utilizadas em derivadas espectrais, filtros de frequência, continuação de campo e transformações magnéticas.

2.7.1 `my_grid_spacing`

A função `my_grid_spacing` estima o espaçamento de uma malha regular a partir dos arrays de coordenadas.

Sintaxe

```
1 my_grid_spacing(x, y)
```

Entrada: `x` e `y` são arrays unidimensionais ou bidimensionais contendo as coordenadas da malha.

Saída: Retorna `dx` e `dy`, correspondentes aos espaçamentos médios nas direções (x) e (y).

Para coordenadas unidimensionais, o espaçamento médio é calculado por

$$\Delta x = \frac{1}{n-1} \sum_{i=1}^{n-1} (x_{i+1} - x_i), \quad (59)$$

e de forma análoga para Δy . Para grades bidimensionais, a diferença é calculada ao longo dos eixos correspondentes.

2.7.2 `my_wavenumber`

A função `my_wavenumber` constrói as grades de números de onda nas direções (x) e (y).

Sintaxe

```
1 my_wavenumber(x, y)
```

Entrada: `x` e `y` são coordenadas unidimensionais ou bidimensionais de uma malha regular.

Saída: Retorna `kx` e `ky`, arrays bidimensionais contendo os números de onda nas direções (x) e (y).

Os números de onda discretos são obtidos a partir das frequências discretas da transformada de Fourier:

$$k_x = 2\pi f_x, \quad (60)$$

$$k_y = 2\pi f_y. \quad (61)$$

No código, as frequências são calculadas usando `numpy.fft.fftfreq`, considerando os espaçamentos Δx e Δy .

2.7.3 `my_wavenumber_modulus`

A função `my_wavenumber_modulus` retorna os números de onda k_x , k_y e o módulo total (k).

Sintaxe

```
1 my_wavenumber_modulus(x, y)
```

Entrada: x e y são coordenadas da malha.

Saída: Retorna k_x , k_y e k .

O módulo do número de onda é

$$k = \sqrt{k_x^2 + k_y^2}. \quad (62)$$

Essa grandeza é fundamental em operadores de continuação, derivada vertical e filtragem espectral.

2.8 Funções de taper e padding para filtros FFT

Quando filtros são aplicados no domínio da frequência, efeitos de borda podem surgir devido à periodicidade implícita da transformada discreta de Fourier. Para reduzir esses efeitos, é comum expandir a grade original por padding e aplicar janelas de suavização nas bordas, conhecidas como tapers.

2.8.1 my_cosine_taper_1d

A função `my_cosine_taper_1d` cria uma janela cossenoidal unidimensional.

Sintaxe

```
1 my_cosine_taper_1d(n, p=0.10)
```

Entrada: n é o número de amostras e p é a fração da janela aplicada em cada borda.

Saída: Retorna w , um vetor de pesos.

A janela possui valores iguais a 1 na parte central e transições suaves nas bordas. A transição é calculada a partir de

$$w(t) = \sin^2(t), \quad (63)$$

com

$$0 \leq t \leq \frac{\pi}{2}. \quad (64)$$

2.8.2 my_cosine_taper_2d

A função `my_cosine_taper_2d` aplica uma janela cossenoidal bidimensional a uma grade.

Sintaxe

```
1 my_cosine_taper_2d(data, p=0.10)
```

Entrada: `data` é uma matriz bidimensional e `p` é a fração de taper aplicada nas bordas.

Saída: Retorna a matriz suavizada `tapered`.

A janela bidimensional é construída pelo produto externo das janelas unidimensionais:

$$\mathbf{W} = \mathbf{w}_y \mathbf{w}_x^T. \quad (65)$$

A grade suavizada é então

$$\mathbf{D}_{tapered} = \mathbf{D} \circ \mathbf{W}, \quad (66)$$

em que $\circ$ representa o produto de Hadamard.

2.8.3 my_pad_grid

A função `my_pad_grid` expande uma grade regular antes da aplicação de filtros no domínio da frequência.

Sintaxe

```
1 my_pad_grid(data, pad_factor=0.50, pad_mode='reflect',
2             ,
taper=True, taper_fraction=0.05)
```

Entrada: `data` é uma matriz bidimensional. O parâmetro `pad_factor` define a fração do tamanho da grade adicionada em cada borda. O parâmetro `pad_mode` define o modo de preenchimento usado pelo `numpy.pad`. O parâmetro `taper` indica se a janela cossenoidal será aplicada após o padding.

Saída: Retorna `data_pad`, a grade expandida; `pad_width`, contendo as informações do padding; e `mean_data`, a média removida da grade original.

Antes do padding, a média do dado é removida:

$$D_0(x, y) = D(x, y) - \bar{D}. \quad (67)$$

Em seguida, a matriz é expandida nas bordas. Se `taper=True`, aplica-se uma janela bidimensional para suavizar a transição nas extremidades.

Motivação do padding

O padding reduz efeitos de borda em filtros FFT, pois diminui a influência da periodicidade artificial imposta pela transformada discreta de Fourier. A remoção da média também contribui para controlar a componente de frequência zero do campo.

2.8.4 `my_crop_grid`

A função `my_crop_grid` remove o padding aplicado anteriormente, recuperando a região original da grade.

Sintaxe

```
1 my_crop_grid(data_pad, pad_width)
```

Entrada: `data_pad` é a matriz expandida e `pad_width` contém as informações de padding retornadas por `my_pad_grid`.

Saída: Retorna a matriz recortada no tamanho original.

Essa função é normalmente aplicada após a filtragem espectral, para descartar as bordas artificiais adicionadas ao dado.

2.9 Funções legadas de preenchimento

O módulo também preserva funções antigas usadas em fluxos de trabalho com `numpy.pad`. Essas funções são mantidas por compatibilidade com códigos anteriores.

2.9.1 `my_padzeros`

A função `my_padzeros` preenche as bordas de um vetor com zeros.

Sintaxe

```
1 my_padzeros(vector, width, ax=None, kwargs=None)
```

Entrada: `vector` é o vetor a ser modificado. O parâmetro `width` define a largura das bordas que serão preenchidas.

Saída: Retorna o vetor com bordas preenchidas por zero.

Essa função pode ser usada como função auxiliar em estratégias antigas de padding.

2.9.2 `my_padones`

A função `my_padones` preenche as bordas de um vetor com valores iguais a um.

Sintaxe

```
1 my_padones(vector, width, ax=None, kwargs=None)
```

Entrada: `vector` é o vetor a ser modificado. O parâmetro `width` define a largura das bordas.

Saída: Retorna o vetor com bordas preenchidas por um.

2.10 Funções de compatibilidade

As últimas funções do módulo são wrappers simples para retornar forma e tamanho de arrays. Elas ajudam a manter compatibilidade com códigos antigos e tornam algumas chamadas mais padronizadas dentro do pacote.

2.10.1 my_shape

A função `my_shape` retorna a forma de um array.

Sintaxe

```
1 my_shape(data)
```

Entrada: `data` é qualquer objeto que possa ser convertido para array NumPy.

Saída: Retorna a forma do array, isto é, sua dimensão computacional.

Se $\mathbf{D}$ é uma matriz com n_y linhas e n_x colunas, então

$$\text{shape}(\mathbf{D}) = (n_y, n_x). \quad (68)$$

2.10.2 my_size

A função `my_size` retorna o número total de elementos de um array.

Sintaxe

```
1 my_size(data)
```

Entrada: `data` é qualquer objeto que possa ser convertido para array NumPy.

Saída: Retorna o número total de elementos.

Para uma matriz $\mathbf{D}$ com n_y linhas e n_x colunas,

$$\text{size}(\mathbf{D}) = n_y n_x. \quad (69)$$

2.11 Síntese técnica do módulo

O módulo `auxiliars.py` constitui uma base numérica e matemática essencial para o pacote `gravmag_codes`. Suas funções não representam, isoladamente, métodos geofísicos completos, mas fornecem operações fundamentais para que os demais módulos possam

executar modelagem, filtragem, inversão, análise estatística e visualização de forma consistente.

Entre as principais contribuições do módulo, destacam-se a padronização de conversões angulares, a implementação de operações seguras para logaritmos, divisões e arctangentes, a construção de matrizes de rotação, o cálculo de direções magnéticas, a geração de ruído sintético, a avaliação de resíduos, a construção de números de onda e a preparação de grades para filtros FFT.

Importância para o pacote

Sem as funções auxiliares reunidas em `auxiliars.py`, os demais módulos precisariam repetir operações matemáticas básicas, aumentando a complexidade do código e o risco de inconsistências. Portanto, esse módulo tem papel estrutural na organização, manutenção e expansão futura do pacote.

Do ponto de vista didático, o módulo também é relevante porque permite introduzir conceitos fundamentais de programação científica aplicada à Geofísica, como arrays, vetorização, funções trigonométricas, rotações, métricas de erro, ruído sintético, números de onda e janelas espectrais. Assim, ele pode ser utilizado como ponto de partida para estudantes compreenderem a base computacional necessária antes de avançarem para modelagem gravimétrica, magnetométrica e inversão geofísica.

3 Descrição técnica do módulo `constants.py`

O arquivo `constants.py` reúne constantes matemáticas, físicas, geofísicas e fatores de conversão de unidades utilizados ao longo do pacote *gravmag_codes*. Esse módulo tem papel estrutural, pois centraliza valores fundamentais empregados em rotinas de gravimetria, magnetometria, geomagnetismo, métodos eletromagnéticos, inversão, filtragem espectral e modelagem direta.

A centralização das constantes em um único arquivo é uma prática importante em programação científica, pois evita a repetição de valores numéricos em diferentes partes do código e reduz o risco de inconsistência dimensional. Em Geofísica Computacional, esse cuidado é essencial, uma vez que erros de unidade podem produzir respostas numericamente plausíveis, mas fisicamente incorretas.

Papel do módulo `constants.py`

O módulo `constants.py` define constantes físicas e fatores de conversão usados por diferentes rotinas do pacote. Ele também preserva nomes antigos de variáveis como aliases, garantindo compatibilidade com scripts e notebooks previamente desenvolvidos.

O módulo utiliza a biblioteca NumPy, importada internamente como `numpy`. Além de constantes numéricas, o arquivo contém funções auxiliares simples para conversão de densidade, distância, aceleração gravimétrica e campo magnético. Essas funções são úteis porque padronizam transformações recorrentes, como converter densidade de g/cm^3 para kg/m^3 , gravidade de unidades SI para mGal e campo magnético de Tesla para nanoTesla.

3.1 Constantes matemáticas fundamentais

O módulo define constantes matemáticas básicas associadas ao número π e à conversão angular entre graus e radianos. Essas constantes são utilizadas em funções trigonométricas, operadores espectrais, transformações geométricas, modelagem magnética e processamento de campos potenciais.

Constantes matemáticas

```
1      PI = numpy.pi
2      TWO_PI = 2.0 * numpy.pi
3      DEG2RAD = numpy.pi / 180.0
4      RAD2DEG = 180.0 / numpy.pi
```

A constante π aparece em diversas formulações físicas, especialmente em integrais de volume, transformadas de Fourier, operadores diferenciais e relações geométricas. A conversão de graus para radianos é dada por

$$\theta_{rad} = \theta_{graus} \frac{\pi}{180}, \quad (70)$$

enquanto a conversão de radianos para graus é dada por

$$\theta_{graus} = \theta_{rad} \frac{180}{\pi}. \quad (71)$$

O arquivo também preserva aliases compatíveis com versões anteriores do código:

Aliases de compatibilidade

```

1      pi = PI
2      deg2rad = DEG2RAD
3      rad2deg = RAD2DEG

```

Compatibilidade com códigos antigos

A presença de aliases como `pi`, `deg2rad` e `rad2deg` permite que notebooks e scripts antigos continuem funcionando mesmo após a reorganização do pacote.

3.2 Constantes gravimétricas

As constantes gravimétricas são utilizadas nas rotinas de modelagem direta, correção gravimétrica, cálculo de anomalias, inversão e interpretação de dados de gravidade. A principal constante física é a constante gravitacional universal.

Constantes gravimétricas

```

1      G = 6.673e-11
2      GRAVITATIONAL_CONSTANT = G
3
4      SI2MGAL = 1.0e5
5      MGAL2SI = 1.0e-5

```

A constante gravitacional universal G é utilizada na lei da gravitação de Newton:

$$F = G \frac{m_1 m_2}{r^2}, \quad (72)$$

em que F é a força gravitacional entre duas massas m_1 e m_2 , separadas por uma distância r . Em modelagem gravimétrica, essa constante aparece na formulação do potencial gravitacional:

$$V(\mathbf{r}) = G \int_{\Omega} \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} dV', \quad (73)$$

e nas componentes do campo gravitacional obtidas a partir das derivadas espaciais do potencial:

$$g_x = \frac{\partial V}{\partial x}, \quad g_y = \frac{\partial V}{\partial y}, \quad g_z = \frac{\partial V}{\partial z}. \quad (74)$$

No código, o valor adotado para G é

$$G = 6.673 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}. \quad (75)$$

Valor preservado de G

O valor de G foi mantido como 6.673e-11 para preservar a compatibilidade numérica com versões anteriores do pacote. Embora existam valores mais recentes recomendados em literatura metrológica, a manutenção desse valor evita alterações nos resultados de notebooks e exemplos antigos.

A conversão entre aceleração em unidades do Sistema Internacional e miligal é dada por

$$1 \text{ m/s}^2 = 10^5 \text{ mGal}. \quad (76)$$

Assim, o fator de conversão de SI para mGal é

$$\text{SI2MGAL} = 10^5, \quad (77)$$

e o fator inverso é

$$\text{MGAL2SI} = 10^{-5}. \quad (78)$$

O módulo também preserva aliases antigos:

Aliases gravimétricos

```

1      si2mGal = SI2MGAL
2      mGal2si = MGAL2SI
```

3.3 Constantes magnéticas

As constantes magnéticas são usadas nas rotinas de modelagem magnética, cálculo de anomalias de campo total, componentes magnéticas e métodos eletromagnéticos. O módulo define uma constante magnética associada ao termo $\mu_0/(4\pi)$, além de fatores de conversão entre Tesla e nanoTesla.

Constantes magnéticas

```

1      CM = 1.0e-7
2      T2NT = 1.0e9
3      NT2T = 1.0e-9
4      MU0 = 4.0e-7 * numpy.pi

```

A constante μ_0 representa a permeabilidade magnética do vácuo:

$$\mu_0 = 4\pi \times 10^{-7} \text{ T m A}^{-1}. \quad (79)$$

A constante

$$C_M = 10^{-7} \quad (80)$$

é equivalente a

$$C_M = \frac{\mu_0}{4\pi}. \quad (81)$$

Essa constante aparece em formulações de campos magnéticos produzidos por fontes magnetizadas. Em termos gerais, o campo magnético produzido por uma fonte dipolar ou por uma distribuição de magnetização pode envolver fatores proporcionais a $\mu_0/(4\pi)$.

A conversão de Tesla para nanoTesla é dada por

$$1 \text{ T} = 10^9 \text{ nT}. \quad (82)$$

Portanto,

$$B_{\text{nT}} = 10^9 B_{\text{T}}, \quad (83)$$

e a conversão inversa é

$$B_T = 10^{-9} B_{nT}. \quad (84)$$

O módulo mantém diversos aliases para compatibilidade:

Aliases magnéticos

```
1      cm = CM
2      T2nT = T2NT
3      nT2T = NT2T
4      t2nT = T2NT
5      nt2T = NT2T
6      mu0 = MU0
```

Importância das unidades magnéticas

Em magnetometria aplicada, os dados observados são frequentemente expressos em nano-Tesla. Entretanto, algumas formulações físicas são naturalmente escritas em unidades SI, envolvendo Tesla. Por isso, os fatores T2NT e NT2T são fundamentais para manter coerência dimensional entre formulação teórica e saída numérica.

3.4 Constantes geodésicas e terrestres

O módulo define parâmetros associados aos elipsoides WGS84 e GRS80, além de parâmetros médios da Terra. Essas constantes são úteis em cálculos geodésicos, gravidade normal, conversões globais e aplicações em geomagnetismo.

Parâmetros do elipsoide WGS84

```
1      WGS84_A = 6378137.0
2      WGS84_F = 1.0 / 298.257223563
3      WGS84_GM = 3986004.418e8
4      WGS84_OMEGA = 7292115.0e-11
```

No elipsoide WGS84, `WGS84_A` representa o semieixo maior a , `WGS84_F` representa o achatamento f , `WGS84_GM` representa a constante gravitacional geocêntrica GM , e `WGS84_OMEGA` representa a velocidade angular da Terra ω .

O achatamento é definido por

$$f = \frac{a - b}{a}, \quad (85)$$

em que a é o semieixo maior e b é o semieixo menor do elipsoide. A partir do achatamento, pode-se obter

$$b = a(1 - f). \quad (86)$$

O módulo também define os parâmetros equivalentes para o elipsoide GRS80:

Parâmetros do elipsoide GRS80

```

1      GRS80_A = 6378137.0
2      GRS80_F = 1.0 / 298.257222101
3      GRS80_GM = 3986005.0e8
4      GRS80_OMEGA = 7292115.0e-11
```

Além dos parâmetros elipsoidais, o arquivo define constantes terrestres médias:

Parâmetros médios da Terra

```

1      EARTH_RADIUS = WGS84_A
2      EARTH_MASS = 5.972e24
3      EARTH_VOLUME_KM3 = 1.08321e12
4      EARTH_ESCAPE_VELOCITY = 11186.0
```

O raio terrestre de referência é adotado como

$$R = 6378137 \text{ m}. \quad (87)$$

A massa média da Terra é

$$M_{\oplus} = 5.972 \times 10^{24} \text{ kg.} \quad (88)$$

A velocidade de escape próxima à superfície terrestre é definida como

$$v_e = 11186 \text{ m/s.} \quad (89)$$

O módulo preserva aliases antigos para essas constantes:

Aliases de parâmetros terrestres

```
1      earth_radius = EARTH_RADIUS
2      earth_mass   = EARTH_MASS
3      volume       = EARTH_VOLUME_KM3
4      vescape      = EARTH_ESCAPE_VELOCITY
```

3.5 Constantes de densidade

As constantes de densidade são particularmente importantes para modelagem gravimétrica e estudos isostáticos. O módulo define densidades médias representativas da Terra, crosta continental, crosta oceânica, água e manto.

Densidades de referência

```
1      RHO_MEAN_EARTH = 5.514
2      RHO_CONTINENTAL_CRUST = 2.673
3      RHO_OCEANIC_CRUST = 2.92
4      RHO_WATER = 1.03
5      RHO_MANTLE = 3.35
```

Essas densidades são expressas em g/cm^3 . Assim, por exemplo, a densidade média da Terra é

$$\rho_{\text{Terra}} = 5.514 \text{ g/cm}^3, \quad (90)$$

a densidade da crosta continental é

$$\rho_{cc} = 2.673 \text{ g/cm}^3, \quad (91)$$

a densidade da crosta oceânica é

$$\rho_{oc} = 2.92 \text{ g/cm}^3, \quad (92)$$

a densidade da água é

$$\rho_w = 1.03 \text{ g/cm}^3, \quad (93)$$

e a densidade do manto é

$$\rho_m = 3.35 \text{ g/cm}^3. \quad (94)$$

A conversão entre g/cm^3 e kg/m^3 é dada por

$$1 \text{ g/cm}^3 = 1000 \text{ kg/m}^3. \quad (95)$$

Logo,

$$\rho_{\text{kg/m}^3} = 1000 \rho_{\text{g/cm}^3}, \quad (96)$$

e a conversão inversa é

$$\rho_{\text{g/cm}^3} = 10^{-3} \rho_{\text{kg/m}^3}. \quad (97)$$

No código, esses fatores são definidos como

Conversões de densidade

```
1      GCM3_TO_KGM3 = 1000.0
2      KGM3_TO_GCM3 = 0.001
```

Os aliases antigos preservados são:

Aliases de densidade

```
1      rhomean = RHO_MEAN_EARTH
2      rho_cc = RHO_CONTINENTAL_CRUST
3      rho_oc = RHO_OCEANIC_CRUST
4      rho_w = RHO_WATER
5      rho_m = RHO_MANTLE
6      gcm3_to_kgm3 = GCM3_TO_KGM3
7      kgm3_to_gcm3 = KGM3_TO_GCM3
```

Uso geofísico das densidades

As densidades de referência são usadas em correções gravimétricas, modelos isostáticos, cálculo de anomalias de Bouguer, estimativa de Moho por modelos do tipo Airy e modelagem direta de corpos geológicos. A escolha da densidade influencia diretamente a amplitude da resposta gravimétrica calculada.

3.6 Conversões de comprimento, área e volume

O módulo também concentra fatores de conversão geométrica. Essas conversões são úteis em rotinas que trabalham com diferentes escalas espaciais, como metros, quilômetros, centímetros e milímetros.

Conversões de comprimento

```
1      M_TO_KM = 1.0e-3
2      KM_TO_M = 1.0e3
3      M_TO_CM = 1.0e2
4      CM_TO_M = 1.0e-2
5      M_TO_MM = 1.0e3
6      MM_TO_M = 1.0e-3
```

As relações matemáticas são

$$1 \text{ m} = 10^{-3} \text{ km}, \quad (98)$$

$$1 \text{ km} = 10^3 \text{ m}, \quad (99)$$

$$1 \text{ m} = 10^2 \text{ cm}, \quad (100)$$

$$1 \text{ cm} = 10^{-2} \text{ m}, \quad (101)$$

$$1 \text{ m} = 10^3 \text{ mm}. \quad (102)$$

O código também define conversões de área:

Conversões de área

```
1      M2_TO_KM2 = 1.0e-6
2      KM2_TO_M2 = 1.0e6
```

Como

$$1 \text{ km}^2 = 10^6 \text{ m}^2, \quad (103)$$

tem-se

$$1 \text{ m}^2 = 10^{-6} \text{ km}^2. \quad (104)$$

As conversões de volume são definidas por:

Conversões de volume

```
1      M3_TO_L = 1000.0
2      L_TO_M3 = 0.001
3      DM3_TO_L = 1.0
4      L_TO_DM3 = 1.0
```

A relação principal é

$$1 \text{ m}^3 = 1000 \text{ L.} \quad (105)$$

Portanto,

$$1 \text{ L} = 10^{-3} \text{ m}^3. \quad (106)$$

Correção de aliases antigos

No arquivo atual, os aliases `m2km`, `km2m`, `m3_L` e `L_m3` foram corrigidos de acordo com seus nomes. O próprio código registra que versões antigas possuíam alguns rótulos invertidos, e que a correção foi feita para garantir coerência dimensional.

3.7 Constantes físicas gerais

Além das constantes geofísicas, o módulo define constantes físicas gerais que podem ser utilizadas em métodos eletromagnéticos, radiometria, física aplicada e formulações auxiliares.

Constantes físicas gerais

```
1      LIGHT_SPEED = 299792458.0
2      PLANCK_CONSTANT = 6.62607015e-34
3      ELEMENTARY_CHARGE = 1.602176634e-19
4      EPSILON0 = 8.8541878128e-12
```

A velocidade da luz no vácuo é

$$c = 299792458 \text{ m/s.} \quad (107)$$

A constante de Planck é

$$h = 6.62607015 \times 10^{-34} \text{ J s.} \quad (108)$$

A carga elementar é

$$e = 1.602176634 \times 10^{-19} \text{ C.} \quad (109)$$

A permissividade elétrica do vácuo é

$$\varepsilon_0 = 8.8541878128 \times 10^{-12} \text{ C}^2/(\text{N m}^2). \quad (110)$$

O módulo preserva os aliases:

Aliases de constantes físicas

```
1      c = LIGHT_SPEED
2      h = PLANCK_CONSTANT
3      ec = ELEMENTARY_CHARGE
4      epsilon0 = EPSILON0
```

Essas constantes ampliam o escopo do pacote para aplicações em métodos eletromagnéticos e modelagens físicas correlatas.

3.8 Pequenos valores numéricos

O módulo define dois pequenos valores numéricos usados para evitar instabilidades em operações matemáticas.

Constantes numéricas pequenas

```
1      EPS = 1.0e-12
2      TINY = 1.0e-30
```

O valor `EPS` pode ser usado como tolerância numérica em comparações, divisões ou verificações de valores próximos de zero. O valor `TINY` representa um número extremamente pequeno, útil para evitar singularidades, divisões por zero ou logaritmos indefinidos.

Uso de tolerâncias numéricas

Em modelagem geofísica, muitos cálculos envolvem distâncias, denominadores, normas, números de onda e operadores espectrais. O uso de tolerâncias como `EPS` e `TINY` ajuda a reduzir problemas associados a singularidades e instabilidades numéricas.

3.9 Função `my_show_constants`

A função `my_show_constants` imprime na tela algumas das principais constantes utilizadas pelo pacote.

Sintaxe

```
1 my_show_constants()
```

Entrada: A função não recebe argumentos.

Saída: Não retorna valores. A saída é impressa diretamente na tela por meio de comandos `print`.

Essa função é útil para diagnóstico, documentação e conferência rápida das constantes adotadas. Ela apresenta constantes gravimétricas, magnéticas, densidades de referência e conversões de comprimento.

Do ponto de vista didático, essa função pode ser utilizada no início de aulas práticas para mostrar aos estudantes quais constantes físicas serão adotadas nos cálculos.

3.10 Funções de conversão de densidade

O módulo contém duas funções para converter densidades entre g/cm^3 e kg/m^3 . Essas conversões são fundamentais porque muitas propriedades geológicas são expressas em g/cm^3 , enquanto as equações físicas em SI exigem densidade em kg/m^3 .

3.10.1 `my_density_gcm3_to_kgm3`

A função `my_density_gcm3_to_kgm3` converte densidade de g/cm^3 para kg/m^3 .

Sintaxe

```
1 my_density_gcm3_to_kgm3(rho)
```

Entrada: rho é um escalar ou array contendo densidades em g/cm³.

Saída: Retorna as densidades convertidas para kg/m³.

A conversão é dada por

$$\rho_{\text{kg/m}^3} = 1000 \rho_{\text{g/cm}^3}. \quad (111)$$

3.10.2 my_density_kgm3_to_gcm3

A função `my_density_kgm3_to_gcm3` converte densidade de kg/m³ para g/cm³.

Sintaxe

```
1 my_density_kgm3_to_gcm3(rho)
```

Entrada: rho é um escalar ou array contendo densidades em kg/m³.

Saída: Retorna as densidades convertidas para g/cm³.

A conversão é dada por

$$\rho_{\text{g/cm}^3} = 10^{-3} \rho_{\text{kg/m}^3}. \quad (112)$$

3.11 Funções de conversão de distância

O módulo contém funções simples para converter distâncias entre metros e quilômetros.

3.11.1 my_m_to_km

A função `my_m_to_km` converte distância de metros para quilômetros.

Sintaxe

```
1 my_m_to_km(distance)
```

Entrada: `distance` é um escalar ou array contendo distâncias em metros.

Saída: Retorna as distâncias convertidas para quilômetros.

A conversão é

$$d_{\text{km}} = 10^{-3} d_{\text{m}}. \quad (113)$$

3.11.2 my_km_to_m

A função `my_km_to_m` converte distância de quilômetros para metros.

Sintaxe

```
1 my_km_to_m(distance)
```

Entrada: `distance` é um escalar ou array contendo distâncias em quilômetros.

Saída: Retorna as distâncias convertidas para metros.

A conversão é

$$d_{\text{m}} = 10^3 d_{\text{km}}. \quad (114)$$

3.12 Funções de conversão gravimétrica

As funções de conversão gravimétrica transformam valores de aceleração entre unidades SI e miligal. Elas são importantes porque a formulação física geralmente produz aceleração em m/s^2 , enquanto a interpretação gravimétrica aplicada utiliza mGal.

3.12.1 my_si_to_mgal

A função `my_si_to_mgal` converte valores de aceleração gravimétrica de SI para mGal.

Sintaxe

```
1 my_si_to_mgal(value)
```

Entrada: `value` é um escalar ou array contendo valores em m/s^2 .

Saída: Retorna os valores convertidos para mGal.

A conversão é

$$g_{\text{mGal}} = 10^5 g_{\text{SI}}. \quad (115)$$

3.12.2 my_mgal_to_si

A função `my_mgal_to_si` converte valores de aceleração gravimétrica de mGal para unidades SI.

Sintaxe

```
1 my_mgal_to_si(value)
```

Entrada: `value` é um escalar ou array contendo valores em mGal.

Saída: Retorna os valores convertidos para m/s^2 .

A conversão é

$$g_{\text{SI}} = 10^{-5} g_{\text{mGal}}. \quad (116)$$

3.13 Funções de conversão magnética

As funções de conversão magnética transformam valores entre Tesla e nanoTesla.

3.13.1 my_tesla_to_nt

A função `my_tesla_to_nt` converte campo magnético de Tesla para nanoTesla.

Sintaxe

```
1 my_tesla_to_nt(value)
```

Entrada: `value` é um escalar ou array contendo valores em Tesla.

Saída: Retorna os valores convertidos para nanoTesla.

A conversão é

$$B_{\text{nT}} = 10^9 B_{\text{T}}. \quad (117)$$

3.13.2 my_nt_to_tesla

A função `my_nt_to_tesla` converte campo magnético de nanoTesla para Tesla.

Sintaxe

```
1 my_nt_to_tesla(value)
```

Entrada: `value` é um escalar ou array contendo valores em nanoTesla.

Saída: Retorna os valores convertidos para Tesla.

A conversão é

$$B_{\text{T}} = 10^{-9} B_{\text{nT}}. \quad (118)$$

3.14 Controle de exportação do módulo

Ao final do arquivo, a variável `__all__` define explicitamente quais constantes, aliases e funções devem ser exportados quando o módulo for importado com a sintaxe

Importação global

```
1 from gravmag_codes.constants import *
```

Essa lista inclui constantes matemáticas, gravimétricas, magnéticas, geodésicas, terrestres, físicas, fatores de conversão, aliases antigos e funções auxiliares. O uso de `__all__` contribui para organizar a interface pública do módulo e reduzir ambiguidades durante a importação.

Interface pública

A variável `__all__` funciona como uma lista de símbolos públicos do módulo. Essa prática é útil em pacotes Python porque deixa explícito quais nomes devem ser considerados parte da API do pacote.

3.15 Síntese técnica do módulo

O módulo `constants.py` possui uma função estrutural dentro do pacote `gravmag_codes`. Embora seja um módulo curto, ele é fundamental para garantir coerência física, dimensional e computacional entre as diferentes rotinas do pacote. Suas constantes aparecem em modelagem direta, correções gravimétricas, cálculo de anomalias, transformações magnéticas, geomagnetismo, métodos eletromagnéticos, inversão e visualização científica.

Do ponto de vista didático, o módulo permite apresentar aos estudantes a importância das unidades físicas na implementação computacional. Em particular, mostra que um código geofísico não depende apenas da correta formulação matemática, mas também da consistência entre unidades, constantes e convenções adotadas.

Importância para ensino e pesquisa

O `constants.py` pode ser utilizado em aulas de Geofísica Computacional para discutir constantes físicas, unidades de medida, fatores de conversão, sistema internacional de unidades e padronização de códigos científicos. Em pesquisa, ele garante que diferentes módulos utilizem os mesmos valores de referência, evitando inconsistências entre resultados.

Em síntese, este módulo fornece a base física e dimensional sobre a qual os demais cálculos do pacote são construídos. Sua documentação é essencial para que futuros usuários compreendam quais valores foram adotados, quais unidades são esperadas e como as conversões são realizadas.

4 Descrição técnica do módulo `cylinder.py`

O arquivo `cylinder.py` reúne rotinas para criação, discretização e modelagem gravimétrica e magnética de cilindros verticais finitos. Esse módulo permite representar corpos geológicos com geometria cilíndrica, calcular seu volume, aproximá-los por prismas retangulares, discretizá-los em células de volume e estimar suas respostas gravimétricas e magnéticas em pontos de observação.

Em termos geofísicos, cilindros verticais podem ser utilizados como modelos simplificados para representar corpos aproximadamente circulares em planta, como intrusões ígneas, domos, condutos, chaminés vulcânicas, estruturas tubulares, corpos mineralizados ou anomalias localizadas com simetria radial aproximada. Embora corpos geológicos reais raramente sejam cilindros perfeitos, essa geometria é didaticamente útil por permitir estudar a relação entre profundidade, raio, contraste de densidade, magnetização e amplitude das anomalias observadas.

Papel do módulo `cylinder.py`

O módulo `cylinder.py` implementa uma abordagem numérica para calcular campos gravitacionais e magnéticos produzidos por cilindros verticais finitos. O cilindro é discretizado em pequenas células de volume, e a resposta total é obtida pela soma das contribuições individuais dessas células. Essa estratégia aproxima a integral de volume por um somatório discreto.

O módulo utiliza a biblioteca NumPy para operações numéricas e importa, quando disponíveis, os módulos `auxiliars.py` e `constants.py`. As constantes utilizadas incluem a constante gravitacional G , o fator de conversão de SI para mGal, a constante magnética $C_M = \mu_0/(4\pi)$, o fator de conversão de Tesla para nanoTesla e uma tolerância numérica ε .

As coordenadas são assumidas em metros e o eixo vertical z é positivo para baixo. A densidade, quando utilizada em modelagem gravimétrica, deve ser fornecida em g/cm^3 , sendo convertida internamente para kg/m^3 . A magnetização, quando utilizada em modelagem magnética, deve ser fornecida em A/m, e as componentes magnéticas calculadas são retornadas em nT.

4.1 Formulação geométrica do cilindro vertical

Um cilindro vertical finito é definido por cinco parâmetros geométricos principais:

$$(x_c, y_c, z_{top}, z_{bottom}, R), \quad (119)$$

em que x_c e y_c são as coordenadas horizontais do centro do cilindro, z_{top} é a profundidade do topo, z_{bottom} é a profundidade da base e R é o raio do cilindro. Como o eixo z é positivo para baixo, a condição física adotada no código é

$$z_{bottom} > z_{top}. \quad (120)$$

A altura do cilindro é dada por

$$H = z_{bottom} - z_{top}, \quad (121)$$

e seu volume é

$$V = \pi R^2 H. \quad (122)$$

Do ponto de vista numérico, o cilindro é dividido em setores radiais, setores angulares e camadas verticais. Cada célula resultante é tratada como um pequeno elemento de volume concentrado em seu centro. Assim, uma integral volumétrica é aproximada por um somatório:

$$\int_{\Omega} f(\mathbf{r}') dV' \approx \sum_{j=1}^N f(\mathbf{r}'_j) \Delta V_j, \quad (123)$$

em que $\mathbf{r}'_j = (x_j, y_j, z_j)$ é o centro da célula j , ΔV_j é o volume da célula e N é o número total de células discretizadas.

Natureza aproximada da modelagem

As respostas gravimétrica e magnética do cilindro neste módulo são calculadas por integração numérica discreta. Portanto, a precisão depende dos parâmetros de discretização **nradial**, **ntheta** e **nz**. Valores maiores tendem a produzir aproximações mais precisas, porém aumentam o custo computacional.

4.2 Funções internas de suporte

O módulo contém algumas funções internas, identificadas pelo prefixo `_`, que não constituem a interface principal para o usuário, mas são importantes para garantir validação, compatibilidade e robustez das rotinas.

4.2.1 `_check_observation_arrays`

A função `_check_observation_arrays` verifica se os arrays de coordenadas x e y dos pontos de observação possuem a mesma forma.

Sintaxe

```
1      _check_observation_arrays(x, y)
```

Entrada: x e y são arrays contendo as coordenadas horizontais dos pontos de observação.

Saída: A função não retorna valores. Caso as formas de x e y sejam diferentes, é gerado um erro do tipo `ValueError`.

Essa verificação é necessária porque os cálculos são realizados ponto a ponto. Portanto, cada coordenada x_i deve estar associada a uma coordenada y_i , formando o ponto de observação x_i, y_i .

4.2.2 `_prepare_z`

A função `_prepare_z` prepara a coordenada vertical dos pontos de observação.

Sintaxe

```
1      _prepare_z(x, z)
```

Entrada: x é usado como referência de forma. O argumento z pode ser escalar ou array.

Saída: Retorna z como array compatível com a forma de x .

Quando z é escalar, a função cria um array constante:

$$z_i = z_0, \quad i = 1, 2, \dots, n. \quad (124)$$

Quando z é array, sua forma deve ser igual à forma de x . Caso contrário, é gerado um erro.

4.2.3 `_get_property`

A função `_get_property` recupera uma propriedade física, como densidade ou magnetização, a partir de uma entrada explícita ou a partir do próprio modelo do cilindro.

Sintaxe

```
1      _get_property(model, value, index, default=None, name  
        ="physical property")
```

Entrada: `model` é a lista que representa o cilindro. `value` é o valor informado diretamente pelo usuário. `index` indica a posição da propriedade no modelo. `default` é o valor padrão opcional. `name` é o nome da propriedade usado na mensagem de erro.

Saída: Retorna a propriedade física escolhida.

A lógica da função é hierárquica: primeiro utiliza o valor informado diretamente; se esse valor for `None`, tenta recuperar a propriedade armazenada no modelo; se não houver propriedade no modelo, utiliza o valor padrão; se nenhuma dessas opções estiver disponível, gera erro.

4.2.4 `_dircos`

A função `_dircos` calcula os cossenos diretores associados a uma inclinação e declinação magnética.

Sintaxe

```
1      _dircos(inc, dec)
```

Entrada: `inc` é a inclinação em graus e `dec` é a declinação em graus.

Saída: Retorna as componentes unitárias `fx`, `fy` e `fz`.

A convenção adotada é x apontando para o norte, y apontando para leste e z positivo para baixo. Assim,

$$f_x = \cos I \cos D, \quad (125)$$

$$f_y = \cos I \sin D, \quad (126)$$

$$f_z = \sin I, \quad (127)$$

em que I é a inclinação e D é a declinação.

4.3 Criação e propriedades geométricas do cilindro

4.3.1 `my_cylinder_model`

A função `my_cylinder_model` cria uma representação computacional para um cilindro vertical finito.

Sintaxe

```
1 my_cylinder_model(xc, yc, top, bottom, radius,
    physical_property=None)
```

Entrada: `xc` e `yc` são as coordenadas horizontais do centro do cilindro. `top` é a profundidade do topo, `bottom` é a profundidade da base, `radius` é o raio e `physical_property` é uma propriedade física opcional, como densidade ou magnetização.

Saída: Retorna uma lista no formato `[xc, yc, top, bottom, radius]` ou `[xc, yc, top, bottom, radius, physical_property]`.

A função verifica duas condições geométricas:

$$R > 0, \quad (128)$$

e

$$z_{bottom} > z_{top}. \quad (129)$$

Caso essas condições não sejam atendidas, o modelo é fisicamente inválido e a função gera um erro.

4.3.2 `my_cylinder_volume`

A função `my_cylinder_volume` calcula o volume de um cilindro vertical finito.

Sintaxe

```
1 my_cylinder_volume(cylinder)
```

Entrada: `cylinder` é uma lista contendo pelo menos os cinco parâmetros geométricos do cilindro.

Saída: Retorna o volume do cilindro em unidades cúbicas compatíveis com as coordenadas de entrada. Se as coordenadas estiverem em metros, o volume será retornado em m³.

A formulação matemática é

$$V = \pi R^2 (z_{bottom} - z_{top}). \quad (130)$$

4.3.3 `my_cylinder_cells`

A função `my_cylinder_cells` discretiza o cilindro em células de volume usando divisões radiais, angulares e verticais.

Sintaxe

```
1 my_cylinder_cells(cylinder, nradial=20, ntheta=72, nz=1)
```

Entrada: `cylinder` é o modelo do cilindro. `nradial` é o número de divisões radiais. `ntheta` é o número de divisões angulares. `nz` é o número de divisões verticais.

Saída: Retorna quatro arrays: `xcell`, `ycell`, `zcell` e `volume`, contendo as coordenadas dos centros das células e seus volumes.

A altura de cada camada vertical é

$$\Delta z = \frac{z_{bottom} - z_{top}}{n_z}. \quad (131)$$

A abertura angular de cada setor é

$$\Delta \theta = \frac{2\pi}{n_\theta}. \quad (132)$$

Para cada anel radial, os limites interno e externo são

$$r_1 = R \frac{i}{n_r}, \quad r_2 = R \frac{i+1}{n_r}, \quad (133)$$

em que $i = 0, 1, \dots, n_r - 1$. O centroide radial do setor anular é aproximado por

$$r_m = \frac{2}{3} \frac{r_2^3 - r_1^3}{r_2^2 - r_1^2}. \quad (134)$$

A área de cada setor anular é

$$A_{setor} = \frac{1}{2} (r_2^2 - r_1^2) \Delta \theta, \quad (135)$$

e o volume de cada célula é

$$\Delta V = A_{setor} \Delta z. \quad (136)$$

As coordenadas horizontais do centro da célula são calculadas por

$$x_j = x_c + r_m \cos \theta_j, \quad (137)$$

$$y_j = y_c + r_m \sin \theta_j. \quad (138)$$

A coordenada vertical do centro da camada é

$$z_j = z_{top} + \left(k + \frac{1}{2}\right) \Delta z. \quad (139)$$

Discretização polar

A discretização polar é adequada para cilindros porque respeita naturalmente a simetria radial do corpo. Em vez de preencher o cilindro com uma malha cartesiana, o código utiliza anéis e setores angulares, produzindo células coerentes com a geometria circular da fonte.

4.3.4 `my_cylinder_to_prisms`

A função `my_cylinder_to_prisms` aproxima um cilindro vertical por um conjunto de prismas retangulares.

Sintaxe

```

1         my_cylinder_to_prisms(cylinder, spacing=None, shape=
2             None,
           physical_property=None, include_boundary=True)
```

Entrada: `cylinder` é o modelo do cilindro. `spacing` define o tamanho horizontal das células cartesianas. `shape` define o número de células ao longo das direções x e y . `physical_property` define a propriedade física associada aos prismas. `include_boundary` controla se células exatamente sobre o raio serão incluídas.

Saída: Retorna uma lista de prismas no formato `[xi, xf, yi, yf, top, bottom, physical_property]`.

A estratégia consiste em construir uma malha cartesiana sobre o quadrado que contém o cilindro. Para cada célula, calcula-se a distância radial do centro da célula ao centro do cilindro:

$$r = \sqrt{(x - x_c)^2 + (y - y_c)^2}. \quad (140)$$

Se `include_boundary=True`, a célula é mantida quando

$$r \leq R. \quad (141)$$

Caso contrário, é mantida apenas quando

$$r < R. \quad (142)$$

Essa função é útil quando se deseja reutilizar rotinas de modelagem baseadas em prismas retangulares.

4.4 Modelagem gravimétrica de um cilindro

As funções gravimétricas do módulo calculam o potencial gravitacional e as componentes da atração gravitacional de um cilindro vertical finito. A abordagem adotada é a discretização do cilindro em células de volume. Cada célula é tratada como uma pequena massa pontual localizada no centro da célula.

Para uma célula j , com volume ΔV_j e densidade ρ , a massa aproximada é

$$\Delta m_j = \rho \Delta V_j. \quad (143)$$

A distância entre o ponto de observação (x, y, z) e a célula x_j, y_j, z_j é

$$r_j = \sqrt{(x - x_j)^2 + (y - y_j)^2 + (z - z_j)^2 + \varepsilon^2}. \quad (144)$$

O termo ε^2 é adicionado para evitar singularidades numéricas quando o ponto de observação coincide ou se aproxima demais do centro de uma célula.

4.4.1 `my_cylinder_potential`

A função `my_cylinder_potential` calcula o potencial gravitacional produzido por um cilindro vertical finito.

Sintaxe

```
1 my_cylinder_potential(x, y, z, cylinder, rho=None,
2 nradial=20, ntheta=72, nz=1)
```

Entrada: **x** e **y** são arrays com as coordenadas horizontais dos pontos de observação. **z** pode ser escalar ou array com as coordenadas verticais. **cylinder** é o modelo do cilindro. **rho** é a densidade em g/cm³. Os parâmetros **nradial**, **ntheta** e **nz** controlam a discretização.

Saída: Retorna o potencial gravitacional em unidades SI.

A densidade é convertida internamente por

$$\rho_{SI} = 1000\rho_{\text{g/cm}^3}. \quad (145)$$

O potencial é aproximado por

$$V(x, y, z) = G\rho_{SI} \sum_{j=1}^N \frac{\Delta V_j}{r_j}. \quad (146)$$

4.4.2 my_cylinder_gx

A função `my_cylinder_gx` calcula a componente x da atração gravitacional produzida por um cilindro.

Sintaxe

```
1 my_cylinder_gx(x, y, z, cylinder, rho=None,
2 nradial=20, ntheta=72, nz=1)
```

Entrada: Os argumentos são equivalentes aos da função `my_cylinder_potential`.

Saída: Retorna a componente g_x em mGal.

A componente horizontal é calculada por

$$g_x(x, y, z) = G\rho_{SI} \sum_{j=1}^N \Delta V_j \frac{x_j - x}{r_j^3}. \quad (147)$$

Em seguida, o resultado é convertido para miligal:

$$g_{x,\text{mGal}} = 10^5 g_{x,\text{SI}}. \quad (148)$$

4.4.3 my_cylinder_gy

A função `my_cylinder_gy` calcula a componente y da atração gravitacional.

Sintaxe

```
1 my_cylinder_gy(x, y, z, cylinder, rho=None,
2 nradial=20, ntheta=72, nz=1)
```

Entrada: Os argumentos são equivalentes aos da função `my_cylinder_potential`.

Saída: Retorna a componente g_y em mGal.

A formulação é

$$g_y(x, y, z) = G\rho_{SI} \sum_{j=1}^N \Delta V_j \frac{y_j - y}{r_j^3}. \quad (149)$$

4.4.4 my_cylinder_gz

A função `my_cylinder_gz` calcula a componente vertical da atração gravitacional.

Sintaxe

```
1 my_cylinder_gz(x, y, z, cylinder, rho=None,
2 nradial=20, ntheta=72, nz=1)
```

Entrada: Os argumentos são equivalentes aos da função `my_cylinder_potential`.

Saída: Retorna a componente g_z em mGal.

Como o eixo vertical é positivo para baixo, a componente vertical é calculada por

$$g_z(x, y, z) = G\rho_{SI} \sum_{j=1}^N \Delta V_j \frac{z_j - z}{r_j^3}. \quad (150)$$

Essa convenção é coerente com o uso geofísico adotado nos demais módulos de corpos geométricos do pacote.

4.4.5 my_cylinder_gxyz

A função `my_cylinder_gxyz` calcula conjuntamente as três componentes gravimétricas g_x , g_y e g_z .

Sintaxe

```
1 my_cylinder_gxyz(x, y, z, cylinder, rho=None,
2 nradial=20, ntheta=72, nz=1)
```

Entrada: Recebe os mesmos argumentos das funções componentes.

Saída: Retorna três arrays: `gx`, `gy` e `gz`, todos em mGal.

Internamente, a função chama separadamente as funções `my_cylinder_gx`, `my_cylinder_gy` e `my_cylinder_gz`. Portanto, é uma função de conveniência para fluxos nos quais as três componentes são necessárias.

4.5 Modelagem magnética de um cilindro

As funções magnéticas do módulo aproximam o cilindro por um conjunto de dipolos magnéticos elementares associados às células de volume. Cada célula possui volume ΔV_j e magnetização $\mathbf{M}$. O momento magnético aproximado da célula é

$$\mathbf{m}_j = \mathbf{M} \Delta V_j. \quad (151)$$

A direção da magnetização é definida por inclinação I_s e declinação D_s . O vetor unitário da magnetização é

$$\hat{\mathbf{m}} = (m_x, m_y, m_z), \quad (152)$$

com

$$m_x = \cos I_s \cos D_s, \quad (153)$$

$$m_y = \cos I_s \sin D_s, \quad (154)$$

$$m_z = \sin I_s. \quad (155)$$

Para um dipolo magnético elementar, o campo magnético pode ser representado de forma proporcional a

$$\mathbf{B} = \frac{\mu_0}{4\pi} \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right], \quad (156)$$

em que $\mathbf{r}$ é o vetor que liga a fonte ao ponto de observação. No código, essa contribuição é somada para todas as células do cilindro e convertida para nanoTesla.

4.5.1 `my_cylinder_bxyz`

A função `my_cylinder_bxyz` calcula as componentes magnéticas B_x , B_y e B_z produzidas por um cilindro magnetizado.

Sintaxe

```

1      my_cylinder_bxyz(x, y, z, cylinder, mag=None, incs
2      =90.0, decs=0.0,
      nradial=20, ntheta=72, nz=1)
```

Entrada: `x`, `y` e `z` definem os pontos de observação. `cylinder` é o modelo do cilindro. `mag` é a intensidade da magnetização em A/m. `incs` e `decs` são a inclinação e a declinação da magnetização da fonte. Os parâmetros `nradial`, `ntheta` e `nz` controlam a discretização.

Saída: Retorna `bx`, `by` e `bz`, correspondentes às componentes magnéticas em nT.

Para cada célula, são calculadas as diferenças

$$r_x = x - x_j, \quad r_y = y - y_j, \quad r_z = z - z_j, \quad (157)$$

e

$$r^2 = r_x^2 + r_y^2 + r_z^2 + \varepsilon^2. \quad (158)$$

O produto escalar entre o vetor posição e a direção da magnetização é

$$\mathbf{r} \cdot \hat{\mathbf{m}} = r_x m_x + r_y m_y + r_z m_z. \quad (159)$$

As componentes acumuladas são calculadas por

$$B_x = C_M \sum_{j=1}^N M \Delta V_j \frac{3(\mathbf{r}_j \cdot \hat{\mathbf{m}})r_x - r^2 m_x}{r^5}, \quad (160)$$

$$B_y = C_M \sum_{j=1}^N M \Delta V_j \frac{3(\mathbf{r}_j \cdot \hat{\mathbf{m}})r_y - r^2 m_y}{r^5}, \quad (161)$$

$$B_z = C_M \sum_{j=1}^N M \Delta V_j \frac{3(\mathbf{r}_j \cdot \hat{\mathbf{m}})r_z - r^2 m_z}{r^5}. \quad (162)$$

Ao final, as componentes são multiplicadas por 10^9 para conversão de Tesla para nanoTesla.

4.5.2 my_cylinder_bx, my_cylinder_by e my_cylinder_bz

As funções my_cylinder_bx, my_cylinder_by e my_cylinder_bz retornam individualmente as componentes magnéticas do cilindro.

Sintaxe

```

1      my_cylinder_bx(x, y, z, cylinder, mag=None, incs
2          =90.0, decs=0.0,
3          nradial=20, ntheta=72, nz=1)

4      my_cylinder_by(x, y, z, cylinder, mag=None, incs
5          =90.0, decs=0.0,
6          nradial=20, ntheta=72, nz=1)

7      my_cylinder_bz(x, y, z, cylinder, mag=None, incs
8          =90.0, decs=0.0,
          nradial=20, ntheta=72, nz=1)

```

Entrada: Recebem os mesmos argumentos da função my_cylinder_bxyz.

Saída: Retornam, respectivamente, bx, by ou bz, em nT.

Essas funções são wrappers de conveniência. Internamente, chamam `my_cylinder_bxyz` e retornam apenas a componente solicitada.

4.5.3 `my_cylinder_tfa`

A função `my_cylinder_tfa` calcula a anomalia magnética de campo total produzida por um cilindro magnetizado.

Sintaxe

```
1 my_cylinder_tfa(x, y, z, cylinder, mag=None,
2 inc=-20.0, dec=0.0, incs=None, decs=None,
3 nradial=20, ntheta=72, nz=1)
```

Entrada: `x`, `y` e `z` definem os pontos de observação. `cylinder` é o modelo do cilindro. `mag` é a magnetização em A/m. `inc` e `dec` definem a direção do campo geomagnético regional. `incs` e `decs` definem a direção da magnetização da fonte. Quando `incs` e `decs` são `None`, assume-se magnetização induzida na direção do campo regional.

Saída: Retorna a anomalia de campo total em nT.

A anomalia de campo total é calculada pela projeção do vetor anômalo $\mathbf{B}$ na direção do campo regional:

$$\Delta T = B_x f_x + B_y f_y + B_z f_z, \quad (163)$$

em que

$$\hat{\mathbf{F}} = (f_x, f_y, f_z) \quad (164)$$

é o vetor unitário do campo geomagnético regional.

Magnetização induzida

Quando `incs=None` e `decs=None`, o código assume que a magnetização da fonte possui a mesma direção do campo regional. Essa é a hipótese de magnetização induzida, frequentemente usada como aproximação inicial em modelagem magnética.

4.5.4 my_cylinder_tf

A função `my_cylinder_tf` é um alias de compatibilidade para `my_cylinder_tfa`.

Sintaxe

```

1      my_cylinder_tf(x, y, z, cylinder, mag=None,
2      inc=-20.0, dec=0.0, incs=None, decs=None,
3      nradial=20, ntheta=72, nz=1)
```

Entrada: Recebe os mesmos argumentos de `my_cylinder_tfa`.

Saída: Retorna a anomalia de campo total em nT.

Sua função principal é preservar compatibilidade com scripts antigos que utilizavam o nome `my_cylinder_tf`.

4.6 Modelagem de múltiplos cilindros

O módulo também contém funções para calcular a resposta total produzida por uma lista de cilindros. Nesses casos, a resposta final é obtida pela superposição linear das respostas individuais:

$$d_{total} = \sum_{k=1}^{N_c} d_k, \quad (165)$$

em que N_c é o número de cilindros e d_k é a resposta produzida pelo cilindro k .

Essa propriedade decorre da linearidade dos campos potenciais em relação à distribuição de massa ou magnetização.

4.6.1 my_cylinders_potential

A função `my_cylinders_potential` calcula o potencial gravitacional produzido por múltiplos cilindros.

Sintaxe

```

1      my_cylinders_potential(x, y, z, cylinders, rho=None,
2      nradial=20, ntheta=72, nz=1)
```

Entrada: `cylinders` é uma lista contendo vários modelos de cilindro. Os demais argumentos seguem a mesma definição usada para um único cilindro.

Saída: Retorna o potencial gravitacional total em unidades SI.

4.6.2 `my_cylinders_gx`, `my_cylinders_gy` e `my_cylinders_gz`

As funções `my_cylinders_gx`, `my_cylinders_gy` e `my_cylinders_gz` calculam as componentes gravimétricas totais produzidas por uma lista de cilindros.

Sintaxe

```

1      my_cylinders_gx(x, y, z, cylinders, rho=None,
2      nradial=20, ntheta=72, nz=1)
3
4      my_cylinders_gy(x, y, z, cylinders, rho=None,
5      nradial=20, ntheta=72, nz=1)
6
7      my_cylinders_gz(x, y, z, cylinders, rho=None,
8      nradial=20, ntheta=72, nz=1)

```

Entrada: `cylinders` é uma lista de cilindros. Os demais argumentos são equivalentes aos das funções de um único cilindro.

Saída: Retornam, respectivamente, as componentes totais g_x , g_y e g_z , em mGal.

A resposta total é dada por

$$g_{\alpha}^{total} = \sum_{k=1}^{N_c} g_{\alpha}^{(k)}, \quad \alpha \in \{x, y, z\}. \quad (166)$$

4.6.3 `my_cylinders_gxyz`

A função `my_cylinders_gxyz` calcula conjuntamente as três componentes gravimétricas produzidas por múltiplos cilindros.

Sintaxe

```
1 my_cylinders_gxyz(x, y, z, cylinders, rho=None,
2 nradial=20, ntheta=72, nz=1)
```

Entrada: Recebe uma lista de cilindros e os parâmetros de observação e discretização.

Saída: Retorna *gx*, *gy* e *gz*, todos em mGal.

4.6.4 my_cylinders_bxyz

A função *my_cylinders_bxyz* calcula as componentes magnéticas totais produzidas por múltiplos cilindros magnetizados.

Sintaxe

```
1 my_cylinders_bxyz(x, y, z, cylinders, mag=None,
2 incs=90.0, decs=0.0,
3 nradial=20, ntheta=72, nz=1)
```

Entrada: *cylinders* é uma lista de cilindros. *mag* é a magnetização em A/m. *incs* e *decs* definem a direção da magnetização.

Saída: Retorna *bx*, *by* e *bz*, em nT.

A resposta magnética total é calculada por

$$\mathbf{B}_{total} = \sum_{k=1}^{N_c} \mathbf{B}^{(k)}. \quad (167)$$

4.6.5 my_cylinders_tfa

A função *my_cylinders_tfa* calcula a anomalia magnética de campo total produzida por múltiplos cilindros.

Sintaxe

```
1 my_cylinders_tfa(x, y, z, cylinders, mag=None,
```

```

2         inc=-20.0, dec=0.0, incs=None, decs=None,
3         nradial=20, ntheta=72, nz=1)

```

Entrada: `cylinders` é uma lista de cilindros. Os demais argumentos seguem a mesma definição da função `my_cylinder_tfa`.

Saída: Retorna a anomalia magnética de campo total em nT.

A superposição é feita por

$$\Delta T_{total} = \sum_{k=1}^{N_c} \Delta T^{(k)}. \quad (168)$$

4.6.6 `my_cylinders_tf`

A função `my_cylinders_tf` é um alias de compatibilidade para `my_cylinders_tfa`.

Sintaxe

```

1     my_cylinders_tf(x, y, z, cylinders, mag=None,
2                     inc=-20.0, dec=0.0, incs=None, decs=None,
3                     nradial=20, ntheta=72, nz=1)

```

Entrada: Recebe os mesmos argumentos de `my_cylinders_tfa`.

Saída: Retorna a anomalia de campo total produzida pela lista de cilindros.

4.7 Aliases de compatibilidade

Ao final do módulo, vários aliases são definidos para preservar compatibilidade com diferentes estilos de chamada usados em versões anteriores do pacote. Entre eles, destacam-se:

Aliases principais

```

1     cylinder_model = my_cylinder_model
2     cylinder_volume = my_cylinder_volume
3     cylinder_cells = my_cylinder_cells

```

```
4         cylinder_to_prisms = my_cylinder_to_prisms
5
6         cylinder_potential = my_cylinder_potential
7         cylinder_gx = my_cylinder_gx
8         cylinder_gy = my_cylinder_gy
9         cylinder_gz = my_cylinder_gz
10        cylinder_gxyz = my_cylinder_gxyz
11
12        cylinder_bx = my_cylinder_bx
13        cylinder_by = my_cylinder_by
14        cylinder_bz = my_cylinder_bz
15        cylinder_bxyz = my_cylinder_bxyz
16        cylinder_tf = my_cylinder_tf
17        cylinder_tfa = my_cylinder_tfa
```

Também são definidos aliases adicionais para anomalia magnética de campo total:

Aliases magnéticos adicionais

```
1         my_cylinder_totalfield = my_cylinder_tfa
2         my_cylinder_total_field = my_cylinder_tfa
3         my_cylinder_magnetic = my_cylinder_tfa
4         my_cylinder_mag = my_cylinder_tfa
5
6         my_cylinders_totalfield = my_cylinders_tfa
7         my_cylinders_total_field = my_cylinders_tfa
8         my_cylinders_magnetic = my_cylinders_tfa
9         my_cylinders_mag = my_cylinders_tfa
```

Preservação dos nomes das funções

A manutenção desses aliases é importante porque evita quebrar notebooks, scripts e exemplos já existentes. Essa estratégia permite reorganizar e documentar o pacote sem comprometer a compatibilidade com fluxos de trabalho anteriores.

4.8 Exemplo conceitual de uso

O exemplo abaixo ilustra a criação de um cilindro vertical, o cálculo de seu volume e a estimativa da componente vertical da gravidade em uma malha de observação.

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import cylinder
3
4      # Cria uma malha de observação
5      x = np.linspace(-5000, 5000, 101)
6      y = np.linspace(-5000, 5000, 101)
7      X, Y = np.meshgrid(x, y)
8      Z = 0.0
9
10     # Cria um cilindro vertical
11     # xc, yc, top, bottom, radius, density
12     cil = cylinder.my_cylinder_model(
13     xc=0.0,
14     yc=0.0,
15     top=1000.0,
16     bottom=4000.0,
17     radius=1000.0,
18     physical_property=0.5
19     )
20
21     # Calcula o volume
22     volume = cylinder.my_cylinder_volume(cil)
23
24     # Calcula a componente vertical da gravidade
25     gz = cylinder.my_cylinder_gz(
26     X, Y, Z,
27     cil,
28     rho=None,
29     nradius=20,
30     ntheta=72,
31     nz=5
32     )
```

Nesse exemplo, a densidade foi armazenada diretamente no modelo do cilindro como `physical_property=0.5`, em g/cm^3 . Como o argumento `rho=None` foi usado na chamada da função `my_cylinder_gz`, o código recupera a densidade diretamente do modelo.

4.9 Síntese técnica do módulo

O módulo `cylinder.py` fornece um conjunto completo de funções para modelagem direta de cilindros verticais finitos. Sua estrutura permite criar modelos geométricos, calcular volumes, discretizar o cilindro em células de volume, aproximá-lo por prismas retangulares e calcular respostas gravimétricas e magnéticas em pontos arbitrários de observação.

A principal característica metodológica do módulo é o uso de integração numérica discreta. O cilindro é dividido em setores radiais, setores angulares e camadas verticais, e cada célula contribui para o campo total. Essa abordagem torna o módulo flexível e fácil de compreender didaticamente, embora exija atenção à escolha dos parâmetros de discretização.

Importância para ensino de Geofísica

O `cylinder.py` é especialmente útil no ensino de modelagem direta porque permite relacionar geometria, propriedade física e resposta geofísica. O estudante pode modificar raio, profundidade, altura, densidade, magnetização e discretização, observando diretamente como esses parâmetros afetam as anomalias gravimétricas e magnéticas.

Do ponto de vista científico, o módulo pode ser usado em testes sintéticos, validação de algoritmos, comparação com rotinas baseadas em prismas e construção de exemplos didáticos para métodos potenciais. Em trabalhos futuros, esse módulo também pode servir como base para discutir precisão numérica, convergência da discretização e custo computacional em modelagem direta.

5 Descrição técnica do módulo `derivative.py`

O arquivo `derivative.py` reúne funções para cálculo de derivadas espaciais, derivadas espectrais, gradientes, Laplaciano, tilt derivative, theta map e operadores derivados aplicados a campos potenciais em malhas regulares. Esse módulo é particularmente importante para o processamento e a interpretação de dados gravimétricos e magnéticos, pois muitos atributos geofísicos são obtidos a partir de derivadas do campo observado.

Em métodos potenciais, derivadas espaciais são usadas para realçar bordas, contatos laterais, fontes rasas, lineamentos, zonas de contraste físico e mudanças abruptas na distribuição de densidade ou magnetização. No contexto do pacote *gravmag_codes*, o módulo **derivative.py** fornece tanto derivadas no domínio da frequência, calculadas por transformada de Fourier, quanto derivadas no domínio espacial, calculadas por diferenças finitas.

Papel do módulo *derivative.py*

O módulo *derivative.py* fornece operadores diferenciais aplicados a campos potenciais organizados em malhas regulares. Ele permite calcular derivadas horizontais e verticais, gradiente horizontal, gradiente total, derivadas mistas, Laplaciano, tilt derivative, theta map e derivadas associadas ao tilt.

As entradas principais das funções são arrays bidimensionais **X**, **Y** e **D**, representando, respectivamente, as coordenadas da malha e o campo geofísico observado ou sintético. A maioria das funções possui o argumento **flatten**, que controla se a saída será retornada como vetor unidimensional ou como matriz bidimensional. O valor padrão **flatten=True** foi preservado para manter compatibilidade com versões anteriores do pacote.

5.1 Fundamentos espectrais das derivadas

Seja $f(x, y)$ um campo potencial definido em uma malha regular. Sua transformada de Fourier bidimensional é representada por

$$F(k_x, k_y) = \mathcal{F}\{f(x, y)\}, \quad (169)$$

em que k_x e k_y são os números de onda nas direções x e y . No domínio da frequência, derivadas espaciais são obtidas por multiplicação espectral. A derivada de ordem n em relação a x é dada por

$$\mathcal{F}\left\{\frac{\partial^n f}{\partial x^n}\right\} = (ik_x)^n F(k_x, k_y), \quad (170)$$

e a derivada de ordem n em relação a y é dada por

$$\mathcal{F} \left\{ \frac{\partial^n f}{\partial y^n} \right\} = (ik_y)^n F(k_x, k_y). \quad (171)$$

Para campos potenciais, a derivada vertical é associada ao módulo do número de onda,

$$k = \sqrt{k_x^2 + k_y^2}. \quad (172)$$

Assim, a derivada vertical de ordem n pode ser representada por

$$\mathcal{F} \left\{ \frac{\partial^n f}{\partial z^n} \right\} = (\pm k)^n F(k_x, k_y), \quad (173)$$

em que o sinal depende da convenção adotada para o eixo vertical. No pacote, o argumento `z_positive_down` controla essa convenção.

Convenção vertical

Quando `z_positive_down=True`, a coordenada vertical é considerada positiva para baixo, seguindo uma convenção comum em modelagem geofísica. Quando `z_positive_down=False`, o sinal da derivada vertical é invertido.

5.2 Funções internas de validação, padding e FFT

O módulo contém funções internas que verificam a consistência das entradas, controlam o formato da saída, estimam espaçamentos da malha, constroem números de onda, aplicam padding, aplicam taper e executam o núcleo das derivadas no domínio da frequência.

5.2.1 `_check_grid_inputs`

A função `_check_grid_inputs` verifica se `x`, `y` e `data` são arrays bidimensionais e possuem a mesma forma.

Sintaxe

1

```
_check_grid_inputs(x, y, data)
```

Entrada: `x` e `y` são arrays de coordenadas; `data` é o campo geofísico.

Saída: Não retorna valores. Caso as entradas sejam incompatíveis, gera erro do tipo `ValueError`.

Essa função garante que cada valor do campo d_{ij} esteja associado a um par de coordenadas x_{ij}, y_{ij} .

5.2.2 `_as_output`

A função `_as_output` controla o formato da saída.

Sintaxe

```
1 _as_output(result, original_shape=None, flatten=True)
```

Entrada: `result` é o resultado calculado; `original_shape` é a forma original da grade; `flatten` controla se a saída será vetorizada.

Saída: Retorna o resultado como vetor unidimensional ou matriz bidimensional.

Quando `flatten=True`, a matriz é reorganizada como vetor:

$$\mathbf{d}_{1D} = \text{reshape}(\mathbf{D}). \quad (174)$$

5.2.3 `_grid_spacing_from_coordinates`

A função `_grid_spacing_from_coordinates` estima os espaçamentos médios Δx e Δy a partir das coordenadas da malha.

Sintaxe

```
1 _grid_spacing_from_coordinates(x, y)
```

Entrada: `x` e `y` são arrays bidimensionais de coordenadas.

Saída: Retorna `dx` e `dy`.

A estimativa é baseada em diferenças médias entre coordenadas adjacentes:

$$\Delta x \approx \text{mean}(x_{i,j+1} - x_{i,j}), \quad (175)$$

$$\Delta y \approx \text{mean}(y_{i+1,j} - y_{i,j}). \quad (176)$$

5.2.4 `_get_wavenumbers`

A função `_get_wavenumbers` constrói os números de onda k_x e k_y usados em operadores espectrais.

Sintaxe

```
1  _get_wavenumbers(x, y)
```

Entrada: `x` e `y` são coordenadas de uma malha regular.

Saída: Retorna `ky` e `kx`.

Os números de onda são calculados a partir das frequências discretas da FFT:

$$k_x = 2\pi f_x, \quad k_y = 2\pi f_y. \quad (177)$$

5.2.5 `_cosine_taper_1d` e `_cosine_taper_2d`

As funções `_cosine_taper_1d` e `_cosine_taper_2d` criam janelas cossenoidais para reduzir efeitos de borda antes da aplicação da FFT.

Sintaxe

```
1  _cosine_taper_1d(n, percent=0.10)
2  _cosine_taper_2d(data, percent=0.10)
```

A janela unidimensional possui transição suave nas bordas, baseada em

$$w(t) = \sin^2(t), \quad 0 \leq t \leq \frac{\pi}{2}. \quad (178)$$

A janela bidimensional é construída por produto externo:

$$\mathbf{W} = \mathbf{w}_y \mathbf{w}_x^T. \quad (179)$$

5.2.6 `_pad_data` e `_crop_data`

A função `_pad_data` expande a grade antes da FFT, enquanto `_crop_data` remove o padding após o processamento.

Sintaxe

```
1      _pad_data(data, pad_factor=0.0, pad_mode="reflect",
2              taper=False)
      _crop_data(data_pad, pad_width)
```

Entrada: `data` é a matriz original; `pad_factor` define a fração de expansão; `pad_mode` define o modo de preenchimento; `taper` controla a aplicação de janela.

Saída: `_pad_data` retorna a matriz expandida e a largura do padding; `_crop_data` retorna a matriz recortada no tamanho original.

Padding em filtros FFT

O padding reduz artefatos de borda associados à periodicidade implícita da transformada discreta de Fourier. O taper suaviza as bordas e reduz descontinuidades artificiais.

5.2.7 `_fft_derivative_operator`

A função `_fft_derivative_operator` é o núcleo computacional das derivadas espectrais.

Sintaxe

```
1      _fft_derivative_operator(x, y, data,
2              operator_function,
              pad_factor=0.0, pad_mode="reflect", taper=False)
```

Entrada: x , y e $data$ definem a malha e o campo; `operator_function` é uma função que recebe k_x , k_y e k , retornando o operador espectral.

Saída: Retorna o campo derivado no domínio espacial.

A operação geral é

$$D_{out}(x, y) = \mathcal{F}^{-1} \{ \mathcal{F}[D(x, y)] O(k_x, k_y, k) \}, \quad (180)$$

em que $O(k_x, k_y, k)$ é o operador espectral definido para cada derivada.

5.3 Derivadas no domínio da frequência

5.3.1 `my_xderiv`

A função `my_xderiv` calcula a derivada de ordem n em relação a x , no domínio da frequência.

Sintaxe

```
1 my_xderiv(x, y, data, n=1, pad_factor=0.0,
2 pad_mode="reflect", taper=False, flatten=True)
```

Entrada: x , y e $data$ são matrizes bidimensionais. n é a ordem da derivada.

Saída: Retorna $\partial^n f / \partial x^n$.

A formulação espectral é

$$\frac{\partial^n f}{\partial x^n} = \mathcal{F}^{-1} [(ik_x)^n F(k_x, k_y)]. \quad (181)$$

5.3.2 `my_yderiv`

A função `my_yderiv` calcula a derivada de ordem n em relação a y .

Sintaxe

```

1      my_yderiv(x, y, data, n=1, pad_factor=0.0,
2      pad_mode="reflect", taper=False, flatten=True)

```

Saída: Retorna $\partial^n f / \partial y^n$.

A formulação é

$$\frac{\partial^n f}{\partial y^n} = \mathcal{F}^{-1} [(ik_y)^n F(k_x, k_y)]. \quad (182)$$

5.3.3 my_zderiv

A função `my_zderiv` calcula a derivada vertical de ordem n .

Sintaxe

```

1      my_zderiv(x, y, data, n=1, pad_factor=0.0,
2      pad_mode="reflect", taper=False, flatten=True,
3      z_positive_down=True)

```

Saída: Retorna $\partial^n f / \partial z^n$.

A formulação é

$$\frac{\partial^n f}{\partial z^n} = \mathcal{F}^{-1} [(sk)^n F(k_x, k_y)], \quad (183)$$

em que

$$s = \begin{cases} 1, & z \text{ positivo para baixo,} \\ -1, & z \text{ positivo para cima.} \end{cases} \quad (184)$$

5.3.4 my_hgrad e my_totalgrad

A função `my_hgrad` calcula a amplitude do gradiente horizontal:

Sintaxe


```

1 my_hgrad(x, y, data, pad_factor=0.0,
2 pad_mode="reflect", taper=False, flatten=True)

```

$$\text{HGA} = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}. \quad (185)$$

A função `my_totalgrad` calcula a amplitude do gradiente total:

Sintaxe

```

1 my_totalgrad(x, y, data, pad_factor=0.0,
2 pad_mode="reflect", taper=False, flatten=True,
3 z_positive_down=True)

```

$$\text{TGA} = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2 + \left(\frac{\partial f}{\partial z}\right)^2}. \quad (186)$$

Gradiente horizontal e gradiente total

O gradiente horizontal é útil para realçar bordas laterais de fontes, enquanto o gradiente total, também conhecido como sinal analítico em algumas aplicações magnéticas, tende a produzir máximos sobre fontes ou contatos, dependendo da geometria e da direção de magnetização.

5.4 Funções com saída em grade

As funções `my_xderiv_grid`, `my_yderiv_grid`, `my_zderiv_grid`, `my_hgrad_grid` e `my_totalgrad_grid` são versões de conveniência das funções anteriores, mas retornam sempre matrizes bidimensionais.

Sintaxe

```
1      my_xderiv_grid(x, y, data, n=1, pad_factor=0.0,
2      pad_mode="reflect", taper=False)
3
4      my_yderiv_grid(x, y, data, n=1, pad_factor=0.0,
5      pad_mode="reflect", taper=False)
6
7      my_zderiv_grid(x, y, data, n=1, pad_factor=0.0,
8      pad_mode="reflect", taper=False, z_positive_down=True
9      )
10
11     my_hgrad_grid(x, y, data, pad_factor=0.0,
12     pad_mode="reflect", taper=False)
13
14     my_totalgrad_grid(x, y, data, pad_factor=0.0,
15     pad_mode="reflect", taper=False,
16     z_positive_down=True)
```

Essas funções são úteis quando o resultado será usado diretamente em mapas de contorno, imagens ou rotinas de visualização.

5.5 Derivadas mistas e de ordem superior

5.5.1 `my_mixed_deriv_fft`

A função `my_mixed_deriv_fft` calcula derivadas mistas gerais no domínio da frequência.

Sintaxe

```
1      my_mixed_deriv_fft(x, y, data, nx_order=0, ny_order
2      =0,
3      nz_order=0, pad_factor=0.0,
4      pad_mode="reflect", taper=False,
5      flatten=True, z_positive_down=True)
```

Entrada: `nx_order`, `ny_order` e `nz_order` definem as ordens das derivadas em x , y e z .

Saída: Retorna a derivada mista especificada.

A formulação geral é

$$\frac{\partial^{p+q+r} f}{\partial x^p \partial y^q \partial z^r} = \mathcal{F}^{-1} [(ik_x)^p (ik_y)^q (sk)^r F(k_x, k_y)]. \quad (187)$$

5.5.2 `my_xzderiv`, `my_yzderiv`, `my_xxderiv`, `my_yyderiv` e `my_zzderiv`

Essas funções são atalhos para derivadas específicas:

Funções de derivadas específicas

```

1      my_xzderiv(x, y, data, ...)
2      my_yzderiv(x, y, data, ...)
3      my_xxderiv(x, y, data, ...)
4      my_yyderiv(x, y, data, ...)
5      my_zzderiv(x, y, data, ...)
```

Elas calculam, respectivamente,

$$\frac{\partial^2 f}{\partial x \partial z}, \quad \frac{\partial^2 f}{\partial y \partial z}, \quad \frac{\partial^2 f}{\partial x^2}, \quad \frac{\partial^2 f}{\partial y^2}, \quad \frac{\partial^2 f}{\partial z^2}. \quad (188)$$

Essas derivadas são importantes em atributos de realce, tensores de gradiente e operadores baseados em derivadas verticais.

5.6 Derivadas no domínio espacial

Além das derivadas espectrais, o módulo implementa derivadas no domínio espacial por meio de diferenças finitas, utilizando a função `numpy.gradient`.

5.6.1 `my_xderiv_space` e `my_yderiv_space`

As funções `my_xderiv_space` e `my_yderiv_space` calculam derivadas horizontais por diferenças finitas.

Sintaxe

```

1         my_xderiv_space(x, y, data, edge_order=2, flatten=
           True)
2         my_yderiv_space(x, y, data, edge_order=2, flatten=
           True)

```

A aproximação por diferenças finitas centrais pode ser representada por

$$\frac{\partial f}{\partial x} \approx \frac{f(x + \Delta x, y) - f(x - \Delta x, y)}{2\Delta x}, \quad (189)$$

$$\frac{\partial f}{\partial y} \approx \frac{f(x, y + \Delta y) - f(x, y - \Delta y)}{2\Delta y}. \quad (190)$$

5.6.2 `my_hgrad_space`, `my_laplacian_space` e `my_totalgrad_space`

A função `my_hgrad_space` calcula o gradiente horizontal por diferenças finitas:

$$\text{HGA}_{space} = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}. \quad (191)$$

A função `my_laplacian_space` calcula o Laplaciano horizontal:

$$\nabla_h^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}. \quad (192)$$

A função `my_totalgrad_space` calcula o gradiente total no domínio espacial, usando uma derivada vertical fornecida pelo usuário:

$$\text{TGA}_{space} = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2 + \left(\frac{\partial f}{\partial z}\right)^2}. \quad (193)$$

Diferença entre derivadas FFT e espaciais

As derivadas por FFT são eficientes e adequadas para malhas regulares, mas podem ser sensíveis a bordas e ruídos de alta frequência. As derivadas espaciais por diferenças finitas são mais diretas, porém dependem fortemente do espaçamento da malha e podem ser menos estáveis em dados ruidosos.

5.7 Atributos derivados: tilt, theta map e THDR

5.7.1 my_tilt_derivative

A função `my_tilt_derivative` calcula o tilt derivative ou ângulo tilt.

Sintaxe

```
1 my_tilt_derivative(x, y, data, pad_factor=0.0,  
2 pad_mode="reflect", taper=False,  
3 eps=1.0e-12, degrees=False,  
4 flatten=True, z_positive_down=True)
```

A formulação é

$$\text{TDR} = \arctan \left(\frac{\frac{\partial f}{\partial z}}{\sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}} \right). \quad (194)$$

Quando `degrees=True`, o resultado é convertido para graus.

5.7.2 my_tilt_vertical_derivative

A função `my_tilt_vertical_derivative` calcula o tilt aplicado à primeira derivada vertical do campo.

Sintaxe

```
1 my_tilt_vertical_derivative(x, y, data, pad_factor  
2 =0.0,  
3 pad_mode="reflect", taper=False,  
4 eps=1.0e-12, degrees=False,  
flatten=True, z_positive_down=True)
```

A formulação é

$$\phi = \arctan \left(\frac{\frac{\partial^2 f}{\partial z^2}}{\sqrt{\left(\frac{\partial^2 f}{\partial x \partial z}\right)^2 + \left(\frac{\partial^2 f}{\partial y \partial z}\right)^2}} \right). \quad (195)$$

5.7.3 my_theta_map

A função `my_theta_map` calcula o theta map, definido como o cosseno do tilt angle.

Sintaxe

```

1      my_theta_map(x, y, data, pad_factor=0.0,
2      pad_mode="reflect", taper=False,
3      eps=1.0e-12, flatten=True,
4      z_positive_down=True)
```

A formulação é

$$\theta = \frac{\text{HGA}}{\sqrt{\text{HGA}^2 + \left(\frac{\partial f}{\partial z}\right)^2}}. \quad (196)$$

5.7.4 my_thdr_tilt

A função `my_thdr_tilt` calcula a derivada horizontal total do tilt.

Sintaxe

```

1      my_thdr_tilt(x, y, data, pad_factor=0.0,
2      pad_mode="reflect", taper=False,
3      flatten=True, z_positive_down=True)
```

Primeiro calcula-se o tilt TDR . Em seguida, calcula-se o gradiente horizontal do tilt:

$$\text{THDR}_{TDR} = \sqrt{\left(\frac{\partial TDR}{\partial x}\right)^2 + \left(\frac{\partial TDR}{\partial y}\right)^2}. \quad (197)$$

5.8 Aliases de compatibilidade

O módulo define aliases para preservar compatibilidade e facilitar a leitura do código:

Aliases principais

```
1      my_hga = my_hgrad
2      my_tga = my_totalgrad
3      my_hgm = my_hgrad
4      my_asa = my_totalgrad
5      my_tilt = my_tilt_derivative
6      my_tdr = my_tilt_derivative
```

Esses aliases associam nomes usuais em interpretação de campos potenciais, como HGA, TGA, HGM, ASA e TDR, às funções implementadas.

5.9 Síntese técnica do módulo

O módulo **derivative.py** é essencial para o processamento de campos potenciais, pois implementa operadores diferenciais que transformam dados observados em atributos interpretativos. As derivadas horizontais, verticais e mistas permitem realçar estruturas geológicas, estimar limites de fontes, construir mapas de atributos e preparar dados para etapas posteriores de filtragem e interpretação.

Importância para ensino de Geofísica

O **derivative.py** permite demonstrar, de forma computacional, como conceitos de cálculo diferencial, transformada de Fourier e métodos numéricos são aplicados diretamente à interpretação geofísica. O módulo é adequado para aulas práticas sobre filtros derivativos, realce de anomalias, gradientes e atributos de campos potenciais.

6 Descrição técnica do módulo **electrical_methods.py**

O arquivo **electrical_methods.py** reúne funções didáticas e computacionais associadas aos métodos elétricos em Geofísica, especialmente resistividade elétrica, condutividade

elétrica, sondagem elétrica vertical, potencial elétrico, campo elétrico, seções geoeletricas e visualização de resultados. O módulo foi desenvolvido para apoiar simulações sintéticas, atividades de ensino e interpretações iniciais de modelos geoeletricos.

Os métodos elétricos são baseados na injeção de corrente elétrica no solo e na medição de diferenças de potencial. A partir da corrente aplicada, da geometria dos eletrodos e da diferença de potencial medida, estima-se a resistividade aparente do meio. Essa resistividade aparente é interpretada em termos de variações litológicas, conteúdo de água, porosidade, salinidade, argilosidade e estruturas subsuperficiais.

Papel do módulo `electrical_methods.py`

O módulo `electrical_methods.py` fornece funções para conversão entre resistividade e condutividade, cálculo de resistividade aparente, fatores geométricos Wenner e Schlumberger, simulação educacional de curvas SEV, potencial elétrico de fontes pontuais, campo elétrico e construção de seções geoeletricas sintéticas.

O módulo utiliza as bibliotecas NumPy, Matplotlib e os. A biblioteca NumPy é usada para cálculo numérico, Matplotlib para geração de gráficos e os para criação automática de diretórios ao salvar figuras.

6.1 Funções auxiliares internas e salvamento de figuras

6.1.1 `_arr`

A função interna `_arr` converte uma entrada para array NumPy do tipo `float`.

Sintaxe

```
1  _arr(a)
```

Entrada: `a` pode ser escalar, lista ou array.

Saída: Retorna `a` convertido para `numpy.ndarray`.

Essa função padroniza as entradas numéricas, permitindo operações vetorizadas.

6.1.2 my_save_figure

A função `my_save_figure` salva uma figura gerada com Matplotlib.

Sintaxe

```
1 my_save_figure(filename, fig=None, dpi=300,  
2 bbox_inches="tight", transparent=True)
```

Entrada: `filename` é o nome do arquivo de saída; `fig` é a figura a ser salva; `dpi` define a resolução; `bbox_inches` controla o recorte; `transparent` define se o fundo será transparente.

Saída: Retorna o nome do arquivo salvo.

Se a pasta indicada não existir, ela é criada automaticamente.

6.2 Resistividade, condutividade e resistividade aparente

6.2.1 my_resistivity_from_conductivity

A função `my_resistivity_from_conductivity` converte condutividade elétrica em resistividade elétrica.

Sintaxe

```
1 my_resistivity_from_conductivity(conductivity)
```

Entrada: `conductivity` é a condutividade elétrica σ , em S/m.

Saída: Retorna a resistividade elétrica ρ , em Ω m.

A relação matemática é

$$\rho = \frac{1}{\sigma}. \quad (198)$$

6.2.2 `my_conductivity_from_resistivity`

A função `my_conductivity_from_resistivity` converte resistividade elétrica em condutividade elétrica.

Sintaxe

```
1 my_conductivity_from_resistivity(resistivity)
```

Entrada: `resistivity` é a resistividade ρ , em Ω m.

Saída: Retorna a condutividade σ , em S/m.

A relação é

$$\sigma = \frac{1}{\rho}. \quad (199)$$

6.2.3 `my_apparent_resistivity_from_k`

A função `my_apparent_resistivity_from_k` calcula a resistividade aparente a partir da diferença de potencial, da corrente e do fator geométrico.

Sintaxe

```
1 my_apparent_resistivity_from_k(delta_v, current,
    geometric_factor)
```

Entrada: `delta_v` é a diferença de potencial ΔV ; `current` é a corrente I ; `geometric_factor` é o fator geométrico K .

Saída: Retorna a resistividade aparente ρ_a .

A formulação é

$$\rho_a = K \frac{\Delta V}{I}. \quad (200)$$

Resistividade aparente

A resistividade aparente não representa necessariamente a resistividade real de uma camada. Ela é uma grandeza equivalente obtida a partir da resposta integrada do subsolo para uma determinada geometria de eletrodos.

6.3 Fatores geométricos Wenner e Schlumberger

6.3.1 `my_wenner_geometric_factor`

A função `my_wenner_geometric_factor` calcula o fator geométrico do arranjo Wenner.

Sintaxe

```
1 my_wenner_geometric_factor(a)
```

Entrada: a é o espaçamento entre eletrodos.

Saída: Retorna o fator geométrico K .

Para o arranjo Wenner, o fator geométrico é

$$K = 2\pi a. \quad (201)$$

6.3.2 `my_schlumberger_geometric_factor`

A função `my_schlumberger_geometric_factor` calcula o fator geométrico do arranjo Schlumberger.

Sintaxe

```
1 my_schlumberger_geometric_factor(AB2, MN2)
```

Entrada: $AB2$ representa $AB/2$, metade da distância entre os eletrodos de corrente. $MN2$ representa $MN/2$, metade da distância entre os eletrodos de potencial.

Saída: Retorna o fator geométrico K .

A formulação implementada é

$$K = \frac{\pi [(AB/2)^2 - (MN/2)^2]}{2(MN/2)}. \quad (202)$$

6.4 Modelo de Terra em camadas e curva SEV

6.4.1 `my_layered_earth_model`

A função `my_layered_earth_model` cria um modelo geoeletrico unidimensional em camadas.

Sintaxe

```
1 my_layered_earth_model(resistivities, thicknesses)
```

Entrada: `resistivities` é o vetor de resistividades das camadas; `thicknesses` é o vetor de espessuras das camadas superiores. A última resistividade representa o semiespaço.

Saída: Retorna um dicionário com resistividades, espessuras, profundidades acumuladas e número de camadas.

A profundidade acumulada das interfaces é dada por

$$z_i = \sum_{j=1}^i h_j, \quad (203)$$

em que h_j é a espessura da camada j .

6.4.2 `my_sev_forward_smooth_transition`

A função `my_sev_forward_smooth_transition` simula uma curva educacional de sondagem elétrica vertical.

Sintaxe

```

1         my_sev_forward_smooth_transition(AB2, resistivities,
2         thicknesses, sharpness=3.0)

```

Entrada: `AB2` é o vetor de aberturas $AB/2$; `resistivities` contém as resistividades das camadas; `thicknesses` contém as espessuras; `sharpness` controla a rapidez das transições.

Saída: Retorna a curva sintética de resistividade aparente.

A função trabalha no domínio logarítmico das resistividades. Para cada interface, aplica uma transição logística:

$$T = \frac{1}{1 + \exp[-s(\log(AB/2) - \log(z_i))]}, \quad (204)$$

em que s é o parâmetro `sharpness` e z_i é a profundidade da interface. A resistividade aparente é construída por mistura suave entre as resistividades das camadas:

$$\log \rho_a^{novo} = (1 - T) \log \rho_a^{antigo} + T \log \rho_{i+1}. \quad (205)$$

Caráter educacional da curva SEV

A função `my_sev_forward_smooth_transition` não implementa um solucionador rigoroso por transformada de Hankel para corrente contínua. Ela fornece uma aproximação prática e didática para gerar curvas sintéticas de SEV, útil para ensino e prototipagem.

6.4.3 `my_classify_sev_curve`

A função `my_classify_sev_curve` classifica uma curva SEV de três camadas nos tipos clássicos H, K, A ou Q.

Sintaxe

```

1         my_classify_sev_curve(resistivities)

```

Entrada: `resistivities` é um vetor com pelo menos três resistividades.

Saída: Retorna uma string com a classe da curva.

A classificação é baseada nas relações entre as três primeiras resistividades:

$$H : \rho_1 > \rho_2 < \rho_3, \quad (206)$$

$$K : \rho_1 < \rho_2 > \rho_3, \quad (207)$$

$$A : \rho_1 < \rho_2 < \rho_3, \quad (208)$$

$$Q : \rho_1 > \rho_2 > \rho_3. \quad (209)$$

6.5 Potencial elétrico e campo elétrico

6.5.1 `my_point_current_potential`

A função `my_point_current_potential` calcula o potencial elétrico gerado por uma fonte pontual de corrente em meio homogêneo infinito.

Sintaxe

```
1 my_point_current_potential(x, y, z, source,
2 current=1.0, resistivity=100.0,
3 eps=1e-12)
```

Entrada: `x`, `y` e `z` são coordenadas de observação; `source` é a posição da fonte; `current` é a corrente; `resistivity` é a resistividade do meio.

Saída: Retorna o potencial elétrico V .

A formulação para espaço completo é

$$V = \frac{\rho I}{4\pi r}, \quad (210)$$

em que

$$r = \sqrt{(x - x_s)^2 + (y - y_s)^2 + (z - z_s)^2}. \quad (211)$$

6.5.2 my_surface_point_current_potential

A função `my_surface_point_current_potential` calcula o potencial elétrico de uma fonte pontual de corrente na superfície de um semiespaço homogêneo.

Sintaxe

```
1      my_surface_point_current_potential(x, y, source ,
2      current=1.0,
3      resistivity=100.0,
4      eps=1e-12)
```

A formulação para semiespaço homogêneo é

$$V = \frac{\rho I}{2\pi r}, \quad (212)$$

em que

$$r = \sqrt{(x - x_s)^2 + (y - y_s)^2}. \quad (213)$$

6.5.3 my_dipole_potential_surface

A função `my_dipole_potential_surface` calcula o potencial produzido por um dipolo de corrente superficial.

Sintaxe

```
1      my_dipole_potential_surface(x, y, A, B,
2      current=1.0, resistivity=100.0)
```

Entrada: A é a posição do eletrodo de corrente positiva e B é a posição do eletrodo de corrente negativa.

Saída: Retorna o potencial total $V_A + V_B$.

A formulação é

$$V(x, y) = \frac{\rho I}{2\pi r_A} - \frac{\rho I}{2\pi r_B}. \quad (214)$$

6.5.4 my_electric_field_from_potential

A função `my_electric_field_from_potential` calcula o campo elétrico a partir do potencial.

Sintaxe

```
1 my_electric_field_from_potential(x, y, potential)
```

Entrada: `x` e `y` são coordenadas da malha; `potential` é o potencial elétrico.

Saída: Retorna `Ex`, `Ey` e `|E|`.

A relação física é

$$\mathbf{E} = -\nabla V. \quad (215)$$

Assim,

$$E_x = -\frac{\partial V}{\partial x}, \quad E_y = -\frac{\partial V}{\partial y}, \quad (216)$$

e a amplitude do campo elétrico é

$$|\mathbf{E}| = \sqrt{E_x^2 + E_y^2}. \quad (217)$$

6.6 Seção geoeletrica

6.6.1 my_geoelectric_section

A função `my_geoelectric_section` constrói uma seção geoeletrica bidimensional a partir de interfaces e resistividades.

Sintaxe

```
1 my_geoelectric_section(x, depths, layer_boundaries,
    resistivities)
```

Entrada: `x` é o eixo horizontal; `depths` é o vetor de profundidades; `layer_boundaries` contém as interfaces; `resistivities` contém as resistividades das unidades.

Saída: Retorna `X`, `Z` e `section`.

A seção é construída atribuindo uma resistividade a cada célula da malha de acordo com a profundidade e as interfaces. Para uma interface $z_b(x)$, a camada superior satisfaz

$$z_{top}(x) \leq z < z_b(x). \quad (218)$$

A última resistividade representa o domínio abaixo da última interface.

6.7 Funções de visualização

O módulo contém três funções de visualização: `my_plot_sev`, `my_plot_map` e `my_plot_geoelectric_section`.

Funções de plotagem

```
1 my_plot_sev(AB2, rhoa, xlabel="AB/2 (m)",
2 ylabel="Apparent resistivity (ohm m)",
3 title="Vertical Electrical Sounding", figsize=(7, 5))
4
5 my_plot_map(x, y, data, xlabel="X (m)", ylabel="Y (m)
6 ",
7 colorbar_label="Value", title="Electrical map",
8 colormap="coolwarm", levels=50, figsize=(7, 6))
9
10 my_plot_geoelectric_section(X, Z, section,
11 xlabel="Distance (m)",
12 ylabel="Depth (m)",
13 colorbar_label="Resistivity (ohm m)",
14 title="Geoelectric section",
15 colormap="viridis",
    levels=50, figsize=(8, 4))
```

A função `my_plot_sev` utiliza escala logarítmica nos dois eixos, adequada para curvas de sondagem elétrica vertical. A função `my_plot_map` gera mapas de contorno preenchido. A função `my_plot_geoelectric_section` gera seções bidimensionais com eixo vertical invertido, de modo que a profundidade aumente para baixo.

6.8 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import electrical_methods as em
3
4      # Modelo geoeletrico 1D
5      resistividades = [100.0, 20.0, 300.0]
6      espessuras = [10.0, 30.0]
7
8      modelo = em.my_layered_earth_model(resistividades,
9                                         espessuras)
10
11     # Aberturas AB/2
12     AB2 = np.logspace(0, 3, 50)
13
14     # Curva SEV sintetica
15     rhoa = em.my_sev_forward_smooth_transition(
16         AB2,
17         resistividades,
18         espessuras,
19         sharpness=3.0
20     )
21
22     # Classificacao da curva
23     classe = em.my_classify_sev_curve(resistividades)
24
25     # Plotagem
26     fig, ax = em.my_plot_sev(AB2, rhoa)
```

Nesse exemplo, a sequência de resistividades $[100, 20, 300] \Omega\text{m}$ representa uma curva do tipo H, pois

$$\rho_1 > \rho_2 < \rho_3. \quad (219)$$

6.9 Síntese técnica do módulo

O módulo **electrical_methods.py** fornece uma base computacional introdutória para métodos elétricos em Geofísica. Ele permite trabalhar com resistividade, condutividade, fatores geométricos, curvas SEV sintéticas, potenciais elétricos, campos elétricos e seções geoeletricas. Seu caráter é fortemente didático, especialmente porque a função de modelagem SEV utiliza transições suaves em vez de um solucionador rigoroso por transformada de Hankel.

Importância para ensino de Geofísica

O **electrical_methods.py** é adequado para introduzir conceitos de eletrorresistividade, arranjos de eletrodos, resistividade aparente, curvas SEV e modelos em camadas. Ele permite que estudantes visualizem como variações de resistividade e espessura afetam respostas elétricas sintéticas.

Do ponto de vista científico, o módulo pode servir como base educacional e como protótipo para futuras implementações mais rigorosas de modelagem direta e inversão elétrica, incluindo solucionadores 1D por transformadas integrais, inversão de SEV e modelagem 2D/3D.

7 Descrição técnica do módulo **electromagnetic_methods.py**

O arquivo **electromagnetic_methods.py** reúne funções didáticas e computacionais associadas aos métodos eletromagnéticos em Geofísica. O módulo inclui rotinas para frequência angular, número de onda eletromagnético, impedância intrínseca, profundidade de penetração, atenuação de ondas planas, velocidade de fase, campo de dipolo magnético, campo de bobina, aplicações introdutórias de GPR, resposta de indução eletromagnética, difusão eletromagnética unidimensional e visualização de mapas, seções e curvas.

Os métodos eletromagnéticos investigam a subsuperfície a partir da interação entre campos elétricos, campos magnéticos e propriedades físicas do meio, especialmente condutividade elétrica, permissividade dielétrica e permeabilidade magnética. Em aplicações geofísicas, esses métodos são usados em estudos ambientais, hidrogeológicos, arqueológicos, geotécnicos, minerais, agrícolas e de caracterização rasa do subsolo.

Papel do módulo `electromagnetic_methods.py`

O módulo `electromagnetic_methods.py` fornece funções introdutórias para simular e visualizar fenômenos eletromagnéticos aplicados à Geofísica. Ele pode ser utilizado em aulas práticas para discutir propagação, atenuação, profundidade de investigação, resposta de bobinas, GPR e difusão eletromagnética.

O módulo utiliza as bibliotecas `NumPy`, `Matplotlib` e `os`. A biblioteca `NumPy` é usada para operações vetorizadas e cálculos físicos, enquanto `Matplotlib` é utilizada para geração de mapas, seções e curvas. A biblioteca `os` é usada para criar diretórios automaticamente no momento de salvar figuras.

7.1 Funções auxiliares internas

7.1.1 `_arr`

A função interna `_arr` converte uma entrada escalar, lista ou array para um array `NumPy` do tipo `float`.

Sintaxe

```
1  _arr(a)
```

Entrada: `a` é a entrada a ser convertida.

Saída: Retorna `a` como array `NumPy`.

Essa função é usada para garantir que os cálculos eletromagnéticos possam ser realizados de forma vetorizada.

7.1.2 my_save_figure

A função `my_save_figure` salva figuras geradas com Matplotlib.

Sintaxe

```
1 my_save_figure(filename, fig=None, dpi=300,  
2 bbox_inches="tight", transparent=True)
```

Entrada: `filename` é o caminho do arquivo de saída; `fig` é a figura a ser salva; `dpi` define a resolução; `bbox_inches` controla o recorte; `transparent` define se o fundo será transparente.

Saída: Retorna o nome do arquivo salvo.

Caso o diretório indicado não exista, ele é criado automaticamente.

7.2 Frequência angular, número de onda e impedância

7.2.1 my_angular_frequency

A função `my_angular_frequency` calcula a frequência angular a partir da frequência linear.

Sintaxe

```
1 my_angular_frequency(frequency)
```

Entrada: `frequency` é a frequência f , em Hz.

Saída: Retorna a frequência angular ω , em rad/s.

A relação matemática é

$$\omega = 2\pi f. \quad (220)$$

7.2.2 my_wave_number

A função `my_wave_number` calcula o número de onda eletromagnético complexo em um meio condutivo.

Sintaxe

```
1 my_wave_number(frequency, mu=4*np.pi*1e-7,  
2 epsilon=8.854187817e-12,  
3 conductivity=0.0)
```

Entrada: `frequency` é a frequência em Hz; `mu` é a permeabilidade magnética; `epsilon` é a permissividade elétrica; `conductivity` é a condutividade elétrica.

Saída: Retorna o número de onda complexo k .

A formulação implementada pode ser escrita como

$$k = \omega \sqrt{\mu \left(\varepsilon - i \frac{\sigma}{\omega} \right)}, \quad (221)$$

em que ω é a frequência angular, μ é a permeabilidade magnética, ε é a permissividade elétrica e σ é a condutividade elétrica.

Número de onda complexo

Em meios condutivos, o número de onda é complexo. Sua parte real está associada à propagação da onda, enquanto sua parte imaginária está associada à atenuação. Essa característica é central em métodos eletromagnéticos, pois controla a profundidade de investigação.

7.2.3 my_intrinsic_impedance

A função `my_intrinsic_impedance` calcula a impedância intrínseca do meio.

Sintaxe

```

1      my_intrinsic_impedance(frequency, mu=4*np.pi*1e-7,
2      epsilon=8.854187817e-12,
3      conductivity=0.0)

```

Entrada: Recebe frequência, permeabilidade, permissividade e condutividade elétrica do meio.

Saída: Retorna a impedância intrínseca complexa η .

A formulação é

$$\eta = \sqrt{\frac{i\omega\mu}{\sigma + i\omega\epsilon}}. \quad (222)$$

A impedância relaciona os campos elétrico e magnético de uma onda plana:

$$\eta = \frac{E}{H}. \quad (223)$$

7.3 Profundidade de penetração, atenuação e velocidade de fase

7.3.1 `my_skin_depth`

A função `my_skin_depth` calcula a profundidade de penetração, também conhecida como *skin depth*.

Sintaxe

```

1      my_skin_depth(frequency, conductivity,
2      mu=4*np.pi*1e-7)

```

Entrada: `frequency` é a frequência em Hz; `conductivity` é a condutividade elétrica em S/m; `mu` é a permeabilidade magnética.

Saída: Retorna a profundidade de penetração δ , em metros.

A formulação é

$$\delta = \sqrt{\frac{2}{\omega\mu\sigma}}. \quad (224)$$

Essa grandeza indica a profundidade na qual a amplitude do campo decai aproximadamente para $1/e$ de seu valor inicial.

7.3.2 `my_plane_wave_attenuation`

A função `my_plane_wave_attenuation` calcula a atenuação exponencial de uma onda plana com a profundidade.

Sintaxe

```
1      my_plane_wave_attenuation(depth, frequency,
2                                conductivity,
3                                mu=4*np.pi*1e-7,
                                amplitude0=1.0)
```

Entrada: `depth` é a profundidade; `frequency` é a frequência; `conductivity` é a condutividade; `amplitude0` é a amplitude inicial.

Saída: Retorna a amplitude atenuada.

A amplitude é calculada por

$$A(z) = A_0 \exp\left(-\frac{z}{\delta}\right), \quad (225)$$

em que δ é a profundidade de penetração.

7.3.3 `my_phase_velocity`

A função `my_phase_velocity` calcula a velocidade de fase de uma onda eletromagnética.

Sintaxe

```
1      my_phase_velocity(frequency, conductivity,
2                        mu=4*np.pi*1e-7,
3                        epsilon=8.854187817e-12)
```

Entrada: Recebe frequência, condutividade, permeabilidade e permissividade.

Saída: Retorna a velocidade de fase v_p .

A função calcula o número de onda complexo e utiliza sua parte real:

$$v_p = \frac{\omega}{\Re(k)}. \quad (226)$$

7.4 Campo de dipolo magnético e bobinas

7.4.1 `my_magnetic_dipole_field`

A função `my_magnetic_dipole_field` calcula o campo magnético produzido por um dipolo magnético.

Sintaxe

```

1      my_magnetic_dipole_field(x, y, z, dipole_position
      =(0,0,0),
2      moment=(0,0,1), mu=4*np.pi*1e-7,
3      eps=1e-12)
```

Entrada: `x`, `y` e `z` são coordenadas de observação; `dipole_position` é a posição do dipolo; `moment` é o momento magnético vetorial.

Saída: Retorna as componentes B_x , B_y , B_z e o módulo $|\mathbf{B}|$.

A formulação do dipolo magnético é

$$\mathbf{B} = \frac{\mu}{4\pi} \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right], \quad (227)$$

em que $\mathbf{m}$ é o momento magnético e $\mathbf{r}$ é o vetor que liga o dipolo ao ponto de observação.

7.4.2 `my_loop_axis_field`

A função `my_loop_axis_field` calcula o campo magnético ao longo do eixo de uma espira circular.

Sintaxe

```
1         my_loop_axis_field(z, radius, current, turns=1,  
2         mu=4*np.pi*1e-7)
```

Entrada: *z* é a coordenada ao longo do eixo; *radius* é o raio da espira; *current* é a corrente; *turns* é o número de voltas.

Saída: Retorna o campo B_z ao longo do eixo.

A formulação é

$$B_z(z) = \frac{\mu N I R^2}{2(R^2 + z^2)^{3/2}}, \quad (228)$$

em que N é o número de voltas, I é a corrente e R é o raio da espira.

7.4.3 `my_coil_moment`

A função `my_coil_moment` calcula o momento magnético de uma bobina circular.

Sintaxe

```
1         my_coil_moment(radius, current, turns=1)
```

Entrada: *radius* é o raio da bobina; *current* é a corrente; *turns* é o número de voltas.

Saída: Retorna o momento magnético m .

A formulação é

$$m = N I A = N I \pi R^2. \quad (229)$$

7.4.4 `my_coil_dipole_field`

A função `my_coil_dipole_field` aproxima uma bobina por um dipolo magnético.

Sintaxe

```

1      my_coil_dipole_field(x, y, z, coil_position=(0,0,0),
2      radius=1.0, current=1.0, turns=1,
3      orientation=(0,0,1),
4      mu=4*np.pi*1e-7)

```

Entrada: Recebe a geometria da bobina, posição, corrente, número de voltas e orientação.

Saída: Retorna as componentes e a amplitude do campo magnético.

A função calcula o momento magnético da bobina e usa a formulação de dipolo:

$$\mathbf{m} = NI\pi R^2 \hat{\mathbf{n}}, \quad (230)$$

em que $\hat{\mathbf{n}}$ é o vetor unitário de orientação da bobina.

7.5 Funções associadas ao GPR

O módulo inclui funções simples para estimativas associadas ao radar de penetração no solo, ou *Ground Penetrating Radar*.

7.5.1 `my_gpr_velocity`

A função `my_gpr_velocity` calcula a velocidade aproximada de propagação em um meio com permissividade relativa ε_r .

Sintaxe

```

1      my_gpr_velocity(relative_permittivity)

```

Saída: Retorna a velocidade v , em m/s.

A formulação é

$$v = \frac{c}{\sqrt{\varepsilon_r}}, \quad (231)$$

em que c é a velocidade da luz no vácuo.

7.5.2 my_gpr_two_way_time

A função `my_gpr_two_way_time` calcula o tempo duplo de percurso de uma reflexão.

Sintaxe

```
1 my_gpr_two_way_time(depth, relative_permittivity)
```

Saída: Retorna o tempo duplo t , em segundos.

A formulação é

$$t = \frac{2z}{v}. \quad (232)$$

7.5.3 my_gpr_depth_from_time

A função `my_gpr_depth_from_time` estima a profundidade a partir do tempo duplo.

Sintaxe

```
1 my_gpr_depth_from_time(two_way_time,
    relative_permittivity)
```

Saída: Retorna a profundidade estimada.

A formulação inversa é

$$z = \frac{vt}{2}. \quad (233)$$

7.5.4 my_gpr_vertical_resolution

A função `my_gpr_vertical_resolution` estima a resolução vertical aproximada.

Sintaxe

```
1 my_gpr_vertical_resolution(frequency ,  
    relative_permittivity)
```

Saída: Retorna a resolução vertical aproximada.

A resolução é estimada como um quarto do comprimento de onda:

$$R_v = \frac{\lambda}{4} = \frac{v}{4f}. \quad (234)$$

7.6 Resposta de indução eletromagnética e imagem sintética

7.6.1 my_em_induction_response

A função `my_em_induction_response` calcula uma resposta simplificada de indução eletromagnética.

Sintaxe

```
1 my_em_induction_response(distance , frequency ,  
2 conductivity , moment=1.0 ,  
3 mu=4*np.pi*1e-7)
```

Entrada: `distance` é a distância fonte-receptor; `frequency` é a frequência; `conductivity` é a condutividade; `moment` é o momento da fonte.

Saída: Retorna uma resposta eletromagnética simplificada.

A função combina decaimento geométrico e atenuação exponencial:

$$R(d) = \frac{m}{d^3} \exp\left(-\frac{d}{\delta}\right), \quad (235)$$

em que d é a distância e δ é a profundidade de penetração.

7.6.2 my_em_image_from_anomalies

A função `my_em_image_from_anomalies` cria uma imagem eletromagnética sintética a partir de anomalias Gaussianas.

Sintaxe

```
1 my_em_image_from_anomalies(x, y, anomalies)
```

Entrada: *x* e *y* são coordenadas da malha; *anomalies* é uma lista de anomalias definidas por posição, amplitude e largura.

Saída: Retorna uma matriz com a imagem sintética.

Cada anomalia pode ser representada por

$$A(x, y) = A_0 \exp \left[-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2} \right]. \quad (236)$$

A imagem final é a soma das contribuições:

$$D(x, y) = \sum_{j=1}^N A_j(x, y). \quad (237)$$

7.7 Difusão eletromagnética unidimensional

7.7.1 my_1d_em_diffusion_step

A função *my_1d_em_diffusion_step* executa uma etapa explícita de difusão eletromagnética 1D.

Sintaxe

```
1 my_1d_em_diffusion_step(field, conductivity, dx, dt,
2 mu=4*np.pi*1e-7)
```

Entrada: *field* é o campo no instante atual; *conductivity* é a condutividade; *dx* é o espaçamento; *dt* é o passo de tempo.

Saída: Retorna o campo atualizado.

A equação de difusão eletromagnética simplificada pode ser escrita como

$$\frac{\partial B}{\partial t} = \eta \frac{\partial^2 B}{\partial x^2}, \quad (238)$$

em que a difusividade magnética é

$$\eta = \frac{1}{\mu\sigma}. \quad (239)$$

A discretização explícita é

$$B_i^{n+1} = B_i^n + \frac{\eta\Delta t}{\Delta x^2} (B_{i+1}^n - 2B_i^n + B_{i-1}^n). \quad (240)$$

7.7.2 my_1d_em_diffusion

A função `my_1d_em_diffusion` executa várias etapas de difusão eletromagnética 1D.

Sintaxe

```
1 my_1d_em_diffusion(field0, conductivity, dx, dt, nt,
2 mu=4*np.pi*1e-7)
```

Entrada: `field0` é o campo inicial; `nt` é o número de passos temporais.

Saída: Retorna uma matriz com o histórico temporal do campo.

O resultado pode ser representado como

$$\mathbf{B} = [B(x, t_0), B(x, t_1), \dots, B(x, t_{nt})]. \quad (241)$$

7.8 Funções de visualização

O módulo contém funções para visualização de mapas, seções e curvas eletromagnéticas.

Funções de plotagem

```
1 my_plot_map(x, y, data, xlabel="X (m)", ylabel="Y (m)
2 ",
3 colorbar_label="Value", title="EM map",
4 colormap="viridis", levels=50, figsize=(7, 6))
5 my_plot_em_section(x, z, data, xlabel="Distance (m)",
```

```
6         ylabel="Depth (m)",
7         colorbar_label="Amplitude",
8         title="EM section", colormap="viridis",
9         levels=50, figsize=(8, 4))
10
11     my_plot_curve(x, y, xlabel="X", ylabel="Y",
12                 title="EM curve", figsize=(7, 5))
```

Essas funções retornam os objetos `fig` e `ax`, permitindo personalização posterior pelo usuário.

7.9 Síntese técnica do módulo

O módulo `electromagnetic_methods.py` fornece uma base introdutória para simulações eletromagnéticas em Geofísica. Ele integra conceitos de propagação de ondas, atenuação, profundidade de penetração, campo de dipolos, bobinas, GPR, indução eletromagnética e difusão.

Importância para ensino de Geofísica

O `electromagnetic_methods.py` permite demonstrar, em Python, conceitos fundamentais de métodos eletromagnéticos, como skin depth, atenuação, resposta de bobinas, tempo duplo de GPR e difusão de campos. É adequado para aulas práticas e para construção de exemplos sintéticos.

Do ponto de vista científico, o módulo possui caráter introdutório e educacional. Ele pode servir como base para futuras implementações mais avançadas, incluindo modelagem eletromagnética 1D, 2D e 3D, inversão de dados EM e integração com dados elétricos e magnéticos.

8 Descrição técnica do módulo `equivalentlayer.py`

O arquivo `equivalentlayer.py` reúne funções para construção, ajuste, predição e transformação de campos potenciais por meio do método da camada equivalente. Esse método é amplamente utilizado em Gravimetria e Magnetometria para interpolação, regularização,

transformação de campos, redução ao polo, redução ao equador, continuação e predição de dados em diferentes níveis de observação.

A ideia central do método consiste em representar o campo observado por uma distribuição de fontes fictícias localizadas em uma camada abaixo dos pontos de observação. Essas fontes não precisam corresponder diretamente à geologia real, mas são ajustadas para reproduzir os dados observados. Uma vez ajustada a camada, ela pode ser usada para prever o campo em outros pontos ou transformar o dado para outras direções de magnetização e campo regional.

Papel do módulo `equivalentlayer.py`

O módulo `equivalentlayer.py` implementa uma abordagem linear de camada equivalente para dados gravimétricos e magnéticos. Ele permite construir uma camada de fontes, ajustar os parâmetros por mínimos quadrados regularizados, prever campos e realizar transformações magnéticas, como redução ao polo e redução ao equador.

Do ponto de vista matemático, o método pode ser escrito como

$$\mathbf{d} = \mathbf{G}\mathbf{p} + \mathbf{e}, \quad (242)$$

em que $\mathbf{d}$ é o vetor de dados observados, $\mathbf{G}$ é a matriz de sensibilidade, $\mathbf{p}$ é o vetor de parâmetros da camada equivalente e $\mathbf{e}$ representa os resíduos. A solução regularizada por Tikhonov minimiza

$$\Phi(\mathbf{p}) = \|\mathbf{G}\mathbf{p} - \mathbf{d}\|_2^2 + \lambda\|\mathbf{p}\|_2^2. \quad (243)$$

A solução é dada por

$$\mathbf{p} = (\mathbf{G}^T \mathbf{G} + \lambda \mathbf{I})^{-1} \mathbf{G}^T \mathbf{d}. \quad (244)$$

8.1 Constantes internas e funções auxiliares

O módulo contém funções internas para recuperar constantes físicas, converter entradas para arrays unidimensionais, verificar dados observados e resolver sistemas regularizados.

8.1.1 `_gravity_constant`, `_si2mgal`, `_cm_constant` e `_t2nt`

As funções `_gravity_constant`, `_si2mgal`, `_cm_constant` e `_t2nt` retornam constantes utilizadas na modelagem gravimétrica e magnética.

Sintaxe

```
1      _gravity_constant()
2      _si2mgal()
3      _cm_constant()
4      _t2nt()
```

Saída: Retornam, respectivamente, a constante gravitacional G , o fator de conversão SI para mGal, a constante magnética C_M e o fator de conversão de Tesla para nanoTesla.

Essas funções tornam o módulo mais robusto, pois tentam recuperar os valores do módulo `constants.py`. Caso não estejam disponíveis, usam valores padrão.

8.1.2 `_as_array_1d`

A função `_as_array_1d` converte uma entrada para array unidimensional.

Sintaxe

```
1      _as_array_1d(data, name)
```

Entrada: `data` é a entrada a ser convertida; `name` é o nome usado em mensagens de erro.

Saída: Retorna um vetor NumPy unidimensional.

Essa função é importante porque o método da camada equivalente organiza os dados e os parâmetros na forma vetorial.

8.1.3 `_check_observations`

A função `_check_observations` verifica a consistência das coordenadas de observação.

Sintaxe

```
1      _check_observations(xo, yo, zo)
```

Entrada: *xo*, *yo* e *zo* são coordenadas dos pontos de observação.

Saída: Retorna os três arrays como vetores unidimensionais.

A função exige que todos tenham o mesmo tamanho:

$$N_x = N_y = N_z. \quad (245)$$

8.1.4 `_check_dataset`

A função `_check_dataset` verifica a estrutura de um conjunto de dados.

Sintaxe

```
1      _check_dataset(dataset)
```

Entrada: *dataset* deve conter quatro elementos: x_o , y_o , z_o e d_o .

Saída: Retorna as coordenadas e os dados observados como vetores unidimensionais.

A estrutura esperada é

$$\text{dataset} = (x_o, y_o, z_o, d_o). \quad (246)$$

8.1.5 `_solve_tikhonov` e `my_tikhonov_solve`

A função interna `_solve_tikhonov` e sua versão pública `my_tikhonov_solve` resolvem um sistema linear regularizado.

Sintaxe

```
1      my_tikhonov_solve(mat, data, regulator)
```

Entrada: `mat` é a matriz de sensibilidade; `data` é o vetor de dados; `regulator` é o parâmetro de regularização.

Saída: Retorna o vetor de parâmetros estimado.

A solução implementada é

$$\mathbf{p} = (\mathbf{G}^T \mathbf{G} + \lambda \alpha \mathbf{I})^{-1} \mathbf{G}^T \mathbf{d}, \quad (247)$$

em que λ é o parâmetro `regulator`. O fator α é uma normalização baseada no traço da matriz $\mathbf{G}^T \mathbf{G}$, usada para tornar a escala da regularização mais compatível com o problema numérico.

Regularização de Tikhonov

A regularização de Tikhonov reduz instabilidades em problemas mal condicionados. Em camada equivalente, ela evita soluções excessivamente oscilatórias ou amplitudes irreais nos parâmetros da camada.

8.2 Construção da camada equivalente

8.2.1 `my_layer`

A função `my_layer` constrói uma camada equivalente regular em uma área retangular.

Sintaxe

```
1 my_layer(area, shape, level)
```

Entrada: `area` define os limites $[x_1, x_2, y_1, y_2]$; `shape` define o número de pontos n_y, n_x ; `level` define a profundidade ou cota da camada.

Saída: Retorna uma lista com as coordenadas x , y e z das fontes equivalentes.

A camada é formada por fontes posicionadas em uma grade regular:

$$x_i \in [x_1, x_2], \quad y_j \in [y_1, y_2], \quad z = z_0. \quad (248)$$

8.2.2 my_layer_array

A função `my_layer_array` é uma versão equivalente que retorna as coordenadas da camada como arrays.

Sintaxe

```
1 my_layer_array(area, shape, level)
```

Entrada: Recebe os mesmos argumentos de `my_layer`.

Saída: Retorna arrays NumPy com as coordenadas da camada.

8.3 Matrizes de sensibilidade gravimétrica

8.3.1 my_gz_layer

A função `my_gz_layer` constrói a matriz de sensibilidade para a componente vertical da gravidade produzida por fontes pontuais equivalentes.

Sintaxe

```
1 my_gz_layer(xo, yo, zo, layer)
```

Entrada: `xo`, `yo` e `zo` são os pontos de observação; `layer` contém as coordenadas das fontes equivalentes.

Saída: Retorna a matriz $\mathbf{G}_{gz}$.

Para uma fonte j e um ponto de observação i , define-se

$$r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}. \quad (249)$$

A sensibilidade da componente vertical é proporcional a

$$G_{ij}^{gz} = G \frac{z_j - z_i}{r_{ij}^3} 10^5. \quad (250)$$

O fator 10^5 converte a resposta para mGal.

8.3.2 my_gxyz_layer

A função `my_gxyz_layer` constrói matrizes de sensibilidade para as três componentes da gravidade.

Sintaxe

```
1 my_gxyz_layer(xo, yo, zo, layer)
```

Saída: Retorna as matrizes $\mathbf{G}_{gx}$, $\mathbf{G}_{gy}$ e $\mathbf{G}_{gz}$.

As componentes são dadas por

$$G_{ij}^{gx} = G \frac{x_j - x_i}{r_{ij}^3} 10^5, \quad (251)$$

$$G_{ij}^{gy} = G \frac{y_j - y_i}{r_{ij}^3} 10^5, \quad (252)$$

$$G_{ij}^{gz} = G \frac{z_j - z_i}{r_{ij}^3} 10^5. \quad (253)$$

8.4 Ajuste e predição gravimétrica

8.4.1 my_predict_grav

A função `my_predict_grav` prediz dados gravimétricos a partir de uma camada equivalente ajustada.

Sintaxe

```
1 my_predict_grav(xo, yo, zo, layermodel, pvec)
```

Entrada: `layermodel` é a camada equivalente; `pvec` é o vetor de parâmetros ajustados.

Saída: Retorna o campo gravimétrico predito.

A predição é calculada por

$$\mathbf{d}_{pred} = \mathbf{G}_{gz}\mathbf{p}. \quad (254)$$

8.4.2 `my_fitdata_grav` e `my_fitdata_grav_full`

As funções `my_fitdata_grav` e `my_fitdata_grav_full` ajustam uma camada equivalente a dados gravimétricos.

Sintaxe

```

1      my_fitdata_grav(dataset, datashape, layermodel,
2      layershape, regulator)
3
4      my_fitdata_grav_full(dataset, datashape, layermodel,
5      layershape, regulator)
```

Entrada: `dataset` contém x_o, y_o, z_o, d_o ; `datashape` é a forma dos dados; `layermodel` é a camada; `layershape` é a forma da camada; `regulator` é a regularização.

Saída: `my_fitdata_grav` retorna o vetor de parâmetros. A versão `my_fitdata_grav_full` retorna um dicionário com parâmetros, dados preditos, resíduos, matriz e informações adicionais.

A solução é obtida por mínimos quadrados regularizados:

$$\mathbf{p} = \left(\mathbf{G}^T \mathbf{G} + \lambda \mathbf{I} \right)^{-1} \mathbf{G}^T \mathbf{d}. \quad (255)$$

8.5 Camada equivalente magnética

8.5.1 my_totalfield_layer

A função `my_totalfield_layer` constrói a matriz de sensibilidade para anomalia magnética de campo total.

Sintaxe

```
1 my_totalfield_layer(xo, yo, zo, layer,
2 inc, dec, incs=None, decs=None)
```

Entrada: `inc` e `dec` definem a direção do campo regional; `incs` e `decs` definem a direção de magnetização das fontes.

Saída: Retorna a matriz de sensibilidade magnética.

O campo magnético anômalo de uma fonte pontual dipolar pode ser escrito como

$$\mathbf{B} = C_M \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right]. \quad (256)$$

A anomalia de campo total é a projeção na direção do campo regional:

$$\Delta T = \mathbf{B} \cdot \hat{\mathbf{F}}. \quad (257)$$

8.5.2 my_predict_mag

A função `my_predict_mag` prediz dados magnéticos a partir de uma camada equivalente ajustada.

Sintaxe

```
1 my_predict_mag(xo, yo, zo, layermodel, pvec,
2 inc, dec, incs=None, decs=None)
```

Saída: Retorna o campo magnético predito em nT.

A predição é

$$\mathbf{d}_{pred} = \mathbf{G}_{mag}\mathbf{p}. \quad (258)$$

8.5.3 `my_fitdata_mag` e `my_fitdata_mag_full`

As funções `my_fitdata_mag` e `my_fitdata_mag_full` ajustam uma camada equivalente a dados magnéticos.

Sintaxe

```
1      my_fitdata_mag(dataset, datashape, layermodel,
2                      layershape, regulator,
3                      inc, dec, incl=None, decl=None)
4
5      my_fitdata_mag_full(dataset, datashape, layermodel,
6                          layershape, regulator,
7                          inc, dec, incl=None, decl=None)
```

Entrada: Recebem o conjunto de dados magnéticos, a camada equivalente, o parâmetro de regularização e as direções do campo e da magnetização.

Saída: A primeira função retorna o vetor de parâmetros. A versão completa retorna também dados preditos, resíduos, matriz de sensibilidade e informações do ajuste.

8.6 Transformações magnéticas

8.6.1 `my_transform_layer`

A função `my_transform_layer` ajusta uma camada equivalente a um conjunto de dados magnéticos e prediz o campo em uma nova direção de campo regional e magnetização.

Sintaxe

```
1      my_transform_layer(datasets, datashape, layermodel,
2                          layershape,
3                          regulator, inc, dec, newinc, newdec,
4                          incl=None, decl=None,
5                          newincl=None, newdecl=None)
```

Entrada: `inc`, `dec`, `incl` e `decl` representam a geometria original. `newinc`, `newdec`, `newincl` e `newdecl` representam a nova geometria desejada.

Saída: Retorna o campo transformado.

A operação pode ser entendida em duas etapas:

$$\mathbf{p} = \mathbf{G}_{orig}^+ \mathbf{d}_{obs}, \quad (259)$$

$$\mathbf{d}_{new} = \mathbf{G}_{new} \mathbf{p}. \quad (260)$$

8.6.2 `my_rtp_layer`

A função `my_rtp_layer` realiza redução ao polo usando camada equivalente.

Sintaxe

```
1         my_rtp_layer(datasets, datashape, layermodel,
2                     layershape,
                     regulator, inc, dec, incl=None, decl=None)
```

Saída: Retorna o campo reduzido ao polo.

Na redução ao polo, a nova direção do campo é assumida como

$$I = 90^\circ, \quad D = 0^\circ. \quad (261)$$

Essa transformação busca reposicionar anomalias magnéticas sobre suas fontes, reduzindo assimetrias associadas à inclinação e declinação do campo geomagnético.

8.6.3 `my_rte_layer`

A função `my_rte_layer` realiza redução ao equador usando camada equivalente.

Sintaxe

```

1         my_rte_layer(datasets, datashape, layermodel,
2         layershape,
           regulator, incf, decf, inceql=None, deceql=None)

```

Saída: Retorna o campo reduzido ao equador.

Na redução ao equador, a nova inclinação do campo é próxima de

$$I = 0^\circ. \quad (262)$$

Essa transformação é útil em regiões de baixa latitude magnética, onde a redução ao polo pode se tornar numericamente instável.

8.7 Resíduos e métricas de ajuste

8.7.1 `my_residual`

A função `my_residual` calcula os resíduos entre dados observados e preditos.

Sintaxe

```

1         my_residual(observed, predicted)

```

Saída: Retorna o vetor de resíduos.

A formulação é

$$\mathbf{r} = \mathbf{d}_{obs} - \mathbf{d}_{pred}. \quad (263)$$

8.7.2 `my_misfit`

A função `my_misfit` calcula o erro quadrático médio normalizado pelo número de dados.

Sintaxe

```

1         my_misfit(observed, predicted)

```

Saída: Retorna o misfit médio.

A formulação é

$$\text{misfit} = \frac{1}{N} \sum_{i=1}^N (d_i^{\text{obs}} - d_i^{\text{pred}})^2. \quad (264)$$

8.8 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import equivalentlayer as eql
3
4      # Dados observados
5      x = np.linspace(0, 10000, 50)
6      y = np.linspace(0, 10000, 50)
7      X, Y = np.meshgrid(x, y)
8      Z = np.zeros_like(X)
9      data = np.random.normal(0, 1, X.size)
10
11     dataset = (X.ravel(), Y.ravel(), Z.ravel(), data)
12
13     # Camada equivalente
14     area = [0, 10000, 0, 10000]
15     shape = (50, 50)
16     layer = eql.my_layer(area, shape, level=1000.0)
17
18     # Ajuste gravimétrico
19     p = eql.my_fitdata_grav(
20         dataset=dataset,
21         datashape=shape,
22         layermodel=layer,
23         layershape=shape,
24         regulator=0.01
25     )
26
27     # Predicao
28     pred = eql.my_predict_grav(
```

```
29         X.ravel(), Y.ravel(), Z.ravel(),  
30         layer,  
31         p  
32     )
```

8.9 Síntese técnica do módulo

O módulo **equivalentlayer.py** implementa uma metodologia importante para processamento e transformação de campos potenciais. A camada equivalente permite ajustar dados observados por meio de fontes fictícias e, posteriormente, predizer ou transformar o campo. Essa abordagem é útil porque estabiliza operações que, no domínio da frequência, podem ser sensíveis a ruído, bordas ou irregularidade espacial dos dados.

Importância para ensino de Geofísica

O **equivalentlayer.py** é muito útil no ensino de problemas inversos lineares, regularização e transformação de campos potenciais. Ele permite mostrar como uma matriz de sensibilidade, um vetor de dados e um parâmetro de regularização podem ser usados para reconstruir e transformar campos geofísicos.

Do ponto de vista científico, o módulo pode ser aplicado à interpolação, predição, redução ao polo, redução ao equador e regularização de dados gravimétricos e magnéticos. Também pode servir como base para discussões sobre não unicidade, instabilidade, condicionamento de matrizes e escolha do parâmetro de regularização.

9 Descrição técnica do módulo **filtering.py**

O arquivo **filtering.py** reúne funções para filtragem, transformação e realce de campos potenciais no domínio da frequência. O módulo inclui rotinas de continuação para cima, continuação para baixo regularizada, filtros Gaussianos passa-baixa, passa-alta e passa-banda, derivada direcional, redução ao polo, redução ao equador, pseudogravidade, tilt derivative, tilt aplicado à derivada vertical, gradiente horizontal total, tilt hiperbólico, theta map, cosseno do tilt e estimativas aproximadas de profundidade baseadas em atributos tilt.

Esse módulo é um dos componentes centrais do pacote *gravmag_codes* para processamento de dados gravimétricos e magnéticos, pois permite transformar um campo observado em produtos interpretativos capazes de realçar bordas, suavizar ruído, separar componentes regionais e residuais, corrigir efeitos direcionais do campo magnético e estimar profundidades aproximadas de fontes.

Papel do módulo *filtering.py*

O módulo *filtering.py* implementa filtros e transformações espectrais aplicados a campos potenciais em malhas regulares. Ele utiliza a transformada de Fourier para operar diretamente sobre os números de onda k_x , k_y e $k = \sqrt{k_x^2 + k_y^2}$.

As entradas principais das funções são matrizes bidimensionais **X**, **Y** e **D**, representando as coordenadas da malha e o campo geofísico. O argumento *flatten* controla se a saída será retornada como vetor unidimensional ou como matriz bidimensional. Várias funções também possuem os argumentos *pad_factor*, *pad_mode* e *taper*, usados para reduzir efeitos de borda durante a aplicação da FFT.

9.1 Base matemática da filtragem espectral

Seja $f(x, y)$ um campo potencial observado em uma malha regular. Sua transformada de Fourier bidimensional é

$$F(k_x, k_y) = \mathcal{F}\{f(x, y)\}. \quad (265)$$

Um filtro espectral genérico pode ser aplicado pela multiplicação

$$F_{filtrado}(k_x, k_y) = F(k_x, k_y)H(k_x, k_y), \quad (266)$$

em que $H(k_x, k_y)$ é o operador espectral. O campo filtrado no domínio espacial é obtido por

$$f_{filtrado}(x, y) = \mathcal{F}^{-1}\{F(k_x, k_y)H(k_x, k_y)\}. \quad (267)$$

O módulo utiliza o módulo do número de onda,

$$k = \sqrt{k_x^2 + k_y^2}, \quad (268)$$

em operadores de continuação, derivadas verticais, filtros Gaussianos e atributos de realce.

9.2 Funções internas de validação, número de onda, padding e FFT

O módulo contém funções internas para verificar a grade de entrada, controlar o formato de saída, calcular espaçamentos, construir números de onda, aplicar janelas cossenoidais, expandir a grade por padding, recortar o resultado e aplicar operadores espectrais.

9.2.1 `_check_grid`

A função `_check_grid` verifica se as entradas `x`, `y` e `data` são matrizes bidimensionais com a mesma forma.

Sintaxe

```
1 _check_grid(x, y, data)
```

Entrada: `x` e `y` são coordenadas da malha; `data` é o campo geofísico.

Saída: Retorna as entradas convertidas para arrays NumPy. Caso as formas sejam incompatíveis, gera erro.

Essa validação garante que cada valor d_{ij} esteja associado a um par de coordenadas x_{ij}, y_{ij} .

9.2.2 `_as_grid`

A função `_as_grid` controla o formato da saída.

Sintaxe

```
1 _as_grid(result, shape, flatten=True)
```

Entrada: `result` é o resultado calculado; `shape` é a forma original da grade; `flatten` controla se o resultado será vetorizado.

Saída: Retorna o resultado como vetor unidimensional ou como matriz bidimensional.

9.2.3 `_grid_spacing` e `_wavenumbers`

A função `_grid_spacing` estima os espaçamentos médios da malha, enquanto `_wavenumbers` constrói os números de onda.

Sintaxe

```
1     _grid_spacing(x, y)
2     _wavenumbers(x, y, shape)
```

Os espaçamentos são estimados por

$$\Delta x \approx \text{mean}(x_{i,j+1} - x_{i,j}), \quad (269)$$

$$\Delta y \approx \text{mean}(y_{i+1,j} - y_{i,j}). \quad (270)$$

Os números de onda são calculados a partir das frequências discretas da FFT:

$$k_x = 2\pi f_x, \quad k_y = 2\pi f_y. \quad (271)$$

9.2.4 `_cosine_taper_1d`, `_cosine_taper_2d`, `_pad_grid` e `_crop_grid`

As funções de taper e padding reduzem artefatos de borda associados à periodicidade implícita da transformada discreta de Fourier.

Sintaxe

```
1     _cosine_taper_1d(n, p=0.10)
2     _cosine_taper_2d(data, p=0.10)
3     _pad_grid(data, pad_factor=0.0, pad_mode="reflect",
4               taper=False)
5     _crop_grid(data_pad, pad_width)
```


A janela bidimensional é construída pelo produto externo de janelas unidimensionais:

$$\mathbf{W} = \mathbf{w}_y \mathbf{w}_x^T. \quad (272)$$

A grade suavizada é

$$\mathbf{D}_{tapered} = \mathbf{D} \circ \mathbf{W}. \quad (273)$$

Padding e taper

O padding aumenta artificialmente a grade antes da FFT, enquanto o taper suaviza as bordas. Ambos ajudam a reduzir oscilações e artefatos introduzidos pela periodicidade da transformada discreta de Fourier.

9.2.5 `_apply_fft_operator`

A função `_apply_fft_operator` é o núcleo de aplicação dos filtros espectrais.

Sintaxe

```

1      _apply_fft_operator(x, y, data, operator_function,
2      pad_factor=0.0, pad_mode="reflect",
3      taper=False, restore_mean=False)
```

Entrada: `operator_function` é uma função que recebe k_x , k_y e k , retornando o filtro espectral.

Saída: Retorna o campo filtrado no domínio espacial.

A operação geral é

$$D_{out}(x, y) = \mathcal{F}^{-1} \{ \mathcal{F}[D(x, y)] O(k_x, k_y, k) \}. \quad (274)$$

9.3 Continuação de campos potenciais

9.3.1 `my_continuation_grid` e `my_continuation`

As funções `my_continuation_grid` e `my_continuation` realizam continuação vertical de campos potenciais.

Sintaxe

```
1      my_continuation_grid(x, y, data, level,
2      pad_factor=0.0, pad_mode="reflect",
3      taper=False)
4
5      my_continuation(x, y, data, level,
6      pad_factor=0.0, pad_mode="reflect",
7      taper=False, flatten=True)
```

Entrada: `level` representa o nível de continuação. Valores positivos suavizam o campo, enquanto valores negativos intensificam componentes de alta frequência.

Saída: Retorna o campo continuado.

O operador espectral é

$$H(k) = e^{-kh}, \quad (275)$$

em que h é a altura de continuação. Para continuação para cima, $h > 0$. Para continuação para baixo, $h < 0$, o que torna o operador instável para grandes k .

9.3.2 `my_upward_continuation`

A função `my_upward_continuation` realiza continuação para cima.

Sintaxe

```
1      my_upward_continuation(x, y, data, height,
2      pad_factor=0.0, pad_mode="reflect",
3      taper=False, flatten=True)
```

A formulação é

$$F_h(k_x, k_y) = F_0(k_x, k_y)e^{-kh}. \quad (276)$$

A continuação para cima suaviza o campo e reduz a contribuição relativa de fontes rasas.

9.3.3 my_downward_continuation_regularized

A função `my_downward_continuation_regularized` realiza continuação para baixo com regularização espectral.

Sintaxe

```

1         my_downward_continuation_regularized(x, y, data,
2             height,
3             k_cut=None,
4             pad_factor=0.0,
5             pad_mode="reflect",
6             taper=False,
7             flatten=True)
```

A continuação para baixo ideal envolve

$$F_{-h}(k_x, k_y) = F_0(k_x, k_y)e^{kh}. \quad (277)$$

Como esse operador amplifica fortemente altas frequências, a função aplica um amortecimento Gaussiano:

$$H(k) = e^{kh} \exp \left[- \left(\frac{k}{k_c} \right)^2 \right], \quad (278)$$

em que k_c é o número de onda de corte.

Instabilidade da continuação para baixo

A continuação para baixo é um problema instável porque amplifica ruído e componentes de pequeno comprimento de onda. Por isso, a regularização espectral é essencial para evitar resultados não físicos.

9.4 Filtros Gaussianos

9.4.1 `my_lowpass_gaussian`

A função `my_lowpass_gaussian` aplica filtro Gaussiano passa-baixa.

Sintaxe

```
1 my_lowpass_gaussian(x, y, data, wavelength_cut,
2 pad_factor=0.0, pad_mode="reflect",
3 taper=False, flatten=True)
```

Entrada: `wavelength_cut` é o comprimento de onda de corte.

Saída: Retorna o campo suavizado.

Define-se

$$k_c = \frac{2\pi}{\lambda_c}. \quad (279)$$

O operador é

$$H_{LP}(k) = \exp \left[- \left(\frac{k}{k_c} \right)^2 \right]. \quad (280)$$

9.4.2 `my_highpass_gaussian`

A função `my_highpass_gaussian` aplica filtro Gaussiano passa-alta.

Sintaxe

```
1 my_highpass_gaussian(x, y, data, wavelength_cut,
```

```

2         pad_factor=0.0, pad_mode="reflect",
3         taper=False, flatten=True)

```

O operador é

$$H_{HP}(k) = 1 - \exp \left[- \left(\frac{k}{k_c} \right)^2 \right]. \quad (281)$$

Esse filtro realça componentes de menor comprimento de onda, geralmente associadas a fontes mais rasas.

9.4.3 my_bandpass_gaussian

A função `my_bandpass_gaussian` aplica filtro passa-banda Gaussiano.

Sintaxe

```

1         my_bandpass_gaussian(x, y, data, wavelength_min,
2         wavelength_max, pad_factor=0.0,
3         pad_mode="reflect", taper=False,
4         flatten=True)

```

A função combina um passa-alta e um passa-baixa, retendo uma faixa de comprimentos de onda:

$$H_{BP}(k) = H_{HP}(k_{max})H_{LP}(k_{min}). \quad (282)$$

9.5 Derivada direcional

A função `my_directional_derivative` calcula a derivada horizontal do campo em uma direção definida por azimuth.

Sintaxe

```

1         my_directional_derivative(x, y, data, azimuth_degrees
2         ,

```

```

2         pad_factor=0.0, pad_mode="reflect",
3         taper=False, flatten=True)

```

Entrada: `azimuth_degrees` é o azimute da direção da derivada.

Saída: Retorna a derivada direcional.

A formulação é

$$\frac{\partial f}{\partial s} = \cos \alpha \frac{\partial f}{\partial x} + \sin \alpha \frac{\partial f}{\partial y}. \quad (283)$$

No domínio da frequência, o operador correspondente é

$$H(k_x, k_y) = i(k_x \cos \alpha + k_y \sin \alpha). \quad (284)$$

9.6 Transformações magnéticas: redução ao polo e redução ao equador

9.6.1 `reduction_grid` e `reduction`

As funções `reduction_grid` e `reduction` aplicam uma transformação magnética entre uma direção original de campo/magnetização e uma nova direção desejada.

Sintaxe

```

1     reduction_grid(x, y, data, inc, dec, incs, decs,
2                   newinc, newdec, newincs, newdecs,
3                   pad_factor=0.0, pad_mode="reflect",
4                   taper=False, eps=1.0e-12)
5
6     reduction(x, y, data, inc, dec, incs, decs,
7              newinc, newdec, newincs, newdecs,
8              pad_factor=0.0, pad_mode="reflect",
9              taper=False, flatten=True)

```

A transformação é baseada na razão entre operadores direcionais magnéticos:

$$H(k_x, k_y) = \frac{\Theta_{newF}(k_x, k_y)\Theta_{newM}(k_x, k_y)}{\Theta_{oldF}(k_x, k_y)\Theta_{oldM}(k_x, k_y) + \varepsilon}. \quad (285)$$

9.6.2 my_rtp e my_rte

A função `my_rtp` realiza redução ao polo, enquanto `my_rte` realiza redução ao equador.

Sintaxe

```

1      my_rtp(x, y, data, inc, dec, incs=None, decs=None,
2          pad_factor=0.0, pad_mode="reflect",
3          taper=False, flatten=True)
4
5      my_rte(x, y, data, inc, dec, incs=None, decs=None,
6          pad_factor=0.0, pad_mode="reflect",
7          taper=False, flatten=True)
```

Na redução ao polo, a nova direção é

$$I = 90^\circ, \quad D = 0^\circ. \quad (286)$$

Na redução ao equador, a nova inclinação é aproximadamente

$$I = 0^\circ. \quad (287)$$

Redução magnética

A redução ao polo busca centralizar anomalias magnéticas sobre suas fontes, enquanto a redução ao equador pode ser mais adequada em regiões de baixa latitude magnética, onde a redução ao polo tende a ser instável.

9.7 Pseudogravidade

As funções `my_pseudograv_grid` e `my_pseudograv` transformam dados magnéticos em um campo equivalente de pseudogravidade.

Sintaxe

```
1      my_pseudograv_grid(x, y, data, inc, dec,  
2      incs=None, decs=None,  
3      rho=1.0, mag=1.0,  
4      pad_factor=0.0, pad_mode="reflect",  
5      taper=False, eps=1.0e-12)  
6  
7      my_pseudograv(x, y, data, inc, dec,  
8      incs=None, decs=None,  
9      rho=1.0, mag=1.0,  
10     pad_factor=0.0, pad_mode="reflect",  
11     taper=False, flatten=True)
```

A transformação utiliza operadores direcionais magnéticos e um fator de escala envolvendo densidade ρ e magnetização M . Conceitualmente, busca produzir um campo análogo ao gravimétrico a partir de dados magnéticos, assumindo relação entre magnetização e densidade.

9.8 Atributos de realce: tilt, THDR, theta map e cosseno do tilt

9.8.1 my_tilt_grid e my_tilt

O tilt derivative é definido por

$$\text{TDR} = \arctan \left(\frac{\partial f / \partial z}{\sqrt{(\partial f / \partial x)^2 + (\partial f / \partial y)^2 + \varepsilon}} \right). \quad (288)$$

As funções são:

Sintaxe

```
1      my_tilt_grid(x, y, data, pad_factor=0.0,  
2      pad_mode="reflect", taper=False,  
3      eps=1.0e-12, degrees=False)  
4  
5      my_tilt(x, y, data, pad_factor=0.0,  
6      pad_mode="reflect", taper=False,
```



```
7 degrees=False, flatten=True)
```

9.8.2 `my_tilt_vdr_grid` e `my_tilt_vdr`

Essas funções calculam o tilt da derivada vertical, combinando derivadas de segunda ordem:

$$\text{TDR}_{VDR} = \arctan \left(\frac{\partial^2 f / \partial z^2}{\sqrt{(\partial^2 f / \partial x \partial z)^2 + (\partial^2 f / \partial y \partial z)^2 + \varepsilon}} \right). \quad (289)$$

9.8.3 `my_thdr_grid` e `my_thdr`

O gradiente horizontal total é

$$\text{THDR} = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}. \quad (290)$$

9.8.4 `my_hyperbolic_tilt_grid` e `my_hyperbolic_tilt`

O tilt hiperbólico é uma variação do tilt derivative baseada na função arco-seno hiperbólico:

$$\text{HTDR} = \text{asinh} \left(\frac{\partial f / \partial z}{\text{THDR} + \varepsilon} \right). \quad (291)$$

9.8.5 `my_thetamap_grid`, `my_thetamap`, `my_cos_tilt_grid` e `my_cos_tilt`

O theta map e o cosseno do tilt são dados por

$$\Theta = \frac{\text{THDR}}{\sqrt{\text{THDR}^2 + (\partial f / \partial z)^2 + \varepsilon}}. \quad (292)$$

Esses atributos são úteis para realçar limites laterais de fontes e reduzir a influência da amplitude absoluta do campo.

9.9 Estimativas aproximadas de profundidade pelo tilt

O módulo contém duas funções simples para estimativas de profundidade com base em distâncias características do tilt derivative.

Sintaxe

```
1 my_tilt_depth_from_pm45(distance_minus45_plus45)
2 my_tilt_depth_from_zero_to_45(distance_zero_to_45)
```

A primeira função estima a profundidade como metade da distância entre os pontos de -45° e $+45^\circ$:

$$z \approx \frac{d_{-45,+45}}{2}. \quad (293)$$

A segunda estima a profundidade a partir da distância entre o ponto de 0° e 45° :

$$z \approx d_{0,45}. \quad (294)$$

Estimativas aproximadas

Essas relações são aproximações interpretativas e devem ser usadas com cautela. A validade depende da geometria da fonte, do ruído, da amostragem e da qualidade do atributo tilt calculado.

9.10 Síntese técnica do módulo

O módulo **filtering.py** fornece um conjunto amplo de filtros e transformações para campos potenciais. Ele permite realizar operações fundamentais em interpretação gravimétrica e magnética, como continuação de campo, separação por frequência, redução magnética, pseudogravidade e cálculo de atributos de realce.

Importância para ensino de Geofísica

O `filtering.py` é adequado para aulas práticas sobre transformada de Fourier, filtragem espectral, realce de anomalias e interpretação de campos potenciais. Ele permite que o estudante visualize como diferentes operadores matemáticos alteram o campo observado e destacam características específicas das fontes.

10 Descrição técnica do módulo `geomagnetism.py`

O arquivo `geomagnetism.py` reúne funções para cálculos, simulações, transformações, visualizações e salvamento de dados geomagnéticos. O módulo inclui conversões de coordenadas geodésicas, coordenadas geocêntricas, cálculo de elementos do campo geomagnético, campo de dipolo, harmônicos esféricos, coeficientes de Gauss sintéticos, variação secular, redução ao polo, perfis geomagnéticos, mapas globais com e sem `Cartopy`, anomalias crustais sintéticas, campos observados sintéticos e campos residuais.

Esse módulo amplia o pacote para aplicações em Geomagnetismo, permitindo trabalhar com o campo principal da Terra, campos sintéticos, componentes X , Y , Z , intensidade total F , declinação D , inclinação I , anomalias crustais e visualização global. Do ponto de vista educacional, ele é útil para introduzir conceitos de campo vetorial, dipolo terrestre, coordenadas esféricas, harmônicos esféricos e representação cartográfica de campos globais.

Papel do módulo `geomagnetism.py`

O módulo `geomagnetism.py` integra formulações de Geomagnetismo com ferramentas computacionais em Python. Ele permite calcular campos sintéticos globais, decompor componentes do campo, gerar anomalias, construir perfis, aplicar filtros e criar mapas cartográficos de componentes geomagnéticas.

O módulo utiliza `NumPy`, `Matplotlib` e, de forma opcional, `Cartopy`. Quando `Cartopy` não está instalado, as funções cartográficas em projeções globais geram erro informativo, enquanto as demais funções continuam disponíveis.

10.1 Funções auxiliares de entrada, salvamento e conversão angular

As funções `_as_array`, `_check_same_shape`, `my_save_figure`, `my_save_table`, `my_deg2rad` e `my_rad2deg` fornecem suporte básico para conversão de arrays, validação de formas, salvamento de figuras, salvamento de tabelas e conversão angular.

Funções auxiliares

```
1      _as_array(a)
2      _check_same_shape(*arrays)
3      my_save_figure(filename, fig=None, dpi=300,
4      bbox_inches="tight", transparent=True)
5      my_save_table(filename, columns, header=None, fmt="
6      %.8e")
7      my_deg2rad(angle)
      my_rad2deg(angle)
```

As conversões angulares são

$$\theta_{rad} = \theta_{graus} \frac{\pi}{180}, \quad \theta_{graus} = \theta_{rad} \frac{180}{\pi}. \quad (295)$$

10.2 Conversões de coordenadas

10.2.1 `my_geodetic_to_ecef`

A função `my_geodetic_to_ecef` converte latitude, longitude e altura geodésica para coordenadas cartesianas geocêntricas ECEF.

Sintaxe

```
1      my_geodetic_to_ecef(lat, lon, height=0.0,
2      a=6378137.0,
3      f=1/298.257223563)
```

Entrada: `lat` e `lon` são latitude e longitude em graus; `height` é a altura em metros; `a` é o semieixo maior; `f` é o achatamento.

Saída: Retorna coordenadas X , Y e Z no sistema ECEF.

A primeira excentricidade ao quadrado é

$$e^2 = f(2 - f). \quad (296)$$

O raio de curvatura no primeiro vertical é

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}. \quad (297)$$

As coordenadas são

$$X = (N + h) \cos \varphi \cos \lambda, \quad (298)$$

$$Y = (N + h) \cos \varphi \sin \lambda, \quad (299)$$

$$Z = [N(1 - e^2) + h] \sin \varphi. \quad (300)$$

10.2.2 `my_ecef_to_spherical`, `my_geodetic_to_spherical` e `my_spherical_to_geocentric_latlon`

A função `my_ecef_to_spherical` converte coordenadas ECEF para coordenadas esféricas r, θ, ϕ , em que r é a distância radial, θ é a colatitude e ϕ é a longitude.

$$r = \sqrt{X^2 + Y^2 + Z^2}, \quad (301)$$

$$\theta = \arccos\left(\frac{Z}{r}\right), \quad (302)$$

$$\phi = \arctan 2(Y, X). \quad (303)$$

A função `my_geodetic_to_spherical` combina a conversão geodésica para ECEF e, em seguida, para coordenadas esféricas. A função `my_spherical_to_geocentric_latlon` converte r, θ, ϕ em latitude e longitude geocêntricas:

$$\varphi_c = 90^\circ - \theta, \quad \lambda = \phi. \quad (304)$$

10.3 Componentes, elementos e direções do campo geomagnético

10.3.1 `my_local_unit_vectors`

A função `my_local_unit_vectors` retorna vetores unitários locais nas direções norte, leste e vertical para baixo.

Sintaxe

```
1 my_local_unit_vectors(lat, lon)
```

Esses vetores permitem projetar campos calculados em coordenadas cartesianas para componentes geomagnéticas locais.

10.3.2 `my_direction_cosines`, `my_components_from_fid` e `my_elements_from_components`

A função `my_direction_cosines` calcula os cossenos diretores de uma direção definida por inclinação I e declinação D :

$$f_x = \cos I \cos D, \quad f_y = \cos I \sin D, \quad f_z = \sin I. \quad (305)$$

A função `my_components_from_fid` converte intensidade total F , inclinação I e declinação D em componentes:

$$X = F \cos I \cos D, \quad (306)$$

$$Y = F \cos I \sin D, \quad (307)$$

$$Z = F \sin I. \quad (308)$$

A função `my_elements_from_components` realiza a operação inversa:

$$H = \sqrt{X^2 + Y^2}, \quad (309)$$

$$F = \sqrt{X^2 + Y^2 + Z^2}, \quad (310)$$

$$D = \arctan 2(Y, X), \quad I = \arctan 2(Z, H). \quad (311)$$

10.3.3 my_total_field_anomaly_from_vector

A função `my_total_field_anomaly_from_vector` calcula a anomalia de campo total a partir de componentes vetoriais B_x , B_y e B_z :

$$\Delta T = B_x f_x + B_y f_y + B_z f_z. \quad (312)$$

10.4 Campo de dipolo magnético e dipolo axial centrado

10.4.1 my_magnetic_dipole_field_cartesian

A função `my_magnetic_dipole_field_cartesian` calcula o campo magnético de um dipolo em coordenadas cartesianas.

Sintaxe

```

1      my_magnetic_dipole_field_cartesian(x, y, z,
2      moment=(0,0,1),
3      source=(0,0,0),
4      output="components")

```

A formulação é

$$\mathbf{B} = \frac{\mu_0}{4\pi} \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right]. \quad (313)$$

10.4.2 my_centered_axial_dipole_elements

A função `my_centered_axial_dipole_elements` calcula elementos de um campo dipolar axial centrado em função da latitude.

Sintaxe

```
1 my_centered_axial_dipole_elements(lat, F_equator
    =30000.0)
```

Para um dipolo axial centrado, a inclinação aproximada satisfaz

$$\tan I = 2 \tan \lambda, \quad (314)$$

em que λ é a latitude. Essa relação é clássica em Geomagnetismo e permite associar inclinação magnética à latitude dipolar.

As funções `my_dip_latitude_from_inclination` e `my_inclination_from_dip_latitude` implementam as relações inversas:

$$\lambda_d = \arctan \left(\frac{1}{2} \tan I \right), \quad (315)$$

$$I = \arctan (2 \tan \lambda_d). \quad (316)$$

10.5 Harmônicos esféricos e coeficientes de Gauss

O campo geomagnético principal é frequentemente representado por harmônicos esféricos. O potencial escalar interno pode ser escrito como

$$V(r, \theta, \phi) = a \sum_{n=1}^N \left(\frac{a}{r} \right)^{n+1} \sum_{m=0}^n [g_n^m \cos(m\phi) + h_n^m \sin(m\phi)] P_n^m(\cos \theta), \quad (317)$$

em que a é o raio de referência, g_n^m e h_n^m são coeficientes de Gauss e P_n^m são funções associadas de Legendre.

10.5.1 `my_schmidt_quasi_normalization`, `my_associated_legendre_schmidt` e `my_associated_legendre_unnormalized`

Essas funções calculam fatores de normalização e funções associadas de Legendre, normalizadas ou não normalizadas.

Sintaxe

```
1 my_schmidt_quasi_normalization(nmax)
2 my_associated_legendre_schmidt(nmax, theta)
3 my_associated_legendre_unnormalized(nmax, x)
```

Essas funções são necessárias para construir o potencial geomagnético interno e suas derivadas.

10.5.2 `my_empty_gauss_coefficients`, `my_set_gauss_coefficient`, `my_synthetic_gauss_coefficients` e `my_gauss_coefficients_from_arrays`

Essas funções criam, preenchem e organizam coeficientes de Gauss.

Funções de coeficientes de Gauss

```
1 my_empty_gauss_coefficients(nmax)
2 my_set_gauss_coefficient(coeffs, n, m, g=None, h=None)
3 my_synthetic_gauss_coefficients(nmax, seed=0,
4 base_dipole=True)
my_gauss_coefficients_from_arrays(g, h)
```

Entrada: `nmax` define o grau máximo da expansão harmônica.

Saída: Retornam dicionários ou estruturas contendo coeficientes g_n^m e h_n^m .

A função de coeficientes sintéticos permite criar campos geomagnéticos artificiais para testes e ensino.

10.5.3 `my_internal_potential_spherical`, `my_internal_field_spherical` e `my_internal_field_elements`

A função `my_internal_potential_spherical` calcula o potencial interno por harmônicos esféricos. A função `my_internal_field_spherical` calcula as componentes do campo por derivadas do potencial, e `my_internal_field_elements` converte esses resultados para elementos geomagnéticos em latitude e longitude.

Sintaxe

```

1      my_internal_potential_spherical(r, theta, phi, coeffs
      , a=6371200.0)
2      my_internal_field_spherical(r, theta, phi, coeffs, a
      =6371200.0)
3      my_internal_field_elements(lat, lon, height, coeffs,
      a=6371200.0)

```

As componentes esféricas são obtidas por

$$B_r = -\frac{\partial V}{\partial r}, \quad B_\theta = -\frac{1}{r} \frac{\partial V}{\partial \theta}, \quad B_\phi = -\frac{1}{r \sin \theta} \frac{\partial V}{\partial \phi}. \quad (318)$$

10.6 Grades, campos sintéticos e variação secular

As funções `my_geomagnetic_grid`, `my_synthetic_geomagnetic_field`, `my_polynomial_geomagnetic_regional`, `my_add_geomagnetic_anomaly`, `my_secular_variation_linear`, `my_interpolate_gauss_coefficients` e `my_time_series_geomagnetic_intensity` permitem criar campos sintéticos e simular variação temporal.

Funções de campos sintéticos e variação secular

```

1      my_geomagnetic_grid(lat_min, lat_max, lon_min,
      lon_max, shape)
2      my_synthetic_geomagnetic_field(lat, lon, height=0.0,
3      nmax=8, seed=0)
4      my_polynomial_geomagnetic_regional(x, y, coefficients
      ,
5      normalize=True)

```

```

6         my_add_geomagnetic_anomaly(regional, anomaly,
7         noise_std=0.0, seed=None)
8         my_secular_variation_linear(field_epoch0,
9         secular_variation,
10        epoch0, epoch)
11        my_interpolate_gauss_coefficients(coeffs0, coeffs1,
12        epoch0, epoch1, epoch)
13        my_time_series_geomagnetic_intensity(F0,
        secular_variation,
        years, epoch0=2020.0)

```

A variação secular linear pode ser representada por

$$F(t) = F(t_0) + \dot{F}(t - t_0), \quad (319)$$

em que $\dot{F}$ é a taxa de variação secular.

10.7 Filtros magnéticos no domínio da frequência

O módulo inclui funções para construir números de onda, filtros direcionais magnéticos, redução ao polo e aplicação de filtros.

Funções de filtragem geomagnética

```

1         my_wavenumbers_from_grid(x, y)
2         my_magnetic_direction_filter(kx, ky, inc, dec, eps=1e
3         -12)
4         my_reduction_to_pole_filter(x, y, inc, dec,
5         inc0=90.0, dec0=0.0,
6         eps=1e-12)
7         my_apply_frequency_filter(data, filt, real_output=
8         True)
9         my_reduce_to_pole(data, x, y, inc, dec, eps=1e-12)

```

O filtro direcional magnético pode ser representado por

$$\Theta(k_x, k_y) = f_z + i \frac{f_x k_x + f_y k_y}{k + \varepsilon}. \quad (320)$$

A redução ao polo usa a razão entre o operador desejado e o operador original:

$$H_{RTP} = \frac{\Theta_{polo}}{\Theta_{original} + \varepsilon}. \quad (321)$$

10.8 Perfis, mapas e visualização geomagnética

O módulo contém funções para extrair perfis e gerar mapas geomagnéticos simples ou cartográficos.

Funções de perfil e visualização

```

1      my_profile_along_latitude(lat0, lon_min, lon_max,
2      npoints, height, coeffs)
3
4      my_profile_along_longitude(lon0, lat_min, lat_max,
5      npoints, height, coeffs)
6
7      my_plot_geomagnetic_map(lon, lat, data, ...)
8      my_plot_geomagnetic_profile(x, y, ...)
9      my_plot_declination_inclination(lon, lat, D, I, ...)
```

As funções com `Cartopy` permitem criar mapas globais ou regionais com projeções cartográficas:

Funções cartográficas

```

1      my_plot_geomagnetic_components_cartopy(lon, lat, X, Y
2      , Z, F,
3      component="F", ...)
4
5      my_plot_all_geomagnetic_components_cartopy(lon, lat,
6      X, Y, Z, F, ...)
7
8      my_plot_magnetic_equator_cartopy(lon, lat, I, ...)
```

```
7
8 my_plot_agonic_line_cartopy(lon, lat, D, ...)
```

A função de equador magnético destaca a linha em que

$$I = 0^\circ, \quad (322)$$

enquanto a função de linha agônica destaca a linha em que

$$D = 0^\circ. \quad (323)$$

Mapas geomagnéticos globais

As funções cartográficas são úteis para representar componentes do campo geomagnético em escala global, incluindo intensidade total, componentes X , Y , Z , equador magnético e linha agônica.

10.9 Anomalias crustais, campo observado e campo residual

As funções finais do módulo permitem criar anomalias crustais sintéticas, compor campos observados e calcular campos residuais.

Funções de anomalias e campos residuais

```
1 my_synthetic_crustal_anomaly_global(lon, lat,
2 anomalies=None,
3 background_waves=True,
4 noise_std=0.0,
5 seed=None)
6
7 my_geomagnetic_observed_field(main_field,
8 crustal_anomaly=None,
9 external_field=None,
10 noise_std=0.0,
11 seed=None)
```

```
12
13         my_geomagnetic_residual_field(observed_field,
14         main_field,
15         external_field=None)
```

O campo observado pode ser representado por

$$F_{obs} = F_{main} + F_{crustal} + F_{external} + \epsilon. \quad (324)$$

O campo residual é obtido por

$$F_{res} = F_{obs} - F_{main} - F_{external}. \quad (325)$$

10.10 Extração e salvamento de dados geomagnéticos

O módulo também permite extrair perfis e salvar campos globais ou elementos geomagnéticos em arquivos de texto.

Funções de extração e salvamento

```
1         my_extract_geomagnetic_profile(lon, lat, data,
2         profile_type="latitude",
3         value=0.0)
4
5         my_save_geomagnetic_global_fields(filename, lat, lon,
6         main_field,
7         crustal_anomaly,
8         observed_field,
9         residual_field,
10        fmt="%.8e")
11
12        my_save_geomagnetic_elements(filename, lat, lon,
13        elements, fmt="%.8e")
```

Essas funções são importantes para reprodutibilidade, exportação de resultados e integração com outros softwares.

10.11 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      from gravmag_codes import geomagnetism as gm
2
3      # Criacao de uma grade global
4      lat, lon = gm.my_geomagnetic_grid(
5          lat_min=-90,
6          lat_max=90,
7          lon_min=-180,
8          lon_max=180,
9          shape=(181, 361)
10     )
11
12     # Campo geomagnetico sintetico
13     field = gm.my_synthetic_geomagnetic_field(
14         lat=lat,
15         lon=lon,
16         height=0.0,
17         nmax=8,
18         seed=10
19     )
20
21     # Componentes e elementos
22     X = field["X"]
23     Y = field["Y"]
24     Z = field["Z"]
25     F = field["F"]
26     D = field["D"]
27     I = field["I"]
28
29     # Mapa simples da intensidade total
30     fig, ax = gm.my_plot_geomagnetic_map(
31         lon, lat, F,
32         colorbar_label="F (nT)",
33         title="Synthetic total geomagnetic field"
34     )
```

10.12 Síntese técnica do módulo

O módulo **geomagnetism.py** fornece uma base computacional ampla para ensino e experimentação em Geomagnetismo. Ele integra conversões de coordenadas, elementos geomagnéticos, dipolos, harmônicos esféricos, coeficientes de Gauss sintéticos, variação secular, redução ao polo, anomalias crustais, campos residuais e visualização cartográfica.

Importância para ensino de Geofísica

O **geomagnetism.py** permite trabalhar conceitos de campo vetorial, dipolo terrestre, harmônicos esféricos, inclinação, declinação, equador magnético, linha agônica e anomalias crustais. É especialmente útil para aulas práticas que combinam física, matemática, programação e visualização global.

Do ponto de vista científico e didático, o módulo também ajuda a demonstrar a separação entre campo principal, campo crustal, campo externo e campo residual, fornecendo uma base para estudos introdutórios de modelagem geomagnética e interpretação de anomalias.

11 Descrição técnica do módulo **gravity.py**

O arquivo **gravity.py** reúne funções associadas à gravidade normal, elipsoides de referência, correções gravimétricas, anomalias gravimétricas, modelos isostáticos e efeitos cinemáticos relacionados à rotação da Terra. Esse módulo é voltado ao tratamento básico de dados gravimétricos observados, permitindo calcular grandezas fundamentais como gravidade normal, correção ar-livre, anomalia ar-livre, correção de Bouguer, anomalia de Bouguer, distúrbio da gravidade, Moho isostática pelo modelo de Airy, anomalia isostática e correção de Eötvös.

Em levantamentos gravimétricos, os valores medidos precisam ser comparados com modelos de referência e corrigidos por diferentes efeitos físicos e geométricos. A gravidade observada em um ponto da superfície terrestre é influenciada pela latitude, altitude, topografia, rotação da Terra, distribuição de massas internas, massas superficiais e movimento da plataforma de aquisição. Por isso, o processamento gravimétrico envolve uma sequência de reduções e correções que buscam isolar a contribuição geológica de interesse.

Papel do módulo `gravity.py`

O módulo `gravity.py` fornece ferramentas para calcular gravidade normal, correções gravimétricas e anomalias clássicas. Ele atua como uma ponte entre dados gravimétricos observados e produtos interpretativos utilizados em Geofísica Aplicada, Geodésia Física e estudos isostáticos.

As entradas principais das funções são latitude, altura, gravidade observada, densidade, topografia e parâmetros de elipsoides de referência. As saídas são geralmente expressas em mGal, unidade tradicionalmente utilizada em Gravimetria Aplicada.

11.1 Funções auxiliares internas

O módulo contém funções internas para conversão de entradas em arrays, controle do formato de saída e verificação de compatibilidade entre variáveis.

11.1.1 `_asarray_float`

A função `_asarray_float` converte uma entrada para array NumPy com tipo `float`.

Sintaxe

```
1 _asarray_float(a)
```

Entrada: `a` pode ser escalar, lista ou array.

Saída: Retorna `a` convertido para array NumPy do tipo `float`.

Essa função padroniza as entradas para permitir operações vetorizadas.

11.1.2 `_return_like_input`

A função `_return_like_input` controla se a saída será retornada como escalar ou array, de acordo com o tipo da entrada original.

Sintaxe

```
1 _return_like_input(value, reference)
```

Entrada: `value` é o resultado calculado; `reference` é a entrada usada como referência.

Saída: Retorna um escalar se a referência original for escalar, ou array se a referência for array.

Essa função torna o módulo mais conveniente, pois permite usar as funções tanto para pontos individuais quanto para conjuntos de dados.

11.1.3 `_check_same_shape`

A função `_check_same_shape` verifica se diferentes arrays possuem a mesma forma.

Sintaxe

```
1 _check_same_shape(*arrays)
```

Entrada: Recebe uma sequência de arrays.

Saída: Não retorna valor. Caso as formas sejam incompatíveis, gera erro.

Essa função é importante para operações ponto a ponto, como cálculo de anomalias entre gravidade observada e gravidade normal.

11.2 Elipsoides de referência

A gravidade normal depende do elipsoide de referência adotado. O módulo inclui funções para retornar parâmetros dos elipsoides WGS84 e GRS80.

11.2.1 `my_WGS84`

A função `my_WGS84` retorna os parâmetros do elipsoide WGS84.

Sintaxe

```
1 my_WGS84()
```

Saída: Retorna um dicionário com parâmetros como semieixo maior, achatamento, constante gravitacional geocêntrica e velocidade angular.

O achatamento do elipsoide é definido por

$$f = \frac{a - b}{a}, \quad (326)$$

em que a é o semieixo maior e b é o semieixo menor.

11.2.2 my_GRS80

A função `my_GRS80` retorna os parâmetros do elipsoide GRS80.

Sintaxe

```
1 my_GRS80 ()
```

Saída: Retorna um dicionário contendo os parâmetros principais do GRS80.

O GRS80 é amplamente utilizado em Geodésia e também serve como referência para cálculos de gravidade normal.

11.2.3 my_get_ellipsoid

A função `my_get_ellipsoid` seleciona um elipsoide de referência a partir de seu nome.

Sintaxe

```
1 my_get_ellipsoid(name="WGS84")
```

Entrada: `name` é o nome do elipsoide, como "WGS84" ou "GRS80".

Saída: Retorna o dicionário de parâmetros do elipsoide selecionado.

11.3 Gravidade normal

A gravidade normal é o valor teórico da gravidade sobre um elipsoide de referência, considerando o achatamento terrestre e a rotação da Terra. Ela é usada como referência

para calcular anomalias.

11.3.1 my_somigliana

A função `my_somigliana` calcula a gravidade normal sobre o elipsoide usando a fórmula de Somigliana.

Sintaxe

```
1 my_somigliana(latitude, ellipsoid="WGS84")
```

Entrada: `latitude` é a latitude geodésica em graus; `ellipsoid` define o elipsoide de referência.

Saída: Retorna a gravidade normal em mGal.

A fórmula de Somigliana pode ser escrita como

$$\gamma(\varphi) = \gamma_e \frac{1 + k \sin^2 \varphi}{\sqrt{1 - e^2 \sin^2 \varphi}}, \quad (327)$$

em que γ_e é a gravidade normal no equador, e é a primeira excentricidade do elipsoide, k é uma constante associada à diferença entre a gravidade polar e equatorial, e φ é a latitude.

11.3.2 my_closedform

A função `my_closedform` calcula a gravidade normal considerando latitude e altura, por meio de uma aproximação fechada.

Sintaxe

```
1 my_closedform(latitude, height, ellipsoid="WGS84")
```

Entrada: `latitude` é a latitude em graus; `height` é a altura em metros.

Saída: Retorna a gravidade normal corrigida para a altura, em mGal.

Conceitualmente, a gravidade normal em altitude pode ser representada por uma expansão do tipo

$$\gamma(\varphi, h) = \gamma(\varphi) + \frac{\partial \gamma}{\partial h} h + \frac{1}{2} \frac{\partial^2 \gamma}{\partial h^2} h^2. \quad (328)$$

11.3.3 my_normal_gravity

A função `my_normal_gravity` é uma interface geral para calcular gravidade normal.

Sintaxe

```
1 my_normal_gravity(latitude, height=0.0,  
2 ellipsoid="WGS84", method="closedform")
```

Entrada: `latitude` é a latitude; `height` é a altura; `ellipsoid` define o elipsoide; `method` define o método de cálculo.

Saída: Retorna a gravidade normal em mGal.

Quando `height=0`, a função pode ser usada para calcular a gravidade normal no elipsoide. Quando a altura é diferente de zero, calcula a gravidade normal no ponto elevado.

11.4 Correção ar-livre e anomalia ar-livre

11.4.1 my_freeair_correction

A função `my_freeair_correction` calcula a correção ar-livre associada à altitude.

Sintaxe

```
1 my_freeair_correction(height)
```

Entrada: `height` é a altitude em metros.

Saída: Retorna a correção ar-livre em mGal.

A correção ar-livre clássica é aproximada por

$$C_{FA} = 0.3086h, \quad (329)$$

em que h é a altitude em metros.

11.4.2 `my_freeair_anomaly`

A função `my_freeair_anomaly` calcula a anomalia ar-livre.

Sintaxe

```
1      my_freeair_anomaly(observed_gravity, latitude, height  
2      ,  
      ellipsoid="WGS84")
```

Entrada: `observed_gravity` é a gravidade observada; `latitude` é a latitude; `height` é a altitude.

Saída: Retorna a anomalia ar-livre em mGal.

A formulação é

$$\Delta g_{FA} = g_{obs} - \gamma + C_{FA}. \quad (330)$$

11.5 Correção de Bouguer e anomalia de Bouguer

11.5.1 `my_bouguer_correction`

A função `my_bouguer_correction` calcula a correção de Bouguer simples para uma placa infinita.

Sintaxe

```
1      my_bouguer_correction(height, density=2.67)
```

Entrada: `height` é a altitude em metros; `density` é a densidade da placa em g/cm^3 .

Saída: Retorna a correção de Bouguer em mGal.

A formulação da placa de Bouguer é

$$C_B = 2\pi G \rho h, \quad (331)$$

em unidades SI. Convertendo para mGal, obtém-se

$$C_{B,\text{mGal}} = 2\pi G \rho_{SI} h \times 10^5. \quad (332)$$

11.5.2 `my_bouguer_anomaly`

A função `my_bouguer_anomaly` calcula a anomalia de Bouguer simples.

Sintaxe

```
1         my_bouguer_anomaly(observed_gravity, latitude, height
2         ,
          density=2.67, ellipsoid="WGS84")
```

Entrada: Recebe gravidade observada, latitude, altura, densidade e elipsoide de referência.

Saída: Retorna a anomalia de Bouguer em mGal.

A formulação é

$$\Delta g_B = g_{obs} - \gamma + C_{FA} - C_B. \quad (333)$$

Interpretação da anomalia de Bouguer

A anomalia de Bouguer busca remover o efeito da altitude e da massa topográfica idealizada entre o ponto de observação e o nível de referência. Ela é amplamente usada para investigar variações laterais de densidade na crosta.

11.6 Modelo isostático de Airy e Moho

11.6.1 my_Airy

A função `my_Airy` calcula a raiz isostática associada à topografia segundo o modelo de Airy.

Sintaxe

```
1 my_Airy(topography, rho_crust=2.67, rho_mantle=3.30)
```

Entrada: `topography` é a topografia; `rho_crust` é a densidade crustal; `rho_mantle` é a densidade do manto.

Saída: Retorna a raiz isostática.

No modelo de Airy, a compensação isostática é dada por

$$r = \frac{\rho_c}{\rho_m - \rho_c} h, \quad (334)$$

em que r é a raiz, h é a topografia, ρ_c é a densidade da crosta e ρ_m é a densidade do manto.

11.6.2 my_moho_airy

A função `my_moho_airy` estima a profundidade da Moho a partir do modelo de Airy.

Sintaxe

```
1 my_moho_airy(topography, normal_thickness=35000.0,  
2 rho_crust=2.67, rho_mantle=3.30)
```

Entrada: `normal_thickness` é a espessura crustal normal de referência.

Saída: Retorna a profundidade estimada da Moho.

A Moho é calculada como

$$z_{Moho} = T_0 + r, \quad (335)$$

em que T_0 é a espessura crustal normal e r é a raiz isostática de Airy.

11.7 Correção e anomalia isostática

11.7.1 `my_isostatic_correction`

A função `my_isostatic_correction` calcula uma correção isostática simplificada.

Sintaxe

```
1 my_isostatic_correction(topography, rho_crust=2.67,  
2 rho_mantle=3.30)
```

Entrada: `topography` é a topografia; `rho_crust` e `rho_mantle` são densidades de referência.

Saída: Retorna a correção isostática em mGal.

A correção está associada ao contraste de densidade entre crosta e manto e à raiz isostática. Em termos conceituais, pode ser associada à atração gravimétrica de uma placa compensadora:

$$C_I \propto 2\pi G(\rho_m - \rho_c)r. \quad (336)$$

11.7.2 `my_isostatic_anomaly`

A função `my_isostatic_anomaly` calcula a anomalia isostática a partir da anomalia de Bouguer e da correção isostática.

Sintaxe

```
1 my_isostatic_anomaly(bouguer_anomaly, topography,  
2 rho_crust=2.67, rho_mantle=3.30)
```

Saída: Retorna a anomalia isostática.

A formulação conceitual é

$$\Delta g_I = \Delta g_B + C_I. \quad (337)$$

11.8 Distúrbio da gravidade

A função `my_gravity_disturbance` calcula o distúrbio da gravidade.

Sintaxe

```
1 my_gravity_disturbance(observed_gravity ,  
    normal_gravity)
```

Entrada: `observed_gravity` é a gravidade observada; `normal_gravity` é a gravidade normal.

Saída: Retorna o distúrbio da gravidade.

A formulação é

$$\delta g = g_{obs} - \gamma. \quad (338)$$

Distúrbio versus anomalia

O distúrbio da gravidade e a anomalia de gravidade não são conceitos idênticos. O distúrbio compara a gravidade observada e a normal no mesmo ponto, enquanto a anomalia envolve reduções para uma superfície de referência.

11.9 Correção de Eötvös

A função `my_eotvos_correction` calcula a correção de Eötvös, associada ao movimento da plataforma de medição sobre a Terra em rotação.

Sintaxe

```
1 my_eotvos_correction(latitude, velocity, heading)
```

Entrada: *latitude* é a latitude; *velocity* é a velocidade da plataforma; *heading* é a direção de deslocamento.

Saída: Retorna a correção de Eötvös em mGal.

A correção depende da componente leste-oeste da velocidade e da rotação da Terra. Conceitualmente, está relacionada ao termo

$$C_E \propto 2\omega V_E \cos \varphi, \quad (339)$$

em que ω é a velocidade angular da Terra, V_E é a componente leste da velocidade e φ é a latitude.

11.10 Aliases de compatibilidade

O módulo define aliases para preservar compatibilidade com códigos anteriores:

Aliases principais

```
1 WGS84 = my_WGS84
2 GRS80 = my_GRS80
3 get_ellipsoid = my_get_ellipsoid
4 somigliana = my_somigliana
5 closedform = my_closedform
6 normal_gravity = my_normal_gravity
7 freeair_correction = my_freeair_correction
8 freeair_anomaly = my_freeair_anomaly
9 bouguer_correction = my_bouguer_correction
10 bouguer_anomaly = my_bouguer_anomaly
11 Airy = my_Airy
12 moho_airy = my_moho_airy
13 isostatic_correction = my_isostatic_correction
14 isostatic_anomaly = my_isostatic_anomaly
15 gravity_disturbance = my_gravity_disturbance
16 eotvos_correction = my_eotvos_correction
```

11.11 Síntese técnica do módulo

O módulo **gravity.py** fornece uma base computacional para reduções e correções gravimétricas clássicas. Ele permite calcular gravidade normal, anomalias ar-livre e Bouguer, distúrbio da gravidade, correções isostáticas e estimativas de Moho pelo modelo de Airy.

Importância para ensino de Geofísica

O **gravity.py** é adequado para aulas práticas sobre processamento gravimétrico. Ele permite demonstrar a diferença entre gravidade observada, gravidade normal, anomalia ar-livre, anomalia de Bouguer, distúrbio da gravidade e anomalia isostática.

Do ponto de vista científico, o módulo serve como base para preparação de dados gravimétricos antes da interpretação, modelagem direta ou inversão. Ele também permite discutir o papel das densidades, da topografia, da altitude, do elipsoide de referência e da isostasia em estudos geofísicos.

12 Descrição técnica do módulo **grids.py**

O arquivo **grids.py** reúne funções para criação, manipulação, interpolação, transformação, exportação e importação de malhas espaciais. Esse módulo é fundamental para o pacote **gravmag_codes**, pois praticamente todas as rotinas de modelagem, processamento, filtragem, visualização e inversão dependem da definição adequada dos pontos de observação e da discretização espacial.

Em Geofísica Computacional, os dados podem estar organizados em perfis, malhas regulares, malhas irregulares, pontos aleatórios, grades circulares, grids radiais, grids triangulares, grids hexagonais ou tabelas com colunas de coordenadas e propriedades físicas. O módulo **grids.py** fornece ferramentas para criar essas geometrias de amostragem e converter entre diferentes formatos.

Papel do módulo **grids.py**

O módulo **grids.py** fornece a base geométrica e espacial para o pacote. Ele permite criar malhas regulares e irregulares, perfis, grades especiais, transformar coordenadas, interpolar

dados, extrair subgrades, localizar pontos próximos e salvar ou carregar grades em arquivos de texto.

As funções do módulo operam principalmente sobre arrays NumPy. A convenção mais comum é representar uma malha regular por arrays bidimensionais **X**, **Y** e **Z**, ou por vetores unidimensionais obtidos a partir dessas malhas.

12.1 Funções internas de validação

12.1.1 `_check_area`

A função `_check_area` verifica se uma área retangular foi definida corretamente.

Sintaxe

```
1 _check_area(area)
```

Entrada: `area` deve conter quatro elementos: $[x_1, x_2, y_1, y_2]$.

Saída: Retorna a área validada como tupla.

A condição geométrica esperada é

$$x_2 > x_1, \quad y_2 > y_1. \quad (340)$$

12.1.2 `_check_shape`

A função `_check_shape` valida a forma de uma grade.

Sintaxe

```
1 _check_shape(shape)
```

Entrada: `shape` deve conter dois inteiros: n_y, n_x .

Saída: Retorna a forma validada.

A condição é

$$n_y > 0, \quad n_x > 0. \quad (341)$$

12.1.3 `_as_level_array`

A função `_as_level_array` transforma um nível vertical escalar ou array em uma matriz compatível com a forma da grade.

Sintaxe

```
1  _as_level_array(z, shape)
```

Entrada: `z` pode ser escalar ou array; `shape` define a forma esperada.

Saída: Retorna uma matriz Z com a forma especificada.

Quando z é escalar, a matriz é

$$Z_{ij} = z_0. \quad (342)$$

12.2 Informações básicas de malhas

12.2.1 `my_shape_to_size`

A função `my_shape_to_size` calcula o número total de pontos de uma malha.

Sintaxe

```
1  my_shape_to_size(shape)
```

Entrada: `shape` é a forma da grade n_y, n_x .

Saída: Retorna o número total de pontos.

A formulação é

$$N = n_y n_x. \quad (343)$$

12.2.2 my_grid_spacing

A função `my_grid_spacing` calcula os espaçamentos de uma grade regular.

Sintaxe

```
1 my_grid_spacing(area, shape)
```

Entrada: `area` define os limites da área; `shape` define o número de pontos.

Saída: Retorna Δx e Δy .

Para uma área $[x_1, x_2, y_1, y_2]$, tem-se

$$\Delta x = \frac{x_2 - x_1}{n_x - 1}, \quad (344)$$

$$\Delta y = \frac{y_2 - y_1}{n_y - 1}. \quad (345)$$

12.2.3 my_grid_area

A função `my_grid_area` calcula os limites de uma grade a partir das coordenadas.

Sintaxe

```
1 my_grid_area(x, y)
```

Entrada: `x` e `y` são arrays de coordenadas.

Saída: Retorna $[x_{min}, x_{max}, y_{min}, y_{max}]$.

12.2.4 my_grid_shape_from_vectors

A função `my_grid_shape_from_vectors` estima a forma de uma grade regular a partir dos vetores de coordenadas.

Sintaxe

```
1 my_grid_shape_from_vectors(x, y)
```

Saída: Retorna a forma n_y, n_x , calculada pelo número de valores únicos em y e x .

12.3 Conversão entre grade e vetor

12.3.1 my_as_grid

A função `my_as_grid` converte um vetor em matriz bidimensional.

Sintaxe

```
1 my_as_grid(vector, shape)
```

Entrada: `vector` é um vetor unidimensional; `shape` é a forma desejada.

Saída: Retorna uma matriz com a forma n_y, n_x .

12.3.2 my_as_vector

A função `my_as_vector` converte uma matriz em vetor.

Sintaxe

```
1 my_as_vector(grid)
```

Saída: Retorna o array achatado.

A operação corresponde a

$$\mathbf{d} = \text{vec}(\mathbf{D}). \quad (346)$$

12.4 Malhas regulares e irregulares

12.4.1 `my_regular` e `my_regular_grid`

As funções `my_regular` e `my_regular_grid` criam uma malha regular em uma área retangular.

Sintaxe

```
1 my_regular(area, shape, z=None)
2 my_regular_grid(area, shape, z=None)
```

Entrada: `area` define $[x_1, x_2, y_1, y_2]$; `shape` define n_y, n_x ; `z` define o nível vertical opcional.

Saída: Retorna X, Y e, opcionalmente, Z .

As coordenadas são construídas por

$$x_i = x_1 + i\Delta x, \quad i = 0, \dots, n_x - 1, \quad (347)$$

$$y_j = y_1 + j\Delta y, \quad j = 0, \dots, n_y - 1. \quad (348)$$

12.4.2 `my_irregular` e `my_random`

A função `my_irregular` cria pontos aleatórios em uma área retangular, enquanto `my_random` é um alias ou função equivalente para geração aleatória.

Sintaxe

```
1 my_irregular(area, size, z=None, seed=None)
2 my_random(area, size, z=None, seed=None)
```

As coordenadas são amostradas por distribuições uniformes:

$$x \sim U(x_1, x_2), \quad y \sim U(y_1, y_2). \quad (349)$$

12.5 Perfis e interpolação

12.5.1 my_profile

A função `my_profile` cria um perfil entre dois pontos.

Sintaxe

```
1 my_profile(point1, point2, size, z=None)
```

Entrada: `point1` e `point2` são os pontos inicial e final; `size` é o número de pontos.

Saída: Retorna as coordenadas do perfil e a distância acumulada.

A parametrização linear é

$$x(t) = x_1 + t(x_2 - x_1), \quad (350)$$

$$y(t) = y_1 + t(y_2 - y_1), \quad (351)$$

com

$$0 \leq t \leq 1. \quad (352)$$

A distância ao longo do perfil é

$$s(t) = t\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}. \quad (353)$$

12.5.2 my_1Dinterpolation

A função `my_1Dinterpolation` interpola dados ao longo de uma dimensão.

Sintaxe

```
1 my_1Dinterpolation(x, y, x_new, kind="linear")
```

Entrada: *x* e *y* são os dados originais; *x_new* são os novos pontos de interpolação; *kind* define o tipo de interpolação.

Saída: Retorna os valores interpolados.

Na interpolação linear entre dois pontos, tem-se

$$y(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1). \quad (354)$$

12.5.3 my_griddata

A função `my_griddata` interpola dados irregulares para uma grade regular.

Sintaxe

```
1 my_griddata(x, y, data, area, shape, method="linear")
```

Entrada: *x*, *y* e *data* são pontos irregulares; *area* e *shape* definem a grade de saída; *method* define o método de interpolação.

Saída: Retorna *X*, *Y* e a grade interpolada.

12.6 Grades especiais

O módulo contém funções para criar diferentes geometrias de amostragem espacial.

Funções de grades especiais

```
1 my_circular_grid(center, radius, ntheta, z=None)
2 my_concentric_circles(center, radii, ntheta, z=None)
3 my_radial_random_grid(center, radius, size, z=None,
4 seed=None)
5 my_triangular_grid(area, spacing, z=None)
6 my_hexagonal_grid(area, spacing, z=None)
7 my_line_grid(x0, x1, y_values, nx, z=None)
8 my_cross_grid(center, length, npoints, z=None)
9 my_spiral_grid(center, max_radius, npoints, turns=3,
10 z=None)
11 my_checkerboard_grid(area, shape, v1=1.0, v2=-1.0)
```

A grade circular é baseada em

$$x = x_c + R \cos \theta, \quad y = y_c + R \sin \theta. \quad (355)$$

A grade em espiral pode ser representada por

$$r(t) = R_{max}t, \quad \theta(t) = 2\pi Nt, \quad (356)$$

$$x = x_c + r(t) \cos \theta(t), \quad y = y_c + r(t) \sin \theta(t). \quad (357)$$

A grade checkerboard alterna dois valores, sendo útil em testes sintéticos de resolução.

Uso de grades especiais

Grades especiais são úteis para simular geometrias de aquisição, criar testes sintéticos, avaliar interpolação, construir perfis e representar padrões espaciais específicos.

12.7 Transformações geométricas

12.7.1 my_rotate_grid

A função `my_rotate_grid` rotaciona coordenadas em torno de um centro.

Sintaxe

1

```
my_rotate_grid(x, y, angle, center=(0.0, 0.0))
```

A rotação é dada por

$$x_r = x_c + (x - x_c) \cos \theta - (y - y_c) \sin \theta, \quad (358)$$

$$y_r = y_c + (x - x_c) \sin \theta + (y - y_c) \cos \theta. \quad (359)$$

12.7.2 my_translate_grid

A função `my_translate_grid` aplica uma translação às coordenadas.

Sintaxe

```
1 my_translate_grid(x, y, dx=0.0, dy=0.0)
```

A transformação é

$$x' = x + \Delta x, \quad y' = y + \Delta y. \quad (360)$$

12.8 Reamostragem, subgrades e ponto mais próximo

12.8.1 my_resample_grid

A função `my_resample_grid` reamostra uma grade para uma nova forma.

Sintaxe

```
1 my_resample_grid(x, y, data, new_shape, method="
    linear")
```

Saída: Retorna uma nova grade interpolada.

12.8.2 my_extract_subgrid

A função `my_extract_subgrid` extrai uma subárea de uma grade.

Sintaxe

```
1 my_extract_subgrid(x, y, data, area)
```

A seleção é feita por uma máscara lógica:

$$x_1 \leq x \leq x_2, \quad y_1 \leq y \leq y_2. \quad (361)$$

12.8.3 `my_nearest_point`

A função `my_nearest_point` localiza o ponto mais próximo de uma coordenada alvo.

Sintaxe

```
1 my_nearest_point(x, y, xp, yp)
```

A distância é

$$d_i = \sqrt{(x_i - x_p)^2 + (y_i - y_p)^2}. \quad (362)$$

O índice retornado corresponde ao menor valor de d_i .

12.9 Conversão para tabela, salvamento e leitura

12.9.1 `my_grid_to_table`

A função `my_grid_to_table` converte uma grade em tabela de colunas.

Sintaxe

```
1 my_grid_to_table(x, y, data=None, z=None)
```

A saída pode conter colunas como

$$[x, y, z, d]. \quad (363)$$

12.9.2 my_save_grid e my_load_grid

As funções `my_save_grid` e `my_load_grid` salvam e carregam grades em arquivos de texto.

Sintaxe

```
1         my_save_grid(filename, x, y, data=None, z=None,
2         header=None, fmt="%.8e")
3
4         my_load_grid(filename, has_z=False, has_data=True)
```

Essas funções são importantes para reprodutibilidade e integração com outros programas.

12.10 Aliases de compatibilidade

O módulo define aliases para preservar compatibilidade:

Aliases principais

```
1         regular = my_regular
2         regular_grid = my_regular_grid
3         irregular = my_irregular
4         random = my_random
5         profile = my_profile
6         griddata = my_griddata
7         rotate_grid = my_rotate_grid
8         translate_grid = my_translate_grid
9         save_grid = my_save_grid
10        load_grid = my_load_grid
```

12.11 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      from gravmag_codes import grids
2
3      # Area e forma da malha
4      area = [0.0, 10000.0, 0.0, 8000.0]
5      shape = (81, 101)
6
7      # Grade regular em z = 0
8      x, y, z = grids.my_regular(area, shape, z=0.0)
9
10     # Campo sintetico simples
11     data = grids.my_checkerboard_grid(area, shape, v1
12         =1.0, v2=-1.0)
13
14     # Conversao para tabela
15     table = grids.my_grid_to_table(x, y, data=data, z=z)
16
17     # Salvamento
18     grids.my_save_grid("grid_sintetico.txt", x, y, data=
19         data, z=z)
```

12.12 Síntese técnica do módulo

O módulo **grids.py** fornece a infraestrutura espacial do pacote **gravmag_codes**. Ele permite construir malhas regulares, irregulares, perfis, grades especiais, transformar coordenadas, interpolar dados, extrair regiões e salvar resultados.

Importância para ensino de Geofísica

O **grids.py** é essencial para ensinar discretização espacial, amostragem, interpolação, criação de perfis e organização de dados em malhas. Ele permite que estudantes compreendam como a geometria dos pontos de observação influencia a modelagem e a interpretação geofísica.

Do ponto de vista científico, o módulo é necessário em praticamente todos os fluxos de trabalho do pacote, desde a criação de modelos sintéticos até a filtragem, visualização, inversão e exportação de dados.

13 Descrição técnica do módulo `hexagonalprism.py`

O arquivo `hexagonalprism.py` reúne funções para criação, discretização e modelagem gravimétrica e magnética de prismas hexagonais verticais. Esse módulo amplia a representação geométrica das fontes geofísicas, permitindo trabalhar com corpos de base hexagonal regular, definidos por centro, raio, topo, base, rotação e propriedade física.

Em métodos potenciais, a geometria da fonte influencia diretamente a forma, a amplitude e a distribuição espacial das anomalias. Embora o prisma retangular seja uma geometria clássica em modelagem gravimétrica e magnética, prismas hexagonais podem ser úteis em discretizações alternativas, modelos sintéticos com simetria poligonal e representações aproximadas de corpos geológicos com contorno mais suave que o retangular.

Papel do módulo `hexagonalprism.py`

O módulo `hexagonalprism.py` implementa rotinas para representar prismas hexagonais verticais, calcular suas propriedades geométricas, discretizá-los em pequenos elementos de volume e calcular suas respostas gravimétricas e magnéticas por integração numérica discreta.

A abordagem adotada no módulo é numérica. O prisma hexagonal é discretizado em uma malha interna de pontos ou células, e a resposta total é obtida pela soma das contribuições elementares. Essa estratégia é semelhante à utilizada em outros módulos de corpos geométricos discretizados, como cilindros e prismas poligonais.

13.1 Formulação geométrica do prisma hexagonal

Um prisma hexagonal vertical é definido por

$$(x_c, y_c, z_{top}, z_{bottom}, R, \alpha), \quad (364)$$

em que x_c e y_c são as coordenadas do centro da base, z_{top} é a profundidade do topo, z_{bottom} é a profundidade da base, R é o raio do hexágono e α é o ângulo de rotação da base.

A condição geométrica fundamental é

$$z_{bottom} > z_{top}, \quad (365)$$

e o raio deve satisfazer

$$R > 0. \quad (366)$$

A altura do prisma é

$$H = z_{bottom} - z_{top}. \quad (367)$$

Para um hexágono regular de raio R , entendido como a distância do centro aos vértices, a área da base é

$$A_h = \frac{3\sqrt{3}}{2}R^2. \quad (368)$$

Assim, o volume do prisma hexagonal é

$$V = A_h H = \frac{3\sqrt{3}}{2}R^2(z_{bottom} - z_{top}). \quad (369)$$

13.2 Funções internas de suporte

O módulo contém funções internas para preparar coordenadas verticais, verificar pontos de observação, recuperar propriedades físicas, calcular cossenos diretores e evitar singularidades.

13.2.1 `_as_array_like_z`

A função `_as_array_like_z` prepara a coordenada vertical dos pontos de observação.

Sintaxe

```
1 _as_array_like_z(z, shape)
```

Entrada: `z` pode ser escalar ou array. `shape` define a forma esperada.

Saída: Retorna `z` como array compatível com a forma dos pontos de observação.

Quando `z` é escalar, a função cria uma matriz constante:

$$z_i = z_0. \quad (370)$$

13.2.2 `_check_observation_points`

A função `_check_observation_points` verifica se os pontos de observação possuem formas compatíveis.

Sintaxe

```
1      _check_observation_points(x, y, z)
```

Entrada: `x`, `y` e `z` são coordenadas dos pontos de observação.

Saída: Retorna os arrays preparados e compatíveis.

Essa verificação é necessária porque os cálculos são realizados ponto a ponto.

13.2.3 `_get_physical_property`

A função `_get_physical_property` recupera uma propriedade física a partir do argumento informado ou do próprio modelo.

Sintaxe

```
1      _get_physical_property(model, value, default=None,  
2      name="physical property")
```

Entrada: `model` é o prisma hexagonal; `value` é a propriedade fornecida diretamente; `default` é um valor padrão opcional.

Saída: Retorna a propriedade física a ser usada no cálculo.

Essa função permite que a densidade ou magnetização seja fornecida diretamente na chamada da função ou armazenada no modelo.

13.2.4 `_direction_cosines`

A função `_direction_cosines` calcula os cossenos diretores associados a uma inclinação e declinação.

Sintaxe

```
1  _direction_cosines(inc, dec)
```

A formulação é

$$f_x = \cos I \cos D, \quad (371)$$

$$f_y = \cos I \sin D, \quad (372)$$

$$f_z = \sin I. \quad (373)$$

13.2.5 `_safe_radius`

A função `_safe_radius` calcula uma distância regularizada para evitar singularidades numéricas.

Sintaxe

```
1  _safe_radius(dx, dy, dz, eps=1e-12)
```

A distância é calculada por

$$r = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2 + \varepsilon^2}. \quad (374)$$

13.3 Criação do modelo e propriedades geométricas

13.3.1 my_hexagonal_prism_model

A função `my_hexagonal_prism_model` cria um modelo de prisma hexagonal vertical.

Sintaxe

```

1      my_hexagonal_prism_model(xc, yc, top, bottom,
2      radius, rotation=0.0,
3      physical_property=None)

```

Entrada: `xc` e `yc` são as coordenadas do centro; `top` e `bottom` definem topo e base; `radius` é o raio do hexágono; `rotation` é o ângulo de rotação; `physical_property` é uma propriedade física opcional.

Saída: Retorna uma lista que representa o prisma hexagonal.

A função verifica se

$$R > 0 \tag{375}$$

e

$$z_{bottom} > z_{top}. \tag{376}$$

13.3.2 my_hexagonal_vertices

A função `my_hexagonal_vertices` calcula os seis vértices da base hexagonal.

Sintaxe

```

1      my_hexagonal_vertices(hexprism)

```

Saída: Retorna arrays com as coordenadas x e y dos vértices.

Os vértices são calculados por

$$x_i = x_c + R \cos\left(\alpha + \frac{2\pi i}{6}\right), \tag{377}$$

$$y_i = y_c + R \sin \left(\alpha + \frac{2\pi i}{6} \right), \quad (378)$$

com $i = 0, 1, \dots, 5$.

13.3.3 `my_hexagonal_area`, `my_hexagonal_volume` e `my_hexagonal_center`

A função `my_hexagonal_area` calcula a área da base:

$$A_h = \frac{3\sqrt{3}}{2} R^2. \quad (379)$$

A função `my_hexagonal_volume` calcula o volume:

$$V = A_h(z_{bottom} - z_{top}). \quad (380)$$

A função `my_hexagonal_center` retorna o centro geométrico do prisma:

$$(x_c, y_c, z_c), \quad (381)$$

com

$$z_c = \frac{z_{top} + z_{bottom}}{2}. \quad (382)$$

13.4 Discretização do prisma hexagonal

13.4.1 `my_point_in_polygon`

A função `my_point_in_polygon` verifica se pontos estão dentro de um polígono.

Sintaxe

```
1 my_point_in_polygon(x, y, xv, yv)
```

Entrada: `x` e `y` são os pontos testados; `xv` e `yv` são os vértices do polígono.

Saída: Retorna uma máscara booleana indicando quais pontos estão dentro do polígono.

Essa função é usada para selecionar células internas ao hexágono durante a discretização.

13.4.2 my_discretize_hexagonal_prism

A função `my_discretize_hexagonal_prism` discretiza o prisma em células de volume.

Sintaxe

```
1 my_discretize_hexagonal_prism(hexprism, spacing=None,
2 shape=None, nz=1)
```

Entrada: `hexprism` é o modelo do prisma; `spacing` define o espaçamento horizontal; `shape` define o número de células horizontais; `nz` define o número de camadas verticais.

Saída: Retorna arrays com coordenadas dos centros das células e seus volumes.

A discretização aproxima a integral volumétrica por um somatório:

$$\int_{\Omega} f(\mathbf{r}') dV' \approx \sum_{j=1}^N f(\mathbf{r}'_j) \Delta V_j. \quad (383)$$

Discretização numérica

A resposta geofísica do prisma hexagonal é calculada pela soma das contribuições de células internas ao polígono. A precisão depende do espaçamento horizontal e do número de camadas verticais.

13.5 Modelagem gravimétrica

A modelagem gravimétrica considera cada célula discretizada como uma pequena massa. Para densidade ρ e volume ΔV_j , a massa elementar é

$$\Delta m_j = \rho \Delta V_j. \quad (384)$$

A distância entre ponto de observação e célula é

$$r_j = \sqrt{(x - x_j)^2 + (y - y_j)^2 + (z - z_j)^2 + \varepsilon^2}. \quad (385)$$

13.5.1 my_hexagonal_prism_potential

A função `my_hexagonal_prism_potential` calcula o potencial gravitacional do prisma.

Sintaxe

```
1         my_hexagonal_prism_potential(x, y, z, hexprism,
2         rho=None, spacing=None,
3         shape=None, nz=1)
```

Entrada: Recebe coordenadas de observação, modelo do prisma, densidade e parâmetros de discretização.

Saída: Retorna o potencial gravitacional em unidades SI.

A formulação aproximada é

$$V(x, y, z) = G\rho \sum_{j=1}^N \frac{\Delta V_j}{r_j}. \quad (386)$$

13.5.2 my_hexagonal_prism_gx, my_hexagonal_prism_gy e my_hexagonal_prism_gz

Essas funções calculam as componentes gravimétricas do prisma hexagonal.

Sintaxe

```
1         my_hexagonal_prism_gx(x, y, z, hexprism, rho=None,
2         ...)
3         my_hexagonal_prism_gy(x, y, z, hexprism, rho=None,
4         ...)
5         my_hexagonal_prism_gz(x, y, z, hexprism, rho=None,
6         ...)
```

As formulações são

$$g_x = G\rho \sum_{j=1}^N \Delta V_j \frac{x_j - x}{r_j^3}, \quad (387)$$

$$g_y = G\rho \sum_{j=1}^N \Delta V_j \frac{y_j - y}{r_j^3}, \quad (388)$$

$$g_z = G\rho \sum_{j=1}^N \Delta V_j \frac{z_j - z}{r_j^3}. \quad (389)$$

As saídas são convertidas para mGal.

13.5.3 my_hexagonal_prism_gxyz

A função `my_hexagonal_prism_gxyz` calcula simultaneamente as três componentes g_x , g_y e g_z .

Sintaxe

```
1 my_hexagonal_prism_gxyz(x, y, z, hexprism,
2 rho=None, spacing=None,
3 shape=None, nz=1)
```

Saída: Retorna g_x , g_y e g_z , em mGal.

13.6 Modelagem magnética

Na modelagem magnética, cada célula é tratada como um dipolo magnético elementar. Para uma magnetização $\mathbf{M}$, o momento magnético da célula é

$$\mathbf{m}_j = \mathbf{M} \Delta V_j. \quad (390)$$

O campo de um dipolo é aproximado por

$$\mathbf{B} = \frac{\mu_0}{4\pi} \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right]. \quad (391)$$

13.6.1 `my_hexagonal_prism_bxyz`

A função `my_hexagonal_prism_bxyz` calcula as componentes magnéticas B_x , B_y e B_z .

Sintaxe

```
1      my_hexagonal_prism_bxyz(x, y, z, hexprism,
2      mag=None, incs=90.0, decs=0.0,
3      spacing=None, shape=None, nz=1)
```

Entrada: `mag` é a intensidade da magnetização em A/m; `incs` e `decs` definem a direção da magnetização.

Saída: Retorna B_x , B_y e B_z , em nT.

13.6.2 `my_hexagonal_prism_bx`, `my_hexagonal_prism_by` e `my_hexagonal_prism_bz`

Essas funções retornam individualmente as componentes magnéticas.

Sintaxe

```
1      my_hexagonal_prism_bx(x, y, z, hexprism, mag=None,
2      ...)
3      my_hexagonal_prism_by(x, y, z, hexprism, mag=None,
4      ...)
5      my_hexagonal_prism_bz(x, y, z, hexprism, mag=None,
6      ...)
```

Saída: Retornam, respectivamente, B_x , B_y ou B_z , em nT.

13.6.3 `my_hexagonal_prism_tfa` e `my_hexagonal_prism_tf`

A função `my_hexagonal_prism_tfa` calcula a anomalia magnética de campo total. A função `my_hexagonal_prism_tf` é um alias de compatibilidade.

Sintaxe

```

1      my_hexagonal_prism_tfa(x, y, z, hexprism,
2      mag=None, inc=-20.0, dec=0.0,
3      incs=None, decs=None,
4      spacing=None, shape=None, nz=1)

```

A anomalia de campo total é dada por

$$\Delta T = B_x f_x + B_y f_y + B_z f_z, \quad (392)$$

em que f_x, f_y, f_z é o vetor unitário na direção do campo geomagnético regional.

13.7 Modelagem de múltiplos prismas hexagonais

O módulo também contém funções para calcular a resposta total de uma lista de prismas hexagonais:

Funções para múltiplos prismas

```

1      my_hexagonal_prisms_potential(...)
2      my_hexagonal_prisms_gx(...)
3      my_hexagonal_prisms_gy(...)
4      my_hexagonal_prisms_gz(...)
5      my_hexagonal_prisms_gxyz(...)
6      my_hexagonal_prisms_bxyz(...)
7      my_hexagonal_prisms_tfa(...)
8      my_hexagonal_prisms_tf(...)

```

A resposta total é obtida por superposição linear:

$$d_{total} = \sum_{k=1}^{N_p} d_k, \quad (393)$$

em que N_p é o número de prismas hexagonais.

13.8 Conversão para prismas retangulares

13.8.1 `my_hexagonal_prism_to_prisms`

A função `my_hexagonal_prism_to_prisms` aproxima o prisma hexagonal por uma lista de prismas retangulares.

Sintaxe

```
1 my_hexagonal_prism_to_prisms(hexprism, spacing=None,
2 shape=None, physical_property=None,
3 include_boundary=True)
```

Entrada: Recebe o prisma hexagonal, parâmetros de discretização e propriedade física.

Saída: Retorna uma lista de prismas retangulares.

Essa função é útil para reutilizar rotinas de modelagem baseadas em prismas retangulares e para comparar respostas obtidas por diferentes estratégias de discretização.

13.9 Síntese técnica do módulo

O módulo `hexagonalprism.py` fornece uma representação alternativa para fontes tridimensionais em métodos potenciais. Ele permite criar prismas hexagonais, calcular suas propriedades geométricas, discretizá-los e estimar suas respostas gravimétricas e magnéticas por integração numérica discreta.

Importância para ensino de Geofísica

O `hexagonalprism.py` permite discutir como a escolha da geometria e da discretização influencia a resposta geofísica. Ele é útil para comparar prismas retangulares, cilindros, prismas poligonais e corpos hexagonais em exercícios de modelagem direta.

14 Descrição técnica do módulo `inversion.py`

O arquivo `inversion.py` reúne funções voltadas à solução de problemas inversos lineares e não lineares, avaliação de resíduos, construção de matrizes de regularização, ajuste polinomial, métodos iterativos, curva L e pequenas matrizes associadas a discretizações diferenciais e elementos finitos. Esse módulo é central para o pacote `gravmag_codes`, pois permite estimar parâmetros de modelos a partir de dados observados.

Em Geofísica, muitos problemas podem ser formulados como problemas inversos. A modelagem direta calcula dados a partir de um modelo conhecido. A inversão realiza o procedimento oposto: busca estimar o modelo que melhor explica os dados observados. Devido à não unicidade, ruído e mau condicionamento, problemas inversos geofísicos geralmente exigem regularização, vínculos físicos e análise criteriosa dos resíduos.

Papel do módulo `inversion.py`

O módulo `inversion.py` fornece ferramentas para resolver sistemas lineares, aplicar regularização, construir operadores diferenciais, ajustar modelos polinomiais, executar inversões não lineares e avaliar a qualidade dos resultados por métricas objetivas.

A formulação linear geral é

$$\mathbf{d} = \mathbf{G}\mathbf{m} + \mathbf{e}, \quad (394)$$

em que $\mathbf{d}$ é o vetor de dados observados, $\mathbf{G}$ é a matriz de sensibilidade, $\mathbf{m}$ é o vetor de parâmetros do modelo e $\mathbf{e}$ representa erros ou resíduos.

14.1 Funções internas de preparação e métricas seguras

14.1.1 `_as_1d` e `_as_2d`

As funções `_as_1d` e `_as_2d` convertem entradas para arrays unidimensionais ou bidimensionais.

Sintaxe

```
1      _as_1d(a, name="array")
2      _as_2d(a, name="matrix")
```

Saída: Retornam arrays NumPy com a dimensão esperada.

Essas funções padronizam a entrada dos algoritmos de inversão.

14.1.2 `_check_matrix_vector`

A função `_check_matrix_vector` verifica se uma matriz **G** e um vetor **d** possuem dimensões compatíveis.

Sintaxe

```
1 _check_matrix_vector(G, d)
```

A condição necessária é

$$N_{linhas}(\mathbf{G}) = N(\mathbf{d}). \quad (395)$$

14.1.3 `_safe_norm` e `_safe_corr`

A função `_safe_norm` calcula norma vetorial com proteção numérica. A função `_safe_corr` calcula correlação de forma segura, evitando divisões por desvios nulos.

A correlação entre dois vetores pode ser escrita como

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}. \quad (396)$$

14.2 Resíduos e métricas de erro

14.2.1 `my_residual`

A função `my_residual` calcula o vetor de resíduos.

Sintaxe

```
1 my_residual(observed, predicted)
```

Saída: Retorna

$$\mathbf{r} = \mathbf{d}_{obs} - \mathbf{d}_{pred}. \quad (397)$$

14.2.2 `my_rmse`, `my_mae` e `my_misfit_report`

A função `my_rmse` calcula

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (d_i^{obs} - d_i^{pred})^2}. \quad (398)$$

A função `my_mae` calcula

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |d_i^{obs} - d_i^{pred}|. \quad (399)$$

A função `my_misfit_report` retorna um conjunto de métricas, incluindo erro médio, erro quadrático, RMSE, MAE, norma dos resíduos e correlação entre observado e predito.

Avaliação do ajuste

As métricas de erro permitem avaliar se o modelo estimado reproduz adequadamente os dados observados. Em inversão geofísica, um bom ajuste numérico deve ser interpretado juntamente com a plausibilidade física do modelo.

14.3 Matrizes de regularização

A regularização é usada para estabilizar problemas inversos. O módulo inclui matrizes de amortecimento, diferenças finitas de primeira e segunda ordem, operadores 2D e Laplacianos.

14.3.1 `my_damping_matrix`

A função `my_damping_matrix` constrói uma matriz identidade para amortecimento.

Sintaxe

```
1 my_damping_matrix(nparams)
```

A matriz é

$$\mathbf{L} = \mathbf{I}. \quad (400)$$

Esse operador penaliza a norma do modelo:

$$\|\mathbf{m}\|_2^2. \quad (401)$$

14.3.2 my_first_difference_matrix e my_second_difference_matrix

A função `my_first_difference_matrix` constrói um operador de primeira diferença:

$$(L\mathbf{m})_i = m_{i+1} - m_i. \quad (402)$$

A função `my_second_difference_matrix` constrói um operador de segunda diferença:

$$(L\mathbf{m})_i = m_{i-1} - 2m_i + m_{i+1}. \quad (403)$$

Esses operadores impõem suavidade ao modelo.

14.3.3 my_2d_first_difference_matrix e my_2d_laplacian_matrix

Essas funções constroem operadores de diferenças finitas para modelos bidimensionais organizados em malhas.

O Laplaciano 2D discreto pode ser representado por

$$\nabla^2 m_{i,j} \approx m_{i-1,j} + m_{i+1,j} + m_{i,j-1} + m_{i,j+1} - 4m_{i,j}. \quad (404)$$

14.3.4 `my_regularization_matrix`

A função `my_regularization_matrix` funciona como uma interface para escolher o tipo de regularização.

Sintaxe

```
1 my_regularization_matrix(nparams, kind="damping",  
2 shape=None)
```

Entrada: `kind` pode indicar amortecimento, primeira diferença, segunda diferença ou operadores 2D.

Saída: Retorna a matriz de regularização $\mathbf{L}$.

14.4 Inversão linear

14.4.1 `my_least_squares`

A função `my_least_squares` resolve o problema de mínimos quadrados ordinários.

Sintaxe

```
1 my_least_squares(G, d, rcond=None)
```

A solução minimiza

$$\Phi(\mathbf{m}) = \|\mathbf{G}\mathbf{m} - \mathbf{d}\|_2^2. \quad (405)$$

Quando $\mathbf{G}^T\mathbf{G}$ é inversível, a solução formal é

$$\mathbf{m} = (\mathbf{G}^T\mathbf{G})^{-1}\mathbf{G}^T\mathbf{d}. \quad (406)$$

14.4.2 `my_weighted_least_squares`

A função `my_weighted_least_squares` resolve mínimos quadrados ponderados.

Sintaxe

```
1 my_weighted_least_squares(G, d, weights, rcond=None)
```

A função objetivo é

$$\Phi(\mathbf{m}) = \|\mathbf{W}(\mathbf{G}\mathbf{m} - \mathbf{d})\|_2^2. \quad (407)$$

14.4.3 my_tikhonov

A função `my_tikhonov` resolve um problema linear com regularização de Tikhonov.

Sintaxe

```
1 my_tikhonov(G, d, lam=0.0, L=None,
2 weights=None, rcond=None)
```

A função objetivo é

$$\Phi(\mathbf{m}) = \|\mathbf{W}(\mathbf{G}\mathbf{m} - \mathbf{d})\|_2^2 + \lambda^2 \|\mathbf{L}\mathbf{m}\|_2^2. \quad (408)$$

A solução é obtida por um sistema aumentado:

$$\begin{bmatrix} \mathbf{WG} \\ \lambda \mathbf{L} \end{bmatrix} \mathbf{m} = \begin{bmatrix} \mathbf{Wd} \\ \mathbf{0} \end{bmatrix}. \quad (409)$$

14.4.4 my_linear_inversion

A função `my_linear_inversion` é uma interface geral para inversão linear.

Sintaxe

```
1 my_linear_inversion(G, d, method="tikhonov",
2 lam=0.0, L=None, weights=None,
3 return_report=True)
```

Saída: Retorna o modelo estimado e, opcionalmente, um relatório com dados preditos, resíduos e métricas.

14.5 Ajuste polinomial e linearização

14.5.1 `my_polynomial_design_matrix`

A função `my_polynomial_design_matrix` constrói uma matriz de desenho polinomial.

Sintaxe

```
1 my_polynomial_design_matrix(x, y=None, degree=1)
```

Para o caso 1D, a matriz contém termos

$$1, x, x^2, \dots, x^n. \quad (410)$$

Para o caso 2D, contém termos

$$x^i y^j, \quad i + j \leq n. \quad (411)$$

14.5.2 `my_polynomial_fit`

A função `my_polynomial_fit` ajusta um polinômio aos dados.

Sintaxe

```
1 my_polynomial_fit(x, data, y=None, degree=1,  
2 lam=0.0, return_report=True)
```

O ajuste é um problema linear:

$$\mathbf{d} = \mathbf{A}\mathbf{c}, \quad (412)$$

em que $\mathbf{A}$ é a matriz de desenho e $\mathbf{c}$ é o vetor de coeficientes.

14.5.3 my_curve_fit_linearized

A função `my_curve_fit_linearized` ajusta modelos que podem ser expressos como combinações lineares de funções base.

Sintaxe

```
1 my_curve_fit_linearized(basis_functions, x, data,  
2 lam=0.0, return_report=True)
```

O modelo é

$$d(x) = \sum_{j=1}^M c_j \phi_j(x), \quad (413)$$

em que $\phi_j(x)$ são funções base.

14.6 Derivadas numéricas e função objetivo

14.6.1 my_numerical_jacobian

A função `my_numerical_jacobian` calcula numericamente a matriz Jacobiana de uma função direta.

Sintaxe

```
1 my_numerical_jacobian(forward_func, m, eps=1e-6,  
2 args=(), kwargs=None)
```

A aproximação por diferenças finitas é

$$J_{ij} \approx \frac{f_i(\mathbf{m} + \varepsilon \mathbf{e}_j) - f_i(\mathbf{m})}{\varepsilon}. \quad (414)$$

14.6.2 my_numerical_gradient

A função `my_numerical_gradient` calcula o gradiente numérico de uma função escalar.

$$\frac{\partial \Phi}{\partial m_j} \approx \frac{\Phi(\mathbf{m} + \varepsilon \mathbf{e}_j) - \Phi(\mathbf{m})}{\varepsilon}. \quad (415)$$

14.6.3 my_objective_function

A função `my_objective_function` calcula uma função objetivo composta por ajuste aos dados e regularização.

Sintaxe

```
1 my_objective_function(m, forward_func, data,
2 weights=None, lam=0.0, L=None,
3 mref=None, args=(), kwargs=None)
```

A formulação é

$$\Phi(\mathbf{m}) = \|\mathbf{W}(\mathbf{f}(\mathbf{m}) - \mathbf{d})\|_2^2 + \lambda^2 \|\mathbf{L}(\mathbf{m} - \mathbf{m}_{ref})\|_2^2. \quad (416)$$

14.7 Inversão não linear

14.7.1 my_gauss_newton

A função `my_gauss_newton` executa o método de Gauss-Newton.

Sintaxe

```
1 my_gauss_newton(forward_func, data, m0, max_iter=20,
2 tol=1e-6, lam=0.0, L=None,
3 weights=None, args=(), kwargs=None)
```

A atualização resolve

$$(\mathbf{J}^T \mathbf{J}) \Delta \mathbf{m} = -\mathbf{J}^T \mathbf{r}, \quad (417)$$

com

$$\mathbf{m}^{k+1} = \mathbf{m}^k + \Delta \mathbf{m}. \quad (418)$$

14.7.2 `my_levenberg_marquardt`

A função `my_levenberg_marquardt` executa o método de Levenberg-Marquardt.

Sintaxe

```

1         my_levenberg_marquardt(forward_func, data, m0,
2         max_iter=30, tol=1e-6,
3         damping=1e-3, weights=None,
4         args=(), kwargs=None)
```

A atualização é controlada por um termo de amortecimento:

$$(\mathbf{J}^T \mathbf{J} + \mu \mathbf{I}) \Delta \mathbf{m} = -\mathbf{J}^T \mathbf{r}. \quad (419)$$

Esse método combina características de Gauss-Newton e descida de gradiente.

14.7.3 `my_nonlinear_inversion`

A função `my_nonlinear_inversion` é uma interface geral para inversão não linear.

Sintaxe

```

1         my_nonlinear_inversion(forward_func, data, m0,
2         method="levenberg",
3         max_iter=30, tol=1e-6,
4         **kwargs)
```

Saída: Retorna o modelo final, histórico de iterações e métricas de ajuste.

14.8 Curva L

14.8.1 `my_lcurve`

A função `my_lcurve` calcula a curva L para diferentes valores de regularização.

Sintaxe

```
1 my_lcurve(G, d, lambdas, L=None, weights=None)
```

A curva L relaciona

$$\|\mathbf{G}\mathbf{m} - \mathbf{d}\|_2 \quad (420)$$

e

$$\|\mathbf{L}\mathbf{m}\|_2. \quad (421)$$

14.8.2 `my_best_lcurve_corner`

A função `my_best_lcurve_corner` estima o canto da curva L.

Sintaxe

```
1 my_best_lcurve_corner(lcurve_result)
```

Saída: Retorna o índice ou parâmetro associado ao ponto de maior curvatura aproximada.

Escolha do parâmetro de regularização

A curva L é uma ferramenta clássica para escolher o parâmetro de regularização, buscando equilíbrio entre ajuste aos dados e suavidade ou estabilidade do modelo.

14.9 Matrizes diferenciais e elementos finitos

O módulo inclui funções adicionais para construir operadores diferenciais e matrizes simples de elementos finitos:

Funções adicionais

```

1      my_first_derivative_matrix_1d(n, dx=1.0)
2      my_second_derivative_matrix_1d(n, dx=1.0)
3      my_laplacian_matrix_2d(shape, dx=1.0, dy=1.0)
4      my_linear_fem_1d_stiffness(nnodes, length=1.0)
5      my_linear_fem_1d_mass(nnodes, length=1.0)

```

A matriz de rigidez 1D para elementos lineares está associada à integral

$$K_{ij} = \int \frac{dN_i}{dx} \frac{dN_j}{dx} dx, \quad (422)$$

enquanto a matriz de massa está associada a

$$M_{ij} = \int N_i N_j dx. \quad (423)$$

14.10 Salvamento de resultados

A função `my_save_inversion_result` salva resultados de inversão em arquivo.

Sintaxe

```

1      my_save_inversion_result(filename, result, fmt="%.8e"
                                )

```

Essa função contribui para reprodutibilidade e documentação dos experimentos.

14.11 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import inversion
3
4      # Matriz de sensibilidade sintetica
5      G = np.random.randn(100, 20)
6
7      # Modelo verdadeiro
8      m_true = np.zeros(20)
9      m_true[5:10] = 1.0
10
11     # Dados observados com ruído
12     d = G @ m_true + np.random.normal(0, 0.05, 100)
13
14     # Regularizacao de suavidade
15     L = inversion.my_second_difference_matrix(20)
16
17     # Inversao de Tikhonov
18     result = inversion.my_linear_inversion(
19         G, d,
20         method="tikhonov",
21         lam=0.1,
22         L=L,
23         return_report=True
24     )
25
26     m_est = result["model"]
```

14.12 Síntese técnica do módulo

O módulo **`inversion.py`** fornece uma base geral para resolução de problemas inversos em Geofísica Computacional. Ele inclui ferramentas para mínimos quadrados, regularização de Tikhonov, inversão linear, inversão não linear, ajuste polinomial, curva L, operadores diferenciais e avaliação de resíduos.

Importância para ensino de Geofísica

O `inversion.py` permite ensinar conceitos fundamentais de problemas inversos, como matriz de sensibilidade, resíduos, regularização, não unicidade, ajuste aos dados, suavidade do modelo e escolha do parâmetro de regularização. Ele é adequado para atividades práticas em que estudantes constroem problemas sintéticos e avaliam diferentes estratégias de inversão.

Do ponto de vista científico, esse módulo pode ser usado como base para metodologias de inversão gravimétrica, magnética, geolétrica e outras aplicações lineares ou não lineares, desde que a função direta e a matriz de sensibilidade sejam devidamente definidas.

15 Descrição técnica do módulo `kernel.py`

O arquivo `kernel.py` reúne funções matemáticas associadas ao núcleo Newtoniano $1/r$ e às suas derivadas espaciais de primeira, segunda e terceira ordem. Esse módulo fornece a base analítica para construção de matrizes de sensibilidade, modelagem por fontes pontuais, camada equivalente, cálculo de gradientes, tensores e respostas gravimétricas e magnéticas simplificadas.

Em métodos potenciais, grande parte das formulações parte da distância entre um ponto de observação e uma fonte. Para uma fonte localizada em x_s, y_s, z_s e um ponto de observação em (x, y, z) , define-se

$$\Delta x = x - x_s, \quad \Delta y = y - y_s, \quad \Delta z = z - z_s, \quad (424)$$

e

$$r = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}. \quad (425)$$

O núcleo fundamental é

$$K_0 = \frac{1}{r}. \quad (426)$$

A partir desse núcleo são obtidos os operadores de campo potencial, gradiente, tensor de gradiente e termos direcionais usados em magnetometria.

O módulo `kernel.py` fornece os blocos matemáticos fundamentais para modelagem de campos potenciais. Ele implementa o núcleo Newtoniano $1/r$, suas derivadas e operadores direcionais, permitindo construir respostas gravimétricas, magnéticas e matrizes de sensibilidade.

O módulo utiliza NumPy para operações vetorizadas. As funções aceitam escalares ou arrays, permitindo calcular kernels para um único ponto de observação ou para conjuntos de pontos.

15.1 Funções internas de preparação

15.1.1 `_as_array`

A função `_as_array` converte uma entrada para array NumPy do tipo `float`.

Sintaxe

```
1  _as_array(a)
```

Entrada: `a` pode ser escalar, lista ou array.

Saída: Retorna `a` como array NumPy.

15.1.2 `_check_observation_arrays`

A função `_check_observation_arrays` verifica se as coordenadas dos pontos de observação possuem a mesma forma.

Sintaxe

```
1  _check_observation_arrays(x, y, z)
```

Entrada: `x`, `y` e `z` são coordenadas dos pontos de observação.

Saída: Retorna os arrays validados.

A condição necessária é

$$\text{shape}(x) = \text{shape}(y) = \text{shape}(z). \quad (427)$$

15.1.3 `_parse_model`

A função `_parse_model` interpreta um modelo de fonte e extrai suas coordenadas e propriedade física.

Sintaxe

```
1  _parse_model(model)
```

Entrada: `model` é uma lista, tupla ou estrutura contendo as coordenadas da fonte e, opcionalmente, uma propriedade física.

Saída: Retorna as coordenadas da fonte e a propriedade física associada.

Essa função permite que diferentes rotinas usem modelos compactos de fonte sem repetir código de leitura e validação.

15.2 Volume de esfera e coordenadas relativas

15.2.1 `my_sphere_volume`

A função `my_sphere_volume` calcula o volume de uma esfera.

Sintaxe

```
1  my_sphere_volume(radius)
```

Entrada: `radius` é o raio da esfera.

Saída: Retorna o volume da esfera.

A formulação é

$$V = \frac{4}{3}\pi R^3. \quad (428)$$

Essa função é útil para converter propriedades volumétricas em massas ou momentos equivalentes quando se usa uma esfera como fonte pontual aproximada.

15.2.2 `my_kernel_coordinates`

A função `my_kernel_coordinates` calcula as diferenças de coordenadas entre observação e fonte.

Sintaxe

```
1 my_kernel_coordinates(x, y, z, xs, ys, zs)
```

Entrada: `x`, `y` e `z` são coordenadas dos pontos de observação. `xs`, `ys` e `zs` são coordenadas da fonte.

Saída: Retorna Δx , Δy e Δz .

$$\Delta x = x - x_s, \quad \Delta y = y - y_s, \quad \Delta z = z - z_s. \quad (429)$$

15.2.3 `my_kernel_r`

A função `my_kernel_r` calcula a distância entre ponto de observação e fonte.

Sintaxe

```
1 my_kernel_r(x, y, z, xs, ys, zs, eps=1e-12)
```

Entrada: Recebe coordenadas de observação, coordenadas da fonte e uma tolerância ε .

Saída: Retorna a distância regularizada r .

A formulação é

$$r = \sqrt{(x - x_s)^2 + (y - y_s)^2 + (z - z_s)^2 + \varepsilon^2}. \quad (430)$$

A inclusão de ε evita singularidades quando o ponto de observação coincide com a fonte.

15.3 Núcleo fundamental e derivadas de primeira ordem

15.3.1 `my_kernel0` e `my_kernel_potential`

A função `my_kernel0` calcula o núcleo fundamental $1/r$. A função `my_kernel_potential` é semanticamente equivalente e representa o núcleo de potencial.

Sintaxe

```

1      my_kernel0(x, y, z, xs, ys, zs, eps=1e-12)
2      my_kernel_potential(x, y, z, xs, ys, zs, eps=1e-12)
```

Saída: Retorna

$$K_0 = \frac{1}{r}. \quad (431)$$

Esse núcleo aparece no potencial gravitacional de fontes pontuais:

$$V = Gm\frac{1}{r}. \quad (432)$$

15.3.2 `my_kernelx`, `my_kernely` e `my_kernelz`

Essas funções calculam as derivadas de primeira ordem do núcleo $1/r$.

Sintaxe

```

1      my_kernelx(x, y, z, xs, ys, zs, eps=1e-12)
2      my_kernely(x, y, z, xs, ys, zs, eps=1e-12)
3      my_kernelz(x, y, z, xs, ys, zs, eps=1e-12)
```

As derivadas são

$$\frac{\partial}{\partial x} \left(\frac{1}{r} \right) = -\frac{\Delta x}{r^3}, \quad (433)$$

$$\frac{\partial}{\partial y} \left(\frac{1}{r} \right) = -\frac{\Delta y}{r^3}, \quad (434)$$

$$\frac{\partial}{\partial z} \left(\frac{1}{r} \right) = -\frac{\Delta z}{r^3}. \quad (435)$$

Sinal das derivadas

As derivadas do núcleo dependem da definição de Δx , Δy e Δz . Por isso, ao combinar essas funções com formulações gravimétricas ou magnéticas, é importante manter a mesma convenção de coordenadas adotada no pacote.

15.4 Derivadas de segunda ordem e tensor Hessiano

As derivadas de segunda ordem do núcleo formam a matriz Hessiana. Elas são usadas em tensores de gradiente gravimétrico, modelagem magnética dipolar e construção de kernels direcionais.

15.4.1 `my_kernelxx`, `my_kernelxy`, `my_kernelxz`, `my_kernelyy`, `my_kernelyz` e `my_kernelzz`

As funções de segunda ordem calculam

$$K_{xx} = \frac{\partial^2}{\partial x^2} \left(\frac{1}{r} \right), \quad K_{xy} = \frac{\partial^2}{\partial x \partial y} \left(\frac{1}{r} \right), \quad (436)$$

e assim por diante.

Para os termos diagonais,

$$K_{xx} = \frac{3\Delta x^2 - r^2}{r^5}, \quad (437)$$

$$K_{yy} = \frac{3\Delta y^2 - r^2}{r^5}, \quad (438)$$

$$K_{zz} = \frac{3\Delta z^2 - r^2}{r^5}. \quad (439)$$

Para os termos cruzados,

$$K_{xy} = \frac{3\Delta x \Delta y}{r^5}, \quad (440)$$

$$K_{xz} = \frac{3\Delta x \Delta z}{r^5}, \quad (441)$$

$$K_{yz} = \frac{3\Delta y \Delta z}{r^5}. \quad (442)$$

Sintaxe

```

1      my_kernelxx(x, y, z, xs, ys, zs, eps=1e-12)
2      my_kernelxy(x, y, z, xs, ys, zs, eps=1e-12)
3      my_kernelxz(x, y, z, xs, ys, zs, eps=1e-12)
4      my_kernelyy(x, y, z, xs, ys, zs, eps=1e-12)
5      my_kernelyz(x, y, z, xs, ys, zs, eps=1e-12)
6      my_kernelzz(x, y, z, xs, ys, zs, eps=1e-12)

```

15.4.2 `my_kernel_gradient`, `my_kernel_hessian`, `my_kernel_tensor_matrix` e `my_kernel_laplacian`

A função `my_kernel_gradient` retorna as três derivadas de primeira ordem:

$$\nabla K_0 = (K_x, K_y, K_z). \quad (443)$$

A função `my_kernel_hessian` retorna os seis termos independentes da Hessiana:

$$\mathbf{H} = \begin{bmatrix} K_{xx} & K_{xy} & K_{xz} \\ K_{xy} & K_{yy} & K_{yz} \\ K_{xz} & K_{yz} & K_{zz} \end{bmatrix}. \quad (444)$$

A função `my_kernel_tensor_matrix` organiza os termos em forma matricial, e `my_kernel_laplacian` calcula

$$\nabla^2 \left(\frac{1}{r} \right) = K_{xx} + K_{yy} + K_{zz}. \quad (445)$$

Fora da singularidade da fonte, vale teoricamente

$$\nabla^2 \left(\frac{1}{r} \right) = 0. \quad (446)$$

15.5 Derivadas de terceira ordem

O módulo também implementa derivadas de terceira ordem do núcleo Newtoniano, úteis em formulações avançadas de tensores, gradientes de tensores e atributos derivados.

Funções de terceira ordem

```

1      my_kernelxxx(...)
2      my_kernelxxy(...)
3      my_kernelxxz(...)
4      my_kernelxyy(...)
5      my_kernelxyz(...)
6      my_kernelxzz(...)
7      my_kernelyyy(...)
8      my_kernelyyz(...)
9      my_kernelyzz(...)
10     my_kernelzzz(...)
11     my_kernel_third_order(...)
```

A função interna `_third_kernel_component` calcula genericamente termos da forma

$$K_{abc} = \frac{\partial^3}{\partial a \partial b \partial c} \left(\frac{1}{r} \right), \quad (447)$$

em que $a, b, c \in \{x, y, z\}$.

De forma geral, essas derivadas combinam produtos das diferenças Δx , Δy , Δz e potências de r , como r^{-5} e r^{-7} . Elas são sensíveis a singularidades e devem ser usadas com cuidado em pontos muito próximos da fonte.

15.6 Operadores direcionais

15.6.1 `my_direction_vector`

A função `my_direction_vector` converte inclinação e declinação em vetor unitário.

Sintaxe

```
1 my_direction_vector(inc, dec)
```

A formulação é

$$u_x = \cos I \cos D, \quad u_y = \cos I \sin D, \quad u_z = \sin I. \quad (448)$$

15.6.2 my_directional_first_kernel

A função `my_directional_first_kernel` calcula a derivada direcional de primeira ordem do núcleo.

Sintaxe

```
1 my_directional_first_kernel(x, y, z, xs, ys, zs,  
2 inc, dec, eps=1e-12)
```

A formulação é

$$K_{\mathbf{u}} = u_x K_x + u_y K_y + u_z K_z. \quad (449)$$

15.6.3 my_directional_second_kernel

A função `my_directional_second_kernel` calcula uma derivada direcional de segunda ordem entre duas direções.

Sintaxe

```
1 my_directional_second_kernel(x, y, z, xs, ys, zs,  
2 inc1, dec1, inc2, dec2,  
3 eps=1e-12)
```

A formulação compacta é

$$K_{\mathbf{uv}} = \mathbf{u}^T \mathbf{H} \mathbf{v}, \quad (450)$$

em que $\mathbf{H}$ é a Hessiana do núcleo, e $\mathbf{u}$ e $\mathbf{v}$ são vetores unitários definidos por inclinação e declinação.

15.7 Kernels gravimétricos e magnéticos

15.7.1 `my_gravity_gx_kernel`, `my_gravity_gy_kernel` e `my_gravity_gz_kernel`

Essas funções retornam kernels para as componentes da atração gravitacional de fontes pontuais.

Sintaxe

```
1 my_gravity_gx_kernel(x, y, z, xs, ys, zs, eps=1e-12)
2 my_gravity_gy_kernel(x, y, z, xs, ys, zs, eps=1e-12)
3 my_gravity_gz_kernel(x, y, z, xs, ys, zs, eps=1e-12)
```

A resposta gravitacional de uma massa m pode ser escrita como

$$\mathbf{g} = Gm \frac{\mathbf{r}_s - \mathbf{r}}{r^3}. \quad (451)$$

Assim, os kernels fornecem os fatores geométricos que, multiplicados por massa ou densidade equivalente, produzem as componentes do campo.

15.7.2 `my_gravity_gradient_kernel`

A função `my_gravity_gradient_kernel` retorna kernels de gradiente gravimétrico, baseados nas derivadas de segunda ordem do potencial.

Sintaxe

```
1 my_gravity_gradient_kernel(x, y, z, xs, ys, zs,
2 component="zz", eps=1e-12)
```

Entrada: `component` indica qual componente do tensor será retornada, como "xx", "xy", "xz", "yy", "yz" ou "zz".

Saída: Retorna o kernel da componente escolhida.

15.7.3 `my_totalfield_kernel` e `my_magnetic_totalfield_kernel`

A função `my_totalfield_kernel` calcula o kernel direcional associado à anomalia magnética de campo total. A função `my_magnetic_totalfield_kernel` representa uma interface equivalente ou mais explícita para o mesmo tipo de cálculo.

A formulação envolve a projeção do campo magnético anômalo na direção do campo regional:

$$\Delta T = \mathbf{B} \cdot \hat{\mathbf{F}}. \quad (452)$$

Para fontes dipolares, o operador magnético pode ser expresso por derivadas direcionais de segunda ordem do núcleo:

$$K_{\Delta T} \propto \hat{\mathbf{F}}^T \mathbf{H} \hat{\mathbf{M}}, \quad (453)$$

em que $\hat{\mathbf{F}}$ é a direção do campo regional e $\hat{\mathbf{M}}$ é a direção da magnetização.

15.8 Matrizes de kernels

15.8.1 `my_kernel_matrix`

A função `my_kernel_matrix` constrói uma matriz de sensibilidade genérica a partir de uma lista de fontes.

Sintaxe

```
1 my_kernel_matrix(x, y, z, sources, kernel_function,
    **kwargs)
```

Entrada: `x`, `y` e `z` são pontos de observação; `sources` é uma lista de fontes; `kernel_function` define o kernel usado.

Saída: Retorna uma matriz **G**.

Cada elemento da matriz é

$$G_{ij} = K(\mathbf{r}_i, \mathbf{s}_j), \quad (454)$$

em que $\mathbf{r}_i$ é o ponto de observação i e $\mathbf{s}_j$ é a fonte j .

15.8.2 my_gravity_kernel_matrix

A função `my_gravity_kernel_matrix` constrói matrizes de sensibilidade para componentes gravimétricas.

Sintaxe

```
1 my_gravity_kernel_matrix(x, y, z, sources,
2 component="gz", eps=1e-12)
```

Entrada: `component` define a componente gravimétrica desejada.

Saída: Retorna a matriz de sensibilidade gravimétrica.

15.8.3 my_totalfield_kernel_matrix

A função `my_totalfield_kernel_matrix` constrói uma matriz de sensibilidade magnética de campo total.

Sintaxe

```
1 my_totalfield_kernel_matrix(x, y, z, sources,
2 inc, dec, incs=None, decs=None,
3 eps=1e-12)
```

Entrada: `inc` e `dec` definem a direção do campo regional; `incs` e `decs` definem a direção da magnetização.

Saída: Retorna a matriz de sensibilidade magnética.

Matrizes de sensibilidade

As matrizes de kernel são fundamentais em problemas inversos, camada equivalente e ajustes lineares. Elas conectam parâmetros das fontes aos dados observados por meio de uma relação do tipo $\mathbf{d} = \mathbf{Gm}$.

15.9 Síntese técnica do módulo

O módulo `kernel.py` fornece a base matemática para diversos cálculos do pacote *gravmag_codes*. Ao implementar o núcleo $1/r$ e suas derivadas, ele permite construir respostas gravimétricas, magnéticas, tensoriais e direcionais de maneira modular.

Importância para ensino de Geofísica

O `kernel.py` é ideal para demonstrar a ligação entre cálculo diferencial, potencial Newtoniano e métodos geofísicos. Ele mostra como derivadas de $1/r$ originam campos, gradientes, tensores e operadores usados em gravimetria e magnetometria.

Do ponto de vista científico, o módulo é útil na construção de matrizes de sensibilidade, problemas inversos, camada equivalente e testes sintéticos baseados em fontes pontuais ou discretizações volumétricas.

16 Descrição técnica do módulo `plotting.py`

O arquivo `plotting.py` reúne funções para visualização científica de dados geofísicos. O módulo inclui rotinas para gráficos de linha, dispersão, histogramas, mapas de contorno, mapas de contorno triangulado, mapas por células, desenhos de formas geométricas, colorbars, normalização de cores, seleção de colormaps, mapas geográficos com `Cartopy` e salvamento de figuras.

A visualização é uma etapa central em Geofísica Computacional. Dados gravimétricos, magnéticos, geoeletricos, sísmicos, radiométricos ou geomagnéticos geralmente são interpretados por meio de mapas, perfis, seções, histogramas e imagens. O módulo `plotting.py` fornece funções padronizadas para gerar esses produtos gráficos com sintaxe simples e consistente.

Papel do módulo `plotting.py`

O módulo `plotting.py` padroniza a visualização dos resultados do pacote. Ele facilita a criação de figuras científicas, mapas, histogramas, perfis, gráficos de dispersão e mapas geográficos, contribuindo para análise, ensino e publicação.

O módulo utiliza principalmente NumPy e Matplotlib. O uso de Cartopy é opcional e ocorre apenas nas funções cartográficas.

16.1 Funções internas de preparação

16.1.1 `_to_array`

A função `_to_array` converte uma entrada para array NumPy.

Sintaxe

```
1 _to_array(a)
```

Entrada: `a` pode ser escalar, lista ou array.

Saída: Retorna um array NumPy.

16.1.2 `_finite_data`

A função `_finite_data` seleciona apenas valores finitos de um conjunto de dados.

Sintaxe

```
1 _finite_data(data)
```

Saída: Retorna os valores que satisfazem

$$\text{isfinite}(d_i) = \text{True}. \quad (455)$$

Essa função é útil para definir escalas de cores e histogramas sem interferência de valores NaN ou infinitos.

16.1.3 `_check_same_shape`

A função `_check_same_shape` verifica se diferentes arrays possuem a mesma forma.

Sintaxe

```
1      _check_same_shape(*arrays)
```

Essa validação é essencial para gráficos em que cada valor de dado deve corresponder a uma coordenada.

16.1.4 `_get_fig_ax`

A função `_get_fig_ax` cria ou reutiliza objetos de figura e eixo do Matplotlib.

Sintaxe

```
1      _get_fig_ax(ax=None, figsize=(7, 5), projection=None)
```

Saída: Retorna `fig` e `ax`.

Essa função permite que as rotinas de plotagem sejam usadas tanto de forma independente quanto dentro de figuras já existentes.

16.1.5 `_format_axis`

A função `_format_axis` aplica rótulos, título, grade e proporção ao eixo.

Sintaxe

```
1      _format_axis(ax, xlabel=None, ylabel=None,  
2      title=None, grid=True, equal=False)
```

Essa função padroniza a aparência dos gráficos.

16.1.6 `_reshape_grid`

A função `_reshape_grid` reorganiza dados vetoriais em formato de grade.

Sintaxe

```
1      _reshape_grid(x, y, data, shape=None)
```

Entrada: `x`, `y` e `data` podem estar em forma vetorial ou matricial.

Saída: Retorna arrays compatíveis para plotagem em grade.

16.2 Colormaps, normalização e barra de cores

16.2.1 `my_get_colormap`

A função `my_get_colormap` retorna um colormap do Matplotlib a partir de seu nome.

Sintaxe

```
1      my_get_colormap(name="viridis")
```

Entrada: `name` é o nome do mapa de cores.

Saída: Retorna o objeto de colormap.

16.2.2 `my_get_norm`

A função `my_get_norm` cria uma normalização para escala de cores.

Sintaxe

```
1 my_get_norm(data=None, vmin=None, vmax=None,  
2 symmetric=False, log=False)
```

Entrada: `data` é o conjunto de dados; `vmin` e `vmax` definem limites; `symmetric` força escala simétrica em torno de zero; `log` ativa escala logarítmica.

Saída: Retorna um objeto de normalização.

Quando `symmetric=True`, o limite é definido por

$$v = \max(|d_{min}|, |d_{max}|), \quad (456)$$

e a escala fica entre $-v$ e v .

16.2.3 my_add_colorbar

A função `my_add_colorbar` adiciona uma barra de cores a um gráfico.

Sintaxe

```
1 my_add_colorbar(fig, ax, mappable, label=None,  
2 orientation="vertical", shrink=1.0, pad=0.05)
```

Saída: Retorna o objeto colorbar.

16.3 Desenho de formas geométricas

O módulo inclui funções para desenhar retângulos, círculos e polígonos sobre mapas ou gráficos.

Funções de desenho

```
1 my_draw_rectangle(ax, area, **kwargs)  
2 my_draw_circle(ax, center, radius, **kwargs)  
3 my_draw_polygon(ax, x, y, **kwargs)
```

Essas funções são úteis para destacar áreas de estudo, limites de modelos, contornos de corpos sintéticos ou regiões de interesse.

Um retângulo é definido por

$$[x_1, x_2, y_1, y_2], \quad (457)$$

um círculo por

$$(x - x_c)^2 + (y - y_c)^2 = R^2, \quad (458)$$

e um polígono por uma sequência de vértices.

16.4 Gráficos básicos

16.4.1 `my_plot`

A função `my_plot` cria um gráfico de linha.

Sintaxe

```
1 my_plot(x, y, ax=None, figsize=(7, 5),  
2 xlabel=None, ylabel=None, title=None,  
3 grid=True, **kwargs)
```

Entrada: `x` e `y` são os dados do gráfico.

Saída: Retorna `fig` e `ax`.

16.4.2 `my_scatter`

A função `my_scatter` cria gráfico de dispersão.

Sintaxe

```
1 my_scatter(x, y, c=None, ax=None, figsize=(7, 5),  
2 xlabel=None, ylabel=None, title=None,
```

```
3         colorbar_label=None, cmap="viridis",  
4         s=20, grid=True, **kwargs)
```

Entrada: *c* pode representar uma variável usada para colorir os pontos.

16.4.3 my_histogram

A função `my_histogram` cria um histograma.

Sintaxe

```
1     my_histogram(data, bins=30, ax=None, figsize=(7, 5),  
2     xlabel=None, ylabel="Frequency",  
3     title=None, grid=True, **kwargs)
```

O histograma estima a distribuição de frequências de um conjunto de dados. É útil para análise de resíduos, erros, densidades estimadas, amplitudes de anomalias e ruído.

16.5 Mapas em grade regular

16.5.1 my_contourf

A função `my_contourf` cria mapa de contorno preenchido.

Sintaxe

```
1     my_contourf(x, y, data, shape=None, levels=30,  
2     ax=None, figsize=(7, 6), cmap="viridis",  
3     colorbar_label=None, xlabel="X", ylabel="Y",  
4     title=None, equal=True, **kwargs)
```

Saída: Retorna figura, eixo e objeto mapeável.

Esse tipo de gráfico representa um campo escalar $d(x, y)$ por regiões coloridas.

16.5.2 my_contour

A função `my_contour` cria linhas de contorno.

Sintaxe

```
1 my_contour(x, y, data, shape=None, levels=15,  
2 ax=None, figsize=(7, 6), colors="k",  
3 xlabel="X", ylabel="Y", title=None,  
4 equal=True, clabel=True, **kwargs)
```

As linhas de contorno representam curvas de nível:

$$d(x, y) = c. \quad (459)$$

16.5.3 my_pcolormesh

A função `my_pcolormesh` cria mapa por células coloridas.

Sintaxe

```
1 my_pcolormesh(x, y, data, shape=None, ax=None,  
2 figsize=(7, 6), cmap="viridis",  
3 colorbar_label=None, xlabel="X",  
4 ylabel="Y", title=None, equal=True,  
5 shading="auto", **kwargs)
```

Esse tipo de visualização é adequado quando se deseja mostrar os valores diretamente associados às células da grade.

16.6 Mapas com triangulação irregular

16.6.1 my_tricontourf e my_tricontour

As funções `my_tricontourf` e `my_tricontour` são usadas para dados irregularmente distribuídos.

Sintaxe

```
1 my_tricontourf(x, y, data, levels=30, ax=None,
2               figsize=(7, 6), cmap="viridis",
3               colorbar_label=None, xlabel="X",
4               ylabel="Y", title=None, equal=True, **kwargs)
5
6 my_tricontour(x, y, data, levels=15, ax=None,
7               figsize=(7, 6), colors="k",
8               xlabel="X", ylabel="Y", title=None,
9               equal=True, clabel=True, **kwargs)
```

Essas funções utilizam triangulação para representar campos em pontos não organizados em grade regular.

Dados irregulares

Em levantamentos geofísicos reais, os pontos nem sempre estão em grade regular. As funções de triangulação permitem visualizar campos diretamente a partir de amostras irregulares, evitando interpolação prévia em alguns casos.

16.7 Funções cartográficas com Cartopy

O módulo inclui funções para plotagem geográfica usando **Cartopy**. Essas funções são úteis quando as coordenadas estão em latitude e longitude.

16.7.1 `_import_cartopy`

A função `_import_cartopy` tenta importar o **Cartopy**.

Sintaxe

```
1 _import_cartopy()
```

Caso o pacote não esteja instalado, a função gera uma mensagem informativa. Essa estratégia evita que todo o módulo falhe quando apenas as funções cartográficas dependem dessa biblioteca.

16.7.2 `_infer_area`

A função `_infer_area` estima os limites geográficos a partir de longitude e latitude.

$$\text{area} = [\lambda_{\min}, \lambda_{\max}, \varphi_{\min}, \varphi_{\max}]. \quad (460)$$

16.7.3 `my_cartopy_base`

A função `my_cartopy_base` cria um mapa base com projeção cartográfica.

Sintaxe

```
1         my_cartopy_base(area=None, projection="PlateCarree",
2           ",
3           ax=None, figsize=(8, 6),
4           land=True, ocean=True, borders=True,
           coastlines=True, gridlines=True)
```

Saída: Retorna `fig`, `ax` e o objeto de projeção.

16.7.4 `my_cartopy_scatter`, `my_cartopy_tricontourf` e `my_cartopy_tricontour`

Essas funções geram mapas geográficos de pontos ou contornos triangulados.

Funções cartográficas

```
1         my_cartopy_scatter(lon, lat, data=None, area=None,
2           ...)
3
4         my_cartopy_tricontourf(lon, lat, data, area=None,
5           levels=30, cmap="viridis", ...)
6
7         my_cartopy_tricontour(lon, lat, data, area=None,
```

```
7 levels=15, colors="k", ...)
```

Essas funções usam coordenadas geográficas λ, φ e são adequadas para mapas regionais ou continentais.

16.8 Salvamento de figuras

16.8.1 my_save_figure

A função `my_save_figure` salva uma figura em arquivo.

Sintaxe

```
1 my_save_figure(filename, fig=None, dpi=300,  
2 bbox_inches="tight", transparent=False)
```

Entrada: `filename` é o caminho de saída; `fig` é a figura; `dpi` define a resolução.

Saída: Retorna o nome do arquivo salvo.

Essa função é importante para padronizar a exportação de figuras em relatórios, artigos e repositórios.

16.9 Exemplo conceitual de uso

Exemplo de uso em Python

```
1 import numpy as np  
2 from gravmag_codes import plotting  
3  
4 # Grade sintetica  
5 x = np.linspace(0, 10000, 101)  
6 y = np.linspace(0, 8000, 81)  
7 X, Y = np.meshgrid(x, y)  
8  
9 # Campo sintetico
```



```
10     D = np.exp(-((X-5000)**2 + (Y-4000)**2)/(2*1000**2))
11
12     # Mapa de contorno preenchido
13     fig, ax, mappable = plotting.my_contourf(
14     X, Y, D,
15     levels=40,
16     cmap="viridis",
17     colorbar_label="Amplitude",
18     xlabel="X (m)",
19     ylabel="Y (m)",
20     title="Campo sintetico"
21     )
22
23     # Salvamento
24     plotting.my_save_figure("campo_sintetico.png", fig=
        fig)
```

16.10 Síntese técnica do módulo

O módulo **plotting.py** fornece uma camada padronizada de visualização para o pacote **gravmag_codes**. Ele simplifica a criação de gráficos, mapas, histogramas, contornos, mapas triangulados, mapas cartográficos e figuras para publicação.

Importância para ensino de Geofísica

O **plotting.py** é essencial para atividades didáticas, pois permite transformar resultados numéricos em representações visuais. Isso facilita a interpretação de anomalias, campos, resíduos, modelos sintéticos e produtos de processamento.

Do ponto de vista científico, o módulo contribui para reprodutibilidade e padronização gráfica, permitindo gerar figuras consistentes para relatórios, artigos, apresentações e documentação do pacote.

17 Descrição técnica do módulo `polygonalprism.py`

O arquivo `polygonalprism.py` reúne funções para criação, discretização e modelagem gravimétrica e magnética de prismas verticais com base poligonal arbitrária. Esse módulo amplia significativamente a flexibilidade geométrica do pacote `gravmag_codes`, pois permite representar fontes cujos contornos horizontais não são necessariamente retangulares, circulares ou regulares.

Em Geofísica, corpos geológicos raramente apresentam geometria perfeitamente retangular ou circular. Intrusões, bacias, lentes mineralizadas, corpos sedimentares, zonas alteradas e estruturas crustais podem apresentar formas laterais irregulares. O uso de prismas poligonais permite aproximar essas geometrias por contornos definidos por vértices, mantendo uma estrutura vertical simples, com topo e base constantes.

Papel do módulo `polygonalprism.py`

O módulo `polygonalprism.py` permite criar corpos tridimensionais verticais cuja base é um polígono arbitrário. A resposta gravimétrica e magnética é calculada por discretização numérica do volume, aproximando a integral física por um somatório de pequenas células internas ao polígono.

A abordagem implementada é numérica. O prisma poligonal é discretizado em pequenas células de volume, e cada célula contribui para o campo observado como uma pequena massa ou dipolo magnético. Essa estratégia torna o módulo flexível e adequado para ensino, testes sintéticos e representação aproximada de corpos geológicos irregulares.

17.1 Formulação geométrica do prisma poligonal

Um prisma poligonal vertical é definido por um conjunto de vértices no plano horizontal,

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N), \quad (461)$$

e por dois níveis verticais,

$$z_{top} \quad \text{e} \quad z_{bottom}. \quad (462)$$

A condição geométrica adotada é

$$z_{bottom} > z_{top}. \quad (463)$$

A altura do prisma é

$$H = z_{bottom} - z_{top}. \quad (464)$$

A área da base poligonal pode ser calculada pela fórmula do polígono, também conhecida como fórmula do cadarço:

$$A = \frac{1}{2} \left| \sum_{i=1}^N (x_i y_{i+1} - x_{i+1} y_i) \right|, \quad (465)$$

com a convenção

$$x_{N+1} = x_1, \quad y_{N+1} = y_1. \quad (466)$$

O volume do prisma é

$$V = AH. \quad (467)$$

17.2 Funções internas de suporte

O módulo contém funções internas para preparar coordenadas, validar pontos de observação, recuperar propriedades físicas e calcular cossenos diretores.

17.2.1 `_as_array_like_z`

A função `_as_array_like_z` prepara a coordenada vertical dos pontos de observação.

Sintaxe

```
1  _as_array_like_z(z, shape)
```

Entrada: `z` pode ser escalar ou array; `shape` define a forma esperada.

Saída: Retorna um array **Z** compatível com a forma dos pontos de observação.

Quando z é escalar, todos os pontos de observação são considerados no mesmo nível:

$$z_i = z_0. \quad (468)$$

17.2.2 `_check_observation_points`

A função `_check_observation_points` verifica se as coordenadas dos pontos de observação possuem formas compatíveis.

Sintaxe

```
1      _check_observation_points(x, y, z)
```

Entrada: `x`, `y` e `z` são coordenadas dos pontos de observação.

Saída: Retorna os arrays preparados e validados.

A condição necessária é

$$\text{shape}(x) = \text{shape}(y) = \text{shape}(z). \quad (469)$$

17.2.3 `_get_physical_property`

A função `_get_physical_property` recupera a propriedade física usada na modelagem, como densidade ou magnetização.

Sintaxe

```
1      _get_physical_property(model, value, default=None,  
2      name="physical property")
```

Entrada: `model` é o prisma poligonal; `value` é a propriedade fornecida diretamente; `default` é um valor padrão opcional.

Saída: Retorna a propriedade física selecionada.

Essa função permite que a propriedade seja armazenada no modelo ou informada diretamente na função de cálculo.

17.2.4 `_direction_cosines`

A função `_direction_cosines` calcula o vetor unitário definido por inclinação e declinação.

$$f_x = \cos I \cos D, \quad f_y = \cos I \sin D, \quad f_z = \sin I. \quad (470)$$

Essa função é usada em cálculos magnéticos, especialmente na projeção da anomalia vetorial sobre a direção do campo geomagnético regional.

17.3 Criação do modelo e propriedades geométricas

17.3.1 `my_polygonal_prism_model`

A função `my_polygonal_prism_model` cria um modelo de prisma poligonal vertical.

Sintaxe

```
1 my_polygonal_prism_model(vertices, top, bottom,  
2 physical_property=None)
```

Entrada: `vertices` é uma sequência de pares (x, y) ; `top` é a profundidade do topo; `bottom` é a profundidade da base; `physical_property` é uma propriedade física opcional.

Saída: Retorna uma estrutura representando o prisma poligonal.

A função verifica se o polígono possui pelo menos três vértices e se

$$z_{bottom} > z_{top}. \quad (471)$$

17.3.2 `my_polygonal_area`

A função `my_polygonal_area` calcula a área da base poligonal.

Sintaxe

```
1 my_polygonal_area(polyprism)
```

Saída: Retorna a área da base.

A formulação é

$$A = \frac{1}{2} \left| \sum_{i=1}^N (x_i y_{i+1} - x_{i+1} y_i) \right|. \quad (472)$$

17.3.3 my_polygonal_centroid

A função `my_polygonal_centroid` calcula o centroide da base poligonal.

Sintaxe

```
1 my_polygonal_centroid(polyprism)
```

Saída: Retorna as coordenadas do centroide x_c, y_c .

Para um polígono simples, o centroide pode ser expresso por

$$C_x = \frac{1}{6A} \sum_{i=1}^N (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i), \quad (473)$$

$$C_y = \frac{1}{6A} \sum_{i=1}^N (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i). \quad (474)$$

17.3.4 my_polygonal_bounds, my_polygonal_volume e my_polygonal_center

A função `my_polygonal_bounds` retorna os limites horizontais do polígono:

$$[x_{min}, x_{max}, y_{min}, y_{max}]. \quad (475)$$

A função `my_polygonal_volume` calcula

$$V = A(z_{bottom} - z_{top}). \quad (476)$$

A função `my_polygonal_center` retorna o centro tridimensional aproximado do prisma:

$$(x_c, y_c, z_c), \quad (477)$$

com

$$z_c = \frac{z_{top} + z_{bottom}}{2}. \quad (478)$$

17.4 Discretização do prisma poligonal

17.4.1 `my_point_in_polygon`

A função `my_point_in_polygon` verifica se pontos estão dentro de um polígono.

Sintaxe

```
1 my_point_in_polygon(x, y, xv, yv)
```

Entrada: `x` e `y` são os pontos testados; `xv` e `yv` são os vértices do polígono.

Saída: Retorna uma máscara booleana.

Essa função é usada para selecionar células localizadas no interior do corpo.

17.4.2 `my_discretize_polygonal_prism`

A função `my_discretize_polygonal_prism` discretiza o volume do prisma em pequenas células.

Sintaxe

```
1 my_discretize_polygonal_prism(polyprism, spacing=None  
2                               ,  
                               shape=None, nz=1)
```

Entrada: `polyprism` é o modelo poligonal; `spacing` define o espaçamento horizontal; `shape` define o número de células horizontais; `nz` define o número de camadas verticais.

Saída: Retorna arrays com coordenadas dos centros das células e volumes.

A integral de volume é aproximada por

$$\int_{\Omega} f(\mathbf{r}') dV' \approx \sum_{j=1}^N f(\mathbf{r}'_j) \Delta V_j. \quad (479)$$

Discretização de corpos irregulares

A discretização de prismas poligonais permite representar corpos com contornos laterais arbitrários. Essa abordagem é útil para modelar fontes com geometria mais realista do que prismas retangulares ou cilindros.

17.5 Modelagem gravimétrica

A modelagem gravimétrica do prisma poligonal é realizada por soma das contribuições das células discretizadas. Para cada célula j , com volume ΔV_j , a massa elementar é

$$\Delta m_j = \rho \Delta V_j. \quad (480)$$

A distância entre o ponto de observação e a célula é

$$r_j = \sqrt{(x - x_j)^2 + (y - y_j)^2 + (z - z_j)^2 + \varepsilon^2}. \quad (481)$$

17.5.1 `my_polygonal_prism_potential`

A função `my_polygonal_prism_potential` calcula o potencial gravitacional.

Sintaxe

```

1      my_polygonal_prism_potential(x, y, z, polyprism,
2      rho=None, spacing=None,
3      shape=None, nz=1)
```


Saída: Retorna o potencial gravitacional em unidades SI.

A formulação aproximada é

$$V(x, y, z) = G\rho \sum_{j=1}^N \frac{\Delta V_j}{r_j}. \quad (482)$$

17.5.2 `my_polygonal_prism_gx`, `my_polygonal_prism_gy` e `my_polygonal_prism_gz`

Essas funções calculam as componentes gravimétricas g_x , g_y e g_z .

Sintaxe

```

1      my_polygonal_prism_gx(x, y, z, polyprism, rho=None,
      ...
2      my_polygonal_prism_gy(x, y, z, polyprism, rho=None,
      ...
3      my_polygonal_prism_gz(x, y, z, polyprism, rho=None,
      ...)
```

As componentes são aproximadas por

$$g_x = G\rho \sum_{j=1}^N \Delta V_j \frac{x_j - x}{r_j^3}, \quad (483)$$

$$g_y = G\rho \sum_{j=1}^N \Delta V_j \frac{y_j - y}{r_j^3}, \quad (484)$$

$$g_z = G\rho \sum_{j=1}^N \Delta V_j \frac{z_j - z}{r_j^3}. \quad (485)$$

As respostas são convertidas para mGal.

17.5.3 `my_polygonal_prism_gxyz`

A função `my_polygonal_prism_gxyz` calcula simultaneamente as componentes gravimétricas.

Sintaxe

```

1      my_polygonal_prism_gxyz(x, y, z, polyprism,
2      rho=None, spacing=None,
3      shape=None, nz=1)

```

Saída: Retorna gx , gy e gz , em mGal.

17.6 Modelagem magnética

Na modelagem magnética, cada célula do prisma poligonal é tratada como um dipolo magnético elementar. Para uma magnetização $\mathbf{M}$, o momento magnético da célula é

$$\mathbf{m}_j = \mathbf{M} \Delta V_j. \quad (486)$$

O campo de dipolo é dado por

$$\mathbf{B} = \frac{\mu_0}{4\pi} \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right]. \quad (487)$$

17.6.1 my_polygonal_prism_bxyz

A função `my_polygonal_prism_bxyz` calcula as componentes magnéticas B_x , B_y e B_z .

Sintaxe

```

1      my_polygonal_prism_bxyz(x, y, z, polyprism,
2      mag=None, incs=90.0, decs=0.0,
3      spacing=None, shape=None, nz=1)

```

Entrada: `mag` é a intensidade de magnetização em A/m; `incs` e `decs` definem sua direção.

Saída: Retorna B_x , B_y e B_z , em nT.

17.6.2 `my_polygonal_prism_bx`, `my_polygonal_prism_by` e `my_polygonal_prism_bz`

Essas funções retornam individualmente as componentes magnéticas.

Sintaxe

```

1      my_polygonal_prism_bx(x, y, z, polyprism, mag=None,
2      ...)
3      my_polygonal_prism_by(x, y, z, polyprism, mag=None,
4      ...)
5      my_polygonal_prism_bz(x, y, z, polyprism, mag=None,
6      ...)
```

Saída: Retornam, respectivamente, B_x , B_y ou B_z , em nT.

17.6.3 `my_polygonal_prism_tfa` e `my_polygonal_prism_tf`

A função `my_polygonal_prism_tfa` calcula a anomalia magnética de campo total. A função `my_polygonal_prism_tf` é um alias de compatibilidade.

Sintaxe

```

1      my_polygonal_prism_tfa(x, y, z, polyprism,
2      mag=None, inc=-20.0, dec=0.0,
3      incs=None, decs=None,
4      spacing=None, shape=None, nz=1)
```

A anomalia de campo total é obtida pela projeção

$$\Delta T = B_x f_x + B_y f_y + B_z f_z, \quad (488)$$

em que f_x, f_y, f_z é a direção unitária do campo regional.

17.7 Modelagem de múltiplos prismas poligonais

O módulo contém funções para calcular a resposta total de uma lista de prismas poligonais:

Funções para múltiplos prismas poligonais

```
1 my_polygonal_prisms_potential(...)
2 my_polygonal_prisms_gx(...)
3 my_polygonal_prisms_gy(...)
4 my_polygonal_prisms_gz(...)
5 my_polygonal_prisms_gxyz(...)
6 my_polygonal_prisms_bxyz(...)
7 my_polygonal_prisms_tfa(...)
8 my_polygonal_prisms_tf(...)
```

A superposição linear é dada por

$$d_{total} = \sum_{k=1}^{N_p} d_k. \quad (489)$$

17.8 Conversão para prismas retangulares

17.8.1 my_polygonal_prism_to_prisms

A função `my_polygonal_prism_to_prisms` aproxima o prisma poligonal por uma lista de prismas retangulares.

Sintaxe

```
1 my_polygonal_prism_to_prisms(polyprism, spacing=None,
2 shape=None, physical_property=None,
3 include_boundary=True)
```

Saída: Retorna uma lista de prismas retangulares que aproximam o volume do prisma poligonal.

Essa função é útil para integração com rotinas baseadas em prismas retangulares e para comparar diferentes estratégias de modelagem.

17.9 Síntese técnica do módulo

O módulo `polygonalprism.py` permite representar fontes geofísicas verticais com contornos horizontais arbitrários. Essa flexibilidade é importante para aproximar corpos geológicos de geometria irregular e para construir modelos sintéticos mais realistas.

Importância para ensino de Geofísica

O `polygonalprism.py` permite demonstrar como a geometria da fonte influencia a resposta gravimétrica e magnética. O módulo também mostra como integrais de volume podem ser aproximadas por discretização numérica, conectando Geofísica, Geometria Computacional e Métodos Numéricos.

18 Descrição técnica do módulo `prism.py`

O arquivo `prism.py` reúne funções para modelagem gravimétrica e magnética de prismas retangulares. Esse é um dos módulos mais importantes do pacote *gravmag_codes*, pois o prisma retangular é uma das geometrias fundamentais em modelagem de campos potenciais. Por meio da combinação de vários prismas, é possível representar corpos geológicos complexos, modelos de densidade em grade, discretizações crustais e modelos tridimensionais do subsolo.

Em métodos potenciais, prismas retangulares são amplamente usados porque permitem discretizar o volume do subsolo em células regulares. Cada célula possui limites definidos em x , y e z , além de propriedades físicas como densidade ou magnetização. A resposta total é obtida pela soma das contribuições individuais dos prismas.

Papel do módulo `prism.py`

O módulo `prism.py` implementa rotinas para criar prismas retangulares, calcular volume, centro, potencial gravitacional, componentes gravimétricas, tensor de gradiente, componentes magnéticas e anomalia magnética de campo total. Também permite calcular a resposta de conjuntos de prismas por superposição linear.

A geometria de um prisma retangular é definida por seus limites:

$$x_1, x_2, y_1, y_2, z_1, z_2, \quad (490)$$

em que $x_1 < x_2$, $y_1 < y_2$ e $z_1 < z_2$. A convenção adotada é que o eixo z é positivo para baixo, usual em modelagem geofísica.

18.1 Formulação geométrica do prisma retangular

O volume de um prisma retangular é

$$V = (x_2 - x_1)(y_2 - y_1)(z_2 - z_1). \quad (491)$$

O centro geométrico é

$$x_c = \frac{x_1 + x_2}{2}, \quad y_c = \frac{y_1 + y_2}{2}, \quad z_c = \frac{z_1 + z_2}{2}. \quad (492)$$

Em modelagem direta, a resposta do prisma em um ponto de observação é obtida por integração volumétrica. Para o potencial gravitacional, por exemplo,

$$V(\mathbf{r}) = G\rho \int_{x_1}^{x_2} \int_{y_1}^{y_2} \int_{z_1}^{z_2} \frac{1}{R} dz' dy' dx', \quad (493)$$

em que

$$R = \sqrt{(x - x')^2 + (y - y')^2 + (z - z')^2}. \quad (494)$$

As soluções analíticas para prismas retangulares envolvem somatórios avaliados nos oito vértices do prisma, com termos logarítmicos e arctangentes.

Importância do prisma retangular

O prisma retangular é uma geometria essencial em modelagem geofísica porque permite discretizar o subsolo em células. Essa discretização é a base de muitos algoritmos de modelagem direta e inversão 3D.

18.2 Funções internas de suporte

O módulo contém funções internas para validação, recuperação de propriedades físicas, cálculo de cossenos diretores e avaliação de termos logarítmicos e arctangentes de forma numericamente estável.

18.2.1 `_check_prism`

A função `_check_prism` verifica se os limites do prisma são geometricamente válidos.

Sintaxe

```
1 _check_prism(prism)
```

A condição esperada é

$$x_2 > x_1, \quad y_2 > y_1, \quad z_2 > z_1. \quad (495)$$

18.2.2 `_check_observation_arrays`

A função `_check_observation_arrays` valida as coordenadas de observação.

Sintaxe

```
1 _check_observation_arrays(x, y, z)
```

As formas de x , y e z devem ser compatíveis para cálculo ponto a ponto.

18.2.3 `_get_property`

A função `_get_property` recupera a densidade ou magnetização informada diretamente ou armazenada no modelo do prisma.

Sintaxe

```
1         _get_property(prism, value, index, default=None,
2         name="physical property")
```

Essa função permite flexibilidade na chamada das rotinas, mantendo compatibilidade com modelos que já armazenam propriedade física.

18.2.4 `_log` e `_atan2`

As funções `_log` e `_atan2` tratam termos logarítmicos e arctangentes usados nas fórmulas analíticas do prisma.

Os termos típicos são

$$\ln(y + R), \quad \ln(x + R), \quad \arctan\left(\frac{xy}{zR}\right). \quad (496)$$

Esses termos exigem cuidado numérico porque seus argumentos podem se aproximar de zero ou mudar de quadrante.

18.3 Criação, volume e centro do prisma

18.3.1 `my_prism_model`

A função `my_prism_model` cria um prisma retangular.

Sintaxe

```
1         my_prism_model(x1, x2, y1, y2, z1, z2,
2         physical_property=None)
```

Entrada: `x1`, `x2`, `y1`, `y2`, `z1` e `z2` definem os limites do prisma. `physical_property` é uma propriedade física opcional.

Saída: Retorna uma lista representando o prisma.

18.3.2 `my_prism_volume`

A função `my_prism_volume` calcula o volume do prisma.

Sintaxe

```
1 my_prism_volume(prism)
```

Saída: Retorna

$$V = (x_2 - x_1)(y_2 - y_1)(z_2 - z_1). \quad (497)$$

18.3.3 my_prism_center

A função `my_prism_center` calcula o centro geométrico do prisma.

Sintaxe

```
1 my_prism_center(prism)
```

Saída: Retorna x_c, y_c, z_c , com

$$x_c = \frac{x_1 + x_2}{2}, \quad y_c = \frac{y_1 + y_2}{2}, \quad z_c = \frac{z_1 + z_2}{2}. \quad (498)$$

18.4 Modelagem gravimétrica de um prisma

A resposta gravimétrica do prisma é calculada por fórmulas analíticas avaliadas nos oito vértices. De forma genérica, muitos termos podem ser escritos como

$$\sum_{i=1}^2 \sum_{j=1}^2 \sum_{k=1}^2 (-1)^{i+j+k} F(x_i, y_j, z_k), \quad (499)$$

em que F contém combinações de logaritmos, arctangentes e produtos de coordenadas relativas.

18.4.1 my_prism_potential

A função `my_prism_potential` calcula o potencial gravitacional produzido por um prisma.

Sintaxe

```
1 my_prism_potential(x, y, z, prism, rho=None)
```

Entrada: *x*, *y* e *z* são pontos de observação; *prism* é o modelo; *rho* é a densidade em g/cm³.

Saída: Retorna o potencial em unidades SI.

A formulação conceitual é

$$V = G\rho \iiint_{\Omega} \frac{1}{R} dV. \quad (500)$$

18.4.2 my_prism_gx, my_prism_gy e my_prism_gz

Essas funções calculam as componentes gravimétricas do prisma.

Sintaxe

```
1 my_prism_gx(x, y, z, prism, rho=None)
2 my_prism_gy(x, y, z, prism, rho=None)
3 my_prism_gz(x, y, z, prism, rho=None)
```

As componentes são derivadas do potencial:

$$g_x = \frac{\partial V}{\partial x}, \quad g_y = \frac{\partial V}{\partial y}, \quad g_z = \frac{\partial V}{\partial z}. \quad (501)$$

As saídas são expressas em mGal.

18.4.3 my_prism_gxyz

A função *my_prism_gxyz* calcula simultaneamente *g_x*, *g_y* e *g_z*.

Sintaxe

```
1 my_prism_gxyz(x, y, z, prism, rho=None)
```

Saída: Retorna as três componentes em mGal.

18.5 Tensor de gradiente gravimétrico

O tensor de gradiente gravimétrico é formado pelas derivadas espaciais das componentes do campo:

$$\mathbf{T} = \begin{bmatrix} g_{xx} & g_{xy} & g_{xz} \\ g_{yx} & g_{yy} & g_{yz} \\ g_{zx} & g_{zy} & g_{zz} \end{bmatrix}. \quad (502)$$

Como o tensor é simétrico em campos potenciais,

$$g_{xy} = g_{yx}, \quad g_{xz} = g_{zx}, \quad g_{yz} = g_{zy}. \quad (503)$$

18.5.1 my_prism_gxx, my_prism_gxy, my_prism_gxz, my_prism_gyy, my_prism_gyz e my_prism_gzz

Essas funções calculam as componentes independentes do tensor de gradiente gravimétrico.

Sintaxe

```
1 my_prism_gxx(x, y, z, prism, rho=None)
2 my_prism_gxy(x, y, z, prism, rho=None)
3 my_prism_gxz(x, y, z, prism, rho=None)
4 my_prism_gyy(x, y, z, prism, rho=None)
5 my_prism_gyz(x, y, z, prism, rho=None)
6 my_prism_gzz(x, y, z, prism, rho=None)
```

A unidade típica dessas componentes é o Eötvös, quando a conversão apropriada é aplicada:

$$1 \text{ E} = 10^{-9} \text{ s}^{-2}. \quad (504)$$

18.6 Modelagem magnética de um prisma

A modelagem magnética do prisma utiliza a relação entre magnetização, tensor de derivadas do potencial e projeção na direção do campo regional. A magnetização é definida por intensidade M , inclinação I_s e declinação D_s .

O vetor unitário de magnetização é

$$\hat{\mathbf{M}} = (m_x, m_y, m_z), \quad (505)$$

com

$$m_x = \cos I_s \cos D_s, \quad m_y = \cos I_s \sin D_s, \quad m_z = \sin I_s. \quad (506)$$

18.6.1 `my_prism_bxyz`

A função `my_prism_bxyz` calcula as componentes magnéticas B_x , B_y e B_z .

Sintaxe

```
1 my_prism_bxyz(x, y, z, prism, mag=None,
2 incs=90.0, decs=0.0)
```

Entrada: `mag` é a intensidade da magnetização; `incs` e `decs` são a inclinação e declinação da magnetização.

Saída: Retorna B_x , B_y e B_z , em nT.

18.6.2 `my_prism_bx`, `my_prism_by` e `my_prism_bz`

Essas funções retornam individualmente as componentes magnéticas.

Sintaxe

```
1 my_prism_bx(x, y, z, prism, mag=None, incs=90.0, decs
   =0.0)
2 my_prism_by(x, y, z, prism, mag=None, incs=90.0, decs
   =0.0)
```

```

3      my_prism_bz(x, y, z, prism, mag=None, incs=90.0, decs
      =0.0)

```

18.6.3 my_prism_tfa e my_prism_tf

A função `my_prism_tfa` calcula a anomalia magnética de campo total. A função `my_prism_tf` é um alias de compatibilidade.

Sintaxe

```

1      my_prism_tfa(x, y, z, prism, mag=None,
2      inc=-20.0, dec=0.0,
3      incs=None, decs=None)

```

A anomalia de campo total é dada por

$$\Delta T = B_x f_x + B_y f_y + B_z f_z, \quad (507)$$

em que $\hat{\mathbf{F}} = (f_x, f_y, f_z)$ é a direção do campo regional.

Quando `incs=None` e `decs=None`, assume-se magnetização induzida na direção do campo regional.

18.7 Modelagem de múltiplos prismas

O módulo contém funções para calcular a resposta de conjuntos de prismas:

Funções para múltiplos prismas

```

1      my_prisms_potential(...)
2      my_prisms_gx(...)
3      my_prisms_gy(...)
4      my_prisms_gz(...)
5      my_prisms_gxyz(...)
6      my_prisms_gxx(...)
7      my_prisms_gxy(...)

```

```

8         my_prisms_gxz(...)
9         my_prisms_gyy(...)
10        my_prisms_gyz(...)
11        my_prisms_gzz(...)
12        my_prisms_bxyz(...)
13        my_prisms_tfa(...)
14        my_prisms_tf(...)

```

A resposta total é obtida por superposição linear:

$$d_{total} = \sum_{k=1}^{N_p} d_k. \quad (508)$$

Essa propriedade decorre da linearidade dos campos potenciais em relação à densidade e à magnetização.

18.8 Aliases de compatibilidade

O módulo define aliases para preservar nomes antigos e simplificar chamadas:

Aliases principais

```

1         prism_model = my_prism_model
2         prism_volume = my_prism_volume
3         prism_center = my_prism_center
4         prism_gz = my_prism_gz
5         prism_tf = my_prism_tf
6         prisms_gz = my_prisms_gz
7         prisms_tf = my_prisms_tf

```

Compatibilidade

A preservação dos aliases permite que notebooks antigos continuem funcionando mesmo após reorganização e documentação do pacote.

18.9 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import prism
3
4      # Malha de observacao
5      x = np.linspace(-5000, 5000, 101)
6      y = np.linspace(-5000, 5000, 101)
7      X, Y = np.meshgrid(x, y)
8      Z = np.zeros_like(X)
9
10     # Prisma com contraste de densidade
11     p = prism.my_prism_model(
12         x1=-1000, x2=1000,
13         y1=-1000, y2=1000,
14         z1=1000, z2=3000,
15         physical_property=0.5
16     )
17
18     # Componente vertical da gravidade
19     gz = prism.my_prism_gz(X, Y, Z, p, rho=None)
20
21     # Anomalia magnetica de campo total
22     tf = prism.my_prism_tfa(
23         X, Y, Z, p,
24         mag=2.0,
25         inc=-20.0,
26         dec=0.0
27     )
```

18.10 Síntese técnica do módulo

O módulo **prism.py** é uma das bases fundamentais do pacote *gravmag_codes*. Ele permite calcular respostas gravimétricas e magnéticas de prismas retangulares individuais ou conjuntos de prismas, incluindo potencial, componentes do campo, tensor de gradiente e anomalia de campo total.

Importância para ensino de Geofísica

O `prism.py` é essencial para ensinar modelagem direta em métodos potenciais. Ele permite mostrar como um corpo com geometria, profundidade e propriedade física definidas gera anomalias observáveis, além de servir como base para discretização 3D e problemas inversos.

Do ponto de vista científico, o módulo pode ser usado em testes sintéticos, modelagem de corpos geológicos, discretização do subsolo, cálculo de matrizes de sensibilidade e desenvolvimento de algoritmos de inversão gravimétrica e magnética.

19 Descrição técnica do módulo `radiometry_welllog_methods.py`

O arquivo `radiometry_welllog_methods.py` reúne funções aplicadas à radiometria gama, análise de razões radiométricas, geração de imagens ternárias, classificação de domínios radiométricos, criação de mapas sintéticos, perfilagem geofísica de poços, cálculo de propriedades petrofísicas e visualização de mapas, histogramas, perfis e crossplots. Esse módulo amplia o escopo do pacote `gravmag_codes` para aplicações integradas entre Geofísica de superfície, Geologia, Petrofísica e interpretação de poços.

A radiometria gama é baseada na medição da radiação natural emitida principalmente pelos radioelementos potássio (*K*), urânio (*U*) e tório (*Th*). Em levantamentos geofísicos, essas variáveis podem indicar variações litológicas, alteração hidrotermal, conteúdo argiloso, intemperismo, fertilidade relativa do solo, domínios geológicos e contrastes geoquímicos superficiais. A perfilagem de poços, por sua vez, utiliza medidas físicas ao longo da profundidade para inferir propriedades como argilosidade, porosidade, saturação, impedância acústica e qualidade de reservatório.

Papel do módulo `radiometry_welllog_methods.py`

O módulo `radiometry_welllog_methods.py` integra ferramentas de radiometria e perfilagem de poço. Ele permite calcular razões entre radioelementos, gerar imagens ternárias RGB, classificar domínios radiométricos, estimar volume de argila, porosidade, saturação de água, impedância acústica, coeficientes de reflexão e indicadores simples de zonas reservatório.

O módulo utiliza `NumPy` para cálculos numéricos, `Matplotlib` para visualização e `os` para salvamento de arquivos e criação de diretórios. Algumas funções podem trabalhar com dados sintéticos ou dados reais organizados em arrays.

19.1 Funções auxiliares internas e salvamento

19.1.1 `_arr`

A função interna `_arr` converte uma entrada para array NumPy do tipo `float`.

Sintaxe

```
1 _arr(a)
```

Entrada: `a` pode ser escalar, lista ou array.

Saída: Retorna `a` como array NumPy.

Essa função padroniza os dados para operações vetorizadas.

19.1.2 `_safe_divide`

A função `_safe_divide` realiza divisão protegida contra zeros e valores não finitos.

Sintaxe

```
1 _safe_divide(a, b, fill_value=0.0)
```

Entrada: `a` é o numerador; `b` é o denominador; `fill_value` é o valor atribuído quando a divisão não é válida.

Saída: Retorna o quociente com substituição de valores indefinidos.

A operação pode ser representada por

$$q_i = \begin{cases} \frac{a_i}{b_i}, & b_i \neq 0, \\ q_0, & b_i = 0, \end{cases} \quad (509)$$

em que q_0 é o valor de preenchimento.

19.1.3 `_normalize_01`

A função `_normalize_01` normaliza um conjunto de dados no intervalo $[0, 1]$.

Sintaxe

```
1      _normalize_01(data)
```

Saída: Retorna os dados normalizados.

A normalização min-max é dada por

$$d_{norm} = \frac{d - d_{min}}{d_{max} - d_{min}}. \quad (510)$$

19.1.4 `my_save_figure`

A função `my_save_figure` salva uma figura gerada pelo Matplotlib.

Sintaxe

```
1      my_save_figure(filename, fig=None, dpi=300,  
2      bbox_inches="tight", transparent=True)
```

Saída: Retorna o nome do arquivo salvo.

19.2 Processamento básico e estatísticas

O módulo contém funções para normalização, remoção de outliers e estatísticas básicas de dados radiométricos ou perfis de poço.

Funções de processamento básico

```

1         my_minmax_normalize(data)
2         my_zscore(data)
3         my_clip_outliers(data, nsigma=3.0)
4         my_basic_statistics(data)

```

A normalização min-max é

$$d_{norm} = \frac{d - d_{min}}{d_{max} - d_{min}}, \quad (511)$$

enquanto o escore padronizado é

$$z_i = \frac{d_i - \bar{d}}{\sigma_d}. \quad (512)$$

A remoção ou suavização de outliers por desvio padrão utiliza a condição

$$|d_i - \bar{d}| \leq n\sigma_d. \quad (513)$$

Essas funções são importantes para preparar dados antes da construção de mapas, imagens ternárias, perfis e classificações.

19.3 Radiometria gama

19.3.1 my_total_count

A função `my_total_count` calcula a contagem total radiométrica a partir dos canais de potássio, urânio e tório.

Sintaxe

```

1         my_total_count(K, U, Th)

```

Entrada: K, U e Th representam os canais radiométricos.

Saída: Retorna a contagem total.

A formulação simples é

$$TC = K + U + Th. \quad (514)$$

19.3.2 `my_radiometric_ratios`

A função `my_radiometric_ratios` calcula razões radiométricas entre os canais K , U e Th .

Sintaxe

```
1 my_radiometric_ratios(K, U, Th)
```

Saída: Retorna razões como K/U , K/Th , U/Th , Th/K e outras combinações.

As razões típicas são

$$R_{KU} = \frac{K}{U}, \quad R_{KTh} = \frac{K}{Th}, \quad R_{UTh} = \frac{U}{Th}. \quad (515)$$

Interpretação das razões radiométricas

As razões radiométricas podem realçar alterações geoquímicas, variações litológicas e domínios superficiais que não são evidentes nos canais individuais. Por exemplo, enriquecimento relativo de potássio pode estar associado a processos de alteração hidrotermal em alguns contextos geológicos.

19.3.3 `my_f_parameter`

A função `my_f_parameter` calcula um parâmetro radiométrico composto, frequentemente usado para destacar enriquecimento relativo de potássio em relação a urânio e tório.

Sintaxe

```
1 my_f_parameter(K, U, Th)
```

Uma formulação comum para esse tipo de parâmetro é

$$F = \frac{K \cdot U}{Th}, \quad (516)$$

ou uma razão composta equivalente, dependendo da convenção adotada na implementação. O objetivo é combinar os radioelementos em um único atributo interpretativo.

19.3.4 `my_ternary_rgb`

A função `my_ternary_rgb` cria uma composição ternária RGB a partir dos canais K , U e Th .

Sintaxe

```
1 my_ternary_rgb(K, U, Th, stretch=True)
```

Entrada: K , U e Th são arrays dos canais radiométricos.

Saída: Retorna uma imagem RGB.

A composição ternária pode associar, por exemplo,

$$R \leftarrow K, \quad G \leftarrow Th, \quad B \leftarrow U, \quad (517)$$

após normalização dos canais. O resultado permite visualizar domínios radiométricos pela combinação de cores.

19.3.5 `my_classify_radiometric_domain`

A função `my_classify_radiometric_domain` classifica domínios radiométricos com base em valores relativos dos canais ou razões.

Sintaxe

```
1 my_classify_radiometric_domain(K, U, Th)
```

Saída: Retorna uma classificação ou mapa de classes radiométricas.

A classificação pode ser baseada em relações como

$$K > U, \quad K > Th, \quad Th > K, \quad U > Th, \quad (518)$$

ou em limiares derivados dos próprios dados.

19.4 Mapas radiométricos sintéticos

O módulo inclui funções para criar campos radiométricos sintéticos, úteis em atividades didáticas e testes de visualização.

Funções sintéticas

```
1 my_synthetic_radiometric_map(x, y, anomalies=None,
2 background=True,
3 noise_std=0.0, seed=None)
```

Uma anomalia radiométrica sintética pode ser representada por uma função Gaussiana:

$$A(x, y) = A_0 \exp \left[-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2} \right]. \quad (519)$$

O campo final pode ser expresso como

$$D(x, y) = D_{regional}(x, y) + \sum_{j=1}^N A_j(x, y) + \epsilon(x, y). \quad (520)$$

19.5 Perfilagem de poços e propriedades petrofísicas

O módulo contém funções para estimar propriedades petrofísicas a partir de perfis de poço. Essas funções são úteis para introduzir relações clássicas entre medidas geofísicas e propriedades de rochas.

19.5.1 my_gamma_ray_index e my_vshale_linear

A função `my_gamma_ray_index` calcula o índice de raio gama:

Sintaxe

```
1 my_gamma_ray_index(GR, GR_min, GR_max)
```

A formulação é

$$I_{GR} = \frac{GR - GR_{min}}{GR_{max} - GR_{min}}. \quad (521)$$

A função `my_vshale_linear` estima o volume de argila pela aproximação linear:

$$V_{sh} \approx I_{GR}. \quad (522)$$

19.5.2 my_density_porosity

A função `my_density_porosity` calcula a porosidade por densidade.

Sintaxe

```
1 my_density_porosity(rhob, rho_matrix=2.65,
2 rho_fluid=1.0)
```

Entrada: `rhob` é a densidade bulk; `rho_matrix` é a densidade da matriz; `rho_fluid` é a densidade do fluido.

Saída: Retorna a porosidade estimada.

A formulação é

$$\phi_D = \frac{\rho_{ma} - \rho_b}{\rho_{ma} - \rho_f}. \quad (523)$$

19.5.3 my_sonic_porosity

A função `my_sonic_porosity` estima a porosidade a partir do perfil sônico.

Sintaxe

```

1      my_sonic_porosity(dt, dt_matrix=55.5,
2      dt_fluid=189.0)

```

A formulação de Wyllie pode ser escrita como

$$\phi_S = \frac{\Delta t - \Delta t_{ma}}{\Delta t_f - \Delta t_{ma}}. \quad (524)$$

19.5.4 my_velocity_from_sonic

A função `my_velocity_from_sonic` converte tempo de trânsito sônico em velocidade.

$$v = \frac{1}{\Delta t}, \quad (525)$$

considerando as conversões adequadas de unidade.

19.5.5 my_archie_water_saturation

A função `my_archie_water_saturation` calcula a saturação de água pela equação de Archie.

Sintaxe

```

1      my_archie_water_saturation(Rt, phi, Rw,
2      a=1.0, m=2.0, n=2.0)

```

A formulação é

$$S_w = \left(\frac{aR_w}{\phi^m R_t} \right)^{1/n}. \quad (526)$$

Limitações da equação de Archie

A equação de Archie é mais adequada para formações limpas, com baixa argilosidade. Em formações argilosas, condutividade superficial e argilas podem alterar significativamente a interpretação da resistividade.

19.5.6 `my_acoustic_impedance` e `my_reflection_coefficient`

A função `my_acoustic_impedance` calcula a impedância acústica:

$$Z = \rho v. \quad (527)$$

A função `my_reflection_coefficient` calcula o coeficiente de reflexão entre duas camadas:

$$R = \frac{Z_2 - Z_1}{Z_2 + Z_1}. \quad (528)$$

19.6 Indicadores de reservatório

O módulo contém funções para identificar zonas reservatório e calcular parâmetros simples, como net-to-gross.

Funções de reservatório

```

1      my_reservoir_flag(GR, phi, Sw,
2      GR_cutoff=75.0,
3      phi_cutoff=0.10,
4      Sw_cutoff=0.60)
5
6      my_net_to_gross(flag)
```

Uma zona pode ser classificada como reservatório quando satisfaz simultaneamente condições como

$$GR < GR_{cutoff}, \quad \phi > \phi_{cutoff}, \quad S_w < S_{w,cutoff}. \quad (529)$$

O net-to-gross é dado por

$$NTG = \frac{h_{net}}{h_{gross}}. \quad (530)$$

19.7 Perfis sintéticos de poço

A função `my_synthetic_well_logs` gera perfis sintéticos ao longo da profundidade.

Sintaxe

```
1 my_synthetic_well_logs(depth, seed=None)
```

Saída: Retorna um conjunto de curvas sintéticas, como gamma ray, densidade, porosidade, resistividade ou velocidade.

Esses perfis permitem construir exemplos didáticos para interpretação de poços e integração com dados radiométricos ou sísmicos.

19.8 Funções de visualização

O módulo contém funções para mapas, histogramas, ternários RGB, perfis de poço e crossplots.

Funções de plotagem

```
1 my_plot_map(x, y, data, ...)
2 my_plot_histogram(data, ...)
3 my_plot_ternary_rgb(rgb, ...)
4 my_plot_well_logs(depth, logs, ...)
5 my_crossplot(x, y, c=None, ...)
```

Essas funções retornam objetos `fig` e `ax`, permitindo personalização posterior. Os perfis de poço geralmente são representados com profundidade aumentando para baixo, o que é coerente com a convenção geológica.

19.9 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import radiometry_welllog_methods
3          as rw
4
5      # Dados radiométricos sintéticos
6      x = np.linspace(0, 10000, 101)
7      y = np.linspace(0, 8000, 81)
8      X, Y = np.meshgrid(x, y)
9
10     K = rw.my_synthetic_radiometric_map(X, Y, seed=1)
11     U = rw.my_synthetic_radiometric_map(X, Y, seed=2)
12     Th = rw.my_synthetic_radiometric_map(X, Y, seed=3)
13
14     # Contagem total e imagem ternária
15     TC = rw.my_total_count(K, U, Th)
16     rgb = rw.my_ternary_rgb(K, U, Th)
17
18     # Perfil sintético de poço
19     depth = np.linspace(0, 1000, 501)
20     logs = rw.my_synthetic_well_logs(depth, seed=10)
21
22     # Porosidade por densidade
23     phi_d = rw.my_density_porosity(logs["RHOB"])
```

19.10 Síntese técnica do módulo

O módulo **`radiometry_welllog_methods.py`** fornece ferramentas para análise radiométrica e interpretação petrofísica básica. Ele permite trabalhar com canais *K*, *U* e *Th*, razões radiométricas, imagens ternárias, mapas sintéticos, perfis de poço, porosidade, saturação, impedância acústica e indicadores simples de reservatório.

Importância para ensino de Geofísica

O **`radiometry_welllog_methods.py`** é especialmente útil para aulas que integram Geofísica, Geologia, Petrofísica e interpretação de dados. Ele permite demonstrar como propriedades físicas medidas em superfície ou em poços podem ser convertidas em atributos interpretativos.

Do ponto de vista científico e educacional, o módulo pode apoiar estudos de caracterização de solos, litologias, reservatórios, domínios radiométricos e integração entre dados geofísicos rasos e informações de poço.

20 Descrição técnica do módulo `regres.py`

O arquivo **`regres.py`** reúne funções para regressão linear, regressão polinomial, regressão robusta, mínimos quadrados, mínimos quadrados ponderados, regularização, interpolação de Lagrange, separação regional-residual, cálculo de métricas de ajuste e salvamento de resultados. Esse módulo é fundamental para o pacote **`gravmag_codes`**, pois muitos problemas de processamento geofísico podem ser formulados como ajuste de modelos matemáticos aos dados observados.

Em Geofísica, regressão é frequentemente utilizada para ajuste de tendências, remoção de campos regionais, calibração de dados, modelagem empírica, análise de correlação, interpolação, suavização e construção de modelos aproximados. No contexto dos métodos potenciais, uma aplicação importante é a separação regional-residual, na qual um campo observado é decomposto em uma componente regional suave e uma componente residual associada a fontes locais.

Papel do módulo `regres.py`

O módulo **`regres.py`** fornece ferramentas matemáticas para ajuste de dados, regressão polinomial, regularização e separação regional-residual. Ele permite construir modelos empíricos e avaliar quantitativamente a qualidade do ajuste.

A formulação geral de uma regressão linear pode ser escrita como

$$\mathbf{d} = \mathbf{A}\mathbf{m} + \mathbf{e}, \quad (531)$$

em que **$\mathbf{d}$** é o vetor de dados observados, **$\mathbf{A}$** é a matriz de desenho, **$\mathbf{m}$** é o vetor de coeficientes do modelo e **$\mathbf{e}$** é o vetor de resíduos.

20.1 Funções auxiliares internas

O módulo contém funções internas para conversão de entradas, validação de dimensões, construção de matrizes e cálculo de métricas.

Funções auxiliares internas

```
1      _as_1d(a, name="array")
2      _as_2d(a, name="matrix")
3      _check_xy_data(x, y)
4      _check_design_data(A, d)
5      _safe_pinv(A, rcond=1e-12)
```

Essas funções garantem que vetores e matrizes tenham formatos compatíveis antes da solução dos sistemas de regressão.

20.2 Métricas de ajuste e resíduos

20.2.1 `my_residual`

A função `my_residual` calcula o vetor de resíduos entre dados observados e preditos.

Sintaxe

```
1      my_residual(observed, predicted)
```

A formulação é

$$\mathbf{r} = \mathbf{d}_{obs} - \mathbf{d}_{pred}. \quad (532)$$

20.2.2 `my_sse`, `my_mse`, `my_rmse` e `my_mae`

A soma dos quadrados dos erros é

$$SSE = \sum_{i=1}^N r_i^2. \quad (533)$$

O erro quadrático médio é

$$MSE = \frac{1}{N} \sum_{i=1}^N r_i^2. \quad (534)$$

O RMSE é

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N r_i^2}. \quad (535)$$

O MAE é

$$MAE = \frac{1}{N} \sum_{i=1}^N |r_i|. \quad (536)$$

20.2.3 `my_r2_score` e `my_correlation`

O coeficiente de determinação é

$$R^2 = 1 - \frac{\sum_i (d_i^{obs} - d_i^{pred})^2}{\sum_i (d_i^{obs} - \bar{d}^{obs})^2}. \quad (537)$$

A correlação de Pearson é

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}. \quad (538)$$

Correlação e R^2

Em alguns fluxos de trabalho geofísicos, especialmente em comparação de mapas e perfis, a correlação pode ser mais interpretável que R^2 , pois mede associação linear entre dois campos sem exigir necessariamente uma interpretação como fração de variância explicada.

20.3 Mínimos quadrados e regularização

20.3.1 `my_least_squares`

A função `my_least_squares` resolve um problema de mínimos quadrados ordinários.

Sintaxe

```
1 my_least_squares(A, d, rcond=1e-12)
```

A solução minimiza

$$\Phi(\mathbf{m}) = \|\mathbf{A}\mathbf{m} - \mathbf{d}\|_2^2. \quad (539)$$

A solução por pseudoinversa é

$$\mathbf{m} = \mathbf{A}^+ \mathbf{d}. \quad (540)$$

20.3.2 my_weighted_least_squares

A função `my_weighted_least_squares` resolve mínimos quadrados ponderados.

Sintaxe

```
1 my_weighted_least_squares(A, d, weights, rcond=1e-12)
```

A função objetivo é

$$\Phi(\mathbf{m}) = \|\mathbf{W}(\mathbf{A}\mathbf{m} - \mathbf{d})\|_2^2. \quad (541)$$

20.3.3 my_tikhonov

A função `my_tikhonov` resolve uma regressão com regularização de Tikhonov.

Sintaxe

```
1 my_tikhonov(A, d, lam=0.0, L=None, rcond=1e-12)
```

A função objetivo é

$$\Phi(\mathbf{m}) = \|\mathbf{A}\mathbf{m} - \mathbf{d}\|_2^2 + \lambda^2 \|\mathbf{L}\mathbf{m}\|_2^2. \quad (542)$$

Quando $\mathbf{L} = \mathbf{I}$, penaliza-se a norma do modelo.

20.4 Regressão linear simples e múltipla

20.4.1 `my_linear_regression`

A função `my_linear_regression` ajusta um modelo linear simples.

Sintaxe

```
1 my_linear_regression(x, y)
```

O modelo é

$$y = ax + b. \quad (543)$$

A saída inclui coeficiente angular, intercepto, dados preditos, resíduos e métricas.

20.4.2 `my_multiple_linear_regression`

A função `my_multiple_linear_regression` ajusta um modelo linear com múltiplas variáveis explicativas.

Sintaxe

```
1 my_multiple_linear_regression(X, y)
```

A formulação é

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p. \quad (544)$$

Em forma matricial,

$$\mathbf{y} = \mathbf{A}\boldsymbol{\beta}. \quad (545)$$

20.5 Matrizes de desenho polinomial

20.5.1 `my_polynomial_matrix_1d`

A função `my_polynomial_matrix_1d` constrói uma matriz de desenho para ajuste polinomial unidimensional.

Sintaxe

```
1 my_polynomial_matrix_1d(x, degree)
```

Para grau n , a matriz contém colunas

$$1, x, x^2, \dots, x^n. \quad (546)$$

20.5.2 `my_polynomial_matrix_2d`

A função `my_polynomial_matrix_2d` constrói matriz de desenho para superfície polinomial bidimensional.

Sintaxe

```
1 my_polynomial_matrix_2d(x, y, degree)
```

Os termos são

$$x^i y^j, \quad i + j \leq n. \quad (547)$$

Por exemplo, para grau 2, a superfície pode ser escrita como

$$z(x, y) = c_0 + c_1 x + c_2 y + c_3 x^2 + c_4 xy + c_5 y^2. \quad (548)$$

20.6 Ajuste polinomial

20.6.1 `my_polynomial_fit_1d`

A função `my_polynomial_fit_1d` ajusta um polinômio a dados unidimensionais.

Sintaxe

```
1 my_polynomial_fit_1d(x, data, degree=1,  
2 lam=0.0, return_report=True)
```

O modelo é

$$d(x) = \sum_{i=0}^n c_i x^i. \quad (549)$$

20.6.2 `my_polynomial_fit_2d`

A função `my_polynomial_fit_2d` ajusta uma superfície polinomial bidimensional.

Sintaxe

```
1 my_polynomial_fit_2d(x, y, data, degree=1,  
2 lam=0.0, return_report=True)
```

O modelo é

$$d(x, y) = \sum_{i+j \leq n} c_{ij} x^i y^j. \quad (550)$$

20.7 Separação regional-residual

A separação regional-residual é uma das aplicações mais importantes do módulo. Em campos potenciais, o dado observado pode ser representado por

$$d(x, y) = r(x, y) + a(x, y), \quad (551)$$

em que $r(x, y)$ é o campo regional e $a(x, y)$ é a anomalia residual.

20.7.1 my_remove_polynomial_regional_2d

A função `my_remove_polynomial_regional_2d` ajusta uma superfície polinomial regional e calcula o residual.

Sintaxe

```
1 my_remove_polynomial_regional_2d(x, y, data,
2 degree=1,
3 lam=0.0,
4 return_report=True)
```

Saída: Retorna o campo regional ajustado, o residual e, opcionalmente, métricas do ajuste.

A componente residual é

$$a_{est}(x, y) = d(x, y) - r_{est}(x, y). \quad (552)$$

Uso em métodos potenciais

A remoção de tendência regional é essencial para destacar anomalias locais. Em gravimetria e magnetometria, o regional pode estar associado a fontes profundas ou tendências amplas, enquanto o residual tende a realçar fontes mais rasas ou localizadas.

20.8 Regressão robusta

O módulo contém funções para regressão robusta, úteis quando os dados contêm outliers.

Funções robustas

```
1 my_huber_weights(residuals, c=1.345)
2 my_robust_regression(A, d, max_iter=50,
```

```
3      tol=1e-6, c=1.345)
```

Na regressão robusta, os pesos são atualizados em função dos resíduos. Para o estimador de Huber, resíduos pequenos recebem peso próximo de 1, enquanto resíduos grandes recebem pesos menores.

20.9 Interpolação de Lagrange

20.9.1 `my_lagrange_basis` e `my_lagrange_interpolation`

A função `my_lagrange_basis` calcula a base de Lagrange:

$$L_j(x) = \prod_{\substack{m=0 \\ m \neq j}}^n \frac{x - x_m}{x_j - x_m}. \quad (553)$$

A interpolação de Lagrange é

$$P(x) = \sum_{j=0}^n y_j L_j(x). \quad (554)$$

Sintaxe

```
1      my_lagrange_basis(x_nodes, j, x_eval)
2      my_lagrange_interpolation(x_nodes, y_nodes, x_eval)
```

20.10 Salvamento de resultados

A função `my_save_regression_result` salva resultados de regressão em arquivo de texto.

Sintaxe

```
1      my_save_regression_result(filename, result, fmt="%.8e
      ")
```

Essa função é importante para documentação de experimentos, reprodutibilidade e integração com relatórios.

20.11 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import regres
3
4      # Grade sintetica
5      x = np.linspace(-10, 10, 101)
6      y = np.linspace(-10, 10, 101)
7      X, Y = np.meshgrid(x, y)
8
9      # Campo regional + anomalia local
10     regional = 10 + 0.5*X - 0.3*Y + 0.02*X**2
11     anomalia = 5*np.exp(-(X**2 + Y**2)/10)
12     dados = regional + anomalia
13
14     # Remocao regional
15     resultado = regres.my_remove_polynomial_regional_2d(
16         X.ravel(),
17         Y.ravel(),
18         dados.ravel(),
19         degree=2,
20         lam=0.0,
21         return_report=True
22     )
23
24     regional_estimado = resultado["predicted"]
25     residual = resultado["residual"]
```

20.12 Síntese técnica do módulo

O módulo **regres.py** fornece ferramentas matemáticas para regressão, ajuste polinomial, regularização, métricas de erro, regressão robusta, interpolação e separação regional-

residual. Ele é importante tanto para processamento de dados quanto para construção de modelos empíricos.

Importância para ensino de Geofísica

O `regres.py` permite ensinar como métodos de álgebra linear e estatística são aplicados à Geofísica. Ele é especialmente útil para demonstrar ajuste de tendências, remoção regional, análise de resíduos, regularização e avaliação quantitativa de modelos.

Do ponto de vista científico, o módulo é aplicável a diversos fluxos de trabalho, incluindo separação regional-residual de campos potenciais, calibração de modelos, ajuste de superfícies, análise de correlação e validação de resultados sintéticos ou reais.

21 Descrição técnica do módulo `seismic_methods.py`

O arquivo `seismic_methods.py` reúne funções introdutórias e didáticas associadas à propagação de ondas sísmicas, sísmica de reflexão, propriedades elásticas, tempos de viagem, coeficientes de reflexão e transmissão, atenuação, energia sísmica, momento sísmico, osciladores, equação da onda, tensor momento e padrões de radiação. Esse módulo amplia o pacote `gravmag_codes` para além dos métodos potenciais, permitindo integrar conceitos de Sismologia e Sísmica Aplicada em um mesmo ambiente computacional.

A sísmica é baseada na propagação de ondas mecânicas no interior da Terra. Essas ondas são influenciadas pelas propriedades físicas dos materiais, como densidade, módulos elásticos, velocidades de onda P e S , contrastes de impedância e atenuação. Em aplicações geofísicas, métodos sísmicos são usados para investigar estruturas crustais, camadas sedimentares, reservatórios, falhas, interfaces geológicas e propriedades dinâmicas do subsolo.

Papel do módulo `seismic_methods.py`

O módulo `seismic_methods.py` fornece funções para simular conceitos básicos de sísmica, incluindo wavelets, traços sintéticos, velocidades elásticas, coeficientes de reflexão, tempos de viagem, atenuação, energia, momento sísmico e propagação de ondas em 1D e 2D.

O módulo utiliza principalmente NumPy para cálculos numéricos e Matplotlib para

visualização. Também inclui funções de salvamento de figuras, permitindo criar produtos didáticos e gráficos para relatórios ou aulas práticas.

21.1 Funções auxiliares e salvamento de figuras

O módulo contém funções auxiliares para conversão de entradas em arrays, controle numérico e salvamento de figuras.

Funções auxiliares

```
1         _arr(a)
2         my_save_figure(filename, fig=None, dpi=300,
3         bbox_inches="tight", transparent=True)
```

A função `_arr` converte uma entrada para array NumPy do tipo `float`, enquanto `my_save_figure` salva uma figura gerada pelo Matplotlib. Essas funções padronizam a entrada de dados e a exportação de resultados.

21.2 Wavelets e traços sintéticos

21.2.1 `my_ricker_wavelet`

A função `my_ricker_wavelet` gera uma wavelet de Ricker, frequentemente usada em sísmica de reflexão como pulso fonte sintético.

Sintaxe

```
1         my_ricker_wavelet(time, frequency)
```

Entrada: `time` é o vetor de tempo; `frequency` é a frequência dominante da wavelet.

Saída: Retorna a amplitude da wavelet de Ricker.

A formulação clássica é

$$w(t) = \left[1 - 2\pi^2 f^2 t^2\right] \exp\left(-\pi^2 f^2 t^2\right), \quad (555)$$

em que f é a frequência dominante.

Wavelet de Ricker

A wavelet de Ricker é amplamente usada em exemplos sísmicos porque possui forma simétrica, média aproximadamente nula e espectro controlado por uma frequência dominante. Ela é adequada para gerar traços sintéticos didáticos.

21.2.2 `my_synthetic_trace`

A função `my_synthetic_trace` gera um traço sísmico sintético por convolução entre uma série de refletividade e uma wavelet.

Sintaxe

```
1 my_synthetic_trace(reflectivity, wavelet, mode="same")
```

Entrada: `reflectivity` é a série de coeficientes de reflexão; `wavelet` é o pulso sísmico; `mode` controla o modo da convolução.

Saída: Retorna o traço sintético.

A formulação é

$$s(t) = r(t) * w(t), \quad (556)$$

em que $s(t)$ é o traço sísmico, $r(t)$ é a refletividade, $w(t)$ é a wavelet e $*$ representa convolução.

21.3 Propriedades elásticas e velocidades sísmicas

O módulo inclui funções para calcular velocidades de ondas P e S , razão de Poisson e velocidades aproximadas de ondas superficiais.

21.3.1 my_vp_from_elastic

A função `my_vp_from_elastic` calcula a velocidade da onda compressional P .

Sintaxe

```
1 my_vp_from_elastic(K, mu, rho)
```

Entrada: K é o módulo de incompressibilidade; μ é o módulo de cisalhamento; ρ é a densidade.

Saída: Retorna a velocidade V_P .

A formulação é

$$V_P = \sqrt{\frac{K + \frac{4}{3}\mu}{\rho}}. \quad (557)$$

21.3.2 my_vs_from_elastic

A função `my_vs_from_elastic` calcula a velocidade da onda cisalhante S .

Sintaxe

```
1 my_vs_from_elastic(mu, rho)
```

A formulação é

$$V_S = \sqrt{\frac{\mu}{\rho}}. \quad (558)$$

21.3.3 my_poisson_ratio

A função `my_poisson_ratio` calcula a razão de Poisson a partir de V_P e V_S .

Sintaxe

```
1 my_poisson_ratio(vp, vs)
```

A formulação é

$$\nu = \frac{V_P^2 - 2V_S^2}{2(V_P^2 - V_S^2)}. \quad (559)$$

21.3.4 my_rayleigh_velocity_approx e my_love_wave_velocity_simple

A função `my_rayleigh_velocity_approx` estima a velocidade de onda Rayleigh por aproximação em função de V_S e ν . Uma aproximação comum é

$$V_R \approx V_S \frac{0.862 + 1.14\nu}{1 + \nu}. \quad (560)$$

A função `my_love_wave_velocity_simple` fornece uma estimativa simplificada para velocidade de onda Love, geralmente relacionada às velocidades de onda S nas camadas envolvidas.

21.4 Tempos de viagem e reflexão sísmica

21.4.1 my_two_way_time

A função `my_two_way_time` calcula o tempo duplo de percurso para uma interface horizontal.

Sintaxe

```
1 my_two_way_time(depth, velocity)
```

Saída: Retorna o tempo duplo t .

A formulação é

$$t = \frac{2z}{v}, \quad (561)$$

em que z é a profundidade e v é a velocidade.

21.4.2 `my_depth_from_two_way_time`

A função `my_depth_from_two_way_time` estima profundidade a partir do tempo duplo.

Sintaxe

```
1 my_depth_from_two_way_time(twt, velocity)
```

A formulação inversa é

$$z = \frac{vt}{2}. \quad (562)$$

21.4.3 `my_normal_moveout`

A função `my_normal_moveout` calcula o tempo de chegada com afastamento fonte-receptor, em um modelo de velocidade constante.

Sintaxe

```
1 my_normal_moveout(t0, offset, velocity)
```

A formulação hiperbólica do NMO é

$$t(x) = \sqrt{t_0^2 + \frac{x^2}{v^2}}, \quad (563)$$

em que t_0 é o tempo de afastamento zero, x é o offset e v é a velocidade.

21.5 Impedância acústica e coeficientes de reflexão

21.5.1 `my_acoustic_impedance`

A função `my_acoustic_impedance` calcula a impedância acústica.

Sintaxe

```
1 my_acoustic_impedance(density, velocity)
```

A formulação é

$$Z = \rho v. \quad (564)$$

21.5.2 my_reflection_coefficient

A função `my_reflection_coefficient` calcula o coeficiente de reflexão normal entre duas camadas.

Sintaxe

```
1 my_reflection_coefficient(Z1, Z2)
```

A formulação é

$$R = \frac{Z_2 - Z_1}{Z_2 + Z_1}. \quad (565)$$

21.5.3 my_transmission_coefficient

A função `my_transmission_coefficient` calcula um coeficiente de transmissão simplificado.

$$T = 1 + R, \quad (566)$$

ou uma forma equivalente associada à razão entre impedâncias, dependendo da convenção adotada na implementação.

21.6 Atenuação, energia e momento sísmico

21.6.1 `my_attenuation_exponential`

A função `my_attenuation_exponential` calcula atenuação exponencial da amplitude com a distância.

Sintaxe

```
1 my_attenuation_exponential(amplitude0, distance,  
                             alpha)
```

A formulação é

$$A(x) = A_0 e^{-\alpha x}, \quad (567)$$

em que α é o coeficiente de atenuação.

21.6.2 `my_seismic_energy`

A função `my_seismic_energy` estima energia sísmica a partir de uma amplitude.

$$E \propto A^2. \quad (568)$$

Essa relação representa a proporcionalidade entre energia e quadrado da amplitude da onda.

21.6.3 `my_moment_magnitude`

A função `my_moment_magnitude` calcula a magnitude de momento a partir do momento sísmico M_0 .

Sintaxe

```
1 my_moment_magnitude(M0)
```

A formulação é

$$M_w = \frac{2}{3} [\log_{10}(M_0) - 9.1], \quad (569)$$

quando M_0 é dado em N m.

21.7 Osciladores e equação da onda

21.7.1 `my_harmonic_oscillator`

A função `my_harmonic_oscillator` calcula a resposta de um oscilador harmônico simples.

Sintaxe

```
1 my_harmonic_oscillator(time, amplitude=1.0,
2 frequency=1.0, phase=0.0)
```

A formulação é

$$u(t) = A \sin(2\pi ft + \phi). \quad (570)$$

21.7.2 `my_wave_equation_1d_step`

A função `my_wave_equation_1d_step` executa uma etapa explícita da equação da onda 1D.

A equação da onda é

$$\frac{\partial^2 u}{\partial t^2} = v^2 \frac{\partial^2 u}{\partial x^2}. \quad (571)$$

Uma discretização explícita pode ser escrita como

$$u_i^{n+1} = 2u_i^n - u_i^{n-1} + \left(\frac{v\Delta t}{\Delta x}\right)^2 (u_{i+1}^n - 2u_i^n + u_{i-1}^n). \quad (572)$$

21.7.3 `my_wave_equation_2d_step`

A função `my_wave_equation_2d_step` executa uma etapa explícita da equação da onda 2D:

$$\frac{\partial^2 u}{\partial t^2} = v^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right). \quad (573)$$

21.8 Tensor momento e padrões de radiação

O módulo inclui funções para representar fontes sísmicas por tensor momento e calcular padrões de radiação simplificados.

Funções de fonte sísmica

```

1      my_double_couple_moment_tensor(strike, dip, rake, MO
      =1.0)
2      my_radiation_pattern_p(moment_tensor, azimuth,
      takeoff_angle)
3      my_radiation_pattern_s(moment_tensor, azimuth,
      takeoff_angle)
```

O tensor momento $\mathbf{M}$ representa a geometria e intensidade da fonte sísmica. A amplitude de radiação para onda P , de forma conceitual, pode ser escrita como

$$A_P = \mathbf{n}^T \mathbf{M} \mathbf{n}, \quad (574)$$

em que $\mathbf{n}$ é o vetor unitário na direção de propagação.

21.9 Visualização sísmica

O módulo contém funções para plotar traços, seções sísmicas, wavelets e curvas.

Funções de visualização

```

1      my_plot_trace(time, trace, ...)
2      my_plot_seismic_section(time, traces, ...)
3      my_plot_wavelet(time, wavelet, ...)
4      my_plot_curve(x, y, ...)
```

Essas funções são úteis para aulas práticas, pois permitem visualizar sinais sísmicos, respostas sintéticas e propagação de ondas.

21.10 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import seismic_methods as sm
3
4      # Vetor de tempo
5      t = np.linspace(-0.1, 0.1, 1001)
6
7      # Wavelet de Ricker
8      w = sm.my_ricker_wavelet(t, frequency=25.0)
9
10     # Serie de refletividade sintetica
11     r = np.zeros_like(t)
12     r[300] = 0.4
13     r[600] = -0.3
14     r[800] = 0.2
15
16     # Traco sintetico
17     trace = sm.my_synthetic_trace(r, w)
18
19     # Plot
20     fig, ax = sm.my_plot_trace(t, trace)
```

21.11 Síntese técnica do módulo

O módulo **seismic_methods.py** fornece uma base introdutória para simulação e visualização de conceitos sísmicos. Ele permite trabalhar com wavelets, traços sintéticos, velocidades elásticas, coeficientes de reflexão, tempos de viagem, atenuação, equação da onda e tensor momento.

Importância para ensino de Geofísica

O `seismic_methods.py` permite integrar conceitos de Física Ondulatória, Elasticidade, Métodos Numéricos e Sísmica Aplicada. Ele é adequado para aulas práticas que buscam demonstrar como ondas se propagam, refletem, atenuam e são registradas em traços sísmicos.

22 Descrição técnica do módulo `sphere.py`

O arquivo `sphere.py` reúne funções para criação e modelagem gravimétrica e magnética de esferas homogêneas. A esfera é uma das geometrias clássicas em métodos potenciais devido à sua simplicidade analítica e ao seu forte valor didático. Em muitos casos, uma esfera homogênea pode ser usada como modelo simplificado para representar corpos aproximadamente compactos, como intrusões, cavidades, domos, massas mineralizadas ou anomalias localizadas.

Em gravimetria, a resposta externa de uma esfera homogênea é equivalente à resposta de uma massa pontual localizada no centro da esfera. Em magnetometria, uma esfera uniformemente magnetizada também pode ser representada, em aproximação externa, por um dipolo magnético localizado em seu centro. Isso torna o módulo `sphere.py` particularmente útil para ensino de modelagem direta, testes sintéticos e análise da influência de profundidade, raio, densidade e magnetização sobre as anomalias.

Papel do módulo `sphere.py`

O módulo `sphere.py` implementa funções para criar modelos de esferas, calcular volume, distância entre fonte e observação, potencial gravitacional, componentes gravimétricas, tensor de gradiente, componentes magnéticas e anomalia magnética de campo total.

A geometria de uma esfera é definida por

$$(x_c, y_c, z_c, R), \quad (575)$$

em que x_c, y_c, z_c é o centro da esfera e R é seu raio.

22.1 Formulação geométrica da esfera

O volume de uma esfera de raio R é

$$V = \frac{4}{3}\pi R^3. \quad (576)$$

Se a esfera possui densidade ρ , sua massa é

$$m = \rho V. \quad (577)$$

Para um ponto de observação (x, y, z) , a distância ao centro da esfera é

$$r = \sqrt{(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2}. \quad (578)$$

O módulo utiliza uma pequena tolerância numérica para evitar singularidades quando o ponto de observação se aproxima muito do centro da fonte.

22.2 Funções internas de suporte

O módulo contém funções internas para validar pontos de observação, preparar coordenadas verticais, recuperar propriedades físicas e calcular cossenos diretores.

Funções internas

```

1      _check_observation_arrays(x, y)
2      _prepare_z(x, z)
3      _get_property(model, value, index, default=None,
4      name="physical property")
5      _dircos(inc, dec)
```

A função `_dircos` calcula

$$f_x = \cos I \cos D, \quad f_y = \cos I \sin D, \quad f_z = \sin I. \quad (579)$$

22.3 Criação do modelo, volume e distância

22.3.1 `my_sphere_model`

A função `my_sphere_model` cria uma representação computacional para uma esfera.

Sintaxe

```
1 my_sphere_model(xc, yc, zc, radius,  
2 physical_property=None)
```

Entrada: `xc`, `yc` e `zc` são as coordenadas do centro; `radius` é o raio; `physical_property` é uma propriedade física opcional, como densidade ou magnetização.

Saída: Retorna uma lista representando a esfera.

A condição geométrica necessária é

$$R > 0. \quad (580)$$

22.3.2 my_sphere_volume

A função `my_sphere_volume` calcula o volume da esfera.

Sintaxe

```
1 my_sphere_volume(sphere)
```

Saída: Retorna

$$V = \frac{4}{3}\pi R^3. \quad (581)$$

22.3.3 my_sphere_distance

A função `my_sphere_distance` calcula a distância entre pontos de observação e o centro da esfera.

Sintaxe

```
1 my_sphere_distance(x, y, z, sphere)
```

Saída: Retorna

$$r = \sqrt{(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2}. \quad (582)$$

22.4 Modelagem gravimétrica de uma esfera

Para pontos externos à esfera, o potencial gravitacional é equivalente ao de uma massa pontual no centro:

$$V = \frac{Gm}{r}. \quad (583)$$

Como $m = \rho V_s$, tem-se

$$V = G\rho V_s \frac{1}{r}. \quad (584)$$

22.4.1 `my_sphere_potential`

A função `my_sphere_potential` calcula o potencial gravitacional da esfera.

Sintaxe

```
1 my_sphere_potential(x, y, z, sphere, rho=None)
```

Entrada: `rho` é a densidade em g/cm^3 , quando não estiver armazenada no modelo.

Saída: Retorna o potencial gravitacional em unidades SI.

A densidade é convertida para kg/m^3 antes do cálculo:

$$\rho_{SI} = 1000\rho_{\text{g/cm}^3}. \quad (585)$$

22.4.2 `my_sphere_gx`, `my_sphere_gy` e `my_sphere_gz`

Essas funções calculam as componentes gravimétricas da esfera.

Sintaxe

```

1      my_sphere_gx(x, y, z, sphere, rho=None)
2      my_sphere_gy(x, y, z, sphere, rho=None)
3      my_sphere_gz(x, y, z, sphere, rho=None)

```

As componentes são

$$g_x = Gm \frac{x_c - x}{r^3}, \quad (586)$$

$$g_y = Gm \frac{y_c - y}{r^3}, \quad (587)$$

$$g_z = Gm \frac{z_c - z}{r^3}. \quad (588)$$

As saídas são convertidas para mGal:

$$g_{\text{mGal}} = 10^5 g_{\text{SI}}. \quad (589)$$

22.4.3 my_sphere_gxyz

A função `my_sphere_gxyz` calcula as três componentes gravimétricas simultaneamente.

Sintaxe

```

1      my_sphere_gxyz(x, y, z, sphere, rho=None)

```

Saída: Retorna `gx`, `gy` e `gz`, em mGal.

22.5 Tensor de gradiente gravimétrico

O tensor de gradiente de uma massa pontual é baseado nas derivadas de segunda ordem de $1/r$. Para uma massa m , os termos diagonais são proporcionais a

$$T_{xx} = Gm \frac{3\Delta x^2 - r^2}{r^5}, \quad (590)$$

$$T_{yy} = Gm \frac{3\Delta y^2 - r^2}{r^5}, \quad (591)$$

$$T_{zz} = Gm \frac{3\Delta z^2 - r^2}{r^5}. \quad (592)$$

Os termos cruzados são

$$T_{xy} = Gm \frac{3\Delta x \Delta y}{r^5}, \quad (593)$$

$$T_{xz} = Gm \frac{3\Delta x \Delta z}{r^5}, \quad (594)$$

$$T_{yz} = Gm \frac{3\Delta y \Delta z}{r^5}. \quad (595)$$

As funções implementadas incluem:

Componentes do tensor

```

1      my_sphere_gxx(...)
2      my_sphere_gxy(...)
3      my_sphere_gxz(...)
4      my_sphere_gyy(...)
5      my_sphere_gyz(...)
6      my_sphere_gzz(...)
7      my_sphere_tensor(...)
```

Essas componentes são úteis em estudos de gradiente gravimétrico e análise de fontes compactas.

22.6 Modelagem magnética de uma esfera

Uma esfera uniformemente magnetizada pode ser aproximada externamente por um dipolo magnético no centro. O momento magnético é

$$\mathbf{m} = MV_s, \quad (596)$$

em que $\mathbf{M}$ é a magnetização e V_s é o volume da esfera.

O campo de dipolo é

$$\mathbf{B} = \frac{\mu_0}{4\pi} \left[\frac{3(\mathbf{m} \cdot \mathbf{r})\mathbf{r}}{r^5} - \frac{\mathbf{m}}{r^3} \right]. \quad (597)$$

22.6.1 my_sphere_bxyz

A função `my_sphere_bxyz` calcula as componentes magnéticas da esfera.

Sintaxe

```
1 my_sphere_bxyz(x, y, z, sphere, mag=None,
2 incs=90.0, decs=0.0)
```

Entrada: `mag` é a intensidade de magnetização em A/m; `incs` e `decs` definem a direção da magnetização.

Saída: Retorna B_x , B_y e B_z , em nT.

22.6.2 my_sphere_bx, my_sphere_by e my_sphere_bz

Essas funções retornam individualmente as componentes magnéticas:

Sintaxe

```
1 my_sphere_bx(x, y, z, sphere, mag=None, incs=90.0,
2 decs=0.0)
3 my_sphere_by(x, y, z, sphere, mag=None, incs=90.0,
4 decs=0.0)
5 my_sphere_bz(x, y, z, sphere, mag=None, incs=90.0,
6 decs=0.0)
```

22.6.3 my_sphere_tfa e my_sphere_tf

A função `my_sphere_tfa` calcula a anomalia magnética de campo total. A função `my_sphere_tf` é um alias de compatibilidade.

Sintaxe

```

1      my_sphere_tfa(x, y, z, sphere, mag=None,
2      inc=-20.0, dec=0.0,
3      incs=None, decs=None)
```

A anomalia é calculada por

$$\Delta T = B_x f_x + B_y f_y + B_z f_z, \quad (598)$$

em que f_x, f_y, f_z define a direção do campo regional.

22.7 Modelagem de múltiplas esferas

O módulo contém funções para calcular a resposta total de várias esferas:

Funções para múltiplas esferas

```

1      my_spheres_potential(...)
2      my_spheres_gx(...)
3      my_spheres_gy(...)
4      my_spheres_gz(...)
5      my_spheres_gxyz(...)
6      my_spheres_tensor(...)
7      my_spheres_bxyz(...)
8      my_spheres_tfa(...)
9      my_spheres_tf(...)
```

A resposta total é obtida por superposição linear:

$$d_{total} = \sum_{k=1}^{N_s} d_k. \quad (599)$$

22.8 Aliases de compatibilidade

O módulo preserva aliases para chamadas mais curtas ou compatibilidade com versões anteriores:

Aliases principais

```
1 sphere_model = my_sphere_model
2 sphere_volume = my_sphere_volume
3 sphere_distance = my_sphere_distance
4 sphere_potential = my_sphere_potential
5 sphere_gx = my_sphere_gx
6 sphere_gy = my_sphere_gy
7 sphere_gz = my_sphere_gz
8 sphere_tf = my_sphere_tf
9 spheres_gz = my_spheres_gz
10 spheres_tf = my_spheres_tf
```

22.9 Exemplo conceitual de uso

Exemplo de uso em Python

```
1 import numpy as np
2 from gravmag_codes import sphere
3
4 # Malha de observacao
5 x = np.linspace(-5000, 5000, 101)
6 y = np.linspace(-5000, 5000, 101)
7 X, Y = np.meshgrid(x, y)
8 Z = np.zeros_like(X)
9
10 # Esfera com contraste de densidade
11 s = sphere.my_sphere_model(
12     xc=0.0,
13     yc=0.0,
14     zc=2000.0,
15     radius=500.0,
```

```
16         physical_property=0.5
17     )
18
19     # Componente vertical da gravidade
20     gz = sphere.my_sphere_gz(X, Y, Z, s, rho=None)
21
22     # Anomalia magnetica
23     tf = sphere.my_sphere_tfa(
24     X, Y, Z, s,
25     mag=2.0,
26     inc=-20.0,
27     dec=0.0
28     )
```

22.10 Síntese técnica do módulo

O módulo **sphere.py** fornece uma formulação simples e didática para modelagem gravimétrica e magnética de esferas homogêneas. Por sua simplicidade analítica, a esfera é uma geometria ideal para introduzir conceitos de modelagem direta, dependência com profundidade, amplitude de anomalias e propriedades físicas.

Importância para ensino de Geofísica

O **sphere.py** é especialmente adequado para aulas iniciais de métodos potenciais, pois permite demonstrar a relação entre massa, densidade, magnetização, profundidade, raio e resposta observada. Ele também serve como referência para comparação com geometrias mais complexas, como prismas, cilindros e corpos poligonais.

Do ponto de vista científico, o módulo pode ser usado em testes sintéticos, validação de algoritmos, construção de exemplos simples e desenvolvimento de intuição física sobre campos potenciais.

23 Descrição técnica do módulo **statistical.py**

O arquivo **statistical.py** reúne funções estatísticas aplicadas ao tratamento, análise, comparação e validação de dados geofísicos. Esse módulo tem papel auxiliar, mas é essencial

para avaliar quantitativamente resultados de modelagem, filtragem, regressão, inversão, interpolação e testes sintéticos. Em Geofísica Computacional, métricas estatísticas são fundamentais para comparar dados observados e preditos, medir erros, avaliar ruído, quantificar correlações e interpretar a qualidade de modelos.

Em aplicações gravimétricas e magnéticas, por exemplo, é comum comparar o campo verdadeiro de um modelo sintético com o campo estimado, ou comparar dados observados com dados calculados por um modelo direto. Para isso, utilizam-se métricas como média, desvio padrão, erro absoluto médio, erro quadrático médio, raiz do erro quadrático médio, correlação, covariância e estatísticas robustas.

Papel do módulo `statistical.py`

O módulo `statistical.py` fornece funções para análise estatística de dados geofísicos, com foco em métricas de erro, estatísticas descritivas, correlação, normalização, padronização, identificação de outliers e comparação entre campos observados, sintéticos e preditos.

As funções desse módulo normalmente recebem arrays NumPy como entrada e retornam escalares, vetores ou dicionários com métricas resumidas. Do ponto de vista didático, o módulo permite mostrar que a interpretação geofísica não depende apenas de mapas visuais, mas também de critérios quantitativos de avaliação.

23.1 Estatísticas descritivas básicas

As estatísticas descritivas permitem resumir um conjunto de dados por meio de medidas de tendência central, dispersão e amplitude. Para um conjunto de dados

$$\mathbf{d} = \{d_1, d_2, \dots, d_N\}, \quad (600)$$

a média é dada por

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N d_i. \quad (601)$$

O desvio padrão é

$$\sigma_d = \sqrt{\frac{1}{N} \sum_{i=1}^N (d_i - \bar{d})^2}. \quad (602)$$

A variância é

$$\sigma_d^2 = \frac{1}{N} \sum_{i=1}^N (d_i - \bar{d})^2. \quad (603)$$

Também são frequentemente calculados os valores mínimo, máximo, mediana e amplitude:

$$A = d_{max} - d_{min}. \quad (604)$$

Uso em dados geofísicos

Estatísticas descritivas são úteis para caracterizar anomalias, resíduos, ruído, densidades estimadas, magnetizações, erros de interpolação e resultados de inversão. Elas permitem identificar rapidamente a escala dos dados e possíveis valores anômalos.

23.2 Normalização e padronização

A normalização transforma os dados para uma escala comum. Uma normalização min-max pode ser escrita como

$$d_{norm} = \frac{d - d_{min}}{d_{max} - d_{min}}. \quad (605)$$

Essa transformação leva os dados para o intervalo $[0, 1]$, sendo útil em visualização, comparação de campos e construção de imagens compostas.

A padronização, ou *escore-z*, é dada por

$$z_i = \frac{d_i - \bar{d}}{\sigma_d}. \quad (606)$$

Nesse caso, os dados passam a ter média aproximadamente zero e desvio padrão unitário. Essa operação é útil para comparar variáveis com unidades e escalas diferentes, como topografia, anomalia de Bouguer, distúrbio da gravidade, densidade estimada e campo magnético.

Cuidados na normalização

A normalização deve ser aplicada com cuidado quando os dados possuem outliers. Valores extremos podem comprimir a variação dos demais pontos e prejudicar a interpretação visual ou estatística.

23.3 Resíduos e métricas de erro

A comparação entre dados observados e dados preditos é feita a partir do vetor de resíduos:

$$r_i = d_i^{obs} - d_i^{pred}. \quad (607)$$

Em forma vetorial,

$$\mathbf{r} = \mathbf{d}_{obs} - \mathbf{d}_{pred}. \quad (608)$$

A média dos resíduos é

$$\bar{r} = \frac{1}{N} \sum_{i=1}^N r_i. \quad (609)$$

O erro absoluto médio é

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |r_i|. \quad (610)$$

O erro quadrático médio é

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N r_i^2. \quad (611)$$

A raiz do erro quadrático médio é

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N r_i^2}. \quad (612)$$

A norma euclidiana dos resíduos é

$$\|\mathbf{r}\|_2 = \sqrt{\sum_{i=1}^N r_i^2}. \quad (613)$$

Interpretação do RMSE e MAE

O RMSE penaliza mais fortemente erros grandes, enquanto o MAE mede a magnitude média absoluta dos desvios. Em testes sintéticos, o uso conjunto dessas métricas permite avaliar tanto o erro médio quanto a presença de discrepâncias localizadas.

23.4 Correlação e covariância

A covariância entre dois conjuntos de dados $\mathbf{x}$ e $\mathbf{y}$ é

$$\text{cov}(x, y) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y}). \quad (614)$$

A correlação linear de Pearson é

$$r = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}}. \quad (615)$$

O valor da correlação varia entre -1 e 1 . Valores próximos de 1 indicam associação linear positiva; valores próximos de -1 indicam associação linear negativa; valores próximos de zero indicam baixa associação linear.

Em Geofísica, a correlação pode ser usada para comparar mapas, perfis, dados observados e calculados, anomalias filtradas, densidades estimadas e modelos sintéticos.

Correlação em interpretação geofísica

Uma alta correlação entre dois campos indica semelhança no padrão espacial ou linear, mas não garante igualdade de amplitude. Por isso, a correlação deve ser analisada juntamente com métricas de erro, mapas de resíduos e interpretação física.

23.5 Identificação de outliers

A identificação de outliers é importante em dados geofísicos porque medições podem ser afetadas por ruído instrumental, erros de posicionamento, falhas de aquisição, interferências

culturais ou problemas de processamento. Uma abordagem simples consiste em usar o escore-z:

$$z_i = \frac{d_i - \bar{d}}{\sigma_d}. \quad (616)$$

Um dado pode ser considerado potencial outlier quando

$$|z_i| > z_{lim}, \quad (617)$$

em que z_{lim} é um limiar, frequentemente escolhido entre 2 e 4.

Outra abordagem robusta usa a mediana e o desvio absoluto mediano:

$$MAD = \text{median}(|d_i - \text{median}(d)|). \quad (618)$$

Essa métrica é menos sensível a valores extremos.

23.6 Relatórios estatísticos

Uma função típica de relatório estatístico pode retornar um dicionário contendo estatísticas como:

$$\bar{d}, \quad \sigma_d, \quad d_{min}, \quad d_{max}, \quad \text{median}(d), \quad RMSE, \quad MAE, \quad r. \quad (619)$$

Esse tipo de saída é útil para automatizar testes, comparar métodos e gerar tabelas de resultados para relatórios ou artigos científicos.

Exemplo conceitual de relatório estatístico

```

1      import numpy as np
2      from gravmag_codes import statistical
3
4      observado = np.array([10.0, 12.0, 13.0, 15.0, 16.0])
5      predito = np.array([10.5, 11.8, 13.2, 14.6, 16.5])
6
7      residuo = observado - predito
8
9      rmse = statistical.my_rmse(observado, predito)
10     mae = statistical.my_mae(observado, predito)
```

```
11 corr = statistical.my_correlation(observado, predito)
```

23.7 Aplicações em testes sintéticos

Em testes sintéticos, as métricas estatísticas permitem comparar o modelo verdadeiro e o modelo estimado. Por exemplo, se $\mathbf{m}_{true}$ é o modelo verdadeiro e $\mathbf{m}_{est}$ é o modelo estimado, o erro do modelo é

$$\mathbf{e}_m = \mathbf{m}_{true} - \mathbf{m}_{est}. \quad (620)$$

A partir desse erro, podem ser calculados RMSE, MAE, erro máximo e histograma dos erros. Essas medidas ajudam a avaliar se o método recuperou adequadamente a amplitude, a forma e a distribuição espacial da propriedade física.

Validação quantitativa

A validação quantitativa por métricas estatísticas é essencial para transformar testes sintéticos em evidência científica. Ela permite demonstrar objetivamente o desempenho de algoritmos de filtragem, modelagem e inversão.

23.8 Síntese técnica do módulo

O módulo **statistical.py** fornece ferramentas estatísticas essenciais para análise de dados geofísicos. Ele permite calcular estatísticas descritivas, normalizações, resíduos, métricas de erro, correlação, identificação de outliers e relatórios de comparação entre campos.

Importância para ensino de Geofísica

O **statistical.py** é importante para mostrar aos estudantes que a interpretação geofísica deve ser acompanhada de avaliação quantitativa. Mapas e perfis são fundamentais, mas métricas estatísticas fornecem uma medida objetiva da qualidade dos resultados.

Do ponto de vista científico, o módulo é útil na validação de testes sintéticos, comparação de métodos, análise de resíduos, calibração de modelos e apresentação de resultados quantitativos em artigos, relatórios e documentação técnica.

24 Descrição técnica do módulo `synthetic.py`

O arquivo **`synthetic.py`** reúne funções para criação de modelos sintéticos, geração de grades de observação, construção de corpos geofísicos simples e compostos, simulação de respostas gravimétricas e magnéticas, adição de ruído, comparação entre dados verdadeiros e estimados e organização de exemplos práticos. Esse módulo atua como uma camada integradora entre os módulos de geometria, modelagem direta, grades e estatística do pacote *gravmag_codes*.

Modelos sintéticos são essenciais em Geofísica Computacional porque permitem testar algoritmos em situações controladas. Diferentemente dos dados reais, nos quais o modelo verdadeiro do subsolo não é conhecido, em um teste sintético todas as propriedades físicas e geometrias são previamente definidas. Isso permite avaliar, de forma objetiva, se uma rotina de modelagem, filtragem, inversão ou interpretação consegue recuperar corretamente as características esperadas.

Papel do módulo `synthetic.py`

O módulo `synthetic.py` organiza exemplos e utilidades para construir modelos sintéticos usando prismas, esferas, cilindros, prismas hexagonais, prismas poligonais, topografias sintéticas, campos de densidade, campos de magnetização, ruído e métricas de comparação.

Esse módulo não substitui os módulos físicos principais, como **`prism.py`**, **`sphere.py`**, **`cylinder.py`**, **`hexagonalprism.py`** e **`polygonalprism.py`**. Em vez disso, ele os utiliza de forma integrada para facilitar a construção de experimentos reproduzíveis.

24.1 Criação de grades de observação

Uma das primeiras etapas em um teste sintético é definir onde o campo será observado. Uma grade de observação regular pode ser definida por uma área

$$[x_{min}, x_{max}, y_{min}, y_{max}], \quad (621)$$

uma forma

$$(n_y, n_x), \quad (622)$$

e um nível vertical

$$z = z_0. \quad (623)$$

A função de criação de grade normalmente retorna matrizes **X**, **Y** e **Z**, ou seus vetores equivalentes. As coordenadas são construídas por

$$x_i = x_{min} + i\Delta x, \quad i = 0, 1, \dots, n_x - 1, \quad (624)$$

$$y_j = y_{min} + j\Delta y, \quad j = 0, 1, \dots, n_y - 1. \quad (625)$$

Os espaçamentos são

$$\Delta x = \frac{x_{max} - x_{min}}{n_x - 1}, \quad (626)$$

$$\Delta y = \frac{y_{max} - y_{min}}{n_y - 1}. \quad (627)$$

Exemplo de grade de observação

```

1      from gravmag_codes import synthetic
2
3      area = [-5000.0, 5000.0, -5000.0, 5000.0]
4      shape = (101, 101)
5
6      X, Y, Z = synthetic.my_make_observation_grid(
7          area=area,
8          shape=shape,
9          level=0.0,
10         flatten=False
11     )

```

24.2 Modelos sintéticos de fontes simples

O módulo permite criar modelos sintéticos baseados em geometrias simples, como prismas retangulares, esferas, cilindros, prismas hexagonais e prismas poligonais. Cada fonte é definida por sua geometria e por uma propriedade física, como densidade ou magnetização.

Para um prisma retangular, a geometria é definida por

$$[x_1, x_2, y_1, y_2, z_1, z_2]. \quad (628)$$

Para uma esfera, a geometria é

$$(x_c, y_c, z_c, R). \quad (629)$$

Para um cilindro vertical, a geometria é

$$(x_c, y_c, z_{top}, z_{bottom}, R). \quad (630)$$

Para um prisma poligonal, a base é definida por um conjunto de vértices:

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N). \quad (631)$$

Uso didático das fontes simples

Fontes simples permitem estudar isoladamente o efeito da profundidade, tamanho, contraste de densidade, magnetização e geometria sobre a anomalia observada. Por isso, são fundamentais em aulas introdutórias de modelagem direta.

24.3 Modelos compostos

Um modelo sintético mais realista pode ser construído pela combinação de várias fontes. Se cada fonte k produz uma resposta d_k , a resposta total é dada por superposição linear:

$$d_{total} = \sum_{k=1}^N d_k. \quad (632)$$

Essa propriedade é válida para campos potenciais porque as equações envolvidas

são lineares em relação à densidade ou à magnetização. Assim, um modelo composto pode incluir, por exemplo, uma esfera rasa, um prisma profundo e um cilindro lateral, todos contribuindo simultaneamente para o campo observado.

Exemplo conceitual de modelo composto

```
1      from gravmag_codes import prism, sphere, cylinder
2
3      p1 = prism.my_prism_model(
4          -1000, 1000, -1000, 1000, 1000, 3000,
5          physical_property=0.4
6      )
7
8      s1 = sphere.my_sphere_model(
9          xc=2500, yc=0, zc=2000,
10         radius=600,
11         physical_property=0.6
12     )
13
14     c1 = cylinder.my_cylinder_model(
15         xc=-2500, yc=0,
16         top=800, bottom=3500,
17         radius=700,
18         physical_property=0.5
19     )
```

24.4 Topografia sintética

O módulo pode incluir funções para criação de topografia sintética. Uma topografia sintética pode combinar feições como montanhas Gaussianas, vales, planos inclinados e ondulações suaves.

Uma montanha Gaussiana pode ser representada por

$$h(x, y) = h_0 \exp \left[-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2} \right]. \quad (633)$$

Um vale Gaussiano pode ser representado por uma amplitude negativa:

$$h_v(x, y) = -h_0 \exp \left[-\frac{(x - x_v)^2 + (y - y_v)^2}{2\sigma_v^2} \right]. \quad (634)$$

Uma topografia composta pode ser escrita como

$$h(x, y) = h_{base} + h_{montanha}(x, y) + h_{vale}(x, y) + h_{ond}(x, y). \quad (635)$$

Importância da topografia sintética

Topografias sintéticas permitem testar correções gravimétricas, modelos isostáticos, relações entre relevo e anomalia, métodos de estimativa de densidade e algoritmos de interpolação em condições controladas.

24.5 Campos sintéticos de densidade e magnetização

Além de fontes geométricas individuais, o módulo pode gerar campos sintéticos de propriedades físicas em grade. Um campo de densidade pode ser definido como

$$\rho(x, y) = \rho_0 + \Delta\rho(x, y), \quad (636)$$

em que ρ_0 é uma densidade de referência e $\Delta\rho(x, y)$ representa variações laterais.

Da mesma forma, um campo de magnetização pode ser escrito como

$$M(x, y) = M_0 + \Delta M(x, y). \quad (637)$$

Esses modelos são úteis para testar inversões espaciais, mapas temáticos e metodologias que buscam recuperar variações laterais de propriedades físicas.

24.6 Geração de dados gravimétricos sintéticos

A resposta gravimétrica sintética é calculada por chamadas aos módulos de modelagem direta. Para uma distribuição de fontes, a componente vertical pode ser escrita como

$$g_z^{synthetic}(x, y, z) = \sum_{k=1}^N g_z^{(k)}(x, y, z). \quad (638)$$

Quando as fontes são prismas, esferas ou cilindros, cada termo é calculado pela função correspondente do módulo específico.

A densidade é geralmente fornecida em g/cm^3 , mas as equações físicas utilizam kg/m^3 . A conversão é

$$\rho_{SI} = 1000\rho_{\text{g/cm}^3}. \quad (639)$$

A saída gravimétrica é normalmente expressa em mGal.

24.7 Geração de dados magnéticos sintéticos

A resposta magnética sintética pode incluir componentes B_x , B_y , B_z ou anomalia de campo total ΔT . Para múltiplas fontes, a anomalia total é

$$\Delta T_{\text{synthetic}} = \sum_{k=1}^N \Delta T^{(k)}. \quad (640)$$

A anomalia de campo total é obtida por projeção do campo anômalo na direção do campo geomagnético regional:

$$\Delta T = B_x f_x + B_y f_y + B_z f_z. \quad (641)$$

O vetor unitário do campo regional é definido por inclinação I e declinação D :

$$f_x = \cos I \cos D, \quad f_y = \cos I \sin D, \quad f_z = \sin I. \quad (642)$$

24.8 Adição de ruído sintético

Para tornar os testes mais realistas, é comum adicionar ruído aos dados sintéticos. O ruído Gaussiano é representado por

$$d_i^{\text{ruído}} = d_i^{\text{limpo}} + \epsilon_i, \quad (643)$$

com

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2). \quad (644)$$

O parâmetro σ controla o nível de ruído. Em testes gravimétricos, por exemplo,

pode-se adicionar ruído em mGal; em testes magnéticos, o ruído pode ser expresso em nT.

Reprodutibilidade do ruído

O uso de uma semente aleatória, ou **seed**, permite gerar sempre a mesma sequência de ruído. Isso é importante para reprodutibilidade de testes, figuras e artigos científicos.

24.9 Comparação entre dados verdadeiros e preditos

O módulo pode integrar funções estatísticas para comparar dados verdadeiros, observados e preditos. Se $\mathbf{d}_{true}$ é o dado sintético sem ruído e $\mathbf{d}_{pred}$ é o dado calculado por algum modelo estimado, o erro é

$$\mathbf{e} = \mathbf{d}_{true} - \mathbf{d}_{pred}. \quad (645)$$

As métricas principais são

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}, \quad (646)$$

$$MAE = \frac{1}{N} \sum_{i=1}^N |e_i|, \quad (647)$$

e a correlação

$$r = \frac{\sum_i (d_i^{true} - \bar{d}^{true})(d_i^{pred} - \bar{d}^{pred})}{\sqrt{\sum_i (d_i^{true} - \bar{d}^{true})^2} \sqrt{\sum_i (d_i^{pred} - \bar{d}^{pred})^2}}. \quad (648)$$

Avaliação de testes sintéticos

A comparação quantitativa entre modelo verdadeiro e modelo estimado permite avaliar objetivamente o desempenho de um método. Essa etapa é essencial para validar algoritmos antes da aplicação em dados reais.

24.10 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      import numpy as np
2      from gravmag_codes import synthetic
3      from gravmag_codes import prism
4      from gravmag_codes import statistical
5
6      # Grade de observacao
7      area = [-5000, 5000, -5000, 5000]
8      shape = (101, 101)
9
10     X, Y, Z = synthetic.my_make_observation_grid(
11         area=area,
12         shape=shape,
13         level=0.0,
14         flatten=False
15     )
16
17     # Fonte sintetica
18     p = prism.my_prism_model(
19         x1=-1000, x2=1000,
20         y1=-1000, y2=1000,
21         z1=1000, z2=3000,
22         physical_property=0.5
23     )
24
25     # Campo gravimetrico sintetico
26     gz_true = prism.my_prism_gz(X, Y, Z, p, rho=None)
27
28     # Adicao de ruido
29     gz_obs = synthetic.my_add_noise(
30         gz_true,
31         noise_std=0.05,
32         seed=10
33     )
34
35     # Comparacao
36     rmse = statistical.my_rmse(gz_true, gz_obs)
```


37

```
mae = statistical.my_mae(gz_true, gz_obs)
```

24.11 Organização de experimentos sintéticos

O módulo **synthetic.py** também pode ser usado para organizar experimentos completos. Um fluxo típico envolve as seguintes etapas: criação da grade de observação, definição das fontes, cálculo do campo verdadeiro, adição de ruído, aplicação de um método de processamento ou inversão, comparação quantitativa e salvamento dos resultados.

Matematicamente, esse fluxo pode ser resumido por

$$\mathbf{m}_{true} \rightarrow \mathbf{d}_{true} \rightarrow \mathbf{d}_{obs} \rightarrow \mathbf{m}_{est} \rightarrow \mathbf{d}_{pred} \rightarrow \mathbf{e}. \quad (649)$$

Nesse esquema, $\mathbf{m}_{true}$ é o modelo verdadeiro, $\mathbf{d}_{true}$ é o dado sintético limpo, $\mathbf{d}_{obs}$ é o dado contaminado por ruído, $\mathbf{m}_{est}$ é o modelo estimado, $\mathbf{d}_{pred}$ é o dado predito e $\mathbf{e}$ é o erro.

24.12 Síntese técnica do módulo

O módulo **synthetic.py** é uma camada de integração voltada à criação de testes sintéticos em Geofísica Computacional. Ele organiza a construção de grades, fontes, topografias, propriedades físicas, respostas gravimétricas e magnéticas, ruído e métricas de comparação.

Importância para ensino de Geofísica

O **synthetic.py** é extremamente útil para aulas práticas porque permite construir experimentos controlados. O estudante pode modificar geometria, profundidade, densidade, magnetização e ruído, observando diretamente como cada parâmetro afeta os dados geofísicos.

Do ponto de vista científico, esse módulo é fundamental para validação de algoritmos, testes de sensibilidade, comparação de métodos, construção de figuras de artigos e documentação de exemplos reprodutíveis para o futuro pacote Python instalável via **pip**.

25 Descrição técnica do módulo `transform.py`

O arquivo `transform.py` reúne funções voltadas à transformação de campos potenciais, especialmente dados gravimétricos e magnéticos organizados em malhas regulares. Esse módulo atua como uma ponte entre operações matemáticas no domínio espacial e no domínio da frequência, permitindo aplicar transformações clássicas como continuação de campo, derivadas espectrais, redução ao polo, redução ao equador, pseudogravidade, separação regional-residual e outros filtros úteis à interpretação geofísica.

Em métodos potenciais, transformar um campo observado significa alterar sua representação matemática para realçar determinadas características das fontes. Por exemplo, a continuação para cima suaviza o campo e enfatiza fontes profundas; a derivada vertical realça fontes rasas; a redução ao polo procura reposicionar anomalias magnéticas sobre suas fontes; e a pseudogravidade converte, sob hipóteses específicas, um campo magnético em um campo análogo ao gravimétrico.

Papel do módulo `transform.py`

O módulo `transform.py` concentra transformações aplicadas a campos potenciais. Ele permite modificar dados geofísicos para facilitar a interpretação estrutural, a separação de fontes, o realce de bordas e a comparação entre respostas gravimétricas e magnéticas.

A formulação geral das transformações espectrais parte da transformada de Fourier bidimensional:

$$F(k_x, k_y) = \mathcal{F}\{f(x, y)\}, \quad (650)$$

em que $f(x, y)$ é o campo observado no domínio espacial e $F(k_x, k_y)$ é sua representação no domínio dos números de onda. Um operador de transformação $H(k_x, k_y)$ é aplicado por multiplicação:

$$F_T(k_x, k_y) = F(k_x, k_y)H(k_x, k_y). \quad (651)$$

O campo transformado é então obtido por transformada inversa:

$$f_T(x, y) = \mathcal{F}^{-1}\{F(k_x, k_y)H(k_x, k_y)\}. \quad (652)$$

25.1 Transformações por continuação vertical

A continuação vertical é uma das transformações mais importantes em campos potenciais. Ela permite estimar como o campo observado se comportaria em outro nível de altitude ou profundidade. Para um campo potencial harmônico, a continuação para cima é dada por

$$F_h(k_x, k_y) = F_0(k_x, k_y)e^{-kh}, \quad (653)$$

em que

$$k = \sqrt{k_x^2 + k_y^2} \quad (654)$$

é o módulo do número de onda, e $h > 0$ representa a altura de continuação.

A continuação para cima reduz componentes de alta frequência, pois o fator exponencial e^{-kh} decai mais rapidamente para grandes valores de k . Assim, fontes rasas e ruídos de pequeno comprimento de onda são suavizados.

A continuação para baixo, por outro lado, envolve

$$F_{-h}(k_x, k_y) = F_0(k_x, k_y)e^{kh}. \quad (655)$$

Essa operação é numericamente instável, pois amplifica fortemente altas frequências. Por isso, quando implementada, deve ser acompanhada de regularização ou filtros de corte.

Continuação para baixo

A continuação para baixo deve ser usada com cautela. Embora possa aproximar o campo das fontes, ela amplifica ruído e erros de amostragem. Em aplicações reais, recomenda-se utilizar filtros estabilizadores.

25.2 Derivadas espectrais

As derivadas espaciais podem ser calculadas no domínio da frequência. A derivada de ordem n em relação a x é dada por

$$\mathcal{F} \left\{ \frac{\partial^n f}{\partial x^n} \right\} = (ik_x)^n F(k_x, k_y), \quad (656)$$

enquanto a derivada em relação a y é

$$\mathcal{F} \left\{ \frac{\partial^n f}{\partial y^n} \right\} = (ik_y)^n F(k_x, k_y). \quad (657)$$

Para campos potenciais, a derivada vertical é associada ao módulo do número de onda:

$$\mathcal{F} \left\{ \frac{\partial^n f}{\partial z^n} \right\} = k^n F(k_x, k_y), \quad (658)$$

considerando a convenção de eixo vertical positivo para baixo. Se a convenção for oposta, o sinal do operador vertical deve ser ajustado.

As derivadas espectrais são usadas para realçar bordas, contatos, fontes rasas e variações abruptas no campo observado.

25.3 Gradiente horizontal, gradiente total e sinal analítico

A partir das derivadas horizontais, define-se a amplitude do gradiente horizontal:

$$\text{HGA} = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}. \quad (659)$$

Esse atributo tende a apresentar máximos próximos às bordas laterais de fontes causadoras de anomalias.

O gradiente total, também associado ao sinal analítico em algumas aplicações magnéticas, é dado por

$$\text{TGA} = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2 + \left(\frac{\partial f}{\partial z}\right)^2}. \quad (660)$$

Uso interpretativo dos gradientes

O gradiente horizontal é útil para localizar limites laterais de fontes, enquanto o gradiente total combina variações horizontais e verticais, destacando zonas de forte contraste físico.

25.4 Redução ao polo e redução ao equador

As transformações magnéticas dependem da direção do campo geomagnético regional e da direção da magnetização das fontes. Para uma direção definida por inclinação I e declinação D , o vetor unitário é

$$\hat{\mathbf{F}} = (f_x, f_y, f_z), \quad (661)$$

com

$$f_x = \cos I \cos D, \quad f_y = \cos I \sin D, \quad f_z = \sin I. \quad (662)$$

No domínio da frequência, um operador direcional magnético pode ser escrito como

$$\Theta(k_x, k_y) = f_z + i \frac{f_x k_x + f_y k_y}{k + \varepsilon}. \quad (663)$$

A redução ao polo busca transformar o campo observado como se a inclinação fosse

$$I = 90^\circ \quad (664)$$

e a declinação, geralmente,

$$D = 0^\circ. \quad (665)$$

O operador conceitual é uma razão entre o operador desejado e o operador original:

$$H_{RTP} = \frac{\Theta_{polo}}{\Theta_{original} + \varepsilon}. \quad (666)$$

A redução ao equador utiliza uma inclinação próxima de

$$I = 0^\circ, \quad (667)$$

sendo útil em regiões de baixa latitude magnética, onde a redução ao polo pode ser instável.

25.5 Pseudogravidade

A pseudogravidade é uma transformação que procura converter uma anomalia magnética em um campo análogo ao gravimétrico, assumindo uma relação entre magnetização e densidade. Essa transformação é útil quando se deseja interpretar dados magnéticos com ferramentas semelhantes às usadas em gravimetria.

De forma conceitual, a pseudogravidade pode ser escrita como

$$g_p(k_x, k_y) = C \frac{F_m(k_x, k_y)}{\Theta_F(k_x, k_y)\Theta_M(k_x, k_y) + \varepsilon}, \quad (668)$$

em que F_m é o espectro do campo magnético, Θ_F representa o operador do campo regional, Θ_M representa o operador da magnetização e C é um fator de escala dependente das propriedades físicas assumidas.

Hipóteses da pseudogravidade

A pseudogravidade depende de hipóteses sobre direção de magnetização, direção do campo regional e relação entre magnetização e densidade. Portanto, sua interpretação deve ser feita com cuidado.

25.6 Separação regional-residual

A separação regional-residual busca decompor um campo observado em duas parcelas:

$$d(x, y) = d_R(x, y) + d_r(x, y), \quad (669)$$

em que d_R é a componente regional e d_r é a componente residual.

No domínio espectral, essa separação pode ser feita por filtros passa-baixa e passa-alta. O regional pode ser obtido por

$$D_R(k_x, k_y) = D(k_x, k_y)H_{LP}(k), \quad (670)$$

enquanto o residual pode ser obtido por

$$D_r(k_x, k_y) = D(k_x, k_y)H_{HP}(k). \quad (671)$$

Como

$$H_{HP}(k) = 1 - H_{LP}(k), \quad (672)$$

tem-se, aproximadamente,

$$d_r(x, y) = d(x, y) - d_R(x, y). \quad (673)$$

25.7 Atributos tilt e theta map

O tilt derivative é definido por

$$\text{TDR} = \arctan \left[\frac{\partial f / \partial z}{\sqrt{(\partial f / \partial x)^2 + (\partial f / \partial y)^2 + \varepsilon}} \right]. \quad (674)$$

Esse atributo normaliza a derivada vertical pela amplitude do gradiente horizontal, reduzindo a influência da amplitude absoluta da anomalia.

O theta map pode ser escrito como

$$\Theta = \frac{\text{HGA}}{\sqrt{\text{HGA}^2 + (\partial f / \partial z)^2 + \varepsilon}}. \quad (675)$$

Esses atributos são úteis para realce de bordas e interpretação estrutural.

25.8 Exemplo conceitual de uso

Exemplo de uso em Python

```

1      import numpy as np
2      from gravmag_codes import transform
3
4      # Campo sintetico em grade
5      x = np.linspace(-5000, 5000, 101)
6      y = np.linspace(-5000, 5000, 101)
7      X, Y = np.meshgrid(x, y)
8
9      D = np.exp(-((X/1200)**2 + (Y/1500)**2))
10

```

```
11      # Continuação para cima
12      D_up = transform.my_upward_continuation(
13          X, Y, D,
14          height=500.0,
15          flatten=False
16      )
17
18      # Derivada vertical
19      Dz = transform.my_zderiv(
20          X, Y, D,
21          n=1,
22          flatten=False
23      )
24
25      # Tilt derivative
26      tilt = transform.my_tilt(
27          X, Y, D,
28          degrees=True,
29          flatten=False
30      )
```

25.9 Síntese técnica do módulo

O módulo **transform.py** fornece ferramentas para transformar campos potenciais no domínio espacial e espectral. Suas funções são fundamentais para realce de fontes, separação regional-residual, continuação de campo, redução magnética, pseudogravidade e cálculo de atributos interpretativos.

Importância para ensino de Geofísica

O **transform.py** permite demonstrar como a transformada de Fourier, derivadas espaciais e operadores direcionais são aplicados diretamente à interpretação de dados gravimétricos e magnéticos. Ele conecta Matemática Aplicada, Física e Geofísica Computacional.

Do ponto de vista científico, o módulo é importante para etapas de pré-processamento, interpretação qualitativa, geração de atributos e preparação de dados para inversão ou modelagem quantitativa.

26 Descrição técnica do módulo `utils.py`

O arquivo `utils.py` reúne funções utilitárias gerais utilizadas por diferentes partes do pacote `gravmag_codes`. Esse módulo normalmente concentra rotinas de apoio que não pertencem exclusivamente a um método geofísico específico, mas que são necessárias para organizar fluxos de trabalho, validar entradas, salvar arquivos, criar diretórios, formatar saídas, controlar mensagens, verificar dependências e padronizar operações recorrentes.

Em pacotes científicos, módulos utilitários são importantes porque reduzem repetição de código e tornam o projeto mais organizado. Em vez de cada módulo implementar sua própria rotina de criação de pastas, salvamento de tabelas, verificação de arrays ou impressão de mensagens, essas tarefas podem ser centralizadas em um único arquivo.

Papel do módulo `utils.py`

O módulo `utils.py` fornece funções auxiliares de caráter geral para organização, validação, entrada e saída de dados, controle de diretórios, mensagens e suporte computacional ao pacote.

Embora suas funções não representem, isoladamente, métodos geofísicos, elas são importantes para garantir robustez, reprodutibilidade e manutenção do pacote.

26.1 Validação de entradas

Uma das tarefas comuns em um módulo utilitário é validar entradas fornecidas pelo usuário. Em computação científica, erros de forma, tamanho ou tipo podem gerar resultados incorretos ou mensagens pouco claras. Por isso, funções de validação verificam se arrays possuem dimensões compatíveis, se parâmetros numéricos estão em intervalos válidos e se entradas obrigatórias foram informadas.

Para arrays usados em operações ponto a ponto, a condição típica é

$$\text{shape}(x) = \text{shape}(y) = \text{shape}(d). \quad (676)$$

Quando essa condição não é satisfeita, a operação perde sentido físico, pois não há correspondência direta entre coordenadas e valores observados.

Exemplo conceitual de validação

```
1         from gravmag_codes import utils
2
3         utils.my_check_same_shape(x, y, data)
4         utils.my_check_positive(radius, name="radius")
5         utils.my_check_area(area)
```

26.2 Conversão e preparação de arrays

Outra função típica do módulo é converter entradas para arrays NumPy. Essa etapa permite que funções aceitem escalares, listas, tuplas ou arrays, mantendo flexibilidade para o usuário.

Uma entrada genérica

$$x = \{x_1, x_2, \dots, x_N\} \quad (677)$$

pode ser convertida para um array NumPy, permitindo operações vetorizadas como

$$y_i = f(x_i). \quad (678)$$

A vetorização é essencial para desempenho computacional em Python, especialmente em grades geofísicas com milhares ou milhões de pontos.

26.3 Criação de diretórios e organização de saídas

Em fluxos de trabalho geofísicos, é comum gerar muitos arquivos de saída, como tabelas, mapas, figuras, modelos e relatórios. O módulo **utils.py** pode incluir funções para criar automaticamente pastas de saída, verificar se diretórios existem e organizar arquivos por tipo.

Exemplo conceitual de criação de diretórios

```
1         from gravmag_codes import utils
2
3         utils.my_make_dir("resultados")
4         utils.my_make_dir("figuras")
```

```
5      utils.my_make_dir("dados")
```

Essa organização é importante para reprodutibilidade, especialmente quando o pacote for disponibilizado no GitHub e instalado via `pip`.

26.4 Salvamento de tabelas e resultados

O módulo também pode conter funções para salvar arrays, tabelas e dicionários de resultados. Uma tabela geofísica simples pode ser organizada como

$$[x, y, z, d], \quad (679)$$

em que x , y e z são coordenadas e d é o dado geofísico.

O salvamento padronizado facilita a reutilização dos resultados em outros programas, notebooks ou módulos.

Exemplo conceitual de salvamento

```
1      from gravmag_codes import utils
2
3      utils.my_save_table(
4          filename="dados/campo_gravimetrico.txt",
5          columns=[x, y, z, gz],
6          header="x y z gz"
7      )
```

26.5 Mensagens, relatórios e impressão organizada

Funções utilitárias também podem ser usadas para imprimir mensagens padronizadas, cabeçalhos, resumos de execução e relatórios simples. Isso é útil quando scripts longos executam várias etapas de modelagem, filtragem ou inversão.

Um relatório computacional pode incluir informações como:

$$N_{\text{dados}}, \quad N_{\text{modelo}}, \quad RMSE, \quad MAE, \quad r, \quad \text{tempo de execução}. \quad (680)$$

A padronização das mensagens melhora a leitura dos resultados e facilita a identificação de erros.

26.6 Controle de dependências

Em pacotes Python, algumas funcionalidades dependem de bibliotecas opcionais. Por exemplo, mapas cartográficos podem depender do **Cartopy**, interpolação pode depender do **SciPy**, e manipulação de dados tabulares pode depender do **pandas**. O módulo utilitário pode incluir funções para verificar se essas bibliotecas estão disponíveis.

Exemplo conceitual de verificação

```
1      from gravmag_codes import utils
2
3      utils.my_check_dependency("cartopy")
4      utils.my_check_dependency("scipy")
```

Quando uma dependência não está instalada, a função pode gerar uma mensagem clara, orientando o usuário a instalar o pacote necessário.

26.7 Medição de tempo de execução

Em Geofísica Computacional, o custo computacional é um aspecto importante, especialmente em modelagem direta 3D, inversão e testes com muitos pontos de observação. Funções utilitárias podem medir o tempo de execução de etapas específicas.

Se t_0 é o tempo inicial e t_1 é o tempo final, o tempo decorrido é

$$\Delta t = t_1 - t_0. \quad (681)$$

Esse valor pode ser usado para comparar algoritmos, avaliar desempenho e otimizar rotinas.

26.8 Reprodutibilidade

A reprodutibilidade é um princípio fundamental em computação científica. O módulo **utils.py** pode auxiliar nesse aspecto por meio de funções para controlar sementes aleatórias, salvar parâmetros de execução e registrar configurações usadas em um experimento.

Por exemplo, ao adicionar ruído Gaussiano a um dado sintético,

$$d_i^{\text{ruidoso}} = d_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (682)$$

o uso de uma semente aleatória fixa garante que o mesmo ruído seja gerado novamente em execuções futuras.

Importância da reprodutibilidade

Ao disponibilizar o pacote no GitHub, exemplos reprodutíveis são essenciais para que outros usuários consigam executar os mesmos testes, obter os mesmos resultados e validar o funcionamento do módulo.

26.9 Exemplo conceitual de uso

Exemplo de uso em Python

```
1      from gravmag_codes import utils
2
3      # Cria pastas de saída
4      utils.my_make_dir("resultados")
5      utils.my_make_dir("figuras")
6
7      # Verifica arrays
8      utils.my_check_same_shape(x, y, data)
9
10     # Salva tabela
11     utils.my_save_table(
12         "resultados/dados_processados.txt",
13         columns=[x, y, data],
14         header="x y data"
15     )
16
17     # Imprime resumo
18     utils.my_print_header("Processamento concluído")
19     utils.my_print_summary({
20         "numero_de_dados": len(data),
```

```
21         "valor_minimo": data.min(),
22         "valor_maximo": data.max()
23     })
```

26.10 Síntese técnica do módulo

O módulo `utils.py` fornece suporte geral ao pacote `gravmag_codes`. Ele concentra funções auxiliares para validação, conversão de dados, organização de arquivos, criação de diretórios, salvamento de resultados, controle de dependências, impressão de mensagens e reprodutibilidade.

Importância para ensino e desenvolvimento

O `utils.py` ajuda a mostrar aos estudantes e usuários que um pacote científico não é composto apenas por fórmulas físicas. A organização computacional, a validação de entradas, o salvamento adequado dos resultados e a reprodutibilidade são partes fundamentais de um projeto científico bem estruturado.

Do ponto de vista do futuro empacotamento no GitHub e instalação via `pip`, esse módulo é importante porque melhora a manutenção do código, reduz repetições e padroniza operações comuns em todo o pacote.

27 Síntese integrada do pacote `gravmag`

O pacote `gravmag` foi estruturado como um conjunto de módulos computacionais voltados ao ensino, à modelagem e ao processamento de dados geofísicos. Embora o nome do pacote remeta diretamente aos métodos gravimétricos e magnéticos, sua estrutura foi ampliada para incluir também ferramentas de métodos elétricos, eletromagnéticos, sísmicos, radiométricos, perfilagem de poços, estatística, regressão, inversão, visualização, construção de malhas e geração de modelos sintéticos.

Do ponto de vista científico e educacional, o pacote pode ser compreendido como uma biblioteca modular para Geofísica Computacional, na qual cada arquivo `.py` desempenha uma função específica dentro de um fluxo de trabalho mais amplo. Essa organização permite que o usuário trabalhe desde conceitos matemáticos fundamentais, como derivadas,

núcleos potenciais e regressão, até aplicações geofísicas completas, como modelagem direta, filtragem, inversão e interpretação de anomalias.

Visão geral do pacote

O pacote **gravmag** integra módulos de constantes físicas, geometria de fontes, modelagem direta, processamento espectral, inversão, estatística, visualização, métodos geofísicos complementares e criação de modelos sintéticos. Essa estrutura favorece tanto o uso em pesquisa quanto o uso em aulas práticas de Geofísica.

27.1 Estrutura conceitual do pacote

A estrutura geral do pacote pode ser organizada em grandes blocos funcionais. O primeiro bloco é formado pelos módulos de suporte matemático e físico, como **constants.py**, **kernel.py**, **derivative.py**, **filtering.py**, **transform.py**, **statistical.py**, **regres.py**, **inversion.py** e **utils.py**. Esses módulos fornecem a base numérica, estatística e física sobre a qual os demais cálculos são construídos.

O segundo bloco é formado pelos módulos de geometria e modelagem direta de fontes, como **sphere.py**, **prism.py**, **cylinder.py**, **hexagonalprism.py** e **polygonalprism.py**. Esses módulos permitem criar corpos geofísicos simples ou compostos e calcular suas respostas gravimétricas e magnéticas.

O terceiro bloco é formado por módulos de métodos geofísicos específicos, como **gravity.py**, **geomagnetism.py**, **electrical_methods.py**, **electromagnetic_methods.py**, **seismic_methods.py** e **radiometry_welllog_methods.py**. Esses arquivos ampliam o escopo do pacote, permitindo trabalhar com diferentes famílias de métodos geofísicos.

O quarto bloco é formado por módulos de organização, criação de modelos e visualização, como **grids.py**, **plotting.py** e **synthetic.py**. Esses módulos ajudam a construir malhas, gerar exemplos, visualizar resultados e organizar experimentos sintéticos.

27.2 Fluxo computacional geral

Um fluxo típico de uso do pacote pode ser representado por uma sequência de etapas. Inicialmente, define-se uma malha de observação por meio do módulo **grids.py**. Em seguida, cria-se uma fonte ou conjunto de fontes usando módulos como **prism.py**, **sphere.py**,

`cylinder.py`, `hexagonalprism.py` ou `polygonalprism.py`. Depois, calcula-se a resposta geofísica direta, como gravidade, campo magnético ou anomalia de campo total.

Matematicamente, a modelagem direta pode ser representada por

$$\mathbf{d} = \mathcal{F}(\mathbf{m}), \quad (683)$$

em que $\mathbf{m}$ representa o modelo físico, $\mathcal{F}$ representa o operador direto e $\mathbf{d}$ representa os dados calculados.

Em problemas lineares, essa relação pode ser escrita como

$$\mathbf{d} = \mathbf{G}\mathbf{m}, \quad (684)$$

em que $\mathbf{G}$ é a matriz de sensibilidade.

Após a geração ou leitura dos dados, o pacote permite aplicar filtros e transformações:

$$\mathbf{d}_{processado} = \mathcal{T}(\mathbf{d}), \quad (685)$$

em que $\mathcal{T}$ pode representar continuação para cima, derivada vertical, redução ao polo, separação regional-residual, tilt derivative ou outro operador.

Por fim, os resultados podem ser analisados estatisticamente, visualizados em mapas ou usados em problemas inversos:

$$\mathbf{m}_{est} = \mathcal{I}(\mathbf{d}_{obs}), \quad (686)$$

em que $\mathcal{I}$ representa o processo de inversão ou estimação de parâmetros.

27.3 Contribuição didática

O pacote possui forte potencial didático porque transforma conceitos teóricos de Geofísica em rotinas computacionais acessíveis. Por exemplo, a lei da gravitação de Newton, a modelagem de prismas, a resposta magnética de dipolos, a transformada de Fourier, as derivadas espaciais, a regularização de Tikhonov e a regressão polinomial podem ser exploradas diretamente em notebooks ou scripts Python.

Essa abordagem favorece o aprendizado ativo, pois o estudante pode alterar parâmetros como profundidade, densidade, magnetização, raio, forma geométrica, ruído e espaçamento da malha, observando imediatamente os efeitos produzidos nos campos

calculados.

Uso em ensino de Geofísica

O pacote pode ser usado em aulas práticas de Geofísica para demonstrar a relação entre modelo físico, formulação matemática, algoritmo computacional e interpretação geológica. Essa integração é especialmente importante em cursos que buscam aproximar Matemática, Física, Programação e Ciências da Terra.

27.4 Contribuição científica

Do ponto de vista científico, o pacote pode servir como base para testes sintéticos, validação de metodologias, desenvolvimento de novos algoritmos e construção de exemplos reprodutíveis. A presença de módulos de modelagem direta, filtragem, inversão e estatística permite elaborar fluxos completos de análise.

Em um artigo científico voltado ao ensino de Geofísica, o pacote pode ser apresentado como uma ferramenta computacional aberta, modular e reprodutível para apoiar o ensino de métodos geofísicos. A contribuição central não está apenas na implementação de fórmulas conhecidas, mas na organização integrada dessas fórmulas em um ambiente didático, expansível e compatível com práticas modernas de ciência aberta.

28 Organização recomendada para disponibilização no GitHub e instalação via `pip`

Para que o pacote `gravmag` possa ser disponibilizado no GitHub e futuramente instalado por outros usuários usando o comando `pip`, é recomendável organizar os arquivos em uma estrutura padronizada de projeto Python. Essa organização facilita a manutenção, a documentação, os testes e a distribuição do código.

Objetivo da organização

A organização do pacote deve permitir que outros usuários possam baixar, instalar, importar e utilizar os módulos de forma simples, por exemplo, com comandos como `pip install gravmag` e `import gravmag`.

28.1 Estrutura básica sugerida

Uma estrutura inicial recomendada é:

Estrutura de diretórios

```
1      gravmag/
2      |
3      +-- gravmag/
4      |   +-- __init__.py
5      |   +-- auxiliars.py
6      |   +-- constants.py
7      |   +-- cylinder.py
8      |   +-- derivative.py
9      |   +-- electrical_methods.py
10     |   +-- electromagnetic_methods.py
11     |   +-- equivalentlayer.py
12     |   +-- filtering.py
13     |   +-- geomagnetism.py
14     |   +-- gravity.py
15     |   +-- grids.py
16     |   +-- hexagonalprism.py
17     |   +-- inversion.py
18     |   +-- kernel.py
19     |   +-- plotting.py
20     |   +-- polygonalprism.py
21     |   +-- prism.py
22     |   +-- radiometry_welllog_methods.py
23     |   +-- regres.py
24     |   +-- seismic_methods.py
25     |   +-- sphere.py
26     |   +-- statistical.py
27     |   +-- synthetic.py
28     |   +-- transform.py
29     |   '-- utils.py
30     |
31     +-- examples/
32     |   +-- exemplo_01_esfera_gravimetria.ipynb
33     |   +-- exemplo_02_prisma_magnetometria.ipynb
34     |   +-- exemplo_03_filtragem_fft.ipynb
35     |   '-- exemplo_04_inversao_linear.ipynb
```

```
36         |
37     +-- tests/
38         |   +-- test_constants.py
39         |   +-- test_grids.py
40         |   +-- test_sphere.py
41         |   '-- test_prism.py
42         |
43     +-- docs/
44         |   +-- index.md
45         |   +-- installation.md
46         |   +-- tutorials.md
47         |   '-- api_reference.md
48         |
49     +-- pyproject.toml
50     +-- README.md
51     +-- LICENSE
52     +-- CITATION.cff
53     '-- requirements.txt
```

Nessa estrutura, a pasta interna **gravmag/** contém os módulos do pacote. A pasta **examples/** armazena notebooks ou scripts demonstrativos. A pasta **tests/** contém testes automatizados. A pasta **docs/** concentra a documentação.

28.2 Arquivo `__init__.py`

O arquivo `__init__.py` transforma a pasta **gravmag/** em um pacote Python importável. Ele pode ser usado para importar módulos principais e definir a versão do pacote.

Exemplo de `__init__.py`

```
1         """
2         gravmag
3         =
4
5         A Python package for teaching and computational
           modeling
6         in geophysics, with emphasis on gravity and magnetic
           methods.
7         """
```

```
8
9      __version__ = "0.1.0"
10
11     from . import constants
12     from . import grids
13     from . import prism
14     from . import sphere
15     from . import cylinder
16     from . import filtering
17     from . import transform
18     from . import inversion
19     from . import plotting
20     from . import synthetic
```

Com isso, o usuário poderá importar o pacote com:

Importação do pacote

```
1      import gravmag
2      from gravmag import prism
3      from gravmag import sphere
4      from gravmag import grids
```

28.3 Arquivo `pyproject.toml`

O arquivo `pyproject.toml` é atualmente uma das formas recomendadas para configurar pacotes Python. Ele informa o nome do pacote, versão, autores, dependências e configurações de empacotamento.

Exemplo de `pyproject.toml`

```
1      [build-system]
2      requires = ["setuptools>=61.0", "wheel"]
3      build-backend = "setuptools.build_meta"
4
5      [project]
```

```
6     name = "gravmag"
7     version = "0.1.0"
8     description = "A Python package for teaching and
9         computational geophysics"
10    readme = "README.md"
11    requires-python = ">=3.9"
12    license = {text = "MIT"}
13    authors = [
14        {name = "Nelson Ribeiro Filho", email = "
15            nelsondelimar@gmail.com"}
16    ]
17    keywords = [
18        "geophysics",
19        "gravity",
20        "magnetics",
21        "geophysical modeling",
22        "education",
23        "python"
24    ]
25    classifiers = [
26        "Programming Language :: Python :: 3",
27        "Intended Audience :: Education",
28        "Intended Audience :: Science/Research",
29        "Topic :: Scientific/Engineering",
30        "License :: OSI Approved :: MIT License"
31    ]
32    dependencies = [
33        "numpy",
34        "scipy",
35        "matplotlib"
36    ]
37
38    [project.optional-dependencies]
39    maps = ["cartopy"]
40    docs = ["mkdocs", "mkdocs-material"]
41    tests = ["pytest"]
42
43    [tool.setuptools.packages.find]
44    where = ["."]
45    include = ["gravmag*"]
```

28.4 Arquivo README.md

O arquivo **README.md** deve apresentar o pacote de forma clara. Recomenda-se incluir: descrição geral, objetivos, instalação, exemplo rápido, lista de módulos, citação sugerida e licença.

Um exemplo de início para o README.md seria:

Exemplo de introdução para README.md

```
1      # gravmag
2
3      'gravmag' is a Python package for teaching and
4      computational
5      geophysics, with emphasis on gravity and magnetic
6      methods.
7
8      The package includes tools for:
9
10     - gravity and magnetic forward modeling;
11     - potential-field filtering and transformations;
12     - equivalent layer methods;
13     - synthetic model generation;
14     - linear and nonlinear inversion;
15     - electrical, electromagnetic, seismic and
16     radiometric examples;
17     - geophysical visualization and statistical
18     validation.
```

28.5 Instalação local para testes

Antes de publicar o pacote, recomenda-se testar a instalação local. Dentro da pasta principal do projeto, o usuário pode executar:

Instalação local editável

```
1 pip install -e .
```

A opção `-e` instala o pacote em modo editável. Isso significa que alterações feitas nos arquivos `.py` serão refletidas imediatamente sem necessidade de reinstalação.

Depois da instalação, pode-se testar:

Teste de importação

```
1 import gravmag
2 from gravmag import sphere
3 from gravmag import prism
4
5 print(gravmag.__version__)
```

28.6 Testes automatizados

A criação de testes automatizados é importante para verificar se as funções continuam funcionando após alterações no código. Um teste simples para o módulo **sphere.py** poderia verificar se o volume de uma esfera está correto:

Exemplo de teste com pytest

```
1 import numpy as np
2 from gravmag import sphere
3
4 def test_sphere_volume():
5     s = sphere.my_sphere_model(0, 0, 1000, 10)
6     volume = sphere.my_sphere_volume(s)
7     expected = 4.0/3.0*np.pi*10**3
8     assert np.isclose(volume, expected)
```

Os testes podem ser executados com:

Execução dos testes

```
1      pytest
```

28.7 Licença e citação

Para permitir uso acadêmico e educacional, recomenda-se incluir uma licença aberta, como MIT, BSD ou GPL. Além disso, o arquivo **CITATION.cff** pode indicar como o pacote deve ser citado.

Exemplo simplificado de CITATION.cff

```
1      cff-version: 1.2.0
2      title: "gravmag: A Python package for teaching and
3             computational geophysics"
4      message: "If you use this software, please cite it as
5             below."
6      type: software
7      authors:
8      - family-names: "Ribeiro Filho"
9      given-names: "Nelson"
10     version: "0.1.0"
11     date-released: "2026-01-01"
```

28.8 Síntese da organização para distribuição

A organização do pacote para GitHub e instalação via pip representa a etapa final de consolidação do projeto. Depois que os módulos estiverem revisados, documentados e testados, o pacote poderá ser disponibilizado publicamente como uma ferramenta aberta para ensino e pesquisa em Geofísica Computacional.

Etapa final do projeto

A disponibilização no GitHub deve ocorrer após a revisão dos módulos, criação dos exemplos, escrita do `README.md`, definição da licença, inclusão de testes automatizados e configuração do `pyproject.toml`. Essa etapa transformará o conjunto de scripts em um pacote Python reutilizável.

Com essa estrutura, o pacote **gravmag** poderá ser usado em aulas, projetos de pesquisa, trabalhos de conclusão, artigos científicos e atividades de extensão envolvendo Matemática Aplicada, Física e Geofísica.